Auto-BI: Automatically Build BI-Models Leveraging Local Join Prediction and Global Schema Graph

Yiming Lin*
University of California, Irvine
yiminl18@uci.edu

Yeye He
Microsoft Research
yeyehe@microsoft.com

Surajit Chaudhuri
Microsoft Research
surajitc@microsoft.com

ABSTRACT

Business Intelligence (BI) is crucial in modern enterprises and billion-dollar business. Traditionally, technical experts like database administrators would manually prepare BI-models (e.g., in star or snowflake schemas) that join tables in data warehouses, before less-technical business users can run analytics using end-user dashboarding tools. However, the popularity of self-service BI (e.g., Tableau and Power-BI) in recent years creates a strong demand for less technical end-users to build BI-models themselves.

We develop an Auto-BI system that can accurately predict BI models given a set of input tables, using a principled graph-based optimization problem we propose called *k-Min-Cost-Arborescence* (k-MCA), which holistically considers both local join prediction and global schema-graph structures, leveraging a graph-theoretical structure called *arborescence*. While we prove k-MCA is intractable and inapproximate in general, we develop novel algorithms that can solve k-MCA optimally, which is shown to be efficient in practice with sub-second latency and can scale to the largest BI-models we encounter (with close to 100 tables).

Auto-BI is rigorously evaluated on a unique dataset with over 100K real BI models we harvested, as well as on 4 popular TPC benchmarks. It is shown to be both efficient and accurate, achieving over 0.9 F1-score on both real and synthetic benchmarks.

PVLDB Reference Format:
Yiming Lin, Yeye He, and Surajit Chaudhuri. Auto-BI: Automatically Build BI-Models
Leveraging Local Join Prediction and Global Schema Graph. PVLDB, 16(10): 2578 - 2590, 2023.
doi:10.14778/3603581.3603596

1 INTRODUCTION

Business Intelligence (BI) is increasingly important in modern enterprises for data-driven decision making, and has grown into a multi-billion dollar business [25]. In traditional BI settings, database administrators (DBAs) typically need to manually prepare BI-models (table schemas and join relationships) in data warehouses, so that less-technical business users can perform ad-hoc analysis using tools like dashboards [16].

In recent years, in a growing trend called "*self-service BI*" [26] that is popularized by vendors like Tableau [11] and Power-BI [9], less-technical business users are increasingly expected to set up and perform BI analysis themselves, without relying on DBAs or central IT. The goal is to democratize BI, so that business users can make agile data-driven decisions themselves, without depending on technical users.

Building BI models: still a pain point. At a high level, there are two main steps in BI project: (1) building BI models, and (2) performing ad-hoc analysis by querying against BI models. While querying BI-models was made simple by vendors like Tableau and Power-BI (through intuitive user interfaces and dashboards) [28, 38], the first step of building "BI-models", a prerequisite for ad-hoc analysis, remains a key pain point for non-technical users.

In the context of self-service BI tools like Tableau and Power-BI, "BI-modeling" refers to the process of preparing raw data and establishing relationships between tables, where a central task is to establish join relationships for a given set of input tables[1]. This closely relates to foreign-key (FK) detection [18, 30, 57] but works specifically in the context of BI, where the resulting schema graphs from the modeling step frequently correspond to structures known as star-schema and snowflake-schema studied in data warehouses [16], like shown in Figure 1.

While self-service BI tools also attempt to improve the usability of the BI-modeling step through better GUI (e.g., allowing users to specify join columns using drag-and-drop) [10, 12], building BI models remains a key pain point. This is because when faced with a large number of tables, even experienced technical users like DBAs can find the task of identifying all possible join relationships challenging and time-consuming. For less-technical enterprise users who are not familiar with concepts like fact/dimension tables, building BI models from scratch can be a daunting challenge.

Foreign-key detection: not yet sufficient. It would clearly be useful, if the join relationships in BI models can be automatically predicted on given input tables (without requiring users to specify them manually). Since joins in BI models are often primary-key (PK) foreign-key (FK) joins, existing FK detection algorithms [18, 30, 57] would seem to apply.

To study this systematically, we harvested over 100K real BI models built using self-service BI tools, from public sources like GitHub and search engines. For each BI model file, we programmatically extract the input tables used in the model, as well as the ground-truth join relationships specified by users, thus creating a real BI benchmark for large-scale evaluation for the first time (prior work mostly use synthetic benchmarks for evaluation instead).

Our large-scale evaluation using these real BI datasets suggests that existing FK-detection algorithms are still insufficient for the

*Work done at Microsoft.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 16, No. 10 ISSN 2150-8097.
doi:10.14778/3603581.3603596

[1]We note that more advanced BI modeling can involve additional steps such as schema redesign and performance optimization, which is usually out of scope for non-technical users in self-service BI tools, and thus not considered in this work.

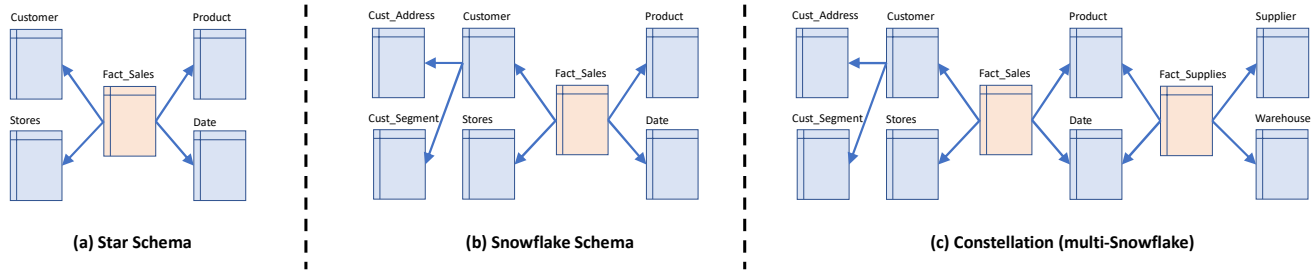


Figure 1: Types of common BI schemas. (a) Star schema, which can be seen as a simplified version of Snowflake. (b) Snowflake schema, where dimension tables like "Customers" can connect to additional dimension tables. (c) Constellation schema, which can be seen as multiple Snowflake schemas.

task, as they frequently produce incorrect results (with ~0.6 precision for all methods we tested). This is because real BI data in the wild tend to be considerably more challenging (e.g., column-headers can often be cryptic with generic names like "name" or "id", and column-values can often overlap accidentally). Crucially, most existing approaches are "local" in nature, that consider two tables at a time to make greedy local decisions, without a principled global optimization, thus unable to produce accurate predictions on challenging real BI test cases (Section 2).

Auto-BI: global optimization using schema graphs. A key insight we develop in Auto-BI, is that because we know users are finding join relationships specifically for building BI-models, and we know the typical structure that schema-graphs in BI-models should generally follow (e.g., variants of snowflake), this gives us a unique opportunity to leverage the *global graph structure* of the resulting schema-graph, to predict joins much more accurately.

We thus formulate the Auto-BI problem as a global optimization problem that holistically considers both the *local decision* of pairwise joinability, as well as the *global decision* of graph structures.

Specifically, we show that the snowflake schema popular in BI-modeling corresponds to a graph-theoretical concept called *Arborescence* [46]. Leveraging this structure, we formulate a new graph problem called *k-MCA (k-Minimum-Cost Arborescence)*, which finds the most probable k-snowflakes, by considering both local joinability and global graph structures, all using a precise probabilistic interpretation based on calibrated probabilities.

We prove that the new k-MCA problem and its variants are in general intractable (NP-hard) and inapproximable (EXP-APX-complete). Nevertheless, we develop novel algorithms based on branch-and-bound principles, which are surprisingly efficient on real test cases and can scale to the largest BI models we encounter, while still being provably optimal. Our extensive evaluations using real BI models and synthetic TPC benchmarks suggest that the proposed Auto-BI is substantially more accurate when compared to existing solutions in the literature.

Contributions. We make the following contributions:

- We are the first to harvest and leverage over 100K real BI models in the wild, for the problem of predicting BI models. This enables a data-driven algorithm design and makes it possible to rigorously evaluate different algorithms on real BI data.
- We are the first to exploit the snowflake-like schema structure in BI settings for our predictions, by mapping snowflakes to a less-known graph-theoretical concept called arboresence. We formulate a set of novel graph-based optimization problems that we call k-MCA, that have precise probabilistic foundations.
- We study the theoretical hardness of k-MCA variants, and propose efficient algorithms that are provably optimal. Extensive evaluations show that our algorithms are both effective (with over 0.9 F1 scores), and efficient (scales to the largest BI models we encounter and runs in a few seconds).

2 RELATED WORKS

BI and dashboarding tools. There are a wide variety of BI and dashboarding tools that aim to help users perform ad-hoc data analysis, with Tableau [11] and Power-BI [9] being the leading vendors [25]. These tools use visual drag-and-drop interfaces [38] (without requiring users to write SQL), and are particularly popular among non-technical users.

Foreign key detection. Foreign key (FK) detection is an important problem in database settings, with many influential methods developed in the literature [18, 30, 47, 57].

MC-FK [57] is a pioneering effort to detect FK using an EMD-based randomness metric for distribution similarity between two columns, which is more reliable to predict true relationships when unrelated key-columns that can frequently have overlapping ranges.

Fast-FK [18] develops an efficient method that selects FKs with the best pre-defined scores until all input tables are connected.

HoPF [30] improves upon prior work by considering not only FK-scores but also PK-scores, which are combined using a predefined scoring function, making this also a global method in spirit. The algorithm enumerates all PK/FK combinations and returns the combination that has the highest total score.

ML-FK [47] proposes an ML-based classifier to predict FK, trained on known FKs. As we will show analytically and experimentally, without principled global optimizations, ML classification alone is still local in nature and not sufficient to achieve high accuracy.

While FK detection methods are clearly related to our problem (we experimentally compare them with these methods), there are also a few key distinctions that we would like to highlight.

First, while FK detection targets general database settings, we focus on predicting joins in BI models, which gives us a unique opportunity to exploit the likely graph structure (e.g., snowflake) to make more accurate predictions, which is a unique direction not considered in prior work.

Second, unlike FK-detection methods that typically consider the canonical PK/FK (1:N) joins, in Auto-BI the types of joins we consider are more general, because real BI models in the wild frequently employ 1:1 joins as well as joins that are not perfectly 1:N, making the prediction problem more challenging.

Lastly, with the exception of [47], most prior FK detection methods primarily rely on hand-crafted scoring functions to make local join predictions (pairwise between two tables). In comparison, we leverage the BI models harvested to first predict local-joinability in a data-driven way, which is then combined into a principled global

Automatic Data Repair: Are We Ready to Deploy?

Wei Ni[*‡1], Xiaoye Miao[*★2], Xiangyu Zhao[‡3], Yangyang Wu[#4], Shuwei Liang[*#5], Jianwei Yin[*#†6]
[*]Center for Data Science, Zhejiang University, Hangzhou, China
[★]The State Key Lab of Brain-Machine Intelligence, Zhejiang University, Hangzhou, China
[‡]School of Data Science, City University of Hong Kong, Hong Kong, China
[#]School of Software Technology, Zhejiang University, Ningbo, China
[†]College of Computer Science, Zhejiang University, Hangzhou, China
{[1]wei.ni, [2]miaoxy, [4]zjuwuyy, [5]lsw5221}@zju.edu.cn [3]xianzhao@cityu.edu.hk [6]zjuyjw@cs.zju.edu.cn

ABSTRACT

Data quality is paramount in today's data-driven world, especially in the era of generative AI. Dirty data with errors and inconsistencies usually leads to flawed insights, unreliable decision-making, and biased or low-quality outputs from generative models. The study of repairing erroneous data has gained significant importance. Existing data repair algorithms differ in information utilization, problem settings, and are tested in limited scenarios. In this paper, we compare and summarize these algorithms with a driven information-based taxonomy. We systematically conduct a comprehensive evaluation of 12 mainstream data repair algorithms on 12 datasets under the settings of various data error rates, error types, and 4 downstream analysis tasks, assessing their error reduction performance with a *novel but practical* metric. We develop an effective and unified repair optimization strategy that substantially benefits the state of the arts. We conclude that, it is always worthy of data repair. The clean data does not determine the upper bound of data analysis performance. We provide valuable guidelines, challenges, and promising directions in the data repair domain. We anticipate this paper enabling researchers and users to well understand and deploy data repair algorithms in practice.

PVLDB Reference Format:
Wei Ni, Xiaoye Miao, Xiangyu Zhao, Yangyang Wu, Shuwei Liang, Jianwei Yin. Automatic Data Repair: Are We Ready to Deploy? . PVLDB, 17(10): 2617 - 2630, 2024. doi:10.14778/3675034.3675051

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/WelkinNi/Automatic-Data-Repair.

1 INTRODUCTION

The global application landscape relies heavily on data, while ubiquitously erroneous information involved in data often compromises the data analysis performance, leading to a huge economic cost [17, 43, 49, 84]. Dirty data may trigger the spread of fraud information, inaccurate decision-making, and even legal repercussions [42, 60, 62, 79, 86]. Among the strategies for efficiently mitigating erroneous information and ensuring data quality, *automatic data cleaning* [39, 86] has become a *pivotal* solution, especially in the age of generative artificial intelligence (GAI), where the large volume of high-quality training data for powerful GAI models like ChatGPT [67] and Midjourney [11] currently often resort to *resource-intensive manual* data cleaning [87].

The promising automatic data cleaning task is generally conducted in two consecutive stages: *error detection* and *error repair*. The error detection stage is to identify all wrong data values or rule violation sets, while the error repair stage is to correct these wrong values into latent right ones [2, 56, 57]. Existing error detection studies [38, 57, 65, 71] have achieved obvious advancements, with an average F1 score exceeding 0.85 on real datasets [57, 71]. In contrast, the error repair is more complex and challenging. Various techniques for error repair have been employed, such as machine learning [56, 72, 85], transfer learning [36, 56], boosting algorithm [47] and few-shot learning [56]. However, within the advanced detection results, the *final* data repair performance of current methods still falls short of the expected ideal, with an average F1 score below 0.7 in real scenarios [32, 56, 72]. Thus, it is crucial and urgent to analyze and explore how to deploy *end-to-end* data cleaning approaches with effective *data repair* algorithms in real-life scenarios.

Table 1 compares and summarizes existing surveys of end-to-end data cleaning study. They either analyze the data repair process without experiments [17, 43, 44, 48], or verify the impact of data repair on downstream tasks while overlooking the inclusion of certain important algorithms [1, 51]. The limitations and analysis perspectives come from the following *three* aspects.

L1: Metric shortfalls in evaluation. Fewer data errors typically lead to higher data quality and reduced biases in subsequent analyses, which constitute the *main goals* of the data cleaning task. Prior data cleaning surveys [1, 43, 44, 48, 51] lack an evaluation of *error reduction* in final data repair results, as indicated in Table 1. They often evaluate the data repair results with metrics such as *precision* and *recall*, which are based on the proportion of correctly repaired cells, disregarding the absolute quantity changes. Thus, such metrics result in a *biased* evaluation. For example, suppose there is a dataset that initially encompasses 10 erroneous cells. One data cleaning approach repairs 100 cells, of which 80 are correct (including the initial 10 error cells). Despite 70 of 80 values being identical to their originals, they are still flagged as erroneous values, and replaced by a value within calculation, thereby should be considered as a repaired cell. It means that, the dataset currently contains

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 17, No. 10 ISSN 2150-8097.
doi:10.14778/3675034.3675051

Table 1: Comparison of previous surveys and ours.

Survey study	Exp. evaluation	Error reduc. evaluation	Error rate		Error type	
			DRE*	DME*	DRE*	DME*
Ilyas et al. [43, 44]	✗	✗	✗	✗	✗	✗
Krishnan et al. [48]	✗	✗	✗	✗	✗	✗
Chu et al. [17]	✗	✗	✗	✗	✗	✗
Li et al. [51]	✓	✗	✗	✗	✗	✓
Abdelaal et al. [1]	✓	✗	✓	✗	✗	✗
Ours	✓	✓	✓	✓	✓	✓

* DRE and DME refer to data repair and downstream model evaluation.

20 erroneous cells, indicating a *doubles* amount of erroneous cells after the repair process. While the precision and recall of this repair both reach as high as 0.8 and 1.0, respectively. The correct repair of more initially right data increases the precision, according to the mediant inequality, but it cannot benefit the data quality. As a result, a higher precision/recall value does not represent a larger degree of error reduction. Furthermore, for distance-based metrics like Jaccard distance or mean squared error (MSE), they only measure the distance between the repaired and clean data, while *ignoring* the distance between the original dirty data and clean data. In a situation where the initial distance between dirty and clean data is 0.1, which then increases to 0.15 after repair. Simply stating the distance as 0.15 fails to directly indicate an increase in data errors. It cannot convey the actual *improvement degree* in data quality, either. Incorporating error reduction degree into evaluations will align repair algorithms more closely with improving data quality, and offer a direct perspective on data quality changes.

L2: Insufficient scenarios exploration in evaluation. The effectiveness of data repair algorithms differs in different scenarios. As depicted in Table 1, existing surveys lack the study of data repair algorithms' capacity under various *error types*, in either the data repair performance evaluation or the assessments of its impact on downstream tasks [1, 17, 44, 48, 51]. Meanwhile, the *error rate* analysis is absent in downstream impact evaluation, even though it is included in repair performance assessments. It results in an inadequate understanding of the proper deployment specific to distinct error types and error rates. It is vital and practical to comprehensively evaluate data repair algorithms with a diverse range of error types and rates in both repair and downstream tasks.

L3: Lacking algorithm deployment guidelines. Blindly deploying data repair algorithms may potentially hurt data quality and task performance. Although existing surveys discuss data repair algorithm performance [17, 43, 44, 48], *three* critical questions regarding algorithm deployment remain unanswered: (i) Is data repair consistently beneficial irrespective of the error rate? (ii) Based on the observation that, the suboptimal performance of data repair primarily results from incorrect repairs, how to develop a practical strategy to mitigate incorrect repairs based on error reduction evaluation? (iii) How to select the proper repair method in different scenarios based on task objectives? The data repair process plays a crucial role in the cleaning task, and addressing these issues will provide practical guidance to deploy data cleaning algorithms.

Therefore, in this survey, we conduct an exhaustive comparative analysis and experimental evaluation of 12 state-of-the-art data repair algorithms. Our main contributions are described below.

- We systematically evaluate *twelve* mainstream and state-of-the-art data repair algorithms based on a driven information-based taxonomy of data repair algorithms, including *cstr-driven* data repair algorithms, *data-driven* ones, and *hybrid-driven* ones. We also state the properly applied scenarios and graphically represent the workflow within each category.
- We define a novel and effective *metric* named *Error Drop Rate* to evaluate the error reduction condition of final data repair results. Surprisingly, in the majority of cases, most data repair algorithms tend to introduce more errors rather than eliminate erroneous information. We propose a *universal* optimization strategy that leverages error detection techniques to prevent the alteration of original right values into wrong ones.
- Extensive experiments on *twelve* real datasets under various error rates/types and data scales demonstrate both the performance of *twelve* data repair algorithms and the impact on *four* common downstream tasks. Particularly, except for five algorithms with publicly available codes, we implement *seven* additional data repair algorithms for comprehensive comparisons. We investigate the effect of *semantic* and *syntactic* data errors (that simulate complex practical scenarios) on the algorithm performance.
- We conclude several interesting findings: *i)* Repair algorithms often *increase* errors. Contrary to expectations, most existing data repair algorithms tend to raise the error rate in the data rather than eliminate it; *ii)* Using the clean data is *not* always the best. Downstream analysis models trained on purely clean data may perform worse than models trained using dirty data. *iii)* Data repair could provide universal benefits for downstream tasks. With proper algorithm selection, data repair boosts downstream tasks in the vast majority of cases; *iv) Semantic* errors pose a more stubborn challenge. Semantic errors within the data are harder to eliminate and have a more negative impact on downstream data analysis than *syntactic* ones, highlighting the need for greater consideration in real-world applications.

In the rest of this paper, Section 2 presents the preliminaries, while the new taxonomy and algorithm analysis are covered in Section 3. Comprehensive evaluations are offered in Section 4. Section 5 discusses current challenges and future directions, followed by related work in Section 6. Finally, Section 7 concludes the study.

2 PRELIMINARIES

In this section, we first introduce the problem definition. We then describe the constraints widely used in automatic repair algorithms.

Data cleaning mainly serves two primary goals: eliminating errors and ensuring data consistency, with the latter being a subgoal of the former. In this survey, we aim to evaluate *error reduction* performance over the final data repair results, as stated below.

Definition 1. **Problem Statement.** Given an instance I of relation R characterized by a set of attributes $Attrs = \{A_1, A_2, \cdots, A_n\}$. Each $t \in I$ comprises a set of cells represented by $Cells[t] = A_i[t]$ corresponding to a distinct attribute in $Attrs$. For each cell c, its unknown true value is denoted by v_c^*, its initial observed value is v_c, and its estimated true value is $\hat{v}_c$. A data repair algorithm aims to make $\hat{v}_c$ equal to v_c^* for all cells c.

This error reduction evaluation defined in *Definition* 1 encompasses correcting all erroneous cells within a dataset. It shares the same definition of *holistic repair*, as studied in most recently proposed data repair algorithms [56, 57, 71, 72].

Can Foundation Models Wrangle Your Data?

Avanika Narayan
Stanford University
avanika@cs.stanford.edu

Ines Chami
Numbers Station
ines.chami@numbersstation.ai

Laurel Orr
Stanford University
lorr1@cs.stanford.edu

Christopher Ré
Stanford University
chrismre@cs.stanford.edu

ABSTRACT

Foundation Models (FMs) are models trained on large corpora of data that, at very large scale, can generalize to new tasks without any task-specific finetuning. As these models continue to grow in size, innovations continue to push the boundaries of what these models can do on language and image tasks. This paper aims to understand an underexplored area of FMs: classical data tasks like cleaning and integration. As a proof-of-concept, we cast five data cleaning and integration tasks as prompting tasks and evaluate the performance of FMs on these tasks. We find that large FMs generalize and achieve SoTA performance on data cleaning and integration tasks, even though they are not trained for these data tasks. We identify specific research challenges and opportunities that these models present, including challenges with private and domain specific data, and opportunities to make data management systems more accessible to non-experts. We make our code and experiments publicly available at: https://github.com/HazyResearch/fm_data_tasks.

PVLDB Reference Format:
Avanika Narayan, Ines Chami, Laurel Orr, and Christopher Ré. Can Foundation Models Wrangle Your Data?. PVLDB, 16(4): 738 - 746, 2022.
doi:10.14778/3574245.3574258

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/HazyResearch/fm_data_tasks.

1 INTRODUCTION

Foundation Models (FMs) [19] are models trained on broad data that can be adapted to a wide range of downstream tasks. These models have achieved substantial gains across many semantically challenging tasks such as question answering [20], knowledge base construction [79], and information retrieval [39]. As they have scaled to hundreds of billions of parameters (e.g. GPT-3 [20], PaLM [22]), large FMs have demonstrated surprising emergent behaviors and good zero-shot generalization to new tasks (i.e. no task-specific finetuning) on domains vastly different from the data they were pretrained on [22]. These large FMs are often autoregressive language models (e.g. GPT-3 and PaLM) that are trained to predict the next word in large text corpora and can be adapted to new tasks given a simple natural language description of the task (see Figure 1). These breakthrough capabilities have led to a race for building bigger and better models, and innovations continue to push the boundaries of what large FMs can do on a variety of hard *language tasks*.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 16, No. 4 ISSN 2150-8097.
doi:10.14778/3574245.3574258

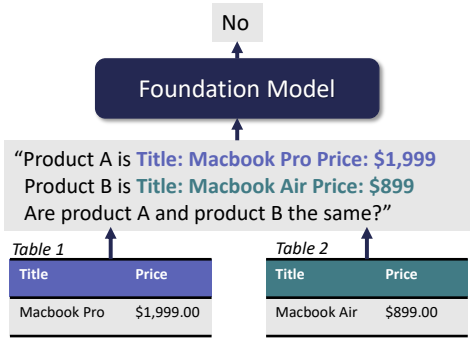


Figure 1: A large FM can address an entity matching task using prompting. Rows are serialized into text and passed to the FM with the question "Are products A and B the same?". The FM then generates a string "Yes" or "No" as the answer.

A natural question that arises is whether these advances can benefit hard classical *data tasks* (e.g. data cleaning and integration). While it is clear that FMs benefit text-intensive tasks, it is not clear whether these models can be applied to data tasks over structured data. The symbols commonly found in structured data (e.g. dates, numbers, alphanumeric codes) are less frequent in natural language text so it is unclear that FMs possess the ability to reason over them. Moreover, since FMs are trained to predict the next word, it is non-obvious that they can work out-of-the-box on complex data tasks. This paper explores the aforementioned question and introduces a new research vision for leveraging FMs for data management, focusing on data cleaning and integration tasks—two keys steps in data-driven enterprise pipelines.

Recently, a large body of research has applied machine learning (ML) [49] and deep learning (DL) [57, 74] methods—namely pretrained language models (PLMs) like BERT [32]—to semantically-complex data tasks. However, these approaches still require a significant amount of engineering effort as they rely on:

- **Task-specific architectures**: Data cleaning and integration encapsulate *many different tasks* such as entity matching [77], schema matching [92], and error detection [41]. Existing approaches, whether they are rule-, ML- or DL-based, vary greatly from one task to the other, often with complex, task-specific architectures. For instance, adapting BERT to data tasks requires architectural changes and finetuning the entire model for each task. This leads to siloed and hard-to-maintain systems.
- **Hard-coded knowledge**: Data tasks often rely on *domain knowledge* (e.g. understanding the relationship between a city and its zip code for data cleaning constraints) and commonsense reasoning. These are usually hard-coded with human-engineered rules or external knowledge bases [24, 84]. Consequently, systems can be brittle and fail to generalize to a diverse set of domains.

- **Labeled data**: ML- and DL-based solutions require copious amounts of hand-labeled data [9]. For instance, PLMs that have achieved state-of-the-art (SoTA) results on data tasks (e.g. Ditto [38]) require a significant amount of task-specific labeled data and fine-tuning to achieve good performance. Labeling data for each task is engineering intensive and adds to the difficulty of maintaining data cleaning and integration systems.

Excitingly, FMs display several useful properties that make them an appealing choice compared to traditional approaches:

- **Task-agnostic architecture**: As a result of their natural language interface, FMs can be applied to a wide-range of tasks. For instance, Figure 1 shows how an entity matching task—which requires identifying whether two table entries refer to the same entity—can be cast as a prompting task. This unifying interface eliminates the need for siloed architectures, in contrast to existing learned approaches where architectures need to be carefully crafted for each task (e.g. task-specific classification layer).
- **Encoded knowledge**: Because FMs are trained on large, generic corpora of data, they contain knowledge about an extensive set of common entities, and thus do not rely on human-engineered rules to acquire knowledge [82].
- **Limited to no labeled data**: FMs can be applied to a breadth of tasks with little to no labeled data (e.g. few-shot and zero-shot). When a FM needs to be fine-tuned, it typically needs dramatically less labeled data to achieve competitive results [46].

Our goal is to better understand if large FMs can be applied to data integration and cleaning tasks. We study the behavior of GPT-3—an early and promising FM. While GPT-3 is already a high quality model, we expect the significant investment in FMs from both academia and industry to lead to more performant and scalable FMs over time. Like many other communities, the data management community stands to benefit from these trends. As such, we aim to understand the advantages and limitations of FMs on data tasks, by focusing on three key questions.

How well do large FMs transfer to data tasks? To answer this, we cast several data tasks as natural language generation tasks (Section 3) and explore whether a single FM can generalize well to these tasks. In Section 4.2, we quantify the zero- and few-shot performance of FMs on five enterprise data tasks: entity matching, error detection, schema matching, data transformation, and data imputation. We find that the largest GPT-3 variant (175B parameters) outperforms SoTA ML-and DL-based approaches on these tasks with few examples. This is particularly surprising since prior approaches are fully-finetuned on task-specific labeled data for these tasks, while GPT-3-175B is simply pretrained to generate text.

What are the caveats in applying FMs to data tasks? In Section 4.3, we unpack the few-shot "prompt tuning" process—serializing tabular data to text, casting data tasks as text generation tasks and constructing demonstrative task examples—for applying FMs to data tasks. We quantify the effects of prompt formatting variations on performance and the differences between manually and randomly selecting task examples. We find that FMs are brittle to differences in prompt formatting and that performance improves when prompts are manually selected versus randomly selected.

What opportunities do FMs present for data tasks and what are the relevant research challenges? Finally, in Section 5, we discuss the potential challenges and related research questions with using FMs in data management pipelines. We discuss the forthcoming shift in how ML systems are built, challenges around updating FM knowledge, and opportunities and considerations pertaining to private, temporal and local data.

We hope that our preliminary exploration will encourage the data management community to explore the effectiveness of FMs for other data tasks and develop techniques to overcome the shortcomings of FMs in this setting.

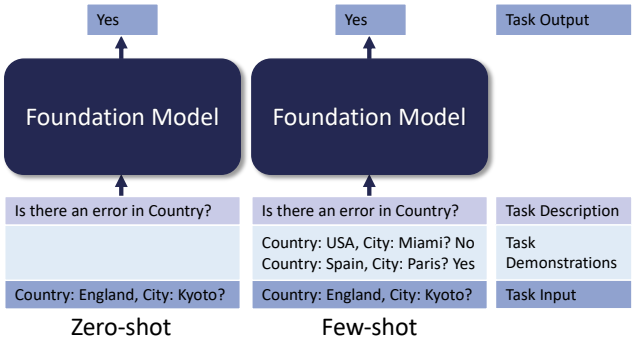


Figure 2: Different ways to use FMs with "in-context" learning [20] on an error detection task. For zero-shot (left), the prompt is the task description and the example to complete. For few-shot (right), the prompt adds task demonstrations.

2 BACKGROUND

We first give some background on the different data tasks considered in this paper and then provide a brief review of FMs.

2.1 Problem Setup

We focus on entity matching (EM), error detection (ED), and data imputation (DI) and describe the setup for these tasks. We denote D, a structured dataset with n entries, such that each entry is represented by a collection of m attribute value pairs: for entry $e_i \in D$ we have $e_i = \{e_{i,j}\}_{1 \le j \le m}$ where for attribute j, $e_{i,j} = \{\texttt{attr}_j, \texttt{val}_j\}$.

Entity Matching The goal of EM is to match entities (real-world objects like people, places and things) across different datasets. Formally, given two structured datasets (D, D') and pairs of entries $e, e' \in D \times D'$, the goal is to predict whether these entries represent the same entity or not. This problem is usually solved as a classification problem, and real-world EM systems are often preceded by blocking heuristics which are used to remove obvious non-matches.

EM has been extensively studied over the past decade (see [77] for a survey) and methods broadly fall into three categories: rule-based, crowd-based [36, 96] and ML/DL-based [49, 74]. Recently, methods relying on PLMs [57] have become SoTA for this task.

Error Detection ED is an important step in data cleaning pipelines. Given an entry e, the goal is to detect attributes j where $\texttt{val}_j$ has an error. The task is framed as a classification problem where the goal is to predict if $\texttt{val}_j$ is correct for a given e.

ED has been studied extensively in both academic and industrial settings [7]. There are a number of successful commercial

Causal Data Integration

Brit Youngmann
CSAIL MIT
brity@mit.edu

Michael Cafarella
CSAIL, MIT
michjc@csail.mit.edu

Babak Salimi
University of California, San Diego
bsalimi@ucsd.edu

Anna Zeng
CSAIL, MIT
annazeng@mit.edu

ABSTRACT

Causal inference is fundamental to empirical scientific discoveries in natural and social sciences; however, in the process of conducting causal inference, data management problems can lead to false discoveries. Two such problems are (i) not having all attributes required for analysis, and (ii) misidentifying which attributes are to be included in the analysis. Analysts often only have access to partial data, and they critically rely on (often unavailable or incomplete) domain knowledge to identify attributes to include for analysis, which is often given in the form of a causal DAG. We argue that data management techniques can surmount both of these challenges. In this work, we introduce the Causal Data Integration (CDI) problem, in which unobserved attributes are mined from external sources and a corresponding causal DAG is automatically built. We identify key challenges and research opportunities in designing a CDI system, and present a system architecture for solving the CDI problem. Our preliminary experimental results demonstrate that solving CDI is achievable and pave the way for future research.

PVLDB Reference Format:
Brit Youngmann, Michael Cafarella, Babak Salimi, and Anna Zeng. Causal Data Integration. PVLDB, 16(10): 2659 - 2665, 2023.
doi:10.14778/3603581.3603602

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/brityoungmann/CDI/blob/main/README.md.

1 INTRODUCTION

Causal inference lies in the heart of empirical research in natural and social sciences and is commonly used in multiple disciplines, including sociology, medicine, and economics [26, 33]. It aims to answer causal queries, such as, "Does being overweight cause coronary heart disease – independent of cholesterol, and diabetes?" or "Do tobacco advertisements entice adolescents to buy more cigarettes regardless of whether their parents smoke?" Causal inference enables analysts to answer causal questions about attributes from a dataset, ultimately enabling them to make real-world discoveries. Pearl's framework [40], which we adapt in this work, provides a principled way to causal inference using structural causal models.

Cause-effect questions are designed to determine whether an *exposure* variable (a pain reliever) causes or affects an *outcome* variable (pain). To correctly estimate causal effects, one must consider *confounding variables*—variables that influence both the exposure and outcome and thus might distort the association between them. Otherwise, one might draw perplexing conclusions due to *confounding bias* [41] that can lead to overestimation or underestimation of the association between exposure and outcome. The internal validity of a study heavily relies on how well confounding bias has been addressed. However, to determine a sufficient set of confounding variables to account for confounding bias, *background knowledge* about the data generative process is required. This knowledge is often given in the form of a *causal DAG* depicting causal relationships between the attributes [40] (and is wildly used in econometric and social sciences [26, 33]). Pearl [40] presented sufficient and necessary conditions for identifying[1] the *adjustment set* of variables to include in the analysis, which can be checked against a causal DAG. However, analysts often lack such a DAG [22]. While associational assumptions are testable from data, causal relations cannot, in general, be fully recovered from data [41].

Table 1: Example Input Dataset.

State	Mask Policy	Confirmed Cases	New Cases	Recovered	Death cases
MA	yes	121046	2740	4980	109
FL	yes	640978	24349	25140	55
CA	no	735235	31150	42170	34
SD	no	15300	1791	2083	49

We identify two fundamental challenges for conducting valid causal inference: First, critical confounding variables may not be included in the data. Second, even if they are included, the background knowledge required to identify the correct adjustment set of variables might be missing. We argue that data management techniques and ideas can surmount both of these challenges. We illustrate that via the following example:

Mary, a data analyst in the WHO organization, aims to estimate the (treatment) effect of a mask policy on the coronavirus mortality rate. To do this, she examines a dataset containing Covid-19-related facts in multiple states in the US (an illustration of this dataset is given in Table 1), and uses a Causal analysis tool (e.g., [49, 53]). To correctly estimate the effectiveness of face masks, Mary must consider confounding variables. However, there are confounding variables that are not included in her data. For example, the weather affects people's willingness to wear masks and Covid-19 death rate [47]. Moreover, to determine a sufficient set of confounding variables, Mary must use background knowledge of causal relationships she may lack.

In this example, variables missing from the dataset are critical for the analysis to avoid confounding bias. Thus, Mary must *integrate her data* with external data sources. This is a laborious task, typically done using data discovery and integration tools (e.g., [36, 59]) by skilled programmers—which is often not the case for social or

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 16, No. 10 ISSN 2150-8097.
doi:10.14778/3603581.3603602

[1] In causal inference, identifiability typically refers to the conditions that permit measuring causal effect from observed data. Here, we discuss the ability to identify a sufficient adjustment set of variables for accurate causal inference.

natural scientists. We argue this is the case for many real-life scenarios. After augmenting the data to include unobserved variables, the next step is determining a sufficient adjustment set of variables to account for confounding bias. However, Mary does not have the required background knowledge to do that.

Our Vision: Our goal is to mine unobserved confounding variables and missing background knowledge needed to answer causal questions. To this end, we introduce the Causal Data Integration (CDI) problem. CDI is an effort to augment an input dataset with unobserved variables and background knowledge required for causal analysis. To address the challenge of finding unobserved variables, the system mines variables from external sources. To overcome the challenge of identifying the right adjustment set of confounders, the system then builds a corresponding causal DAG, ensuring the adjustment set is sufficient for controlling the confounding bias. A CDI system may greatly reduce the manual effort devoted to performing causal analysis, allowing analysts to get all required data and background knowledge quickly and effectively.

CDI may also be beneficial for the data management community. Causal inference has been shown to be useful for various data management tasks, including query result and classifier explanation [8, 42, 56], and hypothetical reasoning [21]. However, some of these studies make strong assumptions about the underlying causal DAG that may not hold in practice [26, 33]. For example, [21] assumes that the DAG is known, and [42] assumes that all confounding variables are present in the input dataset. To address these issues, a CDI system can be utilized to construct a causal DAG for a given dataset while integrating relevant missing attributes. This not only addresses the limitations of existing studies but also enables the use of causal inference in more complex and realistic scenarios.

In this work, we show how a CDI system can be accomplished.

Measure of Success: Distinct from the classical data integration task, where success is defined by its ability to combine data from different sources and provide users with a unified materialized view of them, the success of a CDI system is defined by the ability to discover all relevant unobserved variables and recover the correct set of confounders for a causal question. The CDI task is potentially easier than data integration. However, its impact is greater, as it enables analysts to conduct causal analysis more easily. A CDI system enables analysts to include all confounding variables in the analysis; failing to do so may lead to false discoveries, incorrect medical diagnoses, and erroneous conclusions [26, 33]. In our example, a CDI system mines, among others, attributes describing the weather and population density per state, and infers causal relations among the attributes in the augmented dataset. A successful CDI system would assist Mary in identifying the right adjustment set of attributes to account for confounding bias, allowing her to accurately estimate the effect of an enforced mask policy on the mortality rate.

Challenges & Opportunities: The first challenge is to augment an input dataset with unobserved variables, ensuring the *output causal DAG is complete* (i.e., there are no unobserved confounding variables). Data discovery methods have been extensively researched to identify relevant data sources that can be integrated with an input dataset for attribute extraction [12, 14]. However, these methods are not specifically designed to discover unobserved attributes that are relevant for cause-effect estimation. This presents a unique opportunity to develop dedicated data discovery tools tailored for identifying unobserved confounding variables.

Extracted attributes may contain data quality issues, such as outliers and missing values. These issues may lead to erroneous conclusions, compromising the reliability of causal inference. In particular, missing values pose a significant challenge in causal inference as they may result in *selection bias* [10], where certain samples are preferentially excluded from the data. Other data quality issues, such as inconsistent data formats and data duplication, can also impact the accuracy of causal inference analysis. Therefore, in addition to handling missing values, it is crucial to ensure that the system is robust to various data quality issues to ensure the validity of causal inference results.

The challenge of identifying the correct adjustment set and constructing a causal DAG is particularly difficult when dealing with hundreds of extracted attributes. A human-driven causal DAG construction (from domain expertise) is infeasible in this scenario, and automatic methods for building a full causal DAG may not be feasible either [22]. To this end, we propose the construction of a cluster causal DAG (C-DAG) [9], which groups related attributes and only specifies causal relationships between clusters of attributes. By doing so, the correct adjustment set of attributes can still be identifiable from this C-DAG. This approach reduces cognitive overhead for analysts and makes causal analysis possible for high-dimensional data, which is currently not feasible for existing tools (e.g., [49, 53]). However, a main challenge is how to maintain the necessary level of granularity to ensure accurate causal inference. The limitations of existing solutions and challenges in constructing the desired C-DAG are discussed, and a limited prototype implementation for solving the CDI problem is presented with preliminary experimental results demonstrating that solving CDI is achievable.

2 THE CDI PROBLEM

2.1 Data Model and Background

The input dataset $\mathcal{D}$ contains the exposure (T) and outcome (O) attributes. A *causal DAG* for $\mathcal{D}$ is a DAG whose nodes are the attributes and whose edges capture all causal relationships between the attributes [40]. Generally, because causes must precede effects, the literature considers causal DAGs rather than graphs [40]. Causal DAGs provide a simple way of graphically representing causal relationships and are particularly helpful in understanding potential sources of bias in causal estimations. It is well-known that background knowledge is required to determine a causal DAG for a given dataset [41]. A causal DAG can only be as good as the background information used to create it [38]; a DAG is complete and therefore has a causal interpretation only if it contains all common causes of any two variables. In Section 3.3, we review existing approaches for building a causal DAG for a given dataset and discuss their limitations. We assume the analyst is interested in investigating a causal relationship between T and O. She may wish to estimate, e.g., *the direct, indirect, or total effect* [40]. The total effect of T on O is the total extent to which O is changed by the T. It is equal to the sum of the direct and indirect effects. Causal questions about total and (in)direct effect could be answered by identifying the right set of *confounding* (that influence both T and O) and *mediating* (that

CHORUS: Foundation Models for Unified Data Discovery and Exploration

Moe Kayali
kayali@cs.washington.edu
University of Washington

Anton Lykov
alykov@cs.washington.edu
University of Washington

Ilias Fountalis
ilias.fountalis@relational.ai
RelationalAI

Nikolaos Vasiloglou
nik.vasiloglou@relational.ai
RelationalAI

Dan Olteanu
olteanu@ifi.uzh.ch
University of Zurich

Dan Suciu
suciu@cs.washington.edu
University of Washington

ABSTRACT

We apply foundation models to data discovery and exploration tasks. Foundation models are large language models (LLMs) that show promising performance on a range of diverse tasks unrelated to their training. We show that these models are highly applicable to the data discovery and data exploration domain. When carefully used, they have superior capability on three representative tasks: table-class detection, column-type annotation and join-column prediction. On all three tasks, we show that a foundation-model-based approach outperforms the task-specific models and so the state of the art. Further, our approach often surpasses human-expert task performance. We investigate the fundamental characteristics of this approach including generalizability to several foundation models and the impact of non-determinism on the outputs. All in all, this suggests a future direction in which disparate data management tasks can be unified under foundation models.

PVLDB Reference Format:
Moe Kayali, Anton Lykov, Ilias Fountalis, Nikolaos Vasiloglou, Dan Olteanu, and Dan Suciu. CHORUS: Foundation Models for Unified Data Discovery and Exploration. PVLDB, 17(8): 2104 - 2114, 2024.
doi:10.14778/3659437.3659461

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at http://github.com/mkyl/CHORUS.

1 INTRODUCTION

Data discovery and exploration are major components of the workflow of analysts and data scientists. A survey conducted by the Anaconda data-science platform in 2021 found that analysts spend 40% of their working hours on data loading and cleaning [2]. Even with this colossal effort, 60-70% of data within an enterprise still goes unused for analytics [21], remaining as *dark data* [23, 63].

Recent developments in large language-models (LLMs) have unlocked human-level performance on diverse domain tasks. The discovery that these models can generalize to diverse domain-specific tasks that they have not been trained on [3, 26, 59, 60] has led to emergence of the term *foundation models* [5].

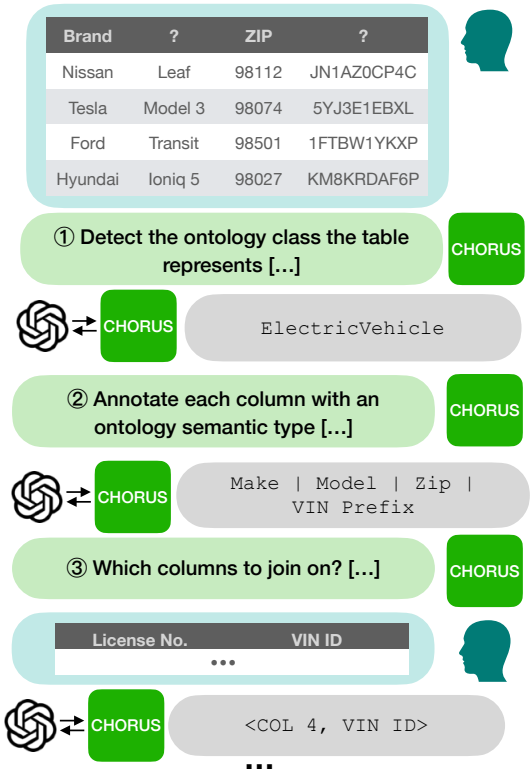


Figure 1: Data discovery tasks considered in this work. Given an ontology, such as DBPedia, ① we assign an overall type to the table and ② we annotate the columns with semantic types. Last, given another table, ③ we predict the join column. The user provides the data while CHORUS interacts with the foundation model. Data from [44], full prompts in Figure 3.

Despite their promise, serious risks have hampered the reception of foundation models. These include: spurious generation (including "hallucination") [24], factual recall limitations [39], bias [19], dataset contamination [14], logical shortcuts [50] and fallacies [38]. Naïve deployment can lead to unanticipated problems: it has already led to legal action [11] and recalls by major corporations [22]. These risks are now acknowledged by the creators of these models [6, 45, 54].

The goal of this paper is to demonstrate the utility of foundation models to the data discovery and exploration while mitigating

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 17, No. 8 ISSN 2150-8097.
doi:10.14778/3659437.3659461

the aforementioned risks. We select three representative tasks to show the promise of foundation models: ① *table-class detection*, ② *column-type annotation* and ③ *join-column prediction*. An outline of our approach is shown in Figure 1. We call our approach CHORUS.

Contributions. We summarize our contributions:

- The first work to use foundation models for the data discovery tasks of table-class detection, column-type annotation and join-column prediction;
- Propose a novel system, CHORUS, whose flexible architecture enables the synthesis of multiple data discovery tasks and deploying risk mitigations;
- Design task-specific approaches that exploit zero- and few-shot strategies and allow information flow between tasks;
- Introduce the novel mitigation of *anchoring* to reduce foundation-model risks specific to this domain;
- Empirically validate CHORUS, comparing its performance with the state-of-the-art baselines across three individual tasks.

Discussion. Prior work has addressed these tasks individually. Landmark approaches like Sherlock [27] trained deep model architectures for a specific task, requiring 100K-1M labeled data points. More recent work such as DoDuo [52] and TaBERT [62] has focused on *representation learning*, learning embeddings for structured data by improving their performance on one or more downstream tasks.

Foundation models allow a substantially different approach: rather than the classical architecture where the outputs of the model are task-specific, the inputs and outputs of the model are natural language text. Training occurs not on tables or data management tasks specifically, but on general text. Performance on domain-specific tasks is solely by generalization.

This results in a high degree of flexibility. Novel tasks can be specified in natural text, without need for expensive data collection—task examples, metadata and constraints are all incorporated into the task easily. Another advantage of this approach is a **unified architecture**: tasks can utilize the overall context and previous outputs. For example, in Figure 1 the table class `ElectricVehicle` helps with deducing the outputs `Make`, `Model` in the next task.

Outline. Section 2 defines the three tasks investigated in this paper. Section 3 describes the architecture of CHORUS and key approaches. We evaluate the performance of CHORUS in Section 4's experiments. In that section, we also investigate the fundamental characteristics of this approach. We offer a discussion of those results in Section 5. This includes a discussion of promising future directions. Finally, we place this work within the literature in Section 6, discussing related works.

2 BACKGROUND

2.1 Tasks

We assume to be given a *data collection* consisting of a number of relational tables T_1,T_2,.... Each table T_i consists of a number of columns, or attributes, A_1,A_2,... and a number of rows, or tuples, r_1,r_2,... The name of a table T_i is, in general, non-informative, for example it may be simply a sequential ID. The columns may optionally have a name H_1,H_2,... or consist only of values.

In addition to the data collection, we are also given a reference ontology of table classes C_1,C_2,..., and a reference ontology of column types τ_1,τ_2,.... For example, the DBPedia.org types for the table classes include https://dbpedia.org/ontology/Actor and https://dbpedia.org/ontology/Continent and column types include https://dbpedia.org/ontology/areaTotal and https://dbpedia.org/ontology/birthDate.

We consider three tasks of interest on the data collection:

Definition 2.1 (① Table-class detection)**.** For each table T_i, determine its appropriate class C_j, such that every row r_1,r_2,... represents an instance of the C_j type. We adopt this definition from [33].

For example, table-class detection on the table given in Figure 1 could output `ElectricVehicle`, since each row of that table is an instance of that class. Alternatively stated, the table is about `ElectricVehicles`.

Definition 2.2 (② Column-type annotation)**.** For each table T_i, find a mapping from its attributes (columns) A_1,A_2,... to the reference column types τ_1,τ_2,..., such that each value in A_i is an instance of the τ_i type. See [1, 13].

For example, column-type annotation on the first column in Figure 1 could output `Manufacturer`, since the values are the respective manufacturers of each `ElectricVehicle`.

Definition 2.3 (③ Join-column prediction)**.** Assume an *execution log* L, a history of user actions including table joins and their join conditions, which maps many $(T_i,T_j) \rightarrow (A_k,A_l)$ where $A_k \in T_i, A_l \in T_j$. Given two tables T and T', with columns A_1,... and A'_1,... respectively, the *join-column prediction* task is to suggest a pair (A_k,A'_l) of columns such that the equality condition $A_k = A'_l$, which can be used to join the the tables, matches with the choice in the execution log L. For more discussion, see [61].

For example, given the table in Figure 1 and another table `car_registration(name, vehicle_id_number)`, join-column prediction could output `(VIN_prefix, vehicle_id_number)`. The correctness of the prediction depends on the ground truth of which columns the user did in-fact join on.

Ontologies Foundation models contain knowledge of ontologies such as DPBedia.org, Freebase and Wikidata. We focus on universal ontologies, that is, ontologies that aim to represent all entities in general. This is in-line with findings that foundation models encode highly technical knowledge, such as clinical reasoning [51] or electrical engineering principles [53].

3 APPROACH

We outline the structure of CHORUS in this section. First, we explore the core idea of ingesting relational data with foundation models and performing data exploration tasks in Subsection 3.1. Next, we describe the necessary post-processing and mitigations we develop in Subsection 3.2.

Figure 2 shows the architecture of the system. CHORUS has a unified architecture which runs multiple tasks in the same context, allowing for information flow. Each task is run sequentially, with the output of one task fed as context into future tasks.

For each task instance, CHORUS generates a prompt by concatenating six inputs: context, demonstration, data samples, metadata,

CM-Explorer: Dissecting Data Ingestion Problems

Niels Bylois
UHasselt
Data Science Institute, ACSL
Belgium
niels.bylois@uhasselt.be

Frank Neven
UHasselt
Data Science Institute, ACSL
Belgium
frank.neven@uhasselt.be

Stijn Vansummeren
UHasselt
Data Science Institute, ACSL
Belgium
stijn.vansummeren@uhasselt.be

ABSTRACT

Data ingestion validation, the task of certifying the quality of continuously collected data, is crucial to ensure trustworthiness of analytics insights. A widely used approach for validating data quality is to specify, either manually or automatically, so-called data unit tests that check whether data quality metrics lie within expected bounds. We employ conditional unit tests based on conditional metrics (CMs) that compute data quality signals over specific parts of the ingestion data and therefore allow for a fine-grained detection of errors. A violated conditional unit test specifies a set of erroneous tuples in a natural way: the subrelation that its CM refers to. Unfortunately, the downside of their fine-grained nature is that violating unit tests are often correlated: a single error in an ingestion batch may cause multiple tests (each referring to different parts of the batch) to fail. The key challenge is therefore to untangle this correlation and filter out the most relevant violated conditional unit tests, i.e., tests that identify a core set of erroneous tuples *and* act as an explanation for the errors. We present CM-Explorer, a system that supports data stewards in quickly finding the most relevant violated conditional unit tests. The system consists of three components: (1) a graph explorer for visualizing the correlation structure of the violated unit tests; (2) a relation explorer for browsing the tuples selected by conditional unit tests; and, (3) a history explorer to get insight why conditional unit tests are violated. In this paper, we discuss these components and present the different scenarios that we make available for the demonstration.

PVLDB Reference Format:
Niels Bylois, Frank Neven, and Stijn Vansummeren. CM-Explorer: Dissecting Data Ingestion Problems. PVLDB, 16(12): 3958 - 3961, 2023.
doi:10.14778/3611540.3611595

1 INTRODUCTION

Modern data analytics pipelines continuously collect and ingest new data. Validating the quality of collected data at ingestion time is crucial in such pipelines, for a number of reasons. First, and foremost, the quality of the derived insights, and the decisions driven by them, depend directly on the quality of the collected data [8]. Second, as more and more of the data analysis process is automated, small errors in source data risk propagating to later data consumers (such as machine learning models), which may themselves act as data sources in other processing pipelines—thereby potentially magnifying the error [3]. Finally, data errors may even cause data processing pipelines to crash (e.g., because of null pointer exceptions due to missing data). Machine learning platforms are therefore including explicit data validation components into their pipelines [1, 2, 5].

In recognition of the importance of data quality validation in modern analytics pipelines, several tools have been proposed to aid in automatic validation [3, 7, 8]. Broadly speaking, these tools allow specification, either manually or automatically, of so-called *data unit tests*. When a new batch of data is to be ingested, the registered tests are executed to gauge the batch's quality where failing tests highlight data quality problems. The tests themselves entail computing certain metrics on the data batch (e.g., the minimum or maximum value appearing in a numerical column or the number of distinct elements appearing in a column) and checking that these fall within an expected range.

Unfortunately, these tools suffer from two limitations. First, the data unit tests that they support are based on *global* metrics: metrics that are computed on the entire data batch, or on an entire column in the batch. As the following example shows, they hence only provide *coarse-grained* signals of data quality and are unable to detect *fine-grained* errors, i.e., errors that occur only in a specific (potentially small) part of the batch. Batches with fine-grained errors hence go unnoticed. Second, even when a data unit test signals that a batch has a data quality problem, it does not provide a principled method to identify the part of the batch that is responsible for a test's failure. As such, either the entire batch must be discarded or a human expert must manually identify and subsequently remove or correct the erroneous tuples—which is time-consuming and labor-intensive.

Example 1.1. A public railway company has equipped all of its trains with measurement sensors that record the train's arrival and departure time at each train station. By comparing these times with the time schedule, the train's software computes the corresponding delays (if any). At the end of each day, the measurements of all trains are collected, and ingested in the railway company's data lake. The delays are used to identify hotspot routes, as well as computation of service quality indicators to the government. Train 5437 runs daily from Hasselt (Belgium) to Blankenberge (Belgium). This route is notorious for the delays that it incurs when it passes through the busy Brussels railway stations. As such, train 5437 normally reports non-zero delay. Due to a hardware malfunction on March 15, however, it consistently reports zero delay for this train.

The metrics used by state-of-the-art tools are unable to detect this error. Indeed: because zero delay is *not* an uncommon value when considering the entire ingestion batch (some trains run on

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 16, No. 12 ISSN 2150-8097.
doi:10.14778/3611540.3611595

time), metrics such as min(delay), max(delay), and avg(delay) will not consider zero delay as an anomaly.

It is important to observe that, even if one of the global metrics signals a data quality problem, it is not clear which train or set of trains in the batch cause the problem. For instance, if the unit tests based on min(delay) or max(delay) signal an anomaly then of course we can easily identify the erroneous trains: simply compute the trains whose delay value is below (or above) the expected minimum (resp. maximum) value. However, if a unit test based on avg(delay) signals a problem, then it is unclear how to identify the trains that caused the delay to deviate. □

We have developed an approach for data quality validation that is not only capable of detecting the *fine-grained errors* illustrated above, but also helps in *identifying* responsible erroneous tuples [4]. Our methodology focuses on the setting where a data pipeline regularly ingests batches of external data. As the main technical vehicle underlying our methodology, we introduce *conditional metrics* (CM for short). In contrast to a global metric, a conditional metric only computes its value on a specified subset of tuples in the ingestion batch. A CM m is of the following form

$$m := \mu(Y \mid X = x) \qquad (\dagger)$$

where μ is a metric like MIN, MAX, SUM, MAXDIGITS, ...; Y and X are attributes of the relation under consideration and x is a domain value for X. We refer to (X, x) as the entity in m. The semantics is as follows: m computes the metric μ over the column Y for the subrelation consisting of all tuples where X equals the value x. In the railway example above, for instance, the conditional metric

$$\textsc{avg}(\mathit{Delay} \mid \mathit{Train} = 5437)$$

computes the average delay of train 5437 in the batch, as opposed to the global metric AVG(*Delay*) which computes the average delay of all trains. A conditional unit test then is simply a CM with an associated function that returns true for admissible values and false otherwise. Because of their ability to calculate values for specific entities (e.g., *Train* = 5437), conditional data unit tests may hence detect data quality issues at a finer level of granularity than data unit tests based on global metrics.

Our methodology consists of two phases: (i) a unit test discovery phase and (ii) a monitoring and error identification phase, as schematically illustrated in Figure 1. In the *unit test discovery phase*, we are given a sequence $\overline{R}$ of previously ingested batches and our objective is to automatically derive a set Θ of CM-based data unit tests from $\overline{R}$ such that a yet-to-be-ingested batch B can be considered to be of acceptable quality if it passes all tests in Θ. Specifically, like [7], we assume that the batches in $\overline{R}$ are themselves of acceptable data quality—which is reasonable because they have already been ingested successfully. Under this assumption, we may derive CM-based data unit tests from $\overline{R}$ by considering the set of all possible CMs, and deriving, for each such CM m, a classifier that allows to distinguish expected values of m from anomalies. If m produces an anomaly on a yet-to-be-ingested batch B, then B will be flagged as having a quality issue. To derive the classifier based on the values of m observed in $\overline{R}$, in principle any anomaly detection method [6] may be used. For illustration purposes, Figure 1 mentions some simple anomaly detection methods that produce classifiers that

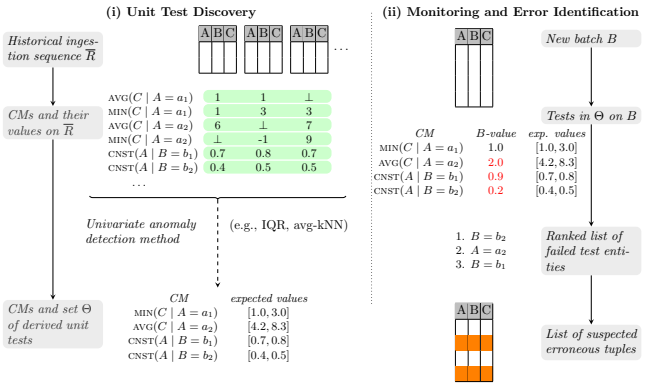


Figure 1: Overview of methodology: (i) *discovery phase* **to derive data unit tests based on conditional metrics; (ii)** *monitoring and error identification phase* **where each new batch is validated and erroneous tuples are identified.**

can be summarized as an expected range of values, where values outside the range are anomalies. Whatever method is chosen, we require that the derived classifier is consistent with our assumption that $\overline{R}$ is of reasonable quality, which implies that the values of m observed in $\overline{R}$ are without significant anomalies.

In the *monitoring and error identification phase*, we take the set Θ of data unit tests derived in the discovery phase, and use this to validate each new ingestion batch B. If all tests in Θ succeed on B then B is deemed to be of acceptable quality. When at least one test in Θ rejects B then our objective is to identify the tuples in B with suspected errors. Every conditional unit test specifies a set of erroneous tuples in a natural way: the subrelation of B that its CM refers to. In our railway example, if a unit test with CM AVG(*Delay* | *Train* = 5437) hence fails, all tuples with *Train* = 5437 could be flagged. Simply flagging *all* the tuples of violated unit tests selects too much, however (i.e., it results in high recall but very low precision). The reason is that violating tests are often correlated: a single error in B may cause multiple tests (each selecting different subrelations) to fail. The key challenge in the monitoring phase, therefore, lies in *ranking* the violated unit tests according to relevance, and from this ranked list of tests *filter* a list of suspected erroneous tuples for further inspection. We have developed multiple metrics to rank and filter violated unit tests in [4].

We illustrate the challenges in analyzing violated unit tests by means of an example. For this, we introduce the bipartite entity-tuple graph as a means to represent the correlation structure of the violated unit tests. This graph consists of two sets of nodes: all entities mentioned in at least one of the violated unit tests and every tuple that is selected by at least one of them. Furthermore, there is an edge from an entity to a tuple when that tuple is selected by the entity. Recall the scenario outlined in Example 1.1, where due to a hardware malfunction train 5437 consistently reports zero delay. Consider the to-be-ingested batch B shown in Figure 2 as well as the failed unit tests ϕ_1, ϕ_2, and ϕ_3 listed there. The entity-tuple graphs is displayed as well. We note that for the sake of this example, we have added an extra attribute *Kind* that lists the kind of service that the train offers. Unit test ϕ_1 fails because the average delay of train

Cross Modal Data Discovery over Structured and Unstructured Data Lakes

Mohamed Y. Eltabakh
Qatar Computing Research Institute (QCRI)
Doha, Qatar
meltabakh@hbku.edu.qa

Mayuresh Kunjir
Amazon Web Services
Berlin, Germany
mkunjir@amazon.de

Ahmed K. Elmagarmid
Qatar Computing Research Institute (QCRI)
Doha, Qatar
aelmagarmid@hbku.edu.qa

Mohammad Shahmeer Ahmad
Qatar Computing Research Institute (QCRI)
Doha, Qatar
mohammadshahmeerah@hbku.edu.qa

ABSTRACT

Organizations are collecting increasingly large amounts of data for data-driven decision making. These data are often dumped into a centralized repository, e.g., a data lake, consisting of thousands of structured and unstructured datasets. Perversely, such mixture makes the problem of *discovering* tables or documents that are relevant to a user's query very challenging. Despite the recent efforts in *data discovery*, the problem remains widely open especially in the two fronts of (1) discovering relationships and relatedness across structured and unstructured datasets–where existing techniques suffer from either scalability, being customized for a specific problem type (e.g., entity matching or data integration), or demolishing the structural properties on its way, and (2) developing a holistic system for integrating various similarity measurements and sketches in an effective way to boost the discovery accuracy.

In this paper, we propose a new data discovery system, named CMDL, for addressing these two limitations. CMDL supports the data discovery process over both structured and unstructured data while retaining the structural properties of tables. As a result, CMDL is the only system to date that empowers end-users to seamlessly pipeline the discovery tasks across the two modalities. We propose a novel multi-modal embedding representation that captures the similarities between text documents and tabular columns. The model training relies on labeled datasets generated though *weak supervision*, and thus the system is domain agnostic and easily generalizable. We evaluate CMDL on three real-world data lakes with diverse applications and show that our system is significantly more effective for cross-modality discovery compared to the search-based baseline techniques. Moreover, CMDL is more accurate and robust to different data types and distributions compared to the state-of-the-art systems that are limited to only the structured datasets.

PVLDB Reference Format:
Mohamed Y. Eltabakh, Mayuresh Kunjir, Ahmed K. Elmagarmid, and Mohammad Shahmeer Ahmad. Cross Modal Data Discovery over Structured and Unstructured Data Lakes. PVLDB, 16(11): 3377 - 3390, 2023.
doi:10.14778/3611479.3611533

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 16, No. 11 ISSN 2150-8097.
doi:10.14778/3611479.3611533

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/qcri/CMDL

1 INTRODUCTION

The democratization of data science has resulted in enterprises drowning in their own collected data. Large enterprises routinely have a huge collection of structured and unstructured data in their data repositories, referred to as *data lakes*. Performing *data discovery* where the goal is to find relevant elements (e.g., tables or documents) for a given user's query or an analytical task from such massive and unorganized data lakes is a major challenge faced by data scientists [3, 17, 39, 42, 56]. Data discovery is one of the initial steps of data preparation that plays an important role in various downstream data processing tasks including data exploration, data integration, and data enrichment [28, 32, 33, 55, 62].

Data discovery is emerging as a practical and core problem to modern enterprises as exemplified by the recent interest from both academia and industry [1, 2, 10, 22, 29, 40]. Nevertheless, it remains a challenging problem due to several factors. First, data is collected and published by disparate sources, and thus do not conform to a common format or structure. A vast amount of this data is in the form of unstructured text documents, e.g., research articles, social media posts, emails, news reports, etc. [12, 19]. As a result, both data modalities (structured and unstructured) are equally critical, and any system that overlooks either of them is lame. The second factor is the lack of consistent and sufficient data description and metadata. Therefore, a robust data discovery system must reason about and establish relationships of relatedness using multiple and diverse similarity signals, where each could be possibly weak. Lastly, a data discovery system needs to scale well to very large volumes of data as more organizations and governments are publishing more data for transparency [54].

The majority of existing work limits the discovery process to only the structured portion of data lakes, e.g., [10, 22, 24, 25, 59]. And thus, these systems do not extend to unstructured data and hit the wall of the first challenge mentioned above. In contrast, there are recent efforts trying to extend the discovery process across both structured and unstructured data sets, however they inherit several limitations. For example, some of these systems shift the focus entirely to the unstructured domain by transforming the

structured data to unstructured documents [12, 13, 19]. This way, they demolish the structural properties and cut the way towards having a holistic discovery system across both modalities. Other systems move the two modalities to a third common space where everything is encoded as a set of subject-predicate-object triplets, e.g., [23]. Again, they lose the ability for supporting discoveries such as table joinability and unioinability in addition to encountering high overheads that hinder its scalability.

The goal of this work is to develop a more holistic data discovery system that can: (1) bring large unstructured data into the framework of data discovery, (2) enable analysts to intermix their structured and unstructured discovery tasks in a single pipeline seamlessly, and (3) be robust through novel integration of various content-based and metadata-based similarity signals–this is especially required because unstructured data inherit very different characteristics compared to structured tables.

Motivation Example (Pharmaceutical Usecase): *Pharmaceutical databases such as DrugBank consist of an exhaustive listing of drugs, enzymes, and genes along with their interactions with other drugs or related proteins [4]. Domain experts periodically curate such databases with findings from research literature, MedLine reports [6], FDA MedWatch reports [5], and other sources. Assume our data lake encompasses all these rich data sets, and the analysts are exploring the data in the wild, i.e., without any prior knowledge of the available data sources. Now, an analyst studying enzyme* "Thymidylate Synthase" *would like at first to find documents related to this enzyme (Q1 in Figure 1). The system returns such documents as depicted in the left side of the figure. Then, she poses the second question (Q2) trying to find curated tables in the database related to the 1st document (or sub-document by highlighting sentences of interest). The system should provide table* "Enzyme_Targets" *as an answer. Then, progressing forward with her exploration, she finds another returned document that talks about drug* "Pemetrexed" *and its relation to enzyme* "Thymidylate Synthase". *As such, she poses Q3 to potentially find more tables in the database supporting this relationship. The system should provide the two tables* "Enzyme_Targets" *and* "Drugs" *as high likelihood candidates to Q3. One step further, the analyst would like to know if there are more relationships in the database around this drug. And thus, she raises Q4 and the system provides two potentially joinable tables (i.e.,* "Enzyme_Targets" *and* "Drug_Interactions"*). The former table seems central to this exploration as it has been reached from multiple paths, and hence the analyst issues Q5 to find more related and unionable tables.*

Evidently, the kind of relationships, similarity measures, and sketches needed to be maintained and exploited to answer Question Q1 (within the unstructured data modality) are very different from those needed to answer Questions Q2 and Q3 (across modalities), and also very distinct from those needed to answer Question Q4 (within the structured data modality). To the best of our knowledge, there is no single holistic system that can support such data discovery chain as highlighted above.

Solution overview: We propose a data discovery system, called CMDL[1], that enables the data discovery tasks demonstrated in the motivation example over a data lake of structured and unstructured data sets. To support cross-modal questions such as Q2 and Q3 (from Figure 1), we introduce a novel embedding-based joint representation that brings both unstructured documents and relational tables, more specifically the individual columns in each table, into a common vector space. We propose several optimizations over the model's loss function to achieve scalability and avoid biases as will be explained later. The model that creates these joint representations relies on supervised learning and requires the presence of a labeled training data set–which may not exist in the first place. Therefore, CMDL is equipped with a weakly-supervised training process that integrates multiple similarity signals over documents and columns to construct a labeled training data set. As such CMDL does not require a pre-existing labeled datasets, and hence, easily generalizable to different applications.

To support other types of data discovery questions such as Q1, Q4, and Q5 (from Figure 1), CMDL utilizes several types of signatures and sketches on both the actual data content and the contextual metadata. For example, for structured data, schema information and tables names are types of metadata. Similarly, for unstructured documents, the documents' names and their sources are the metadata. Then, we build appropriate indexes using state-of-the-art techniques on them, e.g., BM25 elastic search index [49] for keyword search and Locality Sensitive Hashing (LSH) index [61] for the set containment operations. While prior data discovery solutions are also based on similar sketches and indexes, e.g., [10, 13, 22, 24, 25], CMDL goes one step beyond by first leveraging them as similarity signals for the weak-supervised training process mentioned above, and second integrating these complementary elements into a single unified discovery system.

The various relationships discovered by the system form an Enterprise Knowledge Graph (EKG) on which a comprehensive query interface is developed. It extends the *SRQL* query engine from Aurum [22] with APIs and retrieval methods specific to CMDL, e.g., for supporting cross-modality discovery tasks.

In summary, we make the following contributions:

(1) Introducing a holistic data discovery system that treats both structured tabular data and unstructured documents as first-class citizens. Within a single discovery pipeline, end-users can seamlessly formulate discovery questions ranging from simple keyword search over either modality to complex discoveries spanning both modalities.
(2) Proposing a novel embedding-based joint representation for multi-modal data sets. The model training relies on weak supervision by combining various types of similarity signals. As such, the system can be adopted even under the absence of labeled data.
(3) Integrating existing sketches with a few alterations to improve the discovery accuracy even for the traditional joinability and unionability tasks by up to 30%.
(4) Evaluating CMDL on three real-world data lakes ranging from drug discovery to ML data augmentation. CMDL is not only more effective in discovering the complex relationships across the unstructured documents and the structured tables, but also more robust for the traditional discovery tasks (e.g., joinability and unionability) over the structured data compared to the state-of-the-art systems.

[1] CMDL–Cross Modal Data Discovery over Structured and Unstructured Data Lakes–is pronounced as kam·dl.

CtxPipe: Context-aware Data Preparation Pipeline Construction for Machine Learning

HAOTIAN GAO, National University of Singapore, Singapore and NUSRI Chongqing, China
SHAOFENG CAI, National University of Singapore, Singapore
TIEN TUAN ANH DINH, Deakin University, Australia
ZHIYONG HUANG, National University of Singapore, Singapore and NUSRI Chongqing, China
BENG CHIN OOI, National University of Singapore, Singapore

Machine learning models are only as good as their training data. Simple models trained on well-chosen features extracted from the raw data often outperform complex models trained directly on the raw data. Data preparation pipelines, which clean and derive features from the data, are therefore important for machine learning applications. However, constructing such pipelines is a resource-intensive process that involves deep human expertise.

Our goal is to design an efficient framework for automatically finding high-quality data preparation pipelines. The main challenge is how to explore a large search space of pipeline components with the objective of computing features that maximize the performance of the downstream models. Existing solutions are limited in terms of feature quality, which results in low accuracies of the downstream models, while incurring significant runtime overhead. We present CtxPipe, a novel framework that addresses the limitations of previous works by leveraging contextual information to improve the pipeline construction process. Specifically, it uses pre-trained embedding models to capture the data semantics, which are then used to guide the selection of pipeline components. We implement CtxPipe with deep reinforcement learning and evaluate it against state-of-the-art automated pipeline construction solutions. Our comprehensive experiments demonstrate that CtxPipe outperforms all of the baselines in both model performance and runtime cost.

CCS Concepts: • **Information systems** → **Data analytics**; **Data cleaning**; • **Computing methodologies** → **Machine learning**.

Additional Key Words and Phrases: Data Analytics Pipeline, Data Preparation

ACM Reference Format:
Haotian Gao, Shaofeng Cai, Tien Tuan Anh Dinh, Zhiyong Huang, and Beng Chin Ooi. 2024. CtxPipe: Context-aware Data Preparation Pipeline Construction for Machine Learning. *Proc. ACM Manag. Data* 2, 6 (SIGMOD), Article 231 (December 2024), 27 pages. https://doi.org/10.1145/3698831

1 Introduction

The quality of input data determines the quality of Machine Learning (ML) models. For machine learning applications involving relational data, simple tree-based models trained with well-selected features derived from the raw data often have comparable, if not better model performance than that of state-of-the-art deep learning models trained directly on the raw data [19, 20, 40, 48]. The

Authors' Contact Information: Haotian Gao, National University of Singapore, Singapore, Singapore and NUSRI Chongqing, Chongqing, China, gaohaotian@comp.nus.edu.sg; Shaofeng Cai, National University of Singapore, Singapore, Singapore, shaofeng@comp.nus.edu.sg; Tien Tuan Anh Dinh, Deakin University, Melbourne, Victoria, Australia, anh.dinh@deakin.edu.au; Zhiyong Huang, National University of Singapore, Singapore, Singapore and NUSRI Chongqing, Chongqing, China, huangzy@comp.nus.edu.sg; Beng Chin Ooi, National University of Singapore, Singapore, Singapore, ooibc@comp.nus.edu.sg.

This work is licensed under a Creative Commons Attribution International 4.0 License.

© 2024 Copyright held by the owner/author(s).
ACM 2836-6573/2024/12-ART231
https://doi.org/10.1145/3698831

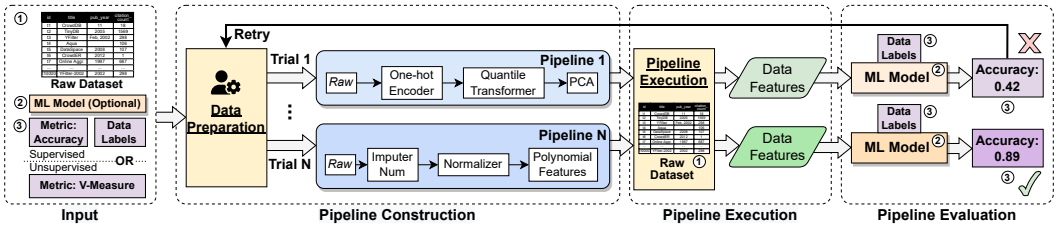


Fig. 1. Constructing a data preparation pipeline to maximize the performance of the generated data features.

tree-based models are not only more efficient to train but also have better interpretability [39, 46]. Extracting high-quality features from raw data requires a *data preparation pipeline*, which comprises multiple components that clean and transform raw data into features [1, 33]. These components either come from the ML frameworks, or are defined by the user.

The process of building a data preparation pipeline that generates high-quality features, or *data preparation pipeline construction*, is illustrated in Figure 1. It is performed by a data scientist, taking as input the raw dataset, an ML model, and its corresponding metric for evaluation. For supervised learning, the data labels are also provided. The process is manual and iterative. At each iteration, the scientist constructs a pipeline, executes it to derive a set of features, trains the model, and observes the model performance. The final output is the pipeline that achieves the best model performance.

Our goal is to design an efficient framework for automatically finding high-quality data preparation pipelines, that is, for automated data preparation pipeline construction. The main challenge is how to efficiently explore a large search space of pipeline components. In particular, the search space of possible component combinations grows exponentially with the number of possible components for different cleaning and transformation tasks [49]. Furthermore, the time cost of evaluating the pipelines is significant, because for each pipeline this cost includes the execution of the pipeline, as well as the model training and testing. With distribution shifts [32, 39, 47] requiring periodic pipeline rebuilding, this overhead accumulates and becomes significant over time.

Existing works on automated pipeline construction fall short of achieving our goal [8, 16, 21, 24, 26, 28–30, 33, 42, 46, 49, 54, 60]. Different techniques, e.g., genetic programming [42, 49], Bayesian optimization [16, 46, 54], reinforcement learning [8, 11, 21, 46], and differential formulation [24, 33, 60] are used to select and permute pipeline components. However, the search space still grows exponentially, resulting in significant runtime overhead. One reason for the large search space is that these works only use table schema and other statistical information to guide the exploration.

Our insight is that we can leverage *contextual information* to significantly reduce the search space. Intuitively, by capturing the *domain expertise* that data scientists use during the manual construction of data preparation pipelines, we can perform a more guided, efficient search. Such domain knowledge from the data, or the contextual information, can be derived from multiple sources (Figure 2). For example, the domain of the datasets (such as healthcare, finance, e-commerce, etc.) can be inferred from the column names and concrete string values in the table. The relationship across columns can be derived from columns with similar names. The high-level semantics of statistical information can also be derived from the type and range of numerical values in the columns. These column semantics, if captured automatically, could enable a more fine-grained separation of dataset scenarios to the pipeline construction solution, thereby improving its predictive performance.

Data Discovery in Data Lakes: Operations, Indexes, Systems

Ziawasch Abedjan
BIFOLD & TU Berlin
Berlin, Germany
abedjan@tu-berlin.de

Mahdi Esmailoghli
HU Berlin
Berlin, Germany
mahdi.esmailoghli@hu-berlin.de

Sainyam Galhotra
Cornell University
Ithaca, USA
sg@cs.cornell.edu

ABSTRACT

Data discovery has gained significant traction in the database community resulting in various discovery operations, index schemes, and discovery systems. This tutorial explores the architecture and components of data discovery systems, focusing on indexing structures and scalable algorithms for typical operations, such as join and union discovery. While giving insights into individual algorithms, we point out open challenges for holistic systems, data discovery evaluation, and discovery in federated setups.

PVLDB Reference Format:
Ziawasch Abedjan, Mahdi Esmailoghli, and Sainyam Galhotra. Data Discovery in Data Lakes: Operations, Indexes, Systems. PVLDB, 18(12): 5455 - 5459, 2025.
doi:10.14778/3750601.3750694

1 INTRODUCTION

Data lakes, whether intentionally created or emerging as a byproduct of organizational processes or open data initiatives, share a common challenge: they are often underspecified and underdocumented [1, 13]. Unlike curated data warehouses, where the schema and purpose are clearly defined, data lakes typically lack predefined objectives. Despite this, there is significant interest in leveraging these repositories for augmenting in-use datasets or pure exploratory purposes. To support such applications, data discovery as a process becomes essential, enabling users to interact with the data lake not by relying on detailed schema knowledge but by using search and matching techniques. By searching with user-provided keywords or table artifacts, users aim to uncover datasets that are relevant to their use cases.

Data discovery is currently an active field of research, as building scalable solutions to effectively capture user intent, such as augmenting a given dataset fragment, presents various challenges. A key issue is the need to index thousands to millions of tables for fast value look-ups and efficient alignment to evaluate joinability [9, 35] and unionability [19] with the user (query) table. After identifying relevant tables, assessing the usefulness of the retrieved results remains a complex problem. Often, users rely on proxy metrics, such as the downstream machine learning performance of the augmented dataset, to assess its effectiveness [7, 8, 22]. Finally, in distributed data lakes, additional constraints, such as privacy regulations and pricing structures, further complicate the process. In such cases, the usefulness and relevance of datasets must often be assessed based on available metadata and summaries, rather than direct access to the full datasets.

This tutorial is based on our previous study of data discovery use cases and our experience in building systems and algorithms that address some of its emergent challenges. Our goal is to give a structured overview of data discovery software components and to dive into technical aspects that differentiate individual prototypes and approaches. A meta-goal is to bridge the language and perspective differences on what is counted as data discovery and how discovery systems in database research can be compared. Figure 1 depicts a conceptual architecture of data discovery systems that we explore in this tutorial. The user formulates a discovery task that typically aims at enhancing an existing table or table fragment through additional features, rows, or individual values. Such tasks are then typically carried our on top of common operations, such as keyword search, join discovery, union discovery, and others that are built on top of dedicated indexes. In our tutorial, we cover the depicted architecture as follows. We first survey and discuss common data discovery use cases from industry and academia. We then present a classification of data discovery systems with layers that distinguish the locality, languages, and index structures of such systems. Locality captures the accessibility of repositories on monoliths and in distributed settings. Discovery languages can differ depending on how many different discovery tasks a system covers. Accordingly, we then dive deep into concrete index structures and how they serve individual discovery operations and languages.

Previous Tutorials: There have been similar tutorials in the past that relate to ours. In particular, the tutorial by Paton and Wu at EDBT 2024 [21] also covers various aspects of dataset search and navigation. In contrast to their tutorial, we have a strong focus on algorithmic details on how index structures and operations have evolved and can be categorized with a projection on how these approaches relate, can extend to distributed scenarios, and can be combined inside holistic systems. There has been a tutorial from the information retrieval perspective on web table extraction [32]. Our tutorial covers more recent advances and discusses the topic from the database management perspective. The tutorial on data exploration [15] is related but does not focus on data lakes.

2 TUTORIAL OUTLINE

In this tutorial, we first discuss the types of data lakes that set the motivation for building data discovery systems. We further categorize existing systems depending on their scope with regard to data locality, i.e., centralized and distributed repositories. We then detail common operations and fundamental indexing schemes in state-of-the-art that support such operations. Finally, we will discuss methods for evaluating and benchmarking data discovery systems and outline open problems and potential future research.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 18, No. 12 ISSN 2150-8097.
doi:10.14778/3750601.3750694

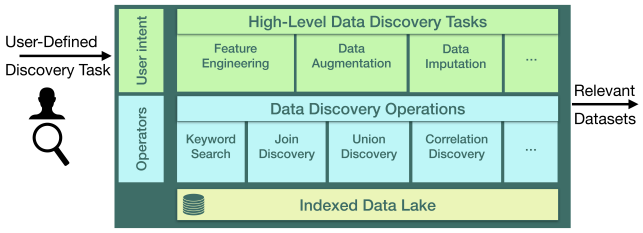


Figure 1: Vanilla data discovery system architecture

2.1 Data Lakes

With the awareness of how valuable data can be even without being currently under active usage, organizations started to accumulate datasets and make them accessible across silos. This is in contrast to the typical data warehousing approach, where handcrafted extract-transform-load pipelines are defined and documented to integrated datasets into a purposeful schema. Data lakes in contrast follow the integrate at application time paradigm. Often times the datasets not only are heterogeneous on schema level but can be subject to heterogenous data models or reside in heterogeneous storage systems. For the sake of scoping, we will focus on the algorithms and systems that have been designed for data lakes with tabular datasets.

2.2 Overview of Data discovery settings

Data discovery is a fundamental capability for organizations that manage large and often distributed collections of datasets. The need for robust data discovery systems has become even more pressing with the widespread adoption of machine learning and AI techniques, which depend critically on access to high-quality, relevant data. Numerous studies have explored different approaches to data discovery, with the optimal solution often varying based on the specific use case. This section provides an overview of the key architectural and interaction paradigms for data discovery.

Centralized vs. Federated Settings. Traditional data discovery systems typically assume *centralized access* to data, where all datasets are consolidated into a single repository, allowing the discovery engine to index and query them directly. However, this assumption is often impractical. In many cases, data is often dispersed across multiple repositories due to concerns related to data sovereignty, privacy regulations, and the high cost of data transfer.

To address these challenges, modern data ecosystems are shifting toward *federated* or *decentralized* discovery architectures. In such settings, data providers retain control over their datasets and share only metadata (or a summary of the dataset) with the discovery system. This model is common in commercial data marketplaces (e.g., Datarade, Dawex) and federated infrastructure initiatives (e.g., Gaia-X, Agora), where discovery must be performed over incomplete information, and access to data is negotiated only after identifying relevant resources.

Data Discovery Tasks and Use Cases. Users express their information needs in various ways, depending on the context and their familiarity with the data. Common interaction modalities include:

- **Keyword-based Discovery:** Users input textual queries along with optional filters. While straightforward and user-friendly, this method is often inadequate when users have nuanced requirements related to data semantics, coverage, or statistical characteristics.
- **Query-by-Example:** Users provide a small sample dataset and search for similar datasets based on structure or content. This approach is helpful when users struggle to verbalize their data needs but can illustrate them via examples.
- **Task-based Discovery:** Users specify a target task, such as training a classifier or evaluating a model, and seek datasets that are suitable for that purpose. This mode is especially relevant for machine learning workflows, where factors like class balance and feature alignment significantly impact performance.

In many real-world scenarios, users may only have a vague or evolving notion of what constitutes a “useful" dataset. Their needs may not be easily captured by keywords, examples, or tasks. To address this, recent work has focused on designing more expressive query languages that allow users to specify fine-grained predicates—such as schema constraints, statistical distributions, or semantic criteria—to better guide the discovery process.

2.3 Discovery Operations

Our tutorial will strongly focus on two major classes of operations that are common in data discovery research: Joins and Unions. note that each of the two can inhibit additional properties, such as fuzziness or conditional utility functions for the correspondingly joinable and unionable tables, as it is relevant for example for task-based discovery tasks. In our tutorial, we will define both operations and discuss the variations of each.

A join discovery operation aims at discovery tables that are joinable with a given user table. In the literature, variations, such as precalculated PK-FK relationships, single column joinability and multi-column joinability, as well as fuzzy joins have been considered. In our tutorial, we will cover the necessary algorithmic steps for each of the aforementioned variations, as well as their use cases and disadvantages. Furthermore, we will dive deep into the class of conditional join discovery approaches, where not only the joinability but also the utility of the obtained tables with regard to downstream tasks have been considered.

A union discovery operation aims at vertically expanding a table with more rows from tables inside a lake. Within this line of work, we will distinguish approaches that check for perfect unionability as in consistent with UNION in SQL as well as approaches that find partially unionable tables.

2.4 Indexing Data Lakes

An efficient index structure is necessary to support any of the aforementioned discovery operators. Index structures have long served as the foundation of traditional information systems, enabling efficient search and retrieval. In the era of large-scale data lakes and advanced data discovery techniques, these structures have evolved to meet two primary objectives: *scalability* and *customizability*. The large size and heterogeneity of data lakes require indexes that can efficiently scale to large volumes of data while supporting various

DataRinse: Semantic Transforms for Data preparation based on Code Mining

Ibrahim Abdelaziz
IBM Research
Yorktown Heights, New York
ibrahim.abdelaziz1@ibm.com

Julian Dolby
IBM Research
Yorktown Heights, New York
dolby@us.ibm.com

Udayan Khurana
IBM Research
Yorktown Heights, New York
ukhurana@us.ibm.com

Horst Samulowitz
IBM Research
Yorktown Heights, New York
samulowitz@us.ibm.com

Kavitha Srinivas
IBM Research
Yorktown Heights, New York
kavitha.srinivas@ibm.com

ABSTRACT

Data preparation is a crucial first step to any data analysis problem. This task is largely manual, performed by a person familiar with the data domain. *DataRinse* is a system designed to extract relevant transforms from large scale static analysis of repositories of code. Our motivation is that in any large enterprise, multiple personas such as data engineers and data scientists work on similar datasets. However, sharing or re-using that code is not obvious and difficult to execute. In this paper, we demonstrate DataRinse to handle data preparation, such that the system recommends code designed to help with the preparation of a column for data analysis more generally. We show that DataRinse does not simply shard expressions observed in code but also uses analysis to group expressions applied to the same field such that related transforms appear coherently to a user. It is a human-in-the-loop system where the users select relevant code snippets produced by DataRinse to apply on their dataset.

PVLDB Reference Format:
Ibrahim Abdelaziz, Julian Dolby, Udayan Khurana, Horst Samulowitz, and Kavitha Srinivas. DataRinse: Semantic Transforms for Data preparation based on Code Mining. PVLDB, 16(12): 4090 - 4093, 2023.
doi:10.14778/3611540.3611628

1 INTRODUCTION

Enterprises often have many data analysts performing modeling and exploration on the same or similar types of datasets. One of the most time consuming parts of any data analysis step is the preparation of data, such as encoding categorical values, handling ordinals, imputation of missing values, data aggregations, amongst others. This step often requires a significant amount of domain knowledge, and is largely carried out manually by a data engineer, data scientist, analyst or a domain expert. The knowledge, once applied, is often buried in code, and this code tends to be stored in enterprise repositories. Listing 1 shows an example of such a transform, where the data scientist is trying to bin `Age` so that it can be combined with the passenger class attribute to make a combined inference about the wealth and age of the passenger, which likely impacted survival on the Titanic. This snippet of code is adapted from a much larger script that performs multiple operations on the titanic dataset[1]. A key goal of DataRinse is to allow a data scientist re-use of code relevant to the dataset without them having to peruse all the code that manipulates the same dataset.

In DataRinse, we perform this by applying static code analysis on Python scripts. We adopt the path of static analysis compared to dynamic analysis because very often the datasets associated with code in repositories are not available readily making dynamic analysis impractical. It works by finding the transformations to fields of a dataframe, which are then extracted, and converted into an intermediate representation, and this representation is traversed to generate code to help the user transform their dataframe. A key goal of DataRinse is to behave as a recommendation system rather than automating data preparation in one shot. Code generation for data preparation is particularly challenging because: (a) the order in which specific transformations are performed is important; (b) code generation has to be sensitive to the 'groups' of transformations that constitute a single operation conceptually. We illustrate both points with a running example. Listing 1 shows a code snippet from an actual script for preparing the titanic dataset for exploratory analysis. Note that when code is generated, it needs to be aware that a number of transforms are being performed as a 'group', so all the contiguous `Age` transforms bin `Age` into buckets. Furthermore, the transformation of `Age * Pclass` is dependent on that binning operation being performed first; that is, there is a dependency between the first function and the second.

DataRinse performs both grouping of conceptually related operations into functions and encodes dependencies between several functions to offer users functions that can be easily incorporated into their own notebooks. We use control flow from static analysis to perform grouping, and we use control and data flow to understand dependencies between the functions that are generated in DataRinse. Furthermore, to make this consumable to the end user, we display a drop down box in the notebook, so that they can choose which field they want to view cleansing functions for. For each function that is extracted, we also show the URL of the

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 16, No. 12 ISSN 2150-8097.
doi:10.14778/3611540.3611628

[1] https://github.com/davified/clean-code-ml/blob/master/notebooks/titanic-original.ipynb

script that was used to generate that function. Listing 4 shows the output of DataRinse of a code snippet for the Age column, where a single function captures all binning changes to Age, and a second separate function shows the creation of a new column using it. The user has a number of such functions per column that can be readily applied to their dataset if they deem it applicable, and the ordering of transformations is encapsulated in the code generated such that a call to transform Age is performed before the use of it in expressions such as `Age * Pclass`, as shown. All the generated functions are dynamically evaluated on the dataframe to ensure that any functions that are not compile-able are filtered out. Similarly, the generated functions that do not result in a change in the values of the dataframe are also eliminated.

We note that ChatBots such as ChatGPT[2] can appear to provide similar functionality for such tasks. There are however key differences between those systems and DataRinse. While ChatGPT generates some useful data operations, it has the following characteristics that are different from DataRinse: (a) It has non-determinism in its generation, it can sometimes find cleansing operations, but produces different results on each run; a well-known problem for ChatBots (see Listings 2, 3 for two different code snippets produced for the same dataset (b) It does not follow basic data operation rules such as being consistent across train and test, and it is not comprehensive - `Fare` is handled only in one snippet. (c) There is no attribution to the source that these are drawn from, (d) It is not focused - the snippets were drawn from a much larger script, where the system produced imports, read CSV files, etc. (e) It can only work off its training dataset - DataRinse can for instance run over any scripts in any repository at any desired frequency (e.g., on every commit or daily) requiring negligible computational resources; ChatBots have to be retrained on enterprise specific code.

2 SYSTEM OVERVIEW

Figure 1 describes the overall DataRinse system for data preparation. Given a new dataset, the system queries code repositories such as GitHub with the table's metadata, specifically the table and column names. One current limitation of DataRinse is that it will only select scripts that match column names and table names - this is a limitation we plan to address in future work through more flexible column-to-concept mappers [7]. The analysis framework is an open source project that we created[3] which performs inter-procedural static analysis to create control flow and data flow graphs for Python scripts [1]. The scope of the analysis is the entire notebook, so dataflow and control flow can be used to order the cleansing operations. We perform interprocedural analysis to trace both data and control flow, as illustrated in Figure 2. The chalky lines for `read_df` and `prep_df` denote function calls, and dataflow follows them, e.g `"titantic_train.csv"` flows to `fl_name`. After analysis, the code generation component for cleansing starts by isolating reads, writes and updates to pandas dataframe. It uses dataflow information to determine which expressions are relevant, e.g. dataflow connects the read of a pandas dataframe at the top of Figure 2 to the update of `dataset` at the bottom.

[2]https://openai.com/blog/chatgpt
[3]https://github.com/wala/graph4code/tree/master/semForms

```python
import pandas as pd

def read_df(fl_name):
  return pd.read_csv(fl_name)

def prep_df(combine):
    for dataset in combine:
        dataset.loc[ dataset['Age'] <= 16, 'Age'] = 0
        dataset.loc[(dataset['Age'] > 16) &
        ↪  (dataset['Age'] <= 32), 'Age'] = 1
        dataset.loc[(dataset['Age'] > 32) &
        ↪  (dataset['Age'] <= 48), 'Age'] = 2
        dataset.loc[(dataset['Age'] > 48) &
        ↪  (dataset['Age'] <= 64), 'Age'] = 3

def main():
    train_df = read_df("titanic_train.csv")
    test_df =  read_df("titanic_test.csv")
    combine = [train_df,test_df]
    prep_df(combine)
    for dataset in combine:
        dataset['Age*Class'] = dataset.Age *
        ↪  dataset.Pclass
```

Listing 1: Example data preparation code, handling the titanic dataset

```python
train_df = train_df.drop(['PassengerId', 'Name',
↪  'Ticket', 'Cabin'], axis=1)
train_df['Age'].fillna( train_df['Age'].median(),
↪  inplace=True)
train_df['Embarked'].fillna(
↪  train_df['Embarked'].mode()[0], inplace=True)
train_df['FamilySize'] = train_df['SibSp'] +
↪  train_df['Parch'] + 1
train_df['IsAlone'] = 1
train_df['IsAlone'].loc[train_df['FamilySize'] > 1]
↪  = 0
train_df['Sex'] = train_df['Sex'].map({'female': 0,
↪  'male': 1}).astype(int)
```

Listing 2: Code snippets from ChatGPT, when prompted to prepare the titanic dataset for analysis, first attempt

The code generation component assembles related code into multiple functions, organized by the fields that they change, as well as records any dependencies in the application of the functions by encoding calls to relevant functions. In Pandas, calls on a dataframe can sometimes mutate the dataframe (e.g. `drop()`), and sometimes not (e.g. `isnull()`). Because there is no way of knowing if the values of a dataframe are actually modified by a call, the filtering component dynamically evaluates the functions returned by the code generation module in the client notebook to filter out functions that do not transform the data frame. Extracted functions across

DeepJoin: Joinable Table Discovery with Pre-trained Language Models

Yuyang Dong
NEC Corporation
dongyuyang@nec.com

Chuan Xiao
Osaka University
Nagoya University
chuanx@ist.osaka-u.ac.jp

Takuma Nozawa
NEC Corporation
nozawa-takuma@nec.com

Masafumi Enomoto
NEC Corporation
masafumi-enomoto@nec.com

Masafumi Oyamada
NEC Corporation
oyamada@nec.com

ABSTRACT

Due to the usefulness in data enrichment for data analysis tasks, joinable table discovery has become an important operation in data lake management. Existing approaches target equi-joins, the most common way of combining tables for creating a unified view, or semantic joins, which tolerate misspellings and different formats to deliver more join results. They are either exact solutions whose running time is linear in the sizes of query column and target table repository, or approximate solutions lacking precision. In this paper, we propose DeepJoin, a deep learning model for accurate and efficient joinable table discovery. Our solution is an embedding-based retrieval, which employs a pre-trained language model (PLM) and is designed as one framework serving both equi- and semantic (with a similarity condition on word embeddings) joins for textual attributes with fairly small cardinalities. We propose a set of contextualization options to transform column contents to a text sequence. The PLM reads the sequence and is fine-tuned to embed columns to vectors such that columns are expected to be joinable if they are close to each other in the vector space. Since the output of the PLM is fixed in length, the subsequent search procedure becomes independent of the column size. With a state-of-the-art approximate nearest neighbor search algorithm, the search time is sublinear in the repository size. To train the model, we devise the techniques for preparing training data as well as data augmentation. The experiments on real datasets demonstrate that by training on a small subset of a corpus, DeepJoin generalizes to large datasets and its precision consistently outperforms other approximate solutions'. DeepJoin is even more accurate than an exact solution to semantic joins when evaluated with labels from experts. Moreover, when equipped with a GPU, DeepJoin is up to two orders of magnitude faster than existing solutions.

PVLDB Reference Format:
Yuyang Dong, Chuan Xiao, Takuma Nozawa, Masafumi Enomoto, and Masafumi Oyamada. DeepJoin: Joinable Table Discovery with Pre-trained Language Models. PVLDB, 16(10): 2458 - 2470, 2023.
doi:10.14778/3603581.3603587

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 16, No. 10 ISSN 2150-8097.
doi:10.14778/3603581.3603587

1 INTRODUCTION

Given a table repository and a query table with a specified join column, joinable table discovery finds the target tables that can be joined with the query. Due to the demonstrated usefulness in data enrichment [10, 16], joinable table discovery has become a key procedure in data lake management and serves various downstream applications, especially those involving data analysis.

For joinable table discovery, early attempts mainly targeted equi-joins [64, 66], which are the most common way of combining tables for creating a unified view [15] and can be easily implemented using SQL. To deliver more joins results for heterogeneous data, recent approaches [16, 17] studied semantic joins, which join on cells with similar meanings via word embedding, so as to handle data with misspellings and discrepancy in formats/terminologies (e.g., "American Indian & Alaska Native" v.s. "Mainland Indigenous"). There are two major limitations in these solutions. First, they only apply to a single join type. Second, most of them are exact algorithms with a worst-case time complexity linear in the product of query column size and table repository size, and thus their scalability is dubious. Despite the existence of an approximate algorithm for equi-joins [66], it is based on MinHash sketches [6] and has to convert the joinability condition to a Jaccard similarity condition, which is non-equivalent and introduces many false positives. Moreover, it is sometimes even slower than an exact algorithm [64].

Seeing the limitations of existing solutions, we propose DeepJoin, a deep learning model designed in a *two-birds-with-one-stone* fashion such that both equi- and semantic joins can be served with one framework. In particular, DeepJoin targets textual attributes with fairly small cardinalities that can fit with language models. To cope with semantic joins, it works on a similarity condition of word embeddings and finds similar textual columns as exactly as possible. To resolve the efficiency issue, it finds joinable tables via an **embedding-based retrieval**. In particular, we employ an embedding model to transform columns to a vector space. By metric learning, columns with high joinability are close to each other in the vector space. Then, to find the top-k target columns ranked by joinability, we resort to a state-of-the-art approximate nearest neighbor search (ANNS) algorithm [38], whose time complexity is *sublinear* in the table repository size.

DeepJoin utilizes a **pre-trained language model** (PLM) for column embedding. PLMs, such as BERT [14], have gained popularity in various data management tasks that involve natural language processing. A salient property of modern PLMs is that they are

transformer networks [55] featuring the attention mechanism, thus not only good at capturing the semantics of column contents for semantic joins, but also able to focus on the cells that are more probable to match in equi-joins, assuming that the query column has a similar distribution to those in the repository. As such, our model gains the capability of handling both join types, only needing the PLM to be fine-tuned on the data labeled for either equi- or semantic joins. In addition, PLMs produce a fixed-length vector, meaning that the following index lookup and search are *independent* of the column size, hence along with the ANNS, addressing the scalability issue in existing solutions. Since PLMs take a text sequence as input, by prompt engineering, we propose a set of options that contextualizes a column to a text sequence. To train the model in a self-supervised manner, we devise a series of techniques to effectively generate positive and negative examples. Moreover, our training features a data augmentation technique, through which our model can learn that the joinability is insensitive to the order of cells in a column.

We conduct experiments on two real datasets and evaluate DeepJoin equipped with two state-of-the-art PLMs. We show that by training on a small subset (30k columns) of the corpus, DeepJoin *generalizes* well to large datasets (1M columns). In particular, DeepJoin outperforms alternative approximate solutions in all the settings, and reports an average precision of 72% for equi-joins and 91% for semantic joins and an average NDCG of 81% for equi-joins and 75% for semantic joins. To test the effectiveness of semantic joins, we also evaluate DeepJoin using data labeled by our database researchers, and the results show that DeepJoin is even better by a margin of 0.105 – 0.165 F1 score than PEXESO [17], an exact solution we use to label DeepJoin's training data. An ablation study demonstrates the usefulness of the proposed contextualization and data augmentation techniques. For scalability test, we vary dataset size from 1M to 5M columns. Even if equipped with a CPU, DeepJoin exhibits superb scalability and is 7 – 57 times faster than existing solutions. With the help of a GPU, DeepJoin outperforms them by up to two orders of magnitude.

Contributions. (1) We propose DeepJoin, a framework for joinable search discovery in a data lake. Our solution targets targets textual attributes with fairly small cardinalities, and is able to detect both equi-joinable and semantically joinable tables. (2) We design the search in DeepJoin as an embedding-based retrieval which employs a fine-tuned PLM for column embedding and ANNS for fast retrieval. The search time complexity is sublinear in the table repository size, and except the column embedding, the search time is independent of the column size. (3) We propose a prompt engineering method to transform column contents to a text sequence fed to the PLM. (4) We devise techniques for training data preparation and data augmentation, as well as a metric learning method to fine-tune the PLM in a self-supervised manner. (5) We conduct experiments to show that our model generalizes well to large datasets and it is accurate and scalable in finding joinable tables.

Furthermore, we would like to mention the following facts: (1) For the embedding-based retrieval in our model, the results of ANNS are directly output as the results of joinable table discovery, whereas more advanced paradigms exist, such as two-stage retrieval [11] which finds a set of candidates by ANNS and ranks the candidates by a more sophisticated model. (2) DeepJoin is not limited to the two PLMs evaluated in our experiments, because the PLM can be regarded as a plug-in in our framework. As such, we expect the performance of DeepJoin can be further improved by using more advanced retrieval paradigms or PLMs.

2 PRELIMINARIES

2.1 Problem Definition

Given a data lake of tables, we extract all the columns in these tables, except those unlikely to appear in a join predicate (e.g., BLOBs), and create a repository of tables, denoted by $\mathcal{X}$. Given a query column Q, our task is to search $\mathcal{X}$ and find the columns joinable to Q. In this paper, we target equi-joins and semantic joins. Next, we define the joinability for these two types, respectively.

Given a query column Q and a target column X in $\mathcal{X}$, the joinability from Q to X is defined by the following equation.

$$jn(Q, X) = \frac{|Q_M|}{|Q|}, \tag{1}$$

where $|\cdot|$ measures the size (i.e., the number of cells) of a column, and Q_M is composed of the cells in Q that have at least one match in X. Here, the term "match" depends on the join type, i.e., an equi-join or a semantic join. We normalize the size of Q_M by the size of Q to return a value between 0 and 1. Moreover, the joinability is not always symmetric, depending on the definition of Q_M.

For equi-joins, we model each column as a set of cells by removing duplicate cell values, and define the equi-joinability as follows.

Definition 2.1 (Equi-Joinability). The equi-joinability from the query column Q to a target column X in $\mathcal{X}$ counts the intersection between Q and X, normalized by the size of Q; i.e., in Equation 1,

$$Q_M = Q \cap X. \tag{2}$$

The equi-joinability defined above counts the the distinct number of cells in Q that match those in X, and thus can be used to measure the equi-joinability [64]. Our method can be also extended to the case when columns are modeled as multisets, so as to support one-to-many, many-to-one, and many-to-many joins. In this case, we may measure the joinability by the number of join results and normalize it by the product of $|Q|$ and $|X|$, instead of $|Q|$ in Equation 1.

For semantic joins, we consider string columns and embed the value of each cell to a metric space $\mathcal{V}$ (e.g., word embedding by fastText [19]). As such, each string column is transformed to a multiset of vectors. We abuse the notation of a column to denote its multiset of vectors. Then, the notion of vector matching is defined as follows.

Definition 2.2 (Vector Matching). Given two vectors v_1 and v_2 in $\mathcal{V}$, a distance function d, and a threshold τ, v_1 matches v_2 if and only if the distance between v_1 and v_2 does not exceed τ. We use notation $M_\tau^d(v_1, v_2)$ to denote if v_1 matches v_2; i.e., $M_\tau^d(v_1, v_2) = 1$, iff. $d(v_1, v_2) \leq \tau$, or 0, otherwise.

Given a query column Q and a target column X, the semantic-joinability is defined using the number of matching vectors [1].

[1] Besides this definition, semantic joins are also investigated in [23], yet it studies the problem of performing joins rather than searching for joinable columns.

DIALITE: Discover, Align and Integrate Open Data Tables

Aamod Khatiwada
Northeastern University
Boston, Massachusetts, USA
khatiwada.a@northeastern.edu

Roee Shraga
Northeastern University
Boston, Massachusetts, USA
r.shraga@northeastern.edu

Renée J. Miller
Northeastern University
Boston, Massachusetts, USA
miller@northeastern.edu

ABSTRACT

We demonstrate a novel table discovery pipeline called DIALITE that allows users to discover, integrate and analyze open data tables. DIALITE has three main stages. First, it allows users to discover tables from open data platforms using state-of-the-art table discovery techniques. Second, DIALITE integrates the discovered tables to produce an integrated table. Finally, it allows users to analyze the integration result by applying different downstreaming tasks over it. Our pipeline is flexible such that the user can easily add and compare additional discovery and integration algorithms.

CCS CONCEPTS

• **Information systems** → **Information integration**.

KEYWORDS

Data Lakes, Data Discovery, Data Integration, Full Disjunction

ACM Reference Format:
Aamod Khatiwada, Roee Shraga, and Renée J. Miller. 2023. DIALITE: Discover, Align and Integrate Open Data Tables. In *Companion of the 2023 International Conference on Management of Data (SIGMOD-Companion '23), June 18–23, 2023, Seattle, WA, USA.* ACM, Seattle, WA, USA, 4 pages. https://doi.org/10.1145/3555041.3589732

1 INTRODUCTION

Data discovery has become an important component in data science pipeline. The discovery process uses techniques such as keyword search [13] and table search [4, 7, 9] to discover a set of tables (datasets). Data scientists use such tables to support decision-making processes, train machine learning models, perform statistical analysis, and so on. After discovery, a natural step is to integrate the discovered tables. An integrated table provides a unified view of the data and allows users to run queries and analyses that go beyond a single table.

While integrating tables, we intend to combine tuples from different tables in a maximal way such that the integrated tuples carry as much information as possible. This enriches the analysis and decision-making process after integration and also improves the quality of downstream applications. The widely known outer-join [12] operator is not associative and does not aim to maximize the connections among the integrated tuples [6]. Accordingly, *Full Disjunction (FD)* [6] has been understood as a natural way of assembling partial pieces of information (facts) such that it maximizes the connections among these facts [11]. FD can be viewed as an associative version of outer join [2] and has been used to integrate information across relational tables [2, 10] and web tables [10]. In a recent paper [8], we proposed ALITE, a new algorithm based on Full Disjunction to integrate tables discovered in data lakes. ALITE was shown to be correct and faster than the existing FD algorithms [2, 10] while integrating real open data lake tables in practice [8]. Additional details on other integration operators and their comparison against FD are available in the full ALITE paper [8] where FD was shown to be a better semantics for integration because it produces a result over which a downstream task like entity-resolution performs more accurately.

In this demonstration, we propose DIALITE, a novel system that discovers, aligns, and integrates open data tables. It extends the aforementioned ALITE [8], which does not perform discovery but instead takes a set of tables as input, aligns the matching columns using holistic schema matching, and applies the FD to get an integrated table. DIALITE offers state-of-the-art systems for different table discovery tasks [7, 14] before applying ALITE to integrate them. Furthermore, DIALITE also offers new downstream analytics (that have not been previously considered) to evaluate the quality of integration. Specifically, DIALITE allows users to

(1) Upload a table or randomly generate one using GPT-3 as a query and discover related tables from a given data lake that are unionable [7] or joinable [14] with the query table. Apart from the available table discovery algorithms, DIALITE also allows users to add new algorithms for table search based on their preference.
(2) Integrate the discovered tables (or upload a set of tables to be integrated) using a novel table integration system called ALITE [8]. Besides ALITE, we allow users to add new integration operators within our extendible architecture.
(3) Analyze and compare the table integrated using ALITE against alternative integration techniques by performing downstream applications. In our demo, we consider outer join as an alternative integration technique (or other queries/methods added by the user) and present data analytics (common aggregations and statistics) and entity-resolution as downstream applications. Both, when applied over real tables (which can be incomplete) will show the dramatic difference between maximally integrating information using FD vs. using outer joins.

Related Work. To the best of our knowledge, DIALITE is the first system that enables a full table search pipeline starting from

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
SIGMOD-Companion '23, June 18–23, 2023, Seattle, WA, USA
© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-9507-6/23/06…$15.00
https://doi.org/10.1145/3555041.3589732

table discovery, followed by table integration and downstream applications over the integrated table. An earlier system called COCOA [3] focuses on finding joinable tables with correlated attributes that expand the query table. After finding the tables, COCOA applies LEFT JOIN between the query table and the discovered table as the integration operator. DIALITE, on the other hand, finds a set of related tables that can be integrated with the query table (including unionable and joinable tables). Auctus [1] also discovers related tables and integrates table pairs by applying inner join or union. Unlike Auctus, DIALITE does not limit the number of tables to be integrated. In addition, DIALITE uses FD to integrate the tables, which has been shown in theory to maximize the connections among the tuples in the tables [11].

Existing table discovery techniques search for joinable, unionable, or related tables. **JOSIE** [14] and **LSH Ensemble** [15], for instance, take a query table as input with a query column marked by the user and returns a set of top-k tables that are joinable with the query table. **SANTOS** [7] takes a query table and discovers a set of top-k semantically unionable tables as output. Other table search techniques [4, 9] also focus on table discovery from large repositories. But the important question of how to integrate the discovered tables is not addressed. **ALITE** [8] provides a solution to this question. As the data lake tables may lack consistent and meaningful column headers, ALITE applies holistic schema matching over the set of searched tables and assigns a dummy column header called an Integration ID to the set of matching columns. Then, it applies a natural FD over the integration IDs to integrate the tables. The integration process outputs an integrated table with maximally integrated tuples [6, 11].

2 SYSTEM DESCRIPTION

DIALITE, outlined in Fig. 1, has three stages (discover, align & integrate, and analyze) which are included in our demo plan (Sec. 3).

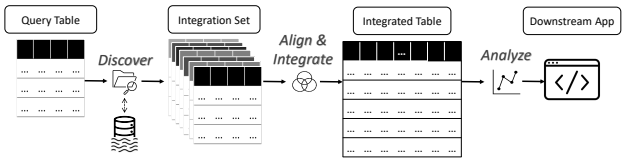


Figure 1: An overview of DIALITE.

DIALITE offers various options for discovery, alignment, integration, and analysis, including the ability for users to implement their own methods. The pipeline begins with a user-provided query table Q. However, we also allow the user to randomly generate a table using GPT-3 [5]. We now detail the components.

2.1 Discover

Given a query table Q, DIALITE uses a data discovery method to find tables in a data lake (table repository) $\mathcal{D}$ that are unionable [7], joinable [14], or simply semantically similar. DIALITE allows the user to choose among existing discovery algorithms including SANTOS [7] and LSH Ensemble [15]. Alternatively, a user can use their own discovery algorithm by implementing a similarity method between two tables. As we apply ALITE [8] for alignment and integration, the discovery phase is agnostic to the type of search. ALITE, aiming to maximize the connections among the tables, decides how the tables should be integrated. The output of discovery is an *integration set* $D \subseteq \mathcal{D}$, a set of tables to be integrated, including the query table. Our discovery techniques allow users to control the number of tables returned, and our system permits users to select a subset of the discovered tables to be integrated.

2.2 Align and Integrate

Given an integration set D, DIALITE uses ALITE [8] to integrate the tables. The integration set can be derived from the discovery part (Sec. 2.1) or provided as input by the user. The latter represents a traditional data integration scenario where the integration set is given. As shown in Fig. 1, this stage outputs an integrated table.

ALITE is composed of two main parts, namely align and integrate. Note that we do not require reliable column headers in the integration set. The first part of ALITE (Align) applies holistic schema matching to identify common columns in the integration set. The matched columns are given the same integration ID. Using these ids as attribute names, the integrate part applies (natural) FD. Specifically, ALITE uses a new holistic schema matching algorithm that was shown to outperform state-of-the-art matchers. ALITE uses a novel algorithm to compute the FD shown to be correct and efficient on real tables (including tables having nulls) [8]. DIALITE also allows users to add alternative integration operators, e.g., outer join, which is included for demonstration.

2.3 Analyze

Given an integrated table (that can also be uploaded as a file by the user), DIALITE allows the user to explore the benefits of integration. Specifically, the user can choose a downstream application to apply over the integrated table. A simple application is an aggregation query that can be applied over the integrated table as we illustrate in Sec. 3.1. In Sec. 3.2 we also present a more complex downstream application, entity-resolution (ER).

2.4 Implementation

DIALITE is implemented in Python 3.8 and the demonstration uses a web application.[1] Within our pipeline, we use SANTOS [2], LSH Ensemble [3] and ALITE [4] using their publicly available code. Also, we use the *py_entitymatching* package to show ER as a downstream application.[5] We allow users to interact with the system after each step so that they can validate the intermediate results.

3 DEMONSTRATION PLAN

Next, we illustrate our demonstration. The link to a demonstration video is available in the github repository.[1] Our demo contains two parts. First, we present the use case of the pipeline as described in Sec. 2. This part is actually composed of three demonstration items, that can be demonstrated independently. Then, we demonstrate DIALITE's extensibility to new algorithms for discovery, integration, and analysis.

[1] https://github.com/northeastern-datalab/dialite
[2] https://github.com/northeastern-datalab/santos
[3] https://github.com/ekzhu/datasketch
[4] https://github.com/northeastern-datalab/alite
[5] https://github.com/anhaidgroup/py_entitymatching

Discovering Similarity Inclusion Dependencies

YOURI KAMINSKY*, Hasso Plattner Institute, University of Potsdam, Germany
EDUARDO H. M. PENA, Federal University of Technology – Paraná, Brazil
FELIX NAUMANN, Hasso Plattner Institute, University of Potsdam, Germany

Inclusion dependencies (INDs) are a well-known type of data dependency, specifying that the values of one column are contained in those of another column. INDs can be used for various purposes, such as foreign-key candidate selection or join partner discovery. The traditional notion of INDs is based on clean data, where the dependencies hold without exceptions. Unfortunately, data often contain errors, preventing otherwise valid INDs from being discovered. A typical response to this problem is to relax the dependency definition using a similarity measure to account for minor data errors, such as typos or different formatting. While this relaxation is known for functional dependencies, for inclusion dependencies no such relaxation has been defined.

We formally introduce *similarity inclusion dependencies*, which relax the inclusion by demanding the existence only of sufficiently similar values. Similarity inclusion dependencies can fulfill traditional IND use cases, such as foreign-key candidate discovery, even in the presence of dirty data. We present Sawfish, the first algorithm to discover all similarity inclusion dependencies in a given dataset efficiently. Our algorithm combines approaches for the discovery of traditional INDs and string similarity joins with a novel sliding-window approach and lazy candidate validation. Our experimental evaluation shows that Sawfish can outperform a baseline by a factor of up to 6.5.

CCS Concepts: • **Information systems** → **Information integration**; *Data mining*.

Additional Key Words and Phrases: data profiling, data lake, joinability

ACM Reference Format:
Youri Kaminsky, Eduardo H. M. Pena, and Felix Naumann. 2023. Discovering Similarity Inclusion Dependencies. *Proc. ACM Manag. Data* 1, 1, Article 75 (May 2023), 24 pages. https://doi.org/10.1145/3588929

1 INCLUSION DEPENDENCY

Data profiling is the process of extracting metadata from datasets. Data dependencies are an important type of metadata and, thus, have a crucial role in data profiling. There are different forms of data dependencies, e.g., functional dependencies (FDs) and inclusion dependencies (INDs). In particular, INDs express that the tuples of one column-combination are contained in the tuples of another column-combination. We call an IND unary, if it holds between two individual columns. INDs help data practitioners to understand and structure unknown data, in particular, in discovering foreign key candidates and joinable partners [20]. Moreover, INDs have assisted schema design in [15] and can improve query execution [12].

*Corresponding author (youri.kaminsky@hpi.de)

Authors' addresses: Youri Kaminsky, Hasso Plattner Institute, University of Potsdam, Germany, youri.kaminsky@hpi.de; Eduardo H. M. Pena, Federal University of Technology – Paraná, Campo Mourão, Paraná, Brazil, eduardopena@utfpr.edu.br; Felix Naumann, Hasso Plattner Institute, University of Potsdam, Germany, felix.naumann@hpi.de.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
2836-6573/2023/5-ART75 $15.00
https://doi.org/10.1145/3588929

Traditional IND discovery assumes clean data: *all* tuples of the dependent column-combination must be exactly *equal* in the referenced column-combination. However, the ever-increasing volume of data also leads to more "dirty" data [18]. Thus, relaxed dependencies have been introduced to deal with erroneous data [4].

One example of relaxed dependencies is matching dependencies (MDs), which generalize the concept of functional dependencies (FDs) [23]. For a traditional FD, the tuples on the dependent side must be equal for all equal tuples on the determinant side. In contrast, MDs use similarity measures instead of the strict equality constraint. In other words, an MD holds if, for all *similar* tuples on the determinant side, all tuples on the dependent side are also *similar*. MDs allow data practitioners to address typical use cases of FDs, such as schema normalization, even in the presence of dirty data. Moreover, MDs can also be used for duplicate detection [23]. A feature of MDs is that they support arbitrary similarity measures and a configurable similarity threshold to balance the error tolerance and over generalization.

There is no corresponding notion yet for INDs to allow for similar values. Thus, we introduce similarity inclusion dependencies (sINDs). In contrast to traditional INDs, sINDs use a similarity measure to define inclusion. An sIND holds if, for all dependent values, there exists a referenced value that is at least *similar*. Like MDs, sINDs support arbitrary similarity measures and configurable similarity thresholds. For this work, we consider representatives of both edit-based and token-based similarity measures: the edit distance and the Jaccard similarity.

sINDs are a natural way to handle dirty data. Besides traditional IND use cases, sINDs can also identify sources of possibly erroneous data in data lakes. If a candidate IND does not hold, but its respective sIND holds, the data can be analyzed and used to identify erroneous relations. Either only one of the relations contains errors, so we can use the other relation to fix these, or both relations contain errors, and the data needs to be cleaned more thoroughly.

To illustrate the usefulness of sINDs, we present an example in Table 1. The tables are for a fictitious soccer tournament. While one table shows the results after all teams played against each other, the other table presents an aggregation of the participation forms of all goalkeepers in the tournament. We would assume that the values of the *club* column of Table 1b are contained in the values of the *name* column of Table 1a. However, multiple goalkeepers made minor spelling mistakes in their club name. Despite these mistakes, we want to discover the dependency. On the one hand, we are interested in joining these columns to know which goalkeeper played for the most successful team. On the other hand, we could perform a data cleaning task, because all mistakes are only in the *club* column of Table 1b. Thus, there is a matching counterpart in the other column to automatically correct the errors.

Table 1. Example relations of a soccer tournament

(a) Final Results

results	
name	points
SpVgg Beelitz	4
Potsdamer SC	9
SpVgg Bernau	4
VfL Potsdam	0

(b) Participating Goalkeepers

goalie	
p_id	club
202	SpVgg Beelitz
216	SpVgg Beelittz
469	Potsdamer SC
617	SpVgg Be_nau
692	ViL Potsdam
853	Potsdamer SC

Effective Entity Augmentation By Querying External Data Sources

Christopher Buss
Oregon State University
bussch@oregonstate.edu

Jasmin Mousavi
Oregon State University
mousavij@oregonstate.edu

Mikhail Tokarev
Oregon State University
tokarevm@oregonstate.edu

Arash Termehchy
Oregon State University
termehca@oregonstate.edu

David Maier
Portland State University
maier@pdx.edu

Stefan Lee
Oregon State University
leestef@oregonstate.edu

ABSTRACT

Users often want to augment and enrich entities in their datasets with relevant information from external data sources. As many external sources are accessible only via keyword-search interfaces, a user usually has to manually formulate a keyword query that extract relevant information for each entity. This approach is challenging as many data sources contain numerous tuples, only a small fraction of which may contain entity-relevant information. Furthermore, different datasets may represent the same information in distinct forms and under different terms (e.g., different data source may use different names to refer to the same person). In such cases, it is difficult to formulate a query that precisely retrieves information relevant to an entity. Current methods for information enrichment mainly rely on lengthy and resource-intensive manual effort to formulate queries to discover relevant information. However, in increasingly many settings, it is important for users to get initial answers quickly and without substantial investment in resources (such as human attention). We propose a progressive approach to discovering entity-relevant information from external sources with minimal expert intervention. It leverages end users' feedback to progressively learn how to retrieve information relevant to each entity in a dataset from external data sources. Our empirical evaluation shows that our approach learns accurate strategies to deliver relevant information quickly.

PVLDB Reference Format:
Christopher Buss, Jasmin Mousavi, Mikhail Tokarev, Arash Termehchy, David Maier, and Stefan Lee. Effective Entity Augmentation By Querying External Data Sources. PVLDB, 16(11): 3404 - 3417, 2023.
doi:10.14778/3611479.3611535

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/bussch/QueryBasedEntityAugmentation.

1 INTRODUCTION

There is a recognized need to collect and connect information from a variety of data sources [14, 18, 23]. As an example, we have recently worked in a large-scale NIH-funded project to augment the information of biomedical entities by querying other biomedical data sources [49]. The main focus of this project is to *repurpose* current drugs to treat or mitigate the symptoms of diseases for which there is insufficient time or resources to develop effective treatments (e.g., new or rare diseases) [2]. Biomedical researchers often have some local dataset of available drugs (e.g., a dataset of *FDA-approved uses* of drugs). Given a drug in the local dataset, a researcher usually needs to query external data sources to find additional information about the drug (e.g., its *off-label uses*).

Due to a lack of access or resources, external information often must be retrieved through querying [14, 47]. Many data sources are only accessible via query interfaces or APIs. Even with access, it may require too much of a resource (e.g., storage space, time) to download and maintain an up-to-date copy of the external dataset. Thus, information relevant to some local entity must often be gathered on a as-needed basis by querying external data sources. For example, as many biomedical data sources are available only via query APIs, the users of the aforementioned drug repurposing data collection system must often query the information relevant to their current drug of interest through query APIs.

However, formulating a query that extracts specific information can be troublesome. Different data sources often represent the same concept in distinct forms [12, 15] such that one needs to tailor their query to specific external data sources. Figure 1 illustrates a case where users have a local dataset of FDA-approved uses of drugs, named *FDA-Approved Uses*, and would like to query an external data source that contains the off-label uses of those drugs, named *Off-Label Uses*. A drug that is identified by one of its brand names (e.g., *Zoloft*) in *FDA-Approved Uses* is referred to by its generic name (e.g., *Sertraline*) in *Off-Label Uses*. Because of heterogeneities, one may not know how to query for a specific external entity prior to investigating the content and structure of the data in the external source. Consider a biomedical researcher who seeks additional information about the drug *Zoloft* in their local dataset. Since they are only aware of the structure and content of their local dataset, they query the external data source for *Zoloft*, but that elicits no results. They try again using a much more general description of *Zoloft* (i.e., being a *serotonin reuptake inhibitor*). However, their under-specified query produces many results, most of which are irrelevant (i.e., contain information about drugs that are not *Zoloft*). After additional trial-and-error, they find a query that retrieves *Sertraline*. More work is required to then merge the local and external entities into one coherent representation.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 16, No. 11 ISSN 2150-8097.
doi:10.14778/3611479.3611535

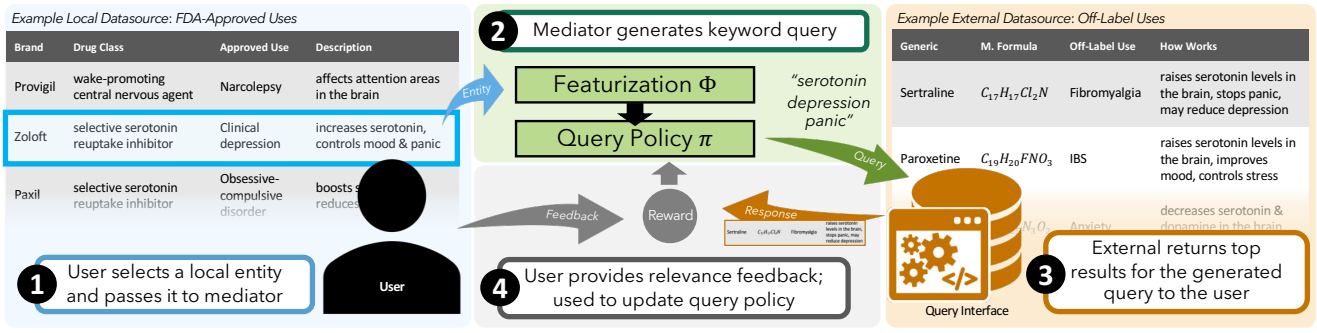


Figure 1: An example of our framework for a single user and single external data source. The user selects (by query, GUI, etc,.) the local entity *Zoloft*. The mediator uses its learned query policy to extract the relevant entity (*Sertraline*) from the external source. The user provides relevance feedback on the results, which is then used to further refine the mediator's querying policy.

Manually querying for specific external entities takes too much time and financial resources. Continuing our example, if the researcher needs additional information for another drug in their local dataset, they will need to repeat the entire process. Moreover, if they need information from additional external data sources, then the work required to query for each drug is greatly exacerbated. Furthermore, any other researcher with a similar information need must *repeat the same such work themselves.*

To alleviate the burden, one can use a **shared system that automates query formulation**. This **mediator system** acts as a go-between for users and external data sources: a user specifies a local entity (e.g., *Zoloft*) perhaps through a query or a graphical user interface, and the mediator maps the local entity to queries that retrieve the relevant external entities (e.g., *Sertraline*) from their respective external sources.

To the best of our knowledge, such mediators are currently created by *manually writing programs* that generate queries for specific external sources to retrieve relevant records to a given local entity. Each program implements a set of manually written rules specific to its external source. These rules *cannot* necessarily be reused across data sources. Thus, the mediator requires a significant amount of labor and expert attention to build and maintain. Instead of conducting their own research, biomedical researchers in our NIH-funded project spend most of their time writing these programs and investigating the content and structure of every external source to ensure that the programs formulate appropriate queries.

In this paper, we examine methods for *learning the mediator's query policy online* through user interaction. As illustrated in Figure 1, after the user specifies a local entity, the mediator formulates a query to retrieve records from an external source according to its *query policy* and shows the returned external records to the user. The user then provides feedback on the relevance of the returned records to the local entity. Our mediator learns to revise its query policy and improves its performance using the user's feedback.

An alternative to this online learning paradigm is to use offline training data to learn query formulation but collecting and labeling such data still requires considerable manual effort from domain experts [14]. Particularly, it is challenging to gather useful training data from external sources. The data collection/labeling might need to be repeated as the external datasets evolve. In many domains (e.g., drug repurposing for emerging viral diseases), users cannot wait long to prepare offline training data.

Of course, online learning of query policies comes with its own set of challenges. First, the mediator should learn to formulate reasonably accurate queries over external sources early on. We assume the mediator must be effective in the *short run* so users will continue to provide feedback. It is particularly difficult to meet this goal over large local or external datasets as the amount of required feedback for accurate learning generally grows with the number of entities. Second, the mediator should improve its querying policy and increase the effectiveness of its results in the *long run*. Online learning literature indicates that a policy that is effective in the *short run* (i.e., meets the first challenge) might not be accurate in the *long run* as it might become biased to early observations or decisions that do not deliver accurate results in the *long run* [42]. Third, due to lack of prior knowledge about the precise content and structure of the relevant external information, the number of candidate queries for a local entity might be enormous. This *large search space* makes finding effective queries difficult.

Due to the wide-spread use of keyword query interfaces over external sources, we develop online learning methods for formulating keyword queries. There are systems for automatic keyword query formulation, but they assume returned results are always relevant, which is not usually true and is the challenge that we address [47] (see Section 8). Our contributions are as follows:

- We present a framework for on-demand collection of relevant external entities only accessible via query interfaces (Section 2).
- We define the problem of online query-policy learning within the context of the aforementioned framework (Section 3).
- We present a method that learns a separate query policy for each individual local entity. We show that this approach does not scale to large local datasets as it might require a great deal of user feedback (Section 4).
- We propose an entity-conditional method that learns a query policy jointly over all local entities. This approach significantly reduces the amount of user feedback required to learn effective query policies. To overcome representational heterogeneity across the local and external sources, we propose techniques to use features and keywords from the external results in our model and queries, respectively (Section 5).
- If the local dataset contains many diverse entities, it might not be possible to learn effective queries for many entities using a shared model. Hence, we propose an approach that gradually

Enriching Relations with Additional Attributes for ER

Mengyi Yan
Beihang University, China
yanmy@act.buaa.edu.cn

Wenfei Fan
Shenzhen Institute of Computing Sciences, China
University of Edinburgh
United Kingdom
Beihang University, China
wenfei@inf.ed.ac.uk

Yaoshu Wang
Shenzhen Institute of Computing Sciences, China
yaoshuw@sics.ac.cn

Min Xie*
Shenzhen Institute of Computing Sciences, China
xiemin@sics.ac.cn

ABSTRACT

This paper studies a new problem of relation enrichment. Given a relation D of schema R and a knowledge graph G with overlapping information, it is to identify a small number of relevant features from G, and extend schema R with the additional attributes, to maximally improve the accuracy of resolving entities represented by the tuples of D. We formulate the enrichment problem and show its intractability. Nonetheless, we propose a method to extract features from G that are diverse from the existing attributes of R, minimize null values, and moreover, reduce false positives and false negatives of entity resolution (ER) models. The method links tuples and vertices that refer to the same entity, learns a robust policy to extract attributes via reinforcement learning, and jointly trains the policy and ER models. Moreover, we develop algorithms for (incrementally) enriching D. Using real-life data, we experimentally verify that relation enrichment improves the accuracy of ER above 15.4% (percentage points) by adding 5 attributes, up to 33%.

PVLDB Reference Format:
Mengyi Yan, Wenfei Fan, Yaoshu Wang, and Min Xie. Enriching Relations with Additional Attributes for ER. PVLDB, 17(11): 3109 - 3123, 2024.
doi:10.14778/3681954.3681987

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/SICS-Fundamental-Research-Center/Enrichment.

1 INTRODUCTION

When we talk about incomplete information, we typically refer to (null) values and tuples missing from a relation D of schema R. However, for an application at hand, schema R may be incomplete; it "may not have all attributes required for analysis" [131]. As a consequence, the tuples in D do not have enough information for the application, *e.g.,* causal inference [131], aggregate SQL queries [84, 130] and movie popularity classification [33]. As exemplified in [131], a data analyst in the WHO organization aimed to estimate the effect of a mask policy on the coronavirus mortality rate, but found critical attributes (*e.g.,* weather that affects people's willingness to wear masks) not included in the data [131].

*Corresponding author
This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 17, No. 11 ISSN 2150-8097.
doi:10.14778/3681954.3681987

Table 1: An Example of Person **Table**

tid	name	gender	email	address	city	age	spouse_name	eid
t_1	James Davis	M	james@example.com	18 Elmwood Rd	Chicago	45	Ava Davis	e_1
t_2	John Wilson	M	john@example.com	18 Maple Avenue	Houston	null	Ava Wilson	e_2
t_3	Ava Davis	F	ava@example.com	12 Pine Lane	Boston	36	John Wilson	e_3
t_4	Ava Wilson	F	a.Wat@example.com	18 Maple Avenue	Houston	36	John Wilson	e_3
t_5	Ava Davis	F	ad123@example.com	18 Elmwood Rd	Chicago	42	James Davis	e_4

Attributes are missing from schema R for several reasons [91, 114]. (1) People may lack full knowledge of desired functionality in large-scale applications. (2) The application world constantly evolves, necessitating enhancements. (3) The scale of tasks often requires incremental design and commissioning. Regardless of the reasons, relation D often misses features needed for our application.

In this paper, we focus on entity resolution (ER) as our target application, which has been commonly used in, *e.g.,* e-commerce and financial institutions. As an example, the insurance industry employs ER as a routine operation in detecting identity fraud, which was responsible for over $17 billion stolen from U.S. consumers in 2017 [1]. As reported in [3], fraudsters forge fake identities. Such a fraud is hard to detect, since it can use information stolen from a real person, and a mix of real and synthetic data [6]. There are usually no unique identities as references, since collecting them is "time-consuming, adds friction to the customer journey, and is also an easy check to bypass" [12]. Similar problems also exist in card fraud (*e.g.,* duplicate applications for gift/credit cards) or money laundering (*e.g.,* move money from one account into another) [5, 111].

To detect fraud, the insurance industry has been advocating "to incorporate external information into the identity verification process" [6]. This has been practiced by, *e.g.,* SEON [13] and SIFT [14].

Example 1: Consider Table 1 with 5 tuples t_1-t_5 for 4 persons e_1-e_4, who are applying for promotional gift cards, where each person is limited to one card (*e.g.,* eGift of Starbucks [16]). The tuples have a schema with attributes name, gender, email, address and city; the store allows one person to have, *e.g.,* multiple different emails.

A not-so-sophisticated ER method $\mathcal{A}_{\mathsf{ER}}$ may predict that t_3 and t_5 are the same person (a false positive, FN), since they have exactly the same name. It may also miss the true match of t_3 and t_4 (a false negative, FN), due to different names, emails and addresses, although Ava Wilson was named Ava Davis before her marriage, and she issued a duplicate application using her old identity.

Fortunately, additional attributes can help us reduce FPs/FNs of ER. (a) As remarked in [131], age or ethnicity are often missing in person datasets. If additional attribute age is available, it provides a strong evidence for $\mathcal{A}_{\mathsf{ER}}$ to tell that t_3 and t_5 are mismatched, since they have different ages. (b) If we further enrich the schema with attribute spouse_name, (*e.g.,* by social networks [12]), $\mathcal{A}_{\mathsf{ER}}$ can identify t_3 and t_4, since they are married to the same person. □

Missing attributes are as damaging as missing data, but it has not received much attention. While there has been a host of work on missing data imputation [25, 26, 29, 40, 49, 50, 54, 55, 66, 77, 100, 105, 116, 128, 129], no prior work has systematically studied how to enrich incomplete schema in order to improve ER accuracy. They either do not target downstream applications or is not developed for ER and thus, miss distinguishing attributes for ER (see Section7).

To enrich dataset D of schema R for ER, we can extract information from external sources *e.g.,* text [63, 64], information space [43], XML [126], data warehouse [22] and Web [59]. In fact, knowledge enrichment has been practiced in medicine [112], network [133], e-commerce [15], recommendation [113] and text generation [132]. Among them, knowledge graphs (KGs) are particularly promising for ER. KGs link related entities and disambiguate entities with similar names [17]; moreover, their attributes are already reconciled and typically highly informative [130, 131], making it feasible to improve the ER accuracy. In particular, financial institutions have been using KGs to detect various types of fraud [5]; indeed, "using a KG for ER, the company can easily detect clusters of fake claims" [3].

In light of this, we study relation enrichment for ER by referencing a KG G. We consider *reliable* G, *i.e.,* a KG that is relatively clean and complete. Several popular KGs are in place, *e.g.,* general-purpose Wikidata [9] and domain-specific DRKG [71]. The issue is highly nontrivial. It requires us to link tuples in D with vertices in G for enrichment. Worse still, a KG typically maintains all sorts of properties of entities and their links to provide a comprehensive picture. If we enrich schema R with all such properties, it may hamper the accuracy of ER. As evidenced by [34], only *relevant* attributes contribute positively to identifying true positives, and "attributes containing null values may affect negatively the ER result". This motivates us to enrich schema with bounded relevant features.

Moreover, real-life datasets and KGs constantly change, *e.g.,* Wikidata publishes hundreds of live updates every minute [8] and the IMDB we used is refreshed daily [11]. In particular, financial institutions often require real-time detection of, *e.g.,* fraud in the online payment of credit cards [18]. It is too costly to conduct relation enrichment starting from scratch in response to the updates. These motivate the need for incremental enrichment.

To make the idea work, several questions have to be answered. What distinguishing features should we add to R to best improve the ER accuracy? For a tuple t in D, where can we find the additional attributes from KG G to complement t? How can we incrementally maintain the enriched D in response to updates to D and G?

Contributions & Organization. This paper studies relation enrichment for improving the accuracy of ER models. Consider a relation D of schema R and assume a reliable knowledge graph G.

(1) An enrichment scheme (Section 3). We formulate *the problem of relation enrichment for ER*. Given a black-box ER model $\mathcal{A}_{\mathsf{ER}}$ and a parameter m, it is to extract at most m features from knowledge graph G and extend schema R with the features as additional attributes, in order to maximize the accuracy of $\mathcal{A}_{\mathsf{ER}}$. We separate schema enrichment from data enrichment, and show that the former is NP-complete and the latter is in PTIME. This said, we propose a scheme ENRICH for enriching both schema R and relation D.

(2) Schema enrichment (Section 4). We propose a method for enriching schema R for ER under ENRICH. We develop a method for heterogeneous entity resolution (HER) to identify top-ranked matches (tuples and vertices) across relation D and KG G. We learn a policy via reinforcement learning to extract at most m features that extends R to schema R_G. Each feature is fetched by a path in G. We pick features that are as diverse from the existing attributes of R as possible, yield as few null values as possible, and maximumly improve the accuracy of ER. To make the policy robust to different data distributions, we jointly train the policy and the model $\mathcal{A}_{\mathsf{ER}}$.

(3) Data enrichment (Section 5). Under ENRICH, we develop algorithms for enriching the relation D to an instance D_G of schema R_G. We support both a batch mode and an incremental mode. In the batch mode, we extend each tuple t of D by identifying vertices v in G that refer to the same entity as t via HER, traversing paths from v to extract the additional features, and adding the features to t. In the incremental mode, we dynamically maintain D_G in response to updates to both relation D and graph G. To scale with large G and D, we parallelize the algorithms and show their parallel scalability, *i.e.,* they guarantee to reduce runtime when more resources are used [81]. We defer the parallelization to [10] for the lack of space.

(4) Experimental study (Section 6). Using real-life data and benchmarks, we empirically find the following. (a) Relation enrichment improves the accuracy of ER models by 15.4% on average, up to 33%, by adding 5 attributes. (b) It is on average 5.2% (resp. 14.6%) more accurate than ML models for feature augmentation (resp. feature selection), up to 18.2% (resp. 32.6%). (c) Batch enrichment is 5.94X faster than the baselines on IMDB on average. (4) The incremental method beats the batch one when updates to D and G are up to 20%, and is 6.28X faster when updates $|\Delta G| = 5\%|G|$.

We discuss related work in Section 7 and future work in Section 8.

2 PRELIMINARIES

In this section, we review basic notations, ER and HER.

Relations. Consider a relation schema $R = (\mathsf{id}, A_1, \ldots, A_n)$, where A_i is an attribute ($i \in [1, n]$), and id is an entity id as introduced by Codd [39], such that each tuple of R represents an entity of type τ with identity id. A relation D of R is a set of tuples of schema R.

Knowledge graphs. Following [68], we represent a knowledge graph as $G = (V, E, L)$. Here (a) V is a finite set of vertices representing entities, (b) $E \subseteq V \times V$ consists of edges representing relationships between entities; and (c) for each vertex $v \in V$, $L(v)$ is its feature or value, and for each edge $e \in E$, $L(e)$ is its label. Between a pair (v, v') in V, there are possibly multiple edges carrying distinct labels.

A *path* ρ *from a vertex* v_0 in graph G is $\rho = (v_0, v_1, \ldots, v_l)$ such that (v_{i-1}, v_i) is an edge in E for $i \in [1, l]$. The *length of* ρ is the number l of edges on ρ. A path is *simple* if each vertex appears on ρ at most once. We consider simple paths, simply referred to as paths.

Entity resolution. Given a relation D, entity resolution (ER) is to identify all pairs of tuples in D that refer to the same real-life entity. It returns a set of pairs (t_1, t_2) of tuples of D that are identified as *matches*. If t_1 does not match t_2, (t_1, t_2) is referred to as a *mismatch*.

A number of ER methods have been developed, based on ML [23, 45, 74, 86, 95, 102, 134], logic rules [24, 27, 32, 48, 61, 79, 127] and hybrid of the two [28, 41, 56]. We focus on ML-based ER models, which take quadratic-time (after the relevant ML models are trained).

Fast and Scalable Data Transfer Across Data Systems

HARALAMPOS GAVRIILIDIS, BIFOLD and Technische Universität Berlin, Germany
KAUSTUBH BEEDKAR, Indian Institute of Technology Delhi, India
MATTHIAS BOEHM, BIFOLD and Technische Universität Berlin, Germany
VOLKER MARKL, BIFOLD, Technische Universität Berlin, and DFKI GmbH, Germany

Fast and scalable data transfer is crucial in today's decentralized data ecosystems and data-driven applications. Example use cases include transferring data from operational systems to consolidated data warehouse environments, or from relational database systems to data lakes for exploratory data analysis or ML model training. Traditional data transfer approaches rely on efficient point-to-point connectors or general middleware with generic intermediate data representations. Physical environments (e.g., on-premise, cloud, or consumer nodes) also have become increasingly heterogeneous. Existing work still struggles to achieve both, fast and scalable data transfer as well as generality in terms of heterogeneous systems and environments. Hence, in this paper, we introduce a holistic data transfer framework. Our XDBC framework splits the data transfer pipeline into logical components and provides a wide variety of physical implementations for these components. This design allows a seamless integration of different systems as well as the automatic optimizations of data transfer configurations according to workload and environment characteristics. Our evaluation shows that XDBC outperforms state-of-the-art generic data transfer tools by up to 5×, while being on par with specialized approaches.

CCS Concepts: • **Information systems** → *Data federation tools*; **Extraction, transformation and loading**; **Data exchange**.

Additional Key Words and Phrases: Data Transfer, Data Loading Optimization, Data System Interoperability

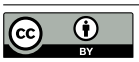
ACM Reference Format:
Haralampos Gavriilidis, Kaustubh Beedkar, Matthias Boehm, and Volker Markl. 2025. Fast and Scalable Data Transfer Across Data Systems. *Proc. ACM Manag. Data* 3, 3 (SIGMOD), Article 157 (June 2025), 28 pages. https://doi.org/10.1145/3725294

1 Introduction

The typical IT landscape of today's organizations comprises various decentralized data ecosystems and many data-driven applications [29, 34]. Over the last decades, there was a trend towards specialized data systems to improve performance and functionality [84, 85]. Apart from processing systems, modern data lakes allow storing raw data in various formats. Since data is distributed across these heterogeneous data representations and systems, efficient data transfer mechanisms are crucial for seamless integration and data analysis.

A Need for Data Transfer: There is a strong need for efficient data transfer because data movement and conversion is often resource-intensive and time-consuming. Accordingly, there is work on eliminating unnecessary data transfer. For example, in-database machine learning aims to conduct training and especially inference inside the DBMS [22, 49, 77] to avoid costly exports from the DBMS.

Authors' Contact Information: Haralampos Gavriilidis, BIFOLD and Technische Universität Berlin, Germany, gavriilidis@tu-berlin.de; Kaustubh Beedkar, Indian Institute of Technology Delhi, India, kbeedkar@cse.iitd.ac.in; Matthias Boehm, BIFOLD and Technische Universität Berlin, Germany, matthias.boehm@tu-berlin.de; Volker Markl, BIFOLD, Technische Universität Berlin, and DFKI GmbH, Germany, volker.markl@tu-berlin.de.



This work is licensed under a Creative Commons Attribution 4.0 International License.
© 2025 Copyright held by the owner/author(s).
ACM 2836-6573/2025/6-ART157
https://doi.org/10.1145/3725294

Similarly, IoT systems push computation down to the edge to reduce unreliable network transfers [31, 92]. However, there are many use cases where data must be moved, either to leverage specialized tools, to integrate systems, or to consolidate data, e.g., data science applications [90], federated systems [26, 41, 44, 76, 78], extraction-transformation loading (ETL) flows [80], or data replication [56]. Overall, many modern data-driven applications would benefit from efficient data transfer mechanisms.

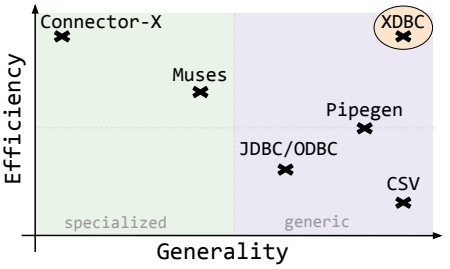


Fig. 1. Data Transfer Approaches.

Data Transfer Challenges: Designing an efficient data transfer mechanism is challenging due to the heterogeneity of data systems, particularly in terms of interfaces, data formats, and varying environment characteristics. Data transfer across heterogeneous systems is commonly realized through custom (i.e., specialized) point-to-point connectors, or generic solutions such as integration middleware or generic JDBC/ODBC drivers [6]. Figure 1 shows this trade-off space of efficiency versus generality. First, specialized connectors provide good performance but lack generality because they are implemented for specific pairs of systems. Second, generic adapters like JDBC/ODBC are primarily designed for lightweight DBMS access and suffer from poor performance [74]. Additionally, data transfer involves multiple steps (e.g., reading, deserializing, compressing, and transmitting), which must be tuned for data, system, and network characteristics. For example, compression may improve the transfer in unreliable consumer-cloud networks, but yield slowdowns in a cloud environment. The complexity of heterogeneous systems and environments renders the optimization of data transfer onerous. Hence, there is a need for a data transfer mechanism that yields robust performance, is easy to use, and general enough for data transfer across heterogeneous systems.

Limitations of Existing Work: Despite work on optimizing ETL [80, 81] and integration flows [28], the literature on efficient data transfer is relatively sparse. Raasveldt et al. analyzed DBMS client protocols and found that JDBC-based protocols are unsuitable for transferring large amounts of data due to their row-based nature and excessive metadata in each protocol buffer [74]. Pipegen generates pipes from existing CSV I/O unit tests in DBMSes, and redirects text I/O to compressed binary communication [48]. Zigzag joins optimize data transfers during joins of data warehouse tables and files in distributed file-system via Bloom filters [86]. Connector-X improves the loading of DBMS tables into pandas dataframes through parallelism and efficient data handling [90]. While these approaches optimize specific aspects of data transfer, there is a lack of a holistic data transfer framework that adapts to different physical environments and supports heterogeneous systems.

Contributions: To address the data transfer challenges and limitations of existing work, we introduce XDBC as a holistic data transfer framework. XDBC offers reader and writer interfaces that can be implemented for arbitrary data systems, eliminating the need for multiple connectors per system pair. XDBC's architecture comprises decoupled components (for read, deserialize, compress, send, receive, decompress, serialize, and write) as well as different physical implementations, which facilitate a flexible configuration and tuning for different environments and data characteristics. In order to free users from the need of manual tuning, XDBC's heuristic optimizer automatically selects effective configurations for given environments. Our detailed technical contributions are:

- **XDBC Architecture:** We describe the architecture and runtime of XDBC in Section 3. The simple yet very effective design allows seamlessly integrating existing data systems as well as tuning the individual data transfer components.

FlexER: Flexible Entity Resolution for Multiple Intents

BAR GENOSSAR, Technion – Israel Institute of Technology, Israel
ROEE SHRAGA, Northeastern University, MA, USA
AVIGDOR GAL, Technion – Israel Institute of Technology, Israel

Entity resolution, a longstanding problem of data cleaning and integration, aims at identifying data records that represent the same real-world entity. Existing approaches treat entity resolution as a universal task, assuming the existence of a **single** interpretation of a real-world entity and focusing only on finding matched records, separating corresponding from non-corresponding ones, with respect to this single interpretation. However, in real-world scenarios, where entity resolution is part of a more general data project, downstream applications may have varying interpretations of real-world entities relating, for example, to various user needs. In what follows, we introduce the problem of *multiple intents entity resolution* (MIER), an extension to the universal (single intent) entity resolution task. As a solution, we propose FlexER, utilizing contemporary solutions to universal entity resolution tasks to solve multiple intents entity resolution. FlexER addresses the problem as a multi-label classification problem. It combines intent-based representations of tuple pairs using a multiplex graph representation that serves as an input to a graph neural network (GNN). FlexER learns intent representations and improves the outcome to multiple resolution problems. A large-scale empirical evaluation introduces a new benchmark and, using also two well-known benchmarks, shows that FlexER effectively solves the MIER problem and outperforms the state-of-the-art for a universal entity resolution.

CCS Concepts: • **Information systems → Entity resolution**.

Additional Key Words and Phrases: entity resolution, entity matching, graph neural networks, supervised learning

ACM Reference Format:
Bar Genossar, Roee Shraga, and Avigdor Gal. 2023. FlexER: Flexible Entity Resolution for Multiple Intents. *Proc. ACM Manag. Data* 1, 1, Article 42 (May 2023), 27 pages. https://doi.org/10.1145/3588722

1 INTRODUCTION

An essential component of any data science lifecycle involves data preparation. Accordingly, contemporary data projects intensively invest in large-scale data cleaning and integration techniques to combine data from multiple heterogeneous sources into meaningful and valuable information. Consider, for example, a data-intensive organization (*e.g.*, an online shopping company) that continuously collects data on user queries to better understand their needs. The (mostly implicit) intents of users may vary. While some users are interested in basketball shoes, regardless of their brand, others may be more interested in buying branded products, caring less whether shoes are designated as running shoes.

At the heart of the data preparation realm lies the *entity resolution* task (with variations of entity matching, record linkage, deduplication, and more), which has been extensively studied over the

Authors' addresses: Bar Genossar, Technion – Israel Institute of Technology, Haifa, Israel, sbargen@campus.technion.ac.il; Roee Shraga, Northeastern University, Boston, MA, USA, r.shraga@northeastern.edu; Avigdor Gal, Technion – Israel Institute of Technology, Haifa, Israel, avigal@technion.ac.il.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
2836-6573/2023/5-ART42 $15.00
https://doi.org/10.1145/3588722

Table 1. Amazon Product Dataset Sample (D_{ex})

r_{id}	**Product title**
r_1	Nike Men's Lunar Force 1 Duckboot
r_2	NIKE Men Lunar Force 1 Duckboot, Black/Dark Loden-BROGHT Crimson
r_3	NIKE Men's Air Max Stutter Step Ankle-High Basketball Shoe
r_4	Nike Men's Air Max 2016 Running Shoe
r_5	adidas Performance Men's D Rose 6 Boost Primeknit Basketball
r_6	The Man Who Tried to Get Away

past decades (see books and surveys [8, 13, 17]). Entity resolution is a data integration task that aims at identifying same real-world entities that are represented by different data records (instances). Specifically, entity resolution can be used to "clean" a dataset from duplicate tuples referring to the same entity, offering a useful post processing tool for integrating multiple data sources.

Entity resolution techniques for resolving duplicates in a dataset were developed over years of research. Traditional methods focused on string similarity [22, 23, 32, 37] and rule-based methods [54, 55]. Learning-based approaches were also suggested [3, 31], followed by deep learning methods in recent years [15, 16, 25, 28, 34, 41, 64]. Following a common practice for text processing, the use of *pre-trained language models*, and specifically, BERT-based models [10], was also introduced for entity resolution [5, 33, 35, 46].

The aforementioned methods share a common assumption regarding the universality of the entity resolution problem, assuming the existence of a **single** interpretation to the notion of a real-world entity. In particular, most solutions aim to create a single clean view of a dataset, by separating corresponding from non-corresponding record pairs, with respect to this single interpretation.

The universal property of entity resolution solutions is being challenged by the development of applications that require the ability to provide a fine-grained (or personalized) analysis, offering services that are tailored to specific user needs and requirements [4, 62]. For example, online shops aim at personalizing shopping experience to the needs of individual users, having to infer such needs from mainly implicit feedback [29], such as clicks and mouse movements. In such a setting, having a universal resolution facility is likely to cripple the resolution process, failing to cater to the needs of all applications at all times. To illustrate the need for a non-universal (flexible) resolution, we use the *AmazonMI* dataset, a newly suggested benchmark based on amazon products data (see Section 5.1), in the following motivating example.

1.1 Motivating Example: From Record Duplication to Entity Resolution Interpretation

Table 1 depicts an excerpt of the AmazonMI dataset to illustrate the differences between the universal approach to entity resolution and the flexible entity resolution we present and address in this work. Figure 1 offers visual illustration to the multiplicity of entity interpretations in the example.

Record duplication is usually the result of discordant representations (*e.g.,* multi-lingual, synonyms, capitalizations), changes in the data over time, typos, *etc.* For example, records r_1 and r_2 in Table 1 refer to the same pair of Nike basketball shoes called "Men's Lunar Force 1 Duckboot." The difference between r_1 and r_2 originates from capitalization issues (Nike vs. NIKE) and additional specification (*e.g.,* color). Contrarily, r_1 and r_6 will most likely be conceived as different, as the latter refers to a book.

GitTables: A Large-Scale Corpus of Relational Tables

MADELON HULSEBOS*, University of Amsterdam, Netherlands
ÇAĞATAY DEMIRALP, Sigma Computing, United States
PAUL GROTH, University of Amsterdam, Netherlands

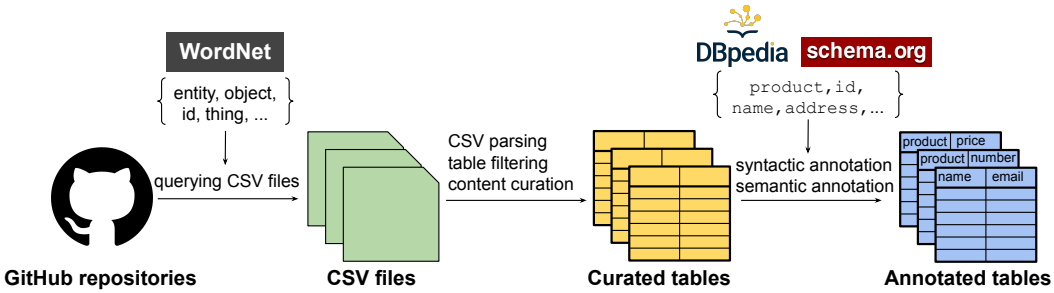


Fig. 1. The pipeline for creating GitTables consists of 1) querying CSV files from GitHub repositories based on WordNet, 2) parsing CSV files and curating tables, and 3) annotating tables with column semantics from DBpedia and Schema.org.

The success of deep learning has sparked interest in improving relational table tasks, like data preparation and search, with table representation models trained on large table corpora. Existing table corpora primarily contain tables extracted from HTML pages, limiting the capability to represent offline database tables. To train and evaluate high-capacity models for applications beyond the Web, we need resources with tables that resemble relational database tables. Here we introduce GitTables, a corpus of 1M relational tables extracted from GitHub. Our continuing curation aims at growing the corpus to at least 10M tables. Analyses of GitTables show that its structure, content, and topical coverage differ significantly from existing table corpora. We annotate table columns in GitTables with semantic types, hierarchical relations and descriptions from Schema.org and DBpedia. The evaluation of our annotation pipeline on the T2Dv2 benchmark illustrates that our approach provides results on par with human annotations. We present three applications of GitTables, demonstrating its value for learned semantic type detection models, schema completion methods, and benchmarks for table-to-KG matching, data search, and preparation. We make the corpus and code available at https://gittables.github.io.

CCS Concepts: • **Information systems** → *Relational database model*; *Information integration*; • **Computing methodologies** → *Information extraction*; *Neural networks*.

Additional Key Words and Phrases: relational tables, data management, deep learning, datasets

ACM Reference Format:
Madelon Hulsebos, Çağatay Demiralp, and Paul Groth. 2023. GitTables: A Large-Scale Corpus of Relational Tables. *Proc. ACM Manag. Data* 1, 1, Article 30 (May 2023), 17 pages. https://doi.org/10.1145/3588710

*Corresponding author (m.hulsebos@uva.nl)

Authors' addresses: Madelon Hulsebos, University of Amsterdam, Amsterdam, 1012 WX, Netherlands, m.hulsebos@uva.nl; Çağatay Demiralp, Sigma Computing, San Francisco, CA 94105, United States, cagatay@sigmacomputing.com; Paul Groth, University of Amsterdam, Amsterdam, 1012 WX, Netherlands, p.t.groth@uva.nl.

This work is licensed under a Creative Commons Attribution International 4.0 License.

© 2023 Copyright held by the owner/author(s).
2836-6573/2023/5-ART30
https://doi.org/10.1145/3588710

1 INTRODUCTION

Deep learning (DL) models, in the past decade, have dramatically improved many long-standing computer vision and natural language processing tasks [25]. The practical success of DL has also spurred interest in its applications to tasks in other domains, including data management, ranging from data cleaning to annotation [45]. To train DL models for relational data, earlier work primarily relied on corpora consisting of tables scraped from HTML pages [10] such as WDC WebTables [46], the largest table corpus to date. These models have played an important role in facilitating data-driven data management research, with Web applications in particular [7, 49].

However, tables extracted from HTML pages on the Web (Web tables) provide a skewed representation of tables in the wild especially those residing in (enterprise) databases [24, 27]. The common attribute "ID", for example, does not appear among the twenty most frequent column names of the WDC WebTables corpus, and the dimensions of Web tables are significantly smaller than typical database tables [46]. It is therefore unsurprising that models based on Web tables have limited applications beyond the Web [7]. To broaden the impact of data-driven data management research, we need new table collections complementing existing corpora with tables resembling typical database tables.

To address this demand, we introduce GitTables: a corpus with 1M relational tables extracted from CSV files in GitHub repositories. We will keep the corpus growing to have at least 10M tables to facilitate the extension of deep transfer learning to the relational domain, analogous to how large corpora of natural language have stimulated pretrained language models like BERT [16] and GPT [6]. We expect GitTables to stimulate similar progress in data management tasks like data search and preparation.

Besides having representative tables, many data management applications benefit from understanding the semantics of table columns. An intelligent data exploration system, for example, could recommend a map chart to visualize two data columns representing countries and sales revenues. Upon encountering missing values, the system could also automatically fill the gaps using a knowledge base. The system could, for instance, impute "France" in an empty cell in a column with country values, if the neighboring cell in a column with capital cities is "Paris".

To facilitate such functionalities, we enhance table columns with semantic types from DBpedia and Schema.org (e.g. `address`) using a syntactic as well as semantic annotation method (an example in Figure 2). Each semantic type is also associated with the expected atomic data type (e.g. `string` and `number`), a description, and hierarchical type relations. These semantically rich annotations together with DL present a unique opportunity to learn table semantics as demonstrated with TURL [15] and TaBERT [48].

Our analysis of GitTables confirms the different nature of the tables: the tables have significantly larger dimensions (rows and columns). A machine learning model for detecting data shift between GitTables and Web tables accurately distinguishes table columns from each source corpus. This reflects the structurally different content of these corpora. The semantic type distribution also deviates significantly from the semantic distribution of Web tables, illustrating its distinctive topical coverage. We make GitTables available through https://gittables.github.io. We contribute:

(1) GitTables, a new large-scale corpus of 1M tables. To the best of our knowledge, GitTables is the first large-scale relational table corpus with topical coverage and content structurally different than tables extracted from HTML pages.
(2) A scalable column-annotation method using distant-supervision. We annotate the columns in GitTables with semantics consisting of semantic types, atomic data types, hierarchical relations, and descriptions, making GitTables the largest annotated table corpus to date.

Ground Truth Inference for Weakly Supervised Entity Matching

RENZHI WU, Georgia Institute of Technology, USA
ALEXANDER BENDECK, Georgia Institute of Technology, USA
XU CHU, Georgia Institute of Technology, USA
YEYE HE, Microsoft Research, USA

Entity matching (EM) refers to the problem of identifying pairs of data records in one or more relational tables that refer to the same entity in the real world. Supervised machine learning (ML) models currently achieve state-of-the-art matching performance; however, they require a large number of labeled examples, which are often expensive or infeasible to obtain. This has inspired us to approach data labeling for EM using *weak supervision*. In particular, we use the labeling function abstraction popularized by Snorkel, where each labeling function (LF) is a user-provided program that can generate many noisy match/non-match labels quickly and cheaply. Given a set of user-written LFs, the quality of data labeling depends on a *labeling model* to accurately infer the ground-truth labels. In this work, we first propose a simple but powerful labeling model for general weak supervision tasks. Then, we tailor the labeling model specifically to the task of entity matching by considering the EM-specific transitivity property.

The general form of our labeling model is simple while substantially outperforming the best existing method across ten general weak supervision datasets. To tailor the labeling model for EM, we formulate an approach to ensure that the final predictions of the labeling model satisfy the transitivity property required in EM, utilizing an exact solution where possible and an ML-based approximation in remaining cases. On two single-table and nine two-table real-world EM datasets, we show that our labeling model results in a 9% higher F1 score on average than the best existing method. We also show that a deep learning EM end model (DeepMatcher) trained on labels generated from our weak supervision approach is comparable to an end model trained using tens of thousands of ground-truth labels, demonstrating that our approach can significantly reduce the labeling efforts required in EM.

CCS Concepts: • **Information systems → Entity resolution**.

Additional Key Words and Phrases: entity matching, weak supervision, labeling model, transitivity

ACM Reference Format:
Renzhi Wu, Alexander Bendeck, Xu Chu, and Yeye He. 2023. Ground Truth Inference for Weakly Supervised Entity Matching. *Proc. ACM Manag. Data* 1, 1, Article 32 (May 2023), 28 pages. https://doi.org/10.1145/3588712

1 INTRODUCTION

Entity matching (EM) refers to the process of determining if a pair of records from two data sources refer to the same real-world entity. EM has many applications, for example, in matching product listings for competitive pricing [2] and in building knowledge graphs [28]. As a long-standing problem, EM has been extensively studied (e.g., see surveys [18, 29, 30, 34, 36, 76]).

Authors' addresses: Renzhi Wu, Georgia Institute of Technology, Atlanta, USA, renzhiwu@gatech.edu; Alexander Bendeck, Georgia Institute of Technology, Atlanta, USA, abendeck3@gatech.edu; Xu Chu, Georgia Institute of Technology, Atlanta, USA, xu.chu@cc.gatech.edu; Yeye He, Microsoft Research, Redmond, USA, yeyehe@microsoft.com.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
2836-6573/2023/5-ART32 $15.00
https://doi.org/10.1145/3588712

The Need for Weak Supervision in EM. The most important requirement for EM solutions is high matching quality as it directly affects downstream application performance. State-of-the-art EM solutions that report the highest matching quality [29, 50, 59, 60] are supervised machine learning (ML) approaches that train a binary classifier to predict the label (match or non-match) for any tuple pair. However, these models require large numbers of labeled tuple pairs, which are often not available or costly to obtain [29]. The high human cost in data labeling has become a main bottleneck in adopting high-quality EM solutions in practice.

To meet the needs of data-hungry supervised ML models, ML practitioners have increasingly turned to *weak supervision* methods, in which a larger volume of cheaply generated, but often noisier, labeled examples is used in lieu of hand-labeled examples. Different forms of weak supervision have been investigated, including the use of non-expert crowd workers [33], pretrained models [24], and rules/patterns/heuristics [75].

To unify different forms of supervision, the *data programming* paradigm [69] has been proposed, in which users write *labeling functions* (LFs) to programmatically label training data, rather than manually labeling each example by hand. Each LF is a small user-provided program (e.g., in Python) that leverages noisy signals in the data or domain knowledge to provide a label (or abstain) for an input example. In binary classification tasks such as EM, the output of each LF is +1 (positive class), -1 (negative class), or 0 (abstain). The data programming approach was first implemented in Snorkel [66], a weakly supervised data labeling system. Snorkel allows users to write LFs and then uses a generative model to combine all LFs to produce probabilistic labels. The data programming approach has been widely adopted in various ML tasks [31, 32, 53, 54, 73, 81, 85].

Example Labeling Functions. The LFs shown in Figure 1 are two user-written LFs developed using an existing tool [84] for matching electronic products [1]. The first LF, "name_overlap", encodes the intuition that matching products should have similar "name" attributes. Specifically, pairs of products with sufficiently high/low word overlap in their names are predicted as matches (+1)/non-matches (-1). When the amount of word overlap does not provide conclusive evidence either way, the LF abstains (0). The second LF, "size_unmatch", uses a regular expression to search the screen size in patterns like "... Samsung **40'** LCD ...". When the screen sizes of a pair of products are different, the LF predicts the pair to be non-match (-1); otherwise, the LF abstains (0).

```python
@labeling_function
def name_overlap(tpl_l, tpl_r):
    x = lower(tpl_l.name)
    y = lower(tpl_r.name)
    x = set(splitBySpace(x))
    y = set(splitBySpace(y))
    itsect = x.intersection(y)
    max_len = max(len(x), len(y))
    score = len(itsect) / max_len
    if score < 0.1:
        return -1
    elif score > 0.8:
        return 1
    else:
        return 0
```

```python
@labeling_function
def size_unmatch(tpl_l, tpl_r):
    l_name_des = str(tpl_l.name) + " " \
                 + str(tpl_l.description)
    r_name_des = str(tpl_r.name) + " " \
                 + str(tpl_r.description)
    x = re.findall(r"[\d\.]+\'", l_name_des)
    y = re.findall(r"[\d\.]+\'", r_name_des)
    if len(x) == 0 or len(y) == 0:
        return 0
    x, y = x[0], y[0]
    if x != y:
        return -1
    else:
        return 0
```

Fig. 1. Two user-written LFs for the abt-buy dataset [1].

The predictions of multiple LFs can then be used to construct a *labeling matrix* X, where each row corresponds to a tuple pair, and each column corresponds to predictions of one LF for all tuple pairs. After constructing the labeling matrix, a *labeling model* is used to consolidate the labeling function predictions into a final label for each tuple pair. Figure 2 shows one instance of a labeling matrix with votes from LFs (LF1, LF2, ...) and inferred labels from a naive labeling model (+1 for match, -1 for non-match) as well as the unknown ground truth (GT) labels for each tuple pair.

Given a set of user-written LFs, the quality of data labeling depends on the *labeling model* that infers the ground-truth labels from the LFs. In this paper, we first present an embarrassingly simple

HAIPipe: Combining Human-generated and Machine-generated Pipelines for Data Preparation

SIBEI CHEN, Renmin University of China, China
NAN TANG, QCRI / HKUST (GZ), Qatar / China
JU FAN*, Renmin University of China, China
XUEMI YAN, Renmin University of China, China
CHENGLIANG CHAI, Beijing Institute of Technology, China
GUOLIANG LI, Tsinghua University, China
XIAOYONG DU, Renmin University of China, China

Data preparation is crucial in achieving optimized results for machine learning (ML). However, having a good data preparation pipeline is highly non-trivial for ML practitioners, which is not only domain-specific, but also dataset-specific. There are two common practices. Human-generated pipelines (**HI-pipelines**) typically use a wide range of *any* operations or libraries but are highly experience- and heuristic-based. In contrast, machine-generated pipelines (**AI-pipelines**), *a.k.a.* AutoML, often adopt a predefined set of sophisticated operations and are search-based and optimized. These two common practices are mutually complementary. In this paper, we study a new problem that, given an HI-pipeline and an AI-pipeline for the same ML task, can we combine them to get a new pipeline (**HAI-pipeline**) that is better than the provided HI-pipeline and AI-pipeline? We propose HAIPipe, a framework to address the problem, which adopts an enumeration-sampling strategy to carefully select the best performing combined pipeline. We also introduce a reinforcement learning (RL) based approach to search an optimized AI-pipeline. Extensive experiments using 1400+ real-world HI-pipelines (Jupyter notebooks from Kaggle) verify that HAIPipe can significantly outperform the approaches using either HI-pipelines or AI-pipelines alone.

CCS Concepts: • **Information systems → Data analytics**.

Additional Key Words and Phrases: pipeline generation, data preparation, reinforcement learning

ACM Reference Format:
Sibei Chen, Nan Tang, Ju Fan, Xuemi Yan, Chengliang Chai, Guoliang Li, and Xiaoyong Du. 2023. HAIPipe: Combining Human-generated and Machine-generated Pipelines for Data Preparation. *Proc. ACM Manag. Data* 1, 1, Article 91 (May 2023), 26 pages. https://doi.org/10.1145/3588945

1 INTRODUCTION

Data preparation (data prep for short) is the process of turning raw data into a format that is ready for use by data science or ML tasks. This process is often implemented by a series of steps (*a.k.a.* a

*Ju Fan is the corresponding author.

Authors' addresses: Sibei Chen, Renmin University of China, Beijing, China, sibei@ruc.edu.cn; Nan Tang, QCRI / HKUST (GZ), Doha / Guangzhou, Qatar / China, ntang@hbku.edu.qa; Ju Fan, Renmin University of China, Beijing, China, fanj@ruc.edu.cn; Xuemi Yan, Renmin University of China, Beijing, China, yxmlpzt@ruc.edu.cn; Chengliang Chai, Beijing Institute of Technology, Beijing, China, ccl@bit.edu.cn; Guoliang Li, Tsinghua University, Beijing, China, liguoliang@tsinghua.edu.cn; Xiaoyong Du, Renmin University of China, Beijing, China, duyong@ruc.edu.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
2836-6573/2023/5-ART91 $15.00
https://doi.org/10.1145/3588945

pipeline) that the data moves through from one step to its subsequent steps. However, data prep is also known to be very hard and time-consuming [26].

Traditional data prep pipelines are human orchestrated, which are abbreviated as HI-pipelines. The pros of HI-pipelines are that users can use "any" operation and can easily inject domain knowledge (*e.g.,* "?" means a missing value for occupation). The cons are that users typically orchestrate pipelines based on their experience, which are heuristic and are rarely optimized, *e.g.,* humans never manually try all different combinations and find the best one.

Recent advances in the ML community have extended AutoML from hyper-parameter tuning and neural architecture search to an attempt that tries to automate the process of data prep for ML (*e.g.,* Auto-SKLearn [9] and DeepLine [12]) – these machine-generated pipelines are called AI-pipelines in this paper. The pros of AI-pipelines are that they use a constrained set of operations, for which they can find the best pipeline through exhaustively evaluating different pipelines within a pre-defined search space. The cons of AI-pipelines are that they lack domain knowledge.

Our **key observation** is that HI-pipelines and AI-pipelines are naturally complementary. That is, the pros of HI-pipelines (*i.e.,* using a wider range of operations based on domain knowledge) are the cons of AI-pipelines, and the cons of HI-pipelines (*i.e.,* experience-based and heuristic-based) are the pros of AI-pipelines (*i.e.,* highly optimized). Therefore, intuitively, HI-pipelines and AI-pipelines should be combined, so as to get the best of both worlds.

In this paper, we study a new problem: Given an HI-pipeline and an AI-pipeline for an ML data prep task, can we generate a better data prep pipeline (*i.e.,* an HAI-pipeline) by combining them?

Example 1. *Consider Figure 1. After loading the dataset (lines 1–4), an HI-pipeline consists of two components h_1 (lines 5-8,* i.e., *dropping rows with missing values) and h_2 (lines 24-28,* i.e., *scaling numerical attributes). Then, we use a machine-generated AI-pipeline that consists of three components a_1 (lines 10–15,* i.e., *encoding categorical attributes), a_2 (lines 17–22,* i.e., *generating interaction features) and a_3 (lines 30–35,* i.e., *removing useless features). A combination of these two pipelines is given in Figure 1, which outperforms both the HI-pipeline and the AI-pipeline.*

To address the problem, we propose HAIPipe, a framework that combines an HI-pipeline with an AI-pipeline for a specific ML task.

There are **two challenges**. (1) How to combine an HI-pipeline and an AI-pipeline? (2) How to obtain an optimized AI-pipeline from various possible combinations?

For Challenge (1), we devise an *enumeration-sampling* approach that first enumerates possible HAI-pipelines, and then employs an active learning strategy to sample a limited number of HAI-pipelines for evaluation, so as to find the one with the highest performance. For Challenge (2), there are a variety of operation families (*e.g.,* `Imputer` and `Scaler`). Within each family, there are many operations (*e.g.,* `MinMaxScaler` and `StandardScaler`). Thus, there exist many possible ways of selecting operations to form an AI-pipeline, which are highly dataset-specific and task-specific. We introduce a reinforcement learning (RL) based approach that learns how to select operations through offline training.

Contributions. Our contributions are summarized as follows.

(1) We formally define data prep pipelines, and introduce HI-pipelines and AI-pipelines (Section 2).

(2) We propose a framework, namely HAIPipe, that combines HI-pipelines and AI-pipelines for data prep (Section 3). We devise an enumeration-sampling strategy that effectively and efficiently combines HI-pipelines and AI-pipelines (Section 4).

(3) We present an RL-based AI-pipeline algorithm that pays particular attention on optimizing data prep operations (Section 5).

(4) We conduct extensive experiments to show that HAIPipe can significantly outperform both the

Integrating Data Lake Tables

Aamod Khatiwada
Northeastern University
Boston, Massachusetts, USA
khatiwada.a@northeastern.edu

Roee Shraga
Northeastern University
Boston, Massachusetts, USA
r.shraga@northeastern.edu

Wolfgang Gatterbauer
Northeastern University
Boston, Massachusetts, USA
w.gatterbauer@northeastern.edu

Renée J. Miller
Northeastern University
Boston, Massachusetts, USA
miller@northeastern.edu

ABSTRACT

We have made tremendous strides in providing tools for data scientists to discover new tables useful for their analyses. But despite these advances, the proper integration of discovered tables has been under-explored. An interesting semantics for integration, called Full Disjunction, was proposed in the 1980's, but there has been little progress in using it for data science to integrate tables culled from data lakes. We provide ALITE, the first proposal for scalable integration of tables that may have been discovered using join, union or related table search. We empirically show that ALITE can outperform previous algorithms for computing the Full Disjunction. ALITE relaxes previous assumptions that tables share common attribute names (which completely determine the join columns), are complete (without null values), and have acyclic join patterns. To evaluate ALITE, we develop and share three new benchmarks for integration that use real data lake tables.

PVLDB Reference Format:
Aamod Khatiwada, Roee Shraga, Wolfgang Gatterbauer, and Renée J. Miller. Integrating Data Lake Tables. PVLDB, 16(4): 932 - 945, 2022.
doi:10.14778/3574245.3574274

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/northeastern-datalab/alite.

1 INTRODUCTION

The number of public datasets has grown immensely in open data platforms [56, 57, 78]. Also, individual corporations have a wealth of data stored in their own data lakes. Analyzing and integrating such datasets can help governments and enterprises in making decisions and plans. Data scientists, as the main users of data, use different techniques to discover datasets such as keyword search [11, 12, 58, 75] and table search (using the data within their table as a query) [10, 28, 48, 50, 57, 80]. Such a process usually outputs a collection of data lake tables that may enrich their analysis [56]. Existing techniques usually discover unionable [12, 41, 48, 57], joinable [26, 55, 75, 77, 78], and related tables [10, 19, 77].

Example 1. *Consider the data lake tables about football stadiums shown in Fig. 1. We have added a* `TID` *(Tuple ID) column in each table to permit us to refer to tuples. Assume that a data scientist uses table* T_1 *as a* query table *to search for the top-2 unionable tables [57] and the top-2 joinable tables [78] from a data lake. Let* T_2 *and* T_3 *be the union search results and* T_4 *and* T_5 *be the join search results. Join search finds tables that join on an indicated column (in this case* `Location`*), but does not discover if there are other common (integratable) columns. For simplicity, assume that the common columns on these tables are already detected and have identical column headers. Note that in practice this will not be the case.*

After discovery, data scientists would often integrate the discovered tables before analyzing and applying statistical tools. Such integration not only extends their data but also allows them to answer queries that go beyond a single table. Consider Tables T_2, T_3 and T_4 in Fig. 1 and assume a football team has data scientists assisting in finding a new coach. Specifically, the team looks for an experienced coach who has handled teams playing in front of large crowds in new stadiums. So, they may use queries such as *"coaches who coach teams having stadiums established after 2000, that accommodate at least 50 thousand spectators"*. The information required here goes beyond a single table. In our example, one needs (at least) to integrate T_2, T_3 and T_4 to obtain such facts, for example, `Dan Campbell` who coaches the `Detroit Lions` that uses `Ford Field` Stadium established in `2002` and having a capacity of `65k` (f_7 in Fig. 2). Prior search methods do not address this "post-discovery" phase and do not answer the important question of how to integrate tables (relations) obtained by table search technique(s).

Example 2. *The standard relational union operator needs all tables to have exactly the same schema. However, this is not the case even for union search results (where tables that union on a subset of attributes can be retrieved) [57]. So to integrate the tables in Fig. 1, one can project out non-common columns and union on only the common columns. For* T_1, T_2, *and* T_3, *this would just leave* `Location`*. For the joinable tables, a join on only* `Location` *of* T_1 *with* T_4 *leads to tuples like* t_{11} *being omitted and the result has two* `Stadium` *attributes. Worse, the natural join operator, i.e.* $T_1 \bowtie T_4 \bowtie T_5$*, returns an empty set because* T_4 *and* T_5 *do not have joining tuples. The problem gets more complicated if we try to integrate all five tables using these operators.*

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 16, No. 4 ISSN 2150-8097.
doi:10.14778/3574245.3574274

T_1

TID	Stadium	Location	Team
t_1	NRG Stadium	Texas	Houston Texans
t_2	AT&T Stadium	Texas	Dallas Cowboys
t_3	Paul Brown	Ohio	±
t_4	Sofi Stadium	California	Angeles Chargers

T_2

TID	Stadium	Location	Opened
t_5	Soldier Field	Chicago	1924
t_6	Ford Field	Michigan	2002

T_3

TID	Team	Location	Coach
t_7	Houston Texans	Texas	Lovie Smith
t_8	Green Bay Packers	Wisconsin	Matt LaFleur
t_9	Detroit Lions	Michigan	Dan Campbell

T_4

TID	Stadium	Location	Capacity
t_{10}	NRG Stadium	Texas	±
t_{11}	Ford Field	Michigan	65k

T_5

TID	Stadium	Location	Team
t_{12}	Lambeau Field	Wisconsin	Green Bay Packers
t_{13}	±	Ohio	Cleveland
t_{14}	Sofi Stadium	California	±

Figure 1: Tables about football stadiums, their locations and home teams. The objective is to integrate the five tables. TID is not a real data column and metadata like column headers may not be available in real data lake tables, but are used for illustration purposes. The symbol ± represents null values present in the input tables ("*missing nulls*").

Within the data integration literature, *Full Disjunction (FD)* [32] has been understood as a natural way of assembling partial pieces of information (facts) such that it maximizes the connections among these facts [65]. Indeed, Rajamaran and Ullman describe FD as a relation with nulls (denoted by ⊥) such that every set of join-consistent tuples appears within an FD tuple, with a concrete value or ⊥ in each attribute not found within the set of tuples [65]. Here, join-consistent is defined as common attributes (attributes with the same name), so this is effectively a *natural FD*. The widely known outer-join [46, 66] is not associative (hence, the result may depend on the order in which tables are integrated) and does not aim to maximize the connections among the integrated tuples [32, 52].

Example 3. *Outer-join and outer-union keep all tuples and columns and pad non-matching tuples (respectively, columns) with nulls [16, 46]. The outer-union of the tables from Fig. 1 is depicted in Fig. 2(a). It does not maximally connect the facts in the original tables. Here, ± indicates a missing value (missing null) in the original tables and ⊥ represents a null introduced by the outer-union operator (produced null). In particular, outer union includes partial facts like t_{10} that are made redundant by more complete facts like t_1. Similar observations can be made of the outer-join results. So, Galindo-Legario defined the Full Disjunction (FD) [32]. Informally, it removes redundant facts and produces, in this case, the first 8 tuples (mustard colored) of Fig. 2(b). FD can be viewed as an associative version of outer join [17] and has been used to integrate information across relational tables [17, 59] and web tables [59]. Notice in this example, FD uses information from all five tables so it is important to be able to compute FD over (possibly large) sets of tables.*

But using FD in data lakes poses several important challenges: 1) We cannot rely on common attributes having the same name as in our example. Instead, we must discover what the common attributes are [56]. We will use schema matching for this purpose [63]. Notice that we are not given just two schemas that need to be matched, rather we have a set of tables all of which could potentially share attributes with some or many of the other tables. Hence, we will use *holistic schema matching* [64]. 2) We cannot assume that integrated datasets are complete (that is, they may actually contain null values or partial facts). 3) Prior work on Full Disjunction has been done on relatively small relations (with only 1000 or so tuples per relation [17]) or assumes the common attributes form graphs with specific acyclic structures [65]. To the best of our knowledge, the only work on using FD on larger datasets requires that all joins be done on attributes having key-to-foreign-key relationships [59], a strict requirement that makes the technique only applicable within well-designed enterprise scenarios, not the possibly messy tables retrieved from data lakes commonly used in data science.

We assume data scientists use table discovery algorithms to identify a set of tables that they wish to integrate. Regardless of the search technique, we wish to find the best way to integrate the tables. Specifically, we propose a table integration technique ALITE (Align and Integrate) that first applies schema matching to identify common columns in a set of tables to be integrated. Matched columns are given the same integration ID. We then apply a natural FD over the tables using integration IDs as attribute names.

Contributions. (1) To the best of our knowledge, we introduce the problem of integrating data lake tables obtained using table discovery algorithms. (2) We propose a new holistic schema matching algorithm for sets of tables that outperforms the state-of-the-art matchers on the real data lake tables in our integration benchmarks. (3) We compare the FD used as the integration semantics in ALITE to several other semantics and show the difference and superiority of FD. Empirically, we show FD's superiority for a downstreaming task of entity resolution [44]. (4) We propose a novel algorithm to compute the FD by using complementation and subsumption operators in a novel way. We show that the use of these operators permits optimizations that make the computation faster than the state-of-the-art techniques, in practice. Specifically, ALITE scales better than the state-of-the-art FD algorithms on data lake tables, which are typically large and may have complex join graphs. (5) We introduce and share several open data integration benchmarks.

2 PRELIMINARIES

We now provide the building blocks for integrating tables in a data lake setting, namely, specifying the notation and the basic integration operators, after which we formally define the problem.

Table 1: Symbols used in this paper and their definitions

Symbol	Definition	Symbol	Definition
$\mathcal{T}$ (n=$\|\mathcal{T}\|$)	Set of Tables	r	Set of tuples
T (T_i)	Table (the i_{th} table in $\mathcal{T}$)	$t[A]$	Value of t on the column A
A ($T.A$)	Column (Column A in table T)	$\mathcal{S}$ (s=$\|\mathcal{S}\|$)	Set of all input tuples
m_i	Arity of Table T_i	$\mathcal{F}$ (f=$\|\mathcal{F}\|$)	Set of output tuples
$\mathcal{A}(T)$	Schema (set of columns) of T	±	Null denoting a missing value
t	Tuple	⊥	Null produced by an operator

Notation. Table 1 summarizes the used notation. For any operator, let $\mathcal{S}$ (and its size s) and $\mathcal{F}$ (and its size f) denote the collective set of all input and output tuples, respectively. We use two types of nulls: ± denotes a *missing null* i.e., missing value from an incomplete input relation to be integrated and ⊥ denotes a *produced null*, a null value that is introduced by an operator during integration.

LakeBench: A Benchmark for Discovering Joinable and Unionable Tables in Data Lakes

Yuhao Deng
dyh18@bit.edu.cn
Beijing Institute of Technology

Chengliang Chai
ccl@bit.edu.cn
Beijing Institute of Technology

Lei Cao
lcao@csail.mit.edu
University of Arizona/MIT

Qin Yuan
yuanq1020@gmail.com
Beijing Institute of Technology

Siyuan Chen
siyuanchen@bit.edu.cn
Beijing Institute of Technology

Yanrui Yu
yanruiyu@bit.edu.cn
Beijing Institute of Technology

Zhaoze Sun
sunzhaoze@bit.edu.cn
Beijing Institute of Technology

Junyi Wang
wangjunyi@bit.edu.cn
Beijing Institute of Technology

Jiajun Li
lijiajun@bit.edu.cn
Beijing Institute of Technology

Ziqi Cao
caoziqi@bit.edu.cn
Beijing Institute of Technology

Kaisen Jin
jks@bit.edu.cn
Beijing Institute of Technology

Chi Zhang
zhangchi315@bit.edu.cn
Beijing Institute of Technology

Yuqing Jiang
jiangyuqing@bit.edu.cn
Beijing Institute of Technology

Yuanfang Zhang
yfzhang@bit.edu.cn
Beijing Institute of Technology

Yuping Wang
wyp_cs@bit.edu.cn
Beijing Institute of Technology

Ye Yuan
yuan-ye@bit.edu.cn
Beijing Institute of Technology

Guoren Wang
wanggr@bit.edu.cn
Beijing Institute of Technology

Nan Tang
nantang@hkust-gz.edu.cn
HKUST (GZ)

ABSTRACT

Discovering tables from poorly maintained data lakes is a significant challenge in data management. Two key tasks are identifying joinable and unionable tables, crucial for data integration, analysis, and machine learning. However, there's a lack of a comprehensive benchmark for evaluating existing methods. To address this, we introduce LakeBench, a large-scale table discovery benchmark. It evaluates effectiveness, efficiency, and scalability of table join & union search methods. With over 16 million real tables, LakeBench is **1,600X** larger than existing datasets and **100X** larger in storage size. It includes synthesized and real queries with ground truth, totaling more than 10 thousand queries – **10X** more than used in any existing evaluation. We spent over 7,500 human hours labeling these queries and constructing diverse query categories for thorough evaluation. Our benchmark thoroughly evaluates state-of-the-art table discovery methods, providing insights into their performance and highlighting research opportunities.

PVLDB Reference Format:
Yuhao Deng, Chengliang Chai, Lei Cao, Qin Yuan, Siyuan Chen, Yanrui Yu, Zhaoze Sun, Junyi Wang, Jiajun Li, Ziqi Cao, Kaisen Jin, Chi Zhang, Yuqing Jiang, Yuanfang Zhang, Yuping Wang, Ye Yuan, Guoren Wang, and Nan Tang. LakeBench: A Benchmark for Discovering Joinable and Unionable Tables in Data Lakes . PVLDB, 17(8): 1925-1938, 2024.
doi:10.14778/3659437.3659448
PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/RLGen/LakeBench.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 17, No. 8 ISSN 2150-8097.
doi:10.14778/3659437.3659448

1 INTRODUCTION

Many governments have acknowledged the importance of providing open access to their data for the public good [30]. However, these open datasets are typically published in formats like image or JSON that don't fit neatly into relational databases. To make sense of this poorly maintained data, data scientists have to go through a pipeline of data curation tasks to discover, merge, and clean data [5, 9–11, 31]. As the first step of this pipeline, data discovery that finds relevant tables is critical for data scientists to effectively and efficiently conduct their work.

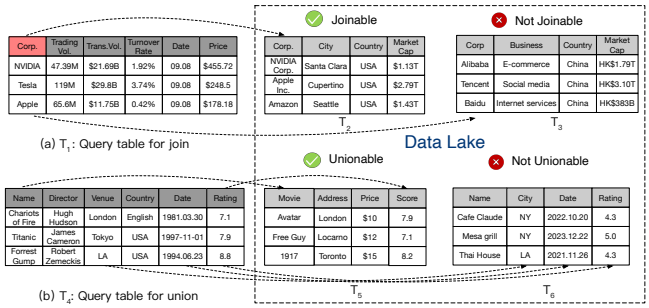


Figure 1: Table Discovery in Data Lake.

Table discovery from data lakes can be generally classified into two categories: keyword-based search [4, 16, 19, 20, 40] and table-based search [3, 13, 14, 17, 24, 32, 45, 46]. Keyword-based search over

tables often relies on the metadata of tables in the lake to identify tables or tuples relevant to the given keywords. However, in the data lake scenario, the metadata is often poorly maintained compared to the database management system (DBMS). Consequently, the incompleteness and inconsistency of metadata in the data lake become bottlenecks for keyword-based search. Thus in this work, we focus on the table-based search that can be further categorized into table join search and table union search. Recently, table-based search has attracted much attention in data management field to augment data instances [7, 27, 43] or attributes [6, 8, 26, 28, 39]. This benefits many downstream applications such as machine learning based data analysis.

Existing Works. To the best of our knowledge, no work has specifically focused on constructing a benchmark for the thorough evaluation of the existing table search methods. Although some works that target inventing new table search methods have done a great job in releasing their datasets and queries used in their evaluation, the scale of their datasets, the number of queries, and the diversity of the workload are not sufficient to evaluate the effectiveness, scalability, and efficiency of the table search methods in diverse scenarios. In particular, TUS [32] and Santos [24] evaluate their *table union search* methods with 1,000 queries on *small* data lakes which contain at most 11,090 tables – in total 11GB data, far from being sufficient to evaluate the scalability of the search algorithms and simulate the diverse real applications. Josie [45], targeting *table join search*, releases 1,000 queries to evaluate if their method is able to efficiently find the tables that have exact overlap with the query tables. In this setting, they do not have to label their queries, e.g., manually discover the tables joinable with the query tables. Therefore, these queries cannot be used to evaluate the effectiveness of other table join search methods that consider the semantic similarity of the data.

Design Goals. To fill this gap, in this work we aim to construct a comprehensive benchmark for table discovery, following the design goals below:

(1) *Scalability.* To thoroughly evaluate the efficiency and scalability of different methods, the benchmark has to involve data lakes of various sizes, especially large-scale ones. The size of the data lake is measured from two aspects: the storage size and the number of total columns. The storage size is highly related to the average table size and the number of tables. The number of total columns matters as well because almost all table discovery algorithms index and search over a large number of columns.

(2) *Variety.* The performance of table discovery largely relies on the characteristics of the datasets and queries. In particular, one algorithm might perform differently on different categories of queries. For example, given a query, a semantic-aware algorithm will show clear advantages if the table semantic matters the most in determining its unionable tables. Hence, plenty of diverse queries are critical to thorough evaluation.

(3) *Comprehensive comparison of solutions.* Understanding the pros/cons of existing table search methods and revealing research opportunities, requires a reliable implementation of all these methods, a thorough and fair experimental evaluation, and a deep analysis of the results.

Our Proposal. Guided by the above three design goals, we construct a benchmark, LakeBench– the first of its kind that supports the thorough evaluation of table union & join methods.

To achieve the first design goal, we collect over 1TB data – **100X** larger than the data lakes used in existing works, and the number of columns is up to **100 million**, on which we evaluate different table union/join search methods. Moreover, we label a sufficient number of diverse queries (more than 10 thousand) – **10X** more than any other works. To this end, we organize a team of 25 graduate students and spent more than *7,500 human hours* on labeling the queries. Because it is prohibitively expensive to manually identify tables in the data lake that are joinable/unionable to a given query, we design a candidate generation strategy to prune the tables that are most unlikely to be the ground truth.

To meet the second design goal, we construct data lakes with diverse characteristics by collecting tables from two data sources, namely OpenData [1] and WebTable [2]. WebTable has a large number of tables (more than 10^8) but each table is small. In contrast, OpenData consists of large tables (on average 1GB in size per table) but has fewer tables than WebTable. Besides, we generate both synthetic (splitting from large lake tables) and real queries (directly sampling from the data lake), which have different properties. We further categorize these queries at finer granularity w.r.t. the key factors that determine the quality of table search.

For the third design goal, we implement many key techniques such as column embeddings, approximate nearest neighbor (ANN) search index, etc., which are the building blocks of these approaches, ensuring that we are able to evaluate the scalability and accuracy of various approaches on different datasets and mostly reuse the code base. More importantly, we evaluate these approaches over the aforementioned fine-grained query categories, so as to reveal multiple insights w.r.t. the performance of different approaches over different categories. In addition, our analysis of the time and space complexity of the search algorithm connects the experiment results to theory, guiding our discussions on the results. In short, LakeBench makes it possible to thoroughly evaluate and understand existing table search methods.

Contributions. We make the following contributions:

(1) We build a comprehensive benchmark, LakeBench, for table discovery, featuring large-scale datasets and a sufficient number of diverse queries. Besides, it addresses critical limitations in existing benchmarking datasets. To be specific, *i*.) The lack of a specific dataset for joinable search with ground truth; *ii*.) The number of ground truth tables for individual queries is small, particularly notable in TUS and SANTOS, thus problematic in accurately assessing recall; *iii*.) Relying heavily on automatically generated ground truth, which may simplify the task and potentially miss ground truth because of lacking sufficient human annotation.

(2) We collect real tables (more than 1 TB in size) from multiple sources, which consist of 4 data lakes with various storage sizes and different numbers of table columns.

(3) We create 10,000+ table queries covering different characteristics and spend 7,500+ human hours on accurately labeling them.

(4) We thoroughly evaluate and analyze the state-of-the-art table join/union search approaches using these created queries on the benchmark datasets. Based on that, we reveal multiple insights and research opportunities. For example, we observe that although

LakeCompass: An End-to-End System for Data Maintenance, Search and Analysis in Data Lakes

Chengliang Chai*
ccl@bit.edu.cn
Beijing Institute of Technology

Yuhao Deng*
dyh18@bit.edu.cn
Beijing Institute of Technology

Yutong Zhan
zyt@bit.edu.cn
Beijing Institute of Technology

Ziqi Cao
caoziqi@bit.edu.cn
Beijing Institute of Technology

Yuanfang Zhang
zyuanfang@bit.edu.cn
Beijing Institute of Technology

Lei Cao
lcao@csail.mit.edu
University of Arizona/MIT

Yuping Wang
wyp_cs@bit.edu.cn
Beijing Institute of Technology

Zhiwei Zhang
zwzhang@bit.edu.cn
Beijing Institute of Technology

Ye Yuan
yuan-ye@bit.edu.cn
Beijing Institute of Technology

Guoren Wang
wanggr@bit.edu.cn
Beijing Institute of Technology

Nan Tang
nantang@hkust-gz.edu.cn
HKUST (GZ)

ABSTRACT

Searching tables from poorly maintained data lakes has long been recognized as a formidable challenge in the realm of data management. There are three pivotal tasks: keyword-based, joinable and unionable table search, which form the backbone of tasks that aim to make sense of diverse datasets, such as machine learning. In this demo, we propose LakeCompass, an end-to-end prototype system that maintains abundant tabular data, supports all above search tasks with high efficacy, and well serves downstream ML modeling. To be specific, LakeCompass manages numerous real tables over which diverse types of indexes are built to support efficient search based on different user requirements. Particularly, LakeCompass could automatically integrate these discovered tables to improve the downstream model performance in an iterative approach. Finally, we provide both Python APIs and Web interface to facilitate flexible user interaction.

PVLDB Reference Format:
Chengliang Chai, Yuhao Deng, Yutong Zhan, Ziqi Cao, Yuanfang Zhang, Lei Cao, Yuping Wang, Zhiwei Zhang, Ye Yuan, Guoren Wang, and Nan Tang. LakeCompass: An End-to-End System for Data Maintenance, Search and Analysis in Data Lakes . PVLDB, 17(12): 4381 - 4384, 2024.
doi:10.14778/3685800.3685880

1 INTRODUCTION

Many organizations have acknowledged the importance of providing open access to their tabular data for the public good [14]. However, the large amount of tables are often poorly maintained (*e.g.*, lacking of index or schema information). To make sense of this data, a key initial step for the data scientists is to search tables relevant to the task at their hand. To this end, we propose to build an end-to-end prototype system, LakeCompass that driven by the need of the downstream analytics tasks, offers efficient and effective table search over data lakes.

At a high level, LakeCompass features the following three key functionalities.

Index building. We build various types of indexes over tables in data lakes, such as inverted index, local sensitive hash (LSH) and the hierarchical navigable small world (HNSW), which taking both text similarities and semantic similarities into consideration, efficiently and effectively support table search.

Table search. We support three typical categories of table search, *i.e.*, keyword-based, joinable and unionable table search. The former one returns relevant tables based on the given keywords. The latter two take as input a query table, and discovering tables that can be integrated with the query table.

Data Selection for Model analysis. LakeCompass offers a data selection component that taking into consideration the specific need of the downstream applications, *i.e.*, the ML models, intelligently selects the tuples and features from the table search result in a more targeted manner.

Overall, LakeCompass has the following advantages.

[1. *High efficiency.*] Various types of indexes allow us to support efficient search over different queries. In terms of the model analysis module, we propose to use the online learning technique to improve the efficiency of iterative model evaluation.

[2. *High effectiveness.*] We maintain the original data associated with their semantic embeddings, together with various indexes, which allow users to search highly relevant tables customized to their needs. For the model analysis, we propose a multi-armed bandit (MAB) technique to judiciously select beneficial tuples/features among the returned relevant tables.

[3. *User-friendly interaction.*] We encapsulate the above functionalities through flexible Python APIs, where the user can upload

*Both authors contributed equally to this research.
Ye Yuan is the corresponding author.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, Vol. 17, No. 12 ISSN 2150-8097.
doi:10.14778/3685800.3685880

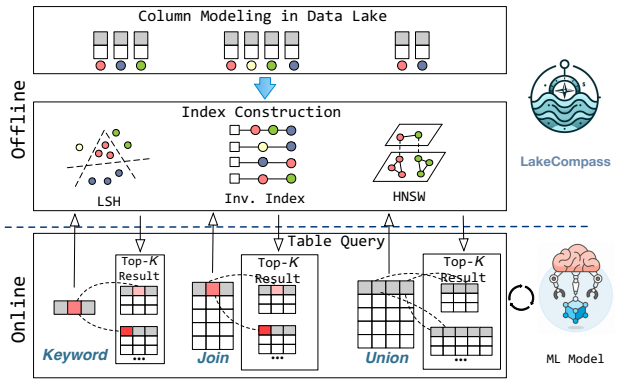


Figure 1: Overall Architecture of LakeCompass.

data, construct indexes, search tables and build ML models. We also provide a Web interface to serve table search and analysis in a more intuitive way.

Existing Works. There exist several prototypes of table search. Aurum [6] takes as input keywords and returns relevant tables based on their schema information. COCOA [5] focuses on searching joinable tables; Auctus [1] supports both joinable and unionable table search; Besides the above table search, Juneau [7] further supports feature extraction and data cleaning based on the returned tables. Similarly, DIALITE [12] can also search tables that can be integrated with the query table, and then conducts some data analysis like entity matching. Although the above prototypes supports table search in data lakes, they do not consider the semantic information of columns/rows of tables, which is rather significant in achieving high effectiveness. What's more, they do not take the machine learning task into consideration, which is the key application for table search in data lakes. Although some works [3, 9] have studied data augmentation for ML, they assume the relevant data is discovered beforehand by another system, hence not offering an end-to-end table search solution.

In summary, LakeCompass is an end-to-end system that is adequate to demonstrate the efficiency and effectiveness of table search for real-world applications.

2 SYSTEM OVERVIEW

At a high level, LakeCompass consists of the following modules: data lake embedding, index construction, online table query processing as well as the model analysis, as shown in Figure 1. The former two modules are offline, which respectively encode the column/schema in the data lake into a vector and index these vectors. Once an online query table arrives, we first encode its column(s) and then use the index to retrieve relevant tables with the highest relevance score from the data lake. If the query is a keyword, we just encode the query and retrieve relevant tables.

Embedding Data Lake. Generally speaking, column representations are significant in table search. The columns of the original tables in data lake are always represented as fixed-length vectors. The vectors are mostly hash codes [6, 17] (*e.g.*, generated by the minhash function) or embeddings [8, 10] (*e.g.*, generated by pre-trained language models) that are effective in finding columns with similar semantics or high overlaps, which are rather significant in searching joinable/unionable tables. However, some methods like Josie and InfoGather [15, 16] directly use the original cell values of each column to search highly overlapping columns rather than using vectors. Besides, we also maintain the metadata of data lake tables (*e.g.*, table name, caption and schema information, etc.) and index them, which are also important in table search, especially for keyword-based search.

Index Construction. Almost all table search methods rely on an index to search large table repositories. Building upon the representations discussed above, several typical ANN indexes like LSH [17] or HNSW [8] can be utilized to index all the embeddings. As another option, inverted index [11, 15, 16] accelerates the process of finding highly overlapping columns, where each cell value is linked to the columns that contain the value. In this case, these colorful circles represent Column IDs rather than vectors. All the types of indexes are supported in LakeCompass.

Online Table Search. For keyword-based search, the user issues the keywords, which are encoded as embeddings, and retrieve tables with similar embeddings on metadata or columns. For joinable search, LakeCompass supports users in issuing a query table through either user-uploaded files or the result of keyword-based search. Once a user issues a table T_J and a specific column c for search, LakeCompass first represents c as a vector. It then uses the index to quickly identify a of top-K columns (tables), denoted by $R(T_J)$, with high relevance scores. The unionable search follows the same procedure. The only difference is that given a table T_U, it has to search over each attribute in T_U and aggregate the results. Specifically, for each column $c_i \in T_U$, similar to the join search, we retrieve from the data lake a set of columns that have high relevance scores with c_i. Then, we compute the union of all retrieved tables. For each table T in the union set, we compute the relevance score using techniques like maximum bipartite graph matching, considering the column relevance between the T_U and T. Finally, a set of top-K tables, denoted by $R(T_U)$ with the highest relevance scores are returned. In LakeCompass, we have implemented various joinable [15–17] and unionable search [8, 11, 15] algorithms.

Iterative Data Selection for Model Analysis. When the query table serves as the train data, due to the data scarcity problem, we expect to augment more tuples/features through unionable/joinable table search. However, although all the returned tables $R(T_J)/R(T_U)$ can be integrated into the query table, integrating all of them may not improve the model performance. The reason is that they come from heterogeneous sources in data lakes, some of which are in fact noisy to the model. Therefore, we apply our proposed tuple/feature augmentation techniques [2, 13] to judiciously select tuples/features (from unionable/joinable tables), iteratively integrate to the query table and evaluate the performance. Here, we summarize the high level idea [2] of selecting tuples from unionable tables.

i). *Clustering.* Given all tables in $R(T_U)$, we first merge all tuples of these tables, and then cluster these tuples based on their feature similarities. Then tuples in each cluster can be seen as homogeneous. Next, we use an iterative MAB solution to judiciously select tuples that can benefit the model from these clusters.

ii). *MAB-based data selection.* We take each cluster as a bandit arm. Each arm is assigned with an upper confidence bound (UCB) score,

Language Models Enable Simple Systems for Generating Structured Views of Heterogeneous Data Lakes

Simran Arora
Stanford University
simarora@stanford.edu

Brandon Yang
Stanford University
bcyang@stanford.edu*

Sabri Eyuboglu
Stanford University
eyuboglu@stanford.edu*

Avanika Narayan
Stanford University
avanikan@stanford.edu

Andrew Hojel
Stanford University
ahojel@stanford.edu

Immanuel Trummer
Cornell University
itrummer@cornell.edu

Christopher Ré
Stanford University
chrismre@stanford.edu

ABSTRACT

A long standing goal in the data management community is developing systems that input documents and output queryable tables without user effort. Given the sheer variety of potential documents, state-of-the art systems make simplifying assumptions and use domain specific training. In this work, we ask whether we can maintain generality by using the in-context learning abilities of large language models (LLMs). We propose and evaluate Evaporate, a prototype system powered by LLMs. We identify two strategies for implementing this system: prompt the LLM to directly extract values from documents or prompt the LLM to synthesize code that performs the extraction. Our evaluations show a cost-quality tradeoff between these two approaches. Code synthesis is cheap, but far less accurate than directly processing each document with the LLM. To improve quality while maintaining low cost, we propose an extended implementation, Evaporate-Code+, which achieves better quality than direct extraction. Our insight is to generate many candidate functions and ensemble their extractions using weak supervision. Evaporate-Code+ outperforms the state-of-the art systems using a *sublinear* pass over the documents with the LLM. This equates to a 110× reduction in the number of documents the LLM needs to process across our 16 real-world evaluation settings.

PVLDB Reference Format:
Simran Arora, Brandon Yang, Sabri Eyuboglu, Avanika Narayan, Andrew Hojel, Immanuel Trummer, and Christopher Ré. Language Models Enable Simple Systems for Generating Structured Views of Heterogeneous Data Lakes. PVLDB, 17(2): 92 - 105, 2023.
doi:10.14778/3626292.3626294
PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/HazyResearch/evaporate.

1 INTRODUCTION

Organizations often seek insights trapped in heterogeneous data lakes (*e.g.* the web, corporate data lakes, and electronic health records) [10, 26, 54]. In their raw form, these data sources cannot easily support analytical queries. A long standing goal of the data management community is to develop systems that automatically convert heterogeneous data lakes into queryable, structured tables [12, 15, 47, 66, inter alia.]. In this work, we investigate whether recent large language models can help address this problem.

We study systems that take as **input** heterogeneous documents (*e.g.* HTML webpages, PDFs, text) and **output** a tabular, structured view of the documents. These systems must identify the schema and perform extraction to populate the table.

> Example 1. Medical researchers frequently use data spanning electronic health records (EHR), clinical trials, knowledge sources (e.g. PubMed), and FDA reports to understand and monitor patients and treatments [8]. Consider the large collection of **FDA 510(k)** reviews for premarket medical devices, which have been the subject of multiple studies [64, 68]. Our objective is to output a table that automatically structures the attributes that are distributed in the ~20-page **PDFs**, for instance the `device classification`, `predicate device code`, and `indications for use`.

Systems designed to tackle this problem must balance a three-way tradeoff between **cost** (data lakes may hold millions of documents), **quality** (output tables should be able to accurately support an analyst's queries), and **generality** (different data lakes have different document types and structure). See Section 2 for a formal task definition and further discussion of this tradeoff.

Given the range of formats, attributes, and domains across documents, prior systems rely on simplifying assumptions (e.g. handling one document format). The majority of works focus on structuring HTML [12, 15, 25], assuming the attributes and values are at specific positions in the HTML-DOM [22, 43, 44, 67]. For unstructured text, current approaches use linguistic tools (e.g., dependency parsers) to introduce structure [15, 25, 31, 48] and then apply heuristic rules over the resulting structure to extract information. The documents in Example 1 highlight the limitations of the prior approaches: they lack (e.g. HTML) structure and, consistent with recent evaluation efforts [67], we find the SoTA approaches for unstructured text perform poorly on long semi-structured PDFs (See [5]). Some systems assume there is a human-in-the-loop, labeling data and writing heuristic rules for extraction [53, 57], while others assume access to annotated training documents from the domain [22, 43, 44]. Researchers manually annotated the reports in Example 1 [64].

* Equal contribution.
This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 17, No. 2 % ISSN 2150-8097.
doi:10.14778/3626292.3626294

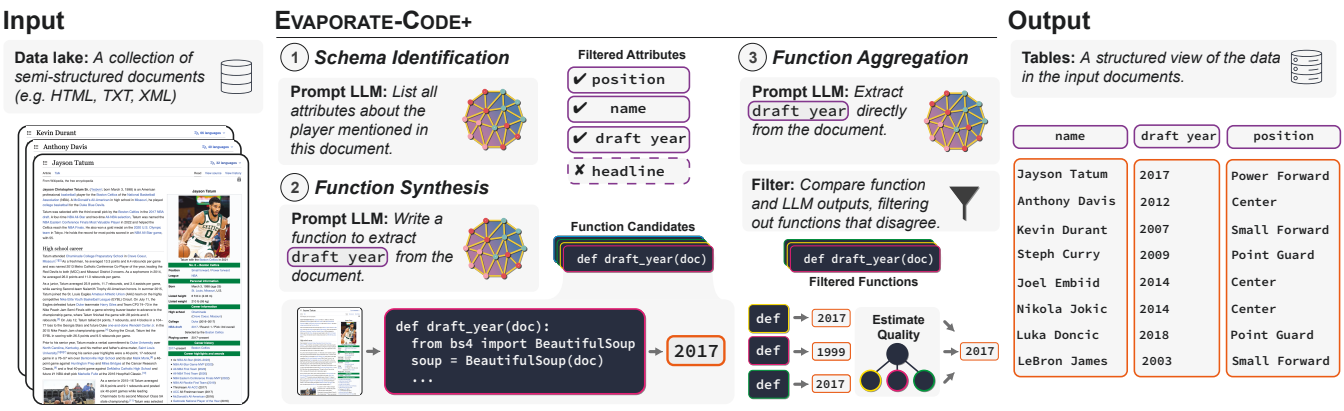


Figure 1: The user provides a collection of documents (e.g. NBA player bios) and EVAPORATE outputs a table by identifying attributes and populating columns. EVAPORATE avoids running expensive LLM inference on all documents by (1) synthesizing the key attributes from a small sample of documents and (2) synthesizing (e.g. Pythonic) functions that then are reused at scale to process documents. Because function quality is variable, EVAPORATE (3) applies an algorithm that generates many candidate functions and ensembles their extractions using weak supervision.

In this work, we explore whether we can improve generality by leveraging *large language models* (LLMs). An LLM is a deep learning model that is pretrained on broad data and can be adapted to diverse tasks, from machine translation to data wrangling [14, 46]. At inference time, the models take as input a natural language task description termed a *prompt* [11, 14] and generate a natural language response. See Section 2.3 for more background on LLMs.

EVAPORATE. (Section 3) We present EVAPORATE, a system that uses LLMs to produce structured views of semi-structured data lakes. Our evaluation spans 16 real-world settings from movie and university websites to *FDA 510(k)* reviews [22, 32, 34, 37, 43, 64, 68].

The user inputs a collection of documents and EVAPORATE automatically identifies the schema and performs extraction to populate the table. Our implementation requires *no customization, training, or human effort* to support the diverse evaluation settings. We propose two fundamental strategies for implementing this interface, identifying a tradeoff between their cost and quality:

(1) EVAPORATE-DIRECT (Figure 2) The LLM directly extracts values from documents.
(2) EVAPORATE-CODE (Figure 4) The LLM *synthesizes code* that is then applied to process documents at scale.

EVAPORATE-CODE is cheap, but underperforms EVAPORATE-DIRECT by 24.9% (13.8 F1 points) averaged across our evaluation settings. We thus seek a new code synthesis approach. We present EVAPORATE-CODE+, which achieves better quality than direct extraction. Our insight is to synthesize many code snippets for extraction and ensemble their outputs using weak supervision.

Direct Extraction (Section 3.1). Our first implementation, EVAPORATE-DIRECT, applies a *single prompt* (included in [5]) to each document in the input. The prompt instructs the LLM to both identify the schema and extract values. Remarkably, we find that in some settings, with a single prompt and no task specific modifications, performance is already competitive with state-of-the-art systems that rely on domain specific assumptions and training.

However, this implementation is very expensive. LLMs are optimized for interactive, human-in-the-loop applications (*e.g.* ChatGPT) [65], not high-throughput data processing tasks [56]. The number of tokens processed by an LLM in EVAPORATE-DIRECT grows *linearly* with the size of the data lake. As of March 2023, applying OpenAI's models to the 55 million Wikipedia articles would cost over $110k (`gpt-3.5`, $0.002/1k tokens) and $1.1M (`text-davinci-003`, $0.02/1k tokens) dollars [1, 49]. There are *billions* of webpages on the broader Internet [35] and the facts change over time. For instance, NBA players are added to Wikipedia, a player's `team` changes after trades, and the `points per game` metric changes after every game. Data processing is a *routine expense* (repeated by multiple data analysts), not a one-time cost [55].

Code Synthesis (Section 3.2). *Can we produce the structured table using a sublinear pass of the LLM over the documents?* We propose EVAPORATE-CODE, which splits the task into two sub-tasks: (1) identify the table schema and (2) extract values. This view allows us to exploit the distinct *redundancies* of each sub-task that occur when running LLM inference on every document:

(1) *Schema Generation.* In order to identify a schema, we only process a small sample of documents with the LLM. This succeeds because there is redundancy in the attributes mentioned across documents. For e.g., in Example 1, most reports mention a `predicate device name`.
(2) *Function Synthesis.* We prompt the LLM to synthesize (*e.g.* Pythonic) *functions*, that can be applied at scale across the documents. This works because of redundancy in the formatting of attribute-value pairs. For e.g., the FDA 510(k)s use the consistent format "`Predicate device name: k`".

The number of tokens processed by the LLM in EVAPORATE-CODE is *fixed* and does not grow with the size of the data lake (as illustrated in Figure 3), addressing the cost issues of EVAPORATE-DIRECT. However, the LLM synthesizes variable quality information extraction functions. The extractions are up to 14 points worse in Pair F1 score than those produced using EVAPORATE-DIRECT.

Lingua Manga : A Generic Large Language Model Centric System for Data Curation

Zui Chen
IIIS, Tsinghua University
Beijing, China
chenzui19@mails.tsinghua.edu.cn

Lei Cao
U of Arizona/MIT
Tucson, Arizona
lcao@csail.mit.edu

Sam Madden
MIT
Cambridge, Massachusetts
madden@csail.mit.edu

ABSTRACT

Data curation is a wide-ranging area which contains many critical but time-consuming data processing tasks. However, the diversity of such tasks makes it challenging to develop a general-purpose data curation system. To address this issue, we present Lingua Manga , a user-friendly and versatile system that utilizes pre-trained large language models. Lingua Manga offers automatic optimization for achieving high performance and label efficiency while facilitating flexible and rapid development. Through three example applications with distinct objectives and users of varying levels of technical proficiency, we demonstrate that Lingua Manga can effectively assist both skilled programmers and low-code or even no-code users in addressing data curation challenges.

PVLDB Reference Format:
Zui Chen, Lei Cao, and Sam Madden. Lingua Manga : A Generic Large Language Model Centric System for Data Curation. PVLDB, 16(12): 4074 - 4077, 2023.
doi:10.14778/3611540.3611624

1 INTRODUCTION

In the era of big data, organizations are collecting vast amounts of data from various sources. However, the data is often messy, incomplete, or contains errors. To effectively perform data analytics or any other applications, it is often necessary to engage in a data curation process [8, 15], which includes tasks like data discovery, integration, and cleaning. Nonetheless, devising a data curation solution for a particular scenario can be a cumbersome and time-consuming process. It entails extensive communication of requirements, a collaboration between domain experts and programmers, multiple rounds of debugging and testing, and active maintenance to accommodate new use cases. This underscores the need for a *generic* system that enables users to efficiently and effectively address diverse data curation challenges and apply solutions to various applications.

However, building a system that addresses diverse data curation problems and applications is challenging. Data curation involves a range of distinct tasks, such as data discovery through table search, schema matching, entity resolution during data integration, and data imputation in data cleaning. Moreover, in real applications, various extra tasks are often involved in the data curation processes, such as event extraction, name extraction, anomaly detection, data summarization, visualization, table semantic parsing, etc. Worst yet, applications in different domains have diverse requirements. Effectively resolving data curation problems often relies heavily on users' domain knowledge, making developing a *general-purpose* solution improbable. For example, entity resolution over *books* focuses on text understanding, while entity resolution over *images* should rather pay special attention to image processing. As a result, existing data curation systems typically focus on only a few tasks on specific data formats. For example, Data Tamer [8] solves schema matching and entity resolution problems; ZenCrowd [4] targets entity linking; while CrowdDB [7] addresses fuzzy SQL semantics.

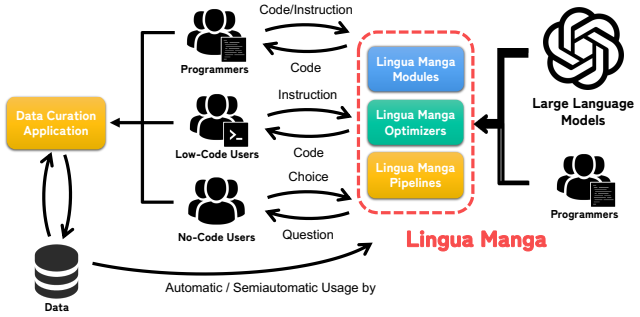


Figure 1: An illustration of the Lingua Manga System.

Recent advances in Large Language Models (LLMs) have shed light on this highly challenging yet critical data curation area. Pre-trained LLMs such as GPT-3 [10], Codex [6], ChatGPT [17], and LLaMA [5] have not only shown the ability to understand human needs and instructions but also to generate code like a programmer. Such ability could be leveraged to solve specific data curation problems like data imputation or entity resolution [2].

However, the current use of LLMs and LLM plugins is still far from the ultimate solution. Repeatedly calling LLM services can be costly and may lead to data privacy issues. For instance, consider a large e-commerce organization with thousands of documents and tables containing billions of rows – it would be neither affordable nor secure to let an LLM access the entire data lake to complete the tasks. LLMs tend to be less effective without access to enterprise data due to the gap between the enterprise data and the public data LLMs trained on. Thus, there is an urgent need to harness the power of LLMs systematically, efficiently, and securely. Properly integrating LLMs and traditional data curation methods could be a promising path toward a general-purpose data curation system.

In this demonstration, we present Lingua Manga , an LLM-centric system that enables the swift development of effective yet efficient data curation solutions. It benefits programmers as well as low-code or even no-code users. Specifically, Lingua Manga shows the following key properties:

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 16, No. 12 ISSN 2150-8097.
doi:10.14778/3611540.3611624

• **User-friendly**: Users can quickly develop a data curation solution by utilizing the templates, built-in modules, and, most importantly, by effectively interacting with LLMs through Lingua Manga as a middleware.

• **Flexible**: Users can effortlessly communicate with Lingua Manga through natural language (NL) to inject domain knowledge or instructions into the data curation solution such that it can meet the needs of the specific applications or domains.

• **Intelligent**: Lingua Manga automatically optimizes the solution to fix errors, improves performance, continuously learns from the data, etc.

• **Highly Performant**: Lingua Manga minimizes the frequency of calling the LLM service, which incurs heavy computation and monetary costs, thus resulting in an economical, efficient, and privacy-preserving solution.

• **Label Efficient**: with our system, users can develop a data curation solution with no or only a few labeled examples from the specific application while still achieving accuracy comparable to the SOTA ML-based methods trained with thousands of labels.

2 PRELIMINARIES: PROMPT-TUNING AND CODE GENERATION IN LLMS

Starting from GPT-3 [10], LLMs' size and computational cost have grown so much that users can no longer afford to fine-tune them for every downstream task. In the meantime, LLMs have also gained significant knowledge through training on large-scale corpora, leading to the emergence of a new paradigm known as prompt-tuning [9]. Using prompt-tuning, users can generalize LLMs to new tasks and data domains without modifying the LLM itself. This is achieved by incorporating prompts, such as descriptions, tags, and learnable components, into the input data.

Moreover, cutting-edge LLMs like ChatGPT [17] have demonstrated their effectiveness in interacting with humans using natural-language-based prompts [3]. These prompts are interpretable by humans. For instance, a straightforward prompt like "Please determine if the following entities are equivalent" can enable LLMs to carry out entity resolution. This capability is especially useful in Lingua Manga since it facilitates LLMs to interactively execute functions vaguely specified by users in natural language.

Furthermore, prompts like "Please write a code for entity resolution" trigger LLMs to generate custom software modules via code generation. GPT-3 [10], ChatGPT [17], Codex, and Copilot [6] are the mainstream LLMs that support code generation. However, existing methods that use these models to generate code have limitations [19]. They are typically inadequate for producing code with complex logic and building large-scale software. This calls for a new methodology to effectively utilize LLM Generated Code (LLMGC).

3 LINGUA MANGA

Lingua Manga comprises a Domain-Specific Language (DSL), a Compiler, an Optimizer, and a set of templates.

Lingua Manga is a **user-friendly** workflow system that enables users to quickly build data curation solutions by composing pipelines of logical operators. The system features a DSL to simplify the workflow-building process.

Lingua Manga is **automatic**. Like a relational database, it auto-compiles each logical operator into a physical, executable *module*.

Lingua Manga is **extensible**. Lingua Manga allows the programmers to implement their own physical modules, which can be seamlessly plugged into the system as long as they follow the standard interfaces of Lingua Manga .

Lingua Manga is **flexible**. To meet the specific need of different applications, the Lingua Manga optimizer interacts with users via LLMs to generate customized physical modules.

Lingua Manga is **intelligent**. The Lingua Manga optimizer uses LLMs to enhance the modules' capability.

In addition, to further facilitate users in building pipelines, Lingua Manga offers templates, essentially groups of pre-built pipelines. Rather than creating a pipeline from scratch, Lingua Manga allows users to start with a pre-defined, well-optimized pipeline that the target application can directly use.

The fundamental observation underlying this research is that while current LLMs may not yet be able to produce exceedingly intricate and large software systems, they are proficient at generating modular code on a smaller scale. Moreover, LLMs possess learning and logical reasoning ability that enable them to complete data curation tasks directly, indicating that LLMs could accelerate the process of developing a data curation solution by automating parts of it and, thus, reduce the workload of software developers.

Next, we discuss the modules and the Lingua Manga optimizer, which are the critical components of Lingua Manga .

3.1 Modules in Lingua Manga

A module is a function $f : X \rightarrow Y$ that takes X as input and returns Y. Modules are usually viewed as black boxes, and their behavior is determined by their expected output. In Lingua Manga , modules can be classified into four types:

• **Custom Module**: As a basic module, a custom module is implemented with manually written code. It can be created by users with programming skills or provided by Lingua Manga as a default built-in module.

• **LLM Module**: An LLM itself can be a module. An LLM module tends to be more powerful than a traditional module because of the common sense knowledge and logical inference abilities possessed by an LLM. However, an LLM module requires a good task description as input; and LLM outputs typically need proper validation, as textual responses generated by the LLMs could be diverse and unstable. For instance, numerical validation that ensures the LLM's output is within a reasonable value range can identify and correct potential errors in LLM output.

• **LLMGC Module**: An LLM can dynamically generate code to implement an LLMGC module, replacing the role of programmers. Lingua Manga allows LLMGC to call other modules in the system or use external tools such as a calculator or another pre-trained LLM. Although an LLMGC module might be less expressive than custom modules written by skilled programmers, it enables no-code or low-code users to implement or customize a module effortlessly.

• **Decorated Module**: As the most advanced module in Lingua Manga , a decorated module can comprise multiple basic modules and be enhanced by the optimizer that leverages the common sense knowledge encoded in the LLMs and their learning ability.

Magneto: Combining Small and Large Language Models for Schema Matching

Yurong Liu*
New York University
yurong.liu@nyu.edu

Eduardo H. M. Pena*†
Federal University of Technology Paraná
eduardopena@utfpr.edu.br

Aécio Santos
New York University
aecio.santos@nyu.edu

Eden Wu
New York University
eden.wu@nyu.edu

Juliana Freire
New York University
juliana.freire@nyu.edu

ABSTRACT

Recent advances in language models (LMs) open new opportunities for schema matching (SM). Recent approaches have shown their potential and key limitations: while small LMs (SLMs) require costly, difficult-to-obtain training data, large LMs (LLMs) demand significant computational resources and face context window constraints. We present `Magneto`, a cost-effective and accurate solution for SM that combines the advantages of SLMs and LLMs to address their limitations. By structuring the SM pipeline in two phases, retrieval and reranking, `Magneto` can use computationally efficient SLM-based strategies to derive candidate matches which can then be reranked by LLMs, thus making it possible to reduce runtime while improving matching accuracy. We propose (1) a self-supervised approach to fine-tune SLMs which uses LLMs to generate syntactically diverse training data, and (2) prompting strategies that are effective for reranking. We also introduce a new benchmark, developed in collaboration with domain experts, which includes real biomedical datasets and presents new challenges for SM methods. Through a detailed experimental evaluation, using both our new and existing benchmarks, we show that `Magneto` is scalable and attains high accuracy for datasets from different domains.

PVLDB Reference Format:
Yurong Liu, Eduardo H. M. Pena, Aécio Santos, Eden Wu, and Juliana Freire. `Magneto`: Combining Small and Large Language Models for Schema Matching. PVLDB, 18(8): 2681 - 2694, 2025.
doi:10.14778/3742728.3742757

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/VIDA-NYU/magneto-matcher.

*Equal contribution.
†Work done as a visiting researcher at New York University.
This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 18, No. 8 ISSN 2150-8097.
doi:10.14778/3742728.3742757

1 INTRODUCTION

The rapid increase in the volume of structured data– from data published in scientific articles [63, 72] and repositories [29, 60, 66, 80] to open government portals [19, 64] – creates new opportunities to answer important questions through analytics and predictive modeling. But often, data from multiple sources must be integrated to answer these questions. Consider the following example.

Example 1.1. In the context of the National Cancer Institute's Clinical Proteomic Tumor Analysis Consortium (CPTAC) [15], Li et al. [52] carried out a comprehensive proteogenomic analysis of cancer tumors. They collected data from ten studies (published as supplementary material in research papers) that cover multiple patient cohorts and cancer types. To facilitate their analysis, they mapped each dataset into the GDC standard, a data model set by the National Cancer Institute's Genomic Data Commons (GDC) for cancer genomic data [38, 60]. Even though the studies had been carried out by members of the CPTAC effort, the datasets containing patient case and sample data used different representations for variable names and values. Integrating these data required a substantial effort to match variables from each dataset source schemata to the target GDC format, which encompasses over 700 attributes. □

Even though the schema matching problem has been extensively studied [8, 22, 48, 58], matching still requires a time-consuming, manual curation process for complex tasks, which like the one described above, involves ambiguity and heterogeneity in the representation of attributes and values (see Table 1). Schema matching approaches that rely on attribute names, data types and values for similarity assessments are likely to fail for such matches. As a point of reference, we assessed the effectiveness of state-of-the-art strategies for the biomedical datasets from Example 1.1. As shown in Figure 1, they perform poorly: the best technique achieves at most 0.45 of mean reciprocal rank (MRR) and incurs high computational costs. We discuss these results in detail in Section 6.

Determining correspondences between columns may require knowledge beyond the schema and contents of a table. Table 1 shows some possible matches for the proteogenomic analysis. Sometimes, selecting the correct match is difficult even for subject matter experts. For example, for the attribute `patient_age` from one of the datasets, there are at least three plausible matches in GDC: `age_at_diagnosis`, `days_to_birth`, and `age_at_index`. Without additional context, it is difficult to determine the correct match.

Table 1: Examples of candidate matches between source tables and the GDC that highlight schema heterogeneity.

Source Column and Values	Target Candidate Columns
Histologic_Grade_FIGO: FIGO grade 1, FIGO grade 2, FIGO grade 3	**tumor_grade**: G1, G2, G3 **who_nte_grade**: G1, G2, G3 **adverse_event_grade**: Grade 1, Grade 2, Grade 3
Path_Stage_Primary_Tumor-pT: pT1a (FIGO IA), pT2 (FIGO II), pT3b (FIGO IIIB)	**uicc_pathologic_t**: T1a, T2, T3b **ajcc_pathologic_t**: T1a, T2, T3b
Age: 65, 72, 49	**age_at_diagnosis**: n/a **days_to_birth**: n/a **age_at_index**: n/a

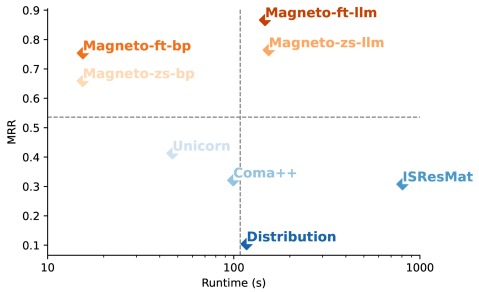


Figure 1: Trade-off between runtime and accuracy (using MRR) for different schema matchers in the task of Example 1.1. `Magneto` variants (orange) outperform traditional methods (blue) in MRR and accuracy-runtime trade-off.

Schema Matching and Language Models. Renewed interest in data integration has emerged due to the capabilities of language models [34, 59]. For schema matching, promising approaches have been proposed [10, 11, 27, 79]. A key challenge in schema matching is estimating the similarity between two columns. Pre-trained Language Models (PLMs), referred to as SLMs in this paper to distinguish them from large language models (LLMs), create column representations (or embeddings) enriched with semantic information. The similarity between two embeddings serve as a proxy for column-matching scores [10, 11]. SLMs have also been fine-tuned for schema matching [27] and general matching tasks [79].

LLMs, in contrast, are trained using large, generic data corpora and thus contain knowledge that can assist in obtaining additional semantics necessary to identify matches. Prompting strategies combined with fine-tuned models have been shown effective to improve table understanding and help in integration tasks [50].

While prior work has shown the usefulness of SLMs and LLMs for schema matching, they also present significant practical challenges:

- *Challenge 1*: Fine-tuning SLMs can lead to significant performance improvements, but this requires the availability of manually curated training data, which can be expensive to create.
- *Challenge 2*: LLMs usually do not need fine-tuning, but face constraints due to fixed context windows, requiring truncation of large prompts and potential loss crucial information. While some models offer larger windows, cost scales with input and output size. Furthermore, accuracy can decline with long prompts and API calls incur high latency, especially for large inputs [51].

Our Approach. We introduce `Magneto`, a framework that combines SLMs and LLMs to derive cost-effective and general schema matching solutions. As illustrated in Figure 2, `Magneto` is structured in two phases: *candidate retrieval* selects a subset of the possible matches; and reranker, which ranks the candidates to make it easier for users to examine and select matches. To address *Challenge 1*, `Magneto` leverages LLMs to automate SLMs fine-tuning. Instead of relying on manually created training data and structure-based augmentation (e.g., row shuffling and sampling [27, 31]), we use LLMs to derive data. Using LLMs, we can add syntactic diversity and capture different representations for column names and their values. `Magneto` addresses *Challenge 2* by using cheaper SLM-based methods for finding candidates thereby reducing the number of matches that need to be checked by more costly LLM-based rerankers. To ensure that all necessary details are included in the prompt while staying within the context window limit, we need to create representative samples of the columns in candidate matches to be ranked by the LLM. In addition, since LLMs are designed for textual data, we must serialize the column content. Selecting the right serialization strategy is still an open research problem that has attracted substantial attention [23, 32]. We explore different alternatives for both sampling and serialization.

Our goal with `Magneto` is to achieve high accuracy for schema matching at a low cost. Figure 1 illustrates the trade-offs between runtime and accuracy for existing schema matching algorithms and different variants of `Magneto`. Note that traditional methods tend to cluster at relatively high accuracy with long runtimes or relatively low runtimes with reduced accuracy. In contrast, `Magneto` variants strike a balance across both dimensions and have the advantage of requiring no training data.

We conduct an extensive experimental study of `Magneto` on various datasets from two benchmarks. The first is a new benchmark, that we developed in collaboration with domain experts, which includes real biomedical datasets. Our experiments show that many existing schema matching solutions struggle with this benchmark, particularly given the complexity and syntactic variability present in its datasets. In contrast, `Magneto` demonstrates superior accuracy and generally faster runtimes. We also evaluate `Magneto` on the Valentine benchmark [48] and observe that it also performs well for the Valentine datasets which cover a variety of domains.

Contributions. Our main contributions are summarized as follows:

- We introduce the `Magneto` framework that effectively combines SLMs and LLMs and allows the creation of schema matching strategies that attain a balance between accuracy and runtime for diverse data and tasks (Section 2).
- We propose an LLM-powered method to generate training data for fine-tuning SLMs for schema matching tasks and a contrastive learning pipeline using triplet loss and online triplet mining to enhance column embedding distinction (Section 3).
- Unlike previous approaches that use LLMs for schema matching, we show that reranking candidate matches with LLMs, derived from less costly methods, achieves high accuracy at a lower cost. (Section 4).
- In collaboration with biomedical researchers, we created a new benchmark that represents a real schema matching effort [52],

Progressive Entity Matching: A Design Space Exploration

JAKUB MACIEJEWSKI, National and Kapodistrian University of Athens, Greece
KONSTANTINOS NIKOLETOS, National and Kapodistrian University of Athens, Greece
GEORGE PAPADAKIS, National and Kapodistrian University of Athens, Greece
YANNIS VELEGRAKIS, University of Trento & Utrecht University, The Netherlands

Entity Resolution (ER) is typically implemented as a batch task that processes all available data before identifying duplicate records. However, applications with time or computational constraints, e.g., those running in the cloud, require a progressive approach that produces results in a pay-as-you-go fashion. Numerous algorithms have been proposed for Progressive ER in the literature. In this work, we propose a novel framework for Progressive Entity Matching that organizes relevant techniques into four consecutive steps: (i) filtering, which reduces the search space to the most likely candidate matches, (ii) weighting, which associates every pair of candidate matches with a similarity score, (iii) scheduling, which prioritizes the execution of the candidate matches so that the real duplicates precede the non-matching pairs, and (iv) matching, which applies a complex, matching function to the pairs in the order defined by the previous step. We associate each step with existing and novel techniques, illustrating that our framework overall generates a superset of the main existing works in the field. We select the most representative combinations resulting from our framework and fine-tune them over 10 established datasets for Record Linkage and 8 for Deduplication, with our results indicating that our taxonomy yields a wide range of high performing progressive techniques both in terms of effectiveness and time efficiency.

CCS Concepts: • **Information systems → Entity resolution**.

Additional Key Words and Phrases: Progressive Entity Resolution, Nearest Neighbor Search, Blocking, Sorted Neighborhood

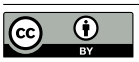
ACM Reference Format:
Jakub Maciejewski, Konstantinos Nikoletos, George Papadakis, and Yannis Velegrakis. 2025. Progressive Entity Matching: A Design Space Exploration. *Proc. ACM Manag. Data* 3, 1 (SIGMOD), Article 65 (February 2025), 25 pages. https://doi.org/10.1145/3709715

1 Introduction

Entity Resolution (ER), often also referred to as Record Linkage (a.k.a. Clean-clean ER) or Deduplication (a.k.a. Dirty ER), is a fundamental task in data management [24]. It deals with the challenge of identifying and linking data structures, typically referred to as *entity profiles*, that represent the same real-world object [7] The linked structures are referred to as duplicates. Detecting the duplicates is crucial for boosting the performance of a wide range of data management tasks, from recommendation to question answering.

An ER application in order to be successful, needs to overcome a series of challenges [3, 7, 13]. The first is *Volume*. ER algorithms should scale to thousands or even millions of entity profiles. This isn't straightforward, due to the the inherently quadratic nature of ER, which is why it is

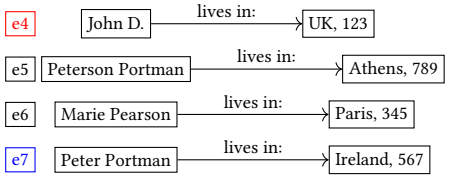
Authors' Contact Information: Jakub Maciejewski, National and Kapodistrian University of Athens, Greece, sdi1700080@di.uoa.gr; Konstantinos Nikoletos, National and Kapodistrian University of Athens, Greece, k.nikoletos@di.uoa.gr; George Papadakis, National and Kapodistrian University of Athens, Greece, gpapadis@di.uoa.gr; Yannis Velegrakis, University of Trento & Utrecht University, The Netherlands, i.velegrakis@uu.nl.



This work is licensed under a Creative Commons Attribution 4.0 International License.
© 2025 Copyright held by the owner/author(s).
ACM 2836-6573/2025/2-ART65
https://doi.org/10.1145/3709715

id	First name	Last name	City	Zip Code
e1	John	Doe	London	123
e2	Peter	Portman	Dublin	567
e3	Donald	Hirsh	New Jersey	486

(a) Data source D_1



(b) Data source D_2

(e1, e4), (e2, e7), (e2, e5), ..., (e3, e6)

(c) Ordered set of candidate pairs

Fig. 1. Two clean data sources and the candidate pairs generated by a Progressive Entity Matching approach.

addressed through filtering [32]. The second is *Variety*. ER should seamlessly apply to data sources that vary in format, schema and structure. Yet, most ER approaches are crafted either for structured, e.g., relational data or semi-structured, e.g., RDF [29]. The third is *Velocity*. Most ER solutions operate in an offline mode, i.e., they produce results only after having processed all the input data. Unfortunately, there are cases where there are restrictions and cost in computational or temporal resources. An example is the cloud environment. In these cases, partial results are required within a specific time frame [9–11, 45].

To address these challenges, we focus on *Schema-agnostic Progressive Entity Matching*. Velocity is addressed by the progressive functionality, which yields results before processing all input data through a pay-as-you-go functionality. Volume is addressed by Filtering, which restricts the computational cost to the most similar entity profiles, disregarding those dissimilar. Variety is addressed by the schema-agnostic functionality, which represents every entity profile through a concatenation of all attribute values, regardless the respective attribute names.

Consider Figure 1 that illustrates a Record Linkage case, i.e., a case of two data sources D_1 and D_2, each one being duplicate-free, meaning that it contains no matching entity profiles, but there are duplicates across the sources. For instance, e_1 matches to e_4 and e_2 to e_7. The two sources need to be merged, and to do so, the duplicates need to be detected. Furthermore, the two data sources exhibit high variety, varying both in format (structured vs semi-structured) and in schema. Typically, an ER process consists of three steps [3, 13]: Filtering, Entity Matching and Clustering. Yet, due to the small size of this example, Filtering is omitted, while Clustering lies out of the scope of our work. Therefore, we exclusively focus on Entity Matching, where a typical batch algorithm considers all pairs of entities, returning $e_1 \equiv e_4$ and $e_2 \equiv e_7$ after processing the entire data sources, i.e., after examining 12 pairs. In contrast, Progressive Entity Matching defines a processing order, as in Figure 1(c), so as to promote the most likely matches and detect most of them even if the processing is terminated prematurely.

Most existing progressive methods are independent of matching decisions, defining a *static* processing order [34, 45, 54]. Yet, they have some limitations: (i) They disregard recent advances in Filtering: they exclusively consider (meta-)blocking as a pre-processing step for addressing volume, but recent studies show that nearest neighbor search achieves a much higher performance [25, 35, 50]. (ii) They disregard recent advances in natural language processing and the dominance of the semantic entity representations that stem from pre-trained language models in both ER tasks, i.e., blocking and matching. Instead, they exclusively rely on the syntactic representation of entity profiles [25, 49, 50, 52, 56]. (iii) There is no generic framework that unifies the traditional, syntactic- and blocking-based approaches with the latest, semantic-based ones that leverage nearest neighbor search.

To address these shortcomings, we propose a novel framework for Progressive Entity Matching that is generic enough to cover any type of entity representation and of Filtering methods. Our goal is not to combine the outcome of different techniques (e.g., the syntactic- and the semantic-based),

ReCG: Bottom-Up JSON Schema Discovery Using a Repetitive Cluster-and-Generalize Framework

Joohyung Yun
POSTECH
Pohang, Republic of Korea
jhyun@dblab.postch.ac.kr

Byungchul Tak*
Kyungpook National University
Daegu, Republic of Korea
bctak@knu.ac.kr

Wook-Shin Han*
Graduate School of AI
POSTECH, Republic of Korea
wshan@dblab.postech.ac.kr

ABSTRACT

The schemalessness, one of the major advantages of JSON representation format, comes with high penalties in querying and operations by denying various critical functions such as query optimizations, indexing, or data verification. There have been continuous efforts to develop an accurate JSON schema discovery algorithm from a bag of JSON documents. Unfortunately, existing schema discovery techniques, being top-down algorithms, face challenges from the lack of visibility into children nodes of JSON tree. With absence of the information about lower-level JSON elements, top-down algorithms need to employ assumptions and heuristics to decide the schema type of nodes. However, such static decisions are often violated in datasets which causes top-down algorithms to perform poorly. To overcome this, we propose an algorithm, called ReCG, that processes JSON documents in a bottom-up manner. It builds up schemas from leaf elements upward in the JSON document tree and, thus, can make more informed decisions of the schema node types. In addition, we adopt MDL (Minimum Description Length) principles systematically while building up the schemas to choose among candidate schemas the most concise yet accurate one with well-balanced generality. Evaluations show that our technique improves the recall and precision of found schemas by as high as 47%, resulting in 46% better F1 score while also performing 2.11× faster on average against the state-of-the-art.

PVLDB Reference Format:
Joohyung Yun, Byungchul Tak, and Wook-Shin Han. ReCG: Bottom-Up JSON Schema Discovery Using a Repetitive Cluster-and-Generalize Framework. PVLDB, 17(11): 3538 - 3550, 2024.
doi:10.14778/3681954.3682019

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/postechdblab/VLDB2024_ReCG.

1 INTRODUCTION

JSON (JavaScript Object Notation) is a widely used data representation format that has become de-facto standard for RESTful Web API's data exchanges [2, 23, 30, 44] and big data analytics [5, 10, 16, 27, 31, 40, 41]. Accordingly, efficient processing of massive volumes of JSON data and querying has become critical for services and operations. However, the efficiency of these operations is often hindered by the lack of schemas for the JSON documents at hand. Although the lack of strict schema enforcement of JSON format offers flexibility and rapid deployability, it instead exacerbates the data management costs by making it difficult to apply various operations [42]. Schemas are essential for many important use cases such as JSON validation [3, 34], JSON parsing acceleration [27, 31], migration of two data sources with different schemas [43], query formulation [15, 48], query optimizations [7, 9, 18, 33], fine-grained access control [42], JSON-to-relational data translation [13, 25, 46], and query answering [47]. Despite these practical benefits, JSON schemas are often nonexistent or unavailable to the users and, thus, have to be derived from a given set of JSON documents.

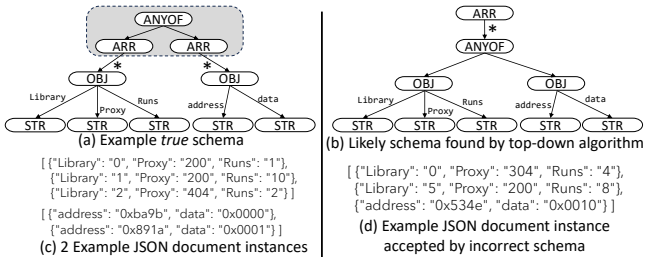


Figure 1: Illustration of top-down approach's limitation. It may lead to incorrect results (b) due to lack of visibility.

Due to the importance of JSON schemas, the JSON schema discovery (JSD) problem and topics related to the JSON schema formalization have been investigated [3, 4, 6, 11, 16, 21, 34, 38, 42]. Despite many efforts, deriving correct JSON schemas from JSON documents remains difficult due to several reasons. First, the ground truth schema set we want to derive may not exist in the first place for the given JSON documents. It is not uncommon that JSON documents are created without explicit schemas defined in advance. Users of JSON data prefer to create, load, and use the data quickly without the burden and delay from the schema construction task [32, 39]. The lack of a predefined schema also implies that the representation of objects in JSON can be inconsistent and has a high degree of variability [24]. Second, the derivation of a huge number of *correct* and *valid* schema sets is possible for a given JSON document set, but with varying degrees of generality. If the derived schemas are too general, the data validation becomes ineffective [38]. Moreover, the specific details of the schemas have to be eventually encoded into the query which increases the query processing overhead during the execution. On the other hand, if too specific, the size of the schema set increases and the schema handling costs rise significantly. A working solution to JSD problem should be able to find the schema with the right balance between representational simplicity and details within the vast search space of valid schemas.

*Corresponding authors
This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 17, No. 11 ISSN 2150-8097.
doi:10.14778/3681954.3682019

A JSON schema can be viewed intuitively as having a tree-like hierarchical structure as shown in Figure 1 (a). The intermediate nodes in this tree structure are either objects or arrays that recursively hold other objects, arrays, or primitive types. The goal of the JSON schema discovery (JSD) problem is to learn this template structure from a given bag of JSON documents. Two example JSON documents are shown in Figure 1 (c). The top-down style of JSON schema discovery, adopted by the state-of-the-art algorithms [4, 6, 38], starts processing from the root and it proceeds down to the subsequent child levels repeatedly. However, this top-down approach faces the following nontrivial challenges. When a new node is encountered, it can be difficult to determine the correct node types (e.g., `ARR`, `OBJ`, `ANYOF`) since it has to be done without the knowledge of yet unseen lower-level nodes. This inevitably leads to the use of heuristics to make the best-effort decision of what the true type of the current node is. For example, at the shaded region of Figure 1 (a), it illustrates one possible case where the top-down algorithm failed to differentiate two types of objects and, thus, comes up with the incorrect (and overly general) schema as shown in (b) which accepts a mixture of objects. In addition, existing algorithms make a decision of node types rigidly even when there are other possible candidates at each decision point. This leaves little chance to rectify incorrect decisions made at the upper level when the processing reaches the descendant nodes where decision errors become evident.

To overcome the limitations of top-down approaches, we have developed a methodology, called `ReCG`, that employs three techniques. *First, we systematically explore candidate JSON schema sets comprehensively by incrementally generalizing them from specific schemas to the most general ones.* As Figure 2 illustrates, there can be schema sets with varying generality for the same JSON document sets at some point during the JSON schema discovery. We do not attempt to guess the correct one during processing as any choice has the potential to grow to be the true schema when schema discovery is completed. *Second, we employ the MDL (Minimum Description Length) metric to our JSON schema discovery problem as a metric that guides us to select the most promising candidate schema set during the navigation of the search space.* The MDL metric, used successfully in `XTRACT` [22] for computing DTD's information size, can be used here to compute the number of bits required to represent the schema and the data (JSON document) instances. We demonstrate in our evaluation its validity as a metric for assessing the goodness of JSON schema set. *Third, we design a novel bottom-up style JSON schema discovery algorithm* that departs from the existing approaches. We argue that the bottom-up style of processing is inherently better suited to the JSON schema discovery problem. At any intermediate node, the determination of a node type is better informed since the knowledge of descendant nodes is already acquired. Any two nodes of identical type can actually be disparate when their children and descendants are taken into account. By using this bottom-up style approach, we can expect to reduce the uncertainty in making decisions and the reliance on heuristics.

We have designed and implemented the prototype of our technique and evaluated it using 20 real-world JSON datasets. Our evaluation showed that `ReCG` delivered the 42.57 ~ 47.80% improved F1 score compared to the state-of-the-art technique. We observed this was due to 39.36 ~ 46.84% improvements in recall and 15.55 ~ 23.33% improvements in precision. In terms of the algorithm running time, `ReCG` outperformed the state-of-the-art by 2.11×.

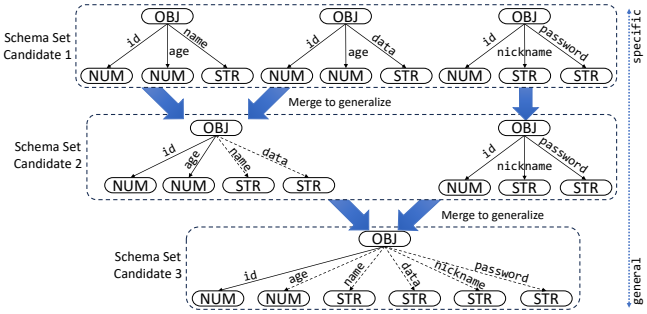


Figure 2: Schema sets with varying generality. More general schema sets are created by merging more specific sets.

J	::=	$P \mid O \mid A$	
P	::=	`null` | `true` | `false` | $n \mid s$	$(n \in \textbf{Number}, s \in \textbf{String})$
O	::=	$\{\ k_1 : J_1, \ldots, k_n : J_n\ \}$	$(n \geq 0, i \neq j \implies k_i \neq k_j)$
A	::=	$[\ J_1, \ldots, J_n\ ]$	$(n \geq 0)$

Figure 3: Grammar of JSON documents

We make the following contributions. First, we design a novel bottom-up algorithm, called `ReCG` for the JSON schema discovery problem that addresses major issues of existing algorithms. Accompanied by a technique for systematic search of schema sets by the specificity and the metric for assessing the goodness, our technique exhibits much-improved precision and recall against competing algorithms. Second, we provide supporting evidence that demonstrates the effectiveness of our technique using real-world data sets. Third, we make a prototype implementation of our technique as well as the dataset publicly available for the research community.

2 BACKGROUND

2.1 Definition

2.1.1 JSON document. We adopt the definition of JSON document introduced in the work by Baazizi et al. [6]. JSON documents are a set of strings that are derived from the grammar described in Figure 3. The derivation starts from a non-terminal J. It transitions to either P (primitive JSON document), O (object), or A (array). The non-terminal P further transitions to one of the following:

- *Number* type JSON document: C-like number. The set of these numbers is denoted as **Number**.
- *String* type JSON document: double-quoted sequence of zero or more Unicode characters. The set of such double-quoted sequences is denoted as **String**.
- *Boolean* type JSON document: `true` or `false`
- *Null* type JSON document: `null`

The non-terminal O is derived to an object, which is an unordered set of (key, JSON document) pairs. An object begins with '`{`' and ends with '`}`', and the pairs are separated by '`,`'. We denote the pairs within an object as *key-value pairs*. A is derived to an array, which is an ordered sequence of JSON documents. We call the JSON documents within the sequence of array as *elements*. An array begins with '`[`' and '`]`', with elements separated by '`,`'. The JSON document can be recursively defined. That is, JSON documents derived from J can recursively occur within objects and arrays.

SANTOS: Relationship-based Semantic Table Union Search

AAMOD KHATIWADA*, Northeastern University, USA
GRACE FAN*, Northeastern University, USA
ROEE SHRAGA, Northeastern University, USA
ZIXUAN CHEN, Northeastern University, USA
WOLFGANG GATTERBAUER, Northeastern University, USA
RENÉE J. MILLER, Northeastern University, USA
MIREK RIEDEWALD, Northeastern University, USA

Existing techniques for unionable table search define unionability using metadata (tables must have the same or similar schemas) or column-based metrics (for example, the values in a table should be drawn from the same domain). In this work, we introduce the use of semantic relationships between pairs of columns in a table to improve the accuracy of the union search. Consequently, we introduce a new notion of unionability that considers relationships between columns, together with the semantics of columns, in a principled way. To do so, we present two new methods to discover the semantic relationships between pairs of columns. The first uses an existing knowledge base (KB), and the second (which we call a "synthesized KB") uses knowledge from the data lake itself. We adopt an existing Table Union Search benchmark and present new (open) benchmarks that represent small and large real data lakes. We show that our new unionability search algorithm, called SANTOS, outperforms a state-of-the-art union search that uses a wide variety of column-based semantics, including word embeddings and regular expressions. We show empirically that our synthesized KB improves the accuracy of union search by representing relationship semantics that may not be contained in an available KB. This result hints at a promising future of creating synthesized KBs from data lakes with limited KB coverage and using them for union search.

CCS Concepts: • **Information systems → Information integration**.

Additional Key Words and Phrases: Data lakes, Table Discovery, Knowledge Graphs

ACM Reference Format:
Aamod Khatiwada*, Grace Fan*, Roee Shraga, Zixuan Chen, Wolfgang Gatterbauer, Renée J. Miller, and Mirek Riedewald. 2023. SANTOS: Relationship-based Semantic Table Union Search. *Proc. ACM Manag. Data* 1, 1, Article 9 (May 2023), 25 pages. https://doi.org/10.1145/3588689

1 INTRODUCTION

Table search is of growing practical relevance [7, 23, 30, 49]. It allows data scientists to find datasets needed for analysis or training of machine learning algorithms [14, 33]. Google's dataset search, for example, uses keyword search over metadata which Google incentivizes data owners to use in

* Khatiwada and Fan are lead authors.

Authors' addresses: Aamod Khatiwada*, Northeastern University, USA, khatiwada.a@northeastern.edu; Grace Fan*, Northeastern University, USA, fan.gr@northeastern.edu; Roee Shraga, Northeastern University, USA, r.shraga@northeastern.edu; Zixuan Chen, Northeastern University, USA, chen.zixu@northeastern.edu; Wolfgang Gatterbauer, Northeastern University, USA, w.gatterbauer@northeastern.edu; Renée J. Miller, Northeastern University, USA, miller@northeastern.edu; Mirek Riedewald, Northeastern University, USA, m.riedewald@northeastern.edu.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
2836-6573/2023/5-ART9 $15.00
https://doi.org/10.1145/3588689

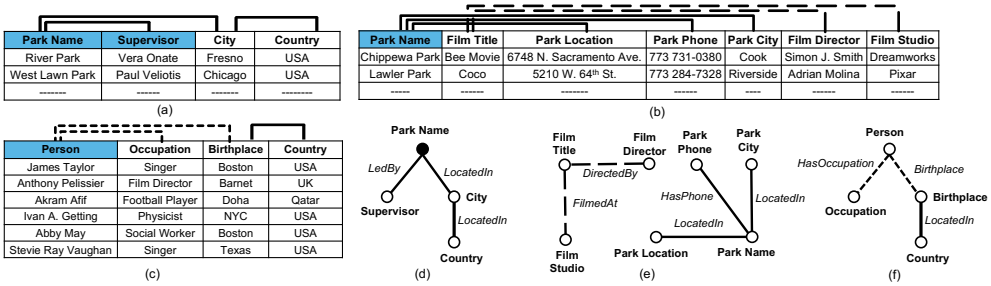


Fig. 1. (a) A table about parks. Relationships between `Park Name` and `Supervisor` (`ledBy`) and between `Park Name` and `City` (`locatedIn`) are found and depicted by the solid lines above the table. We have also found a relationship between `Country` and `City` (`locatedIn`). (b) A table about parks and films shown in the park. Some of the relationships we found are shown by solid and dashed lines above the table. (c) A table about famous people. (d) A semantic graph for Table (a) with the relationships found related to the root `Park Name`. (e), (f) Semantic graphs for Tables (b), (c), respectively.

a standard way [4]. However, when dealing with enterprise data or open data, keyword search via textual queries or over metadata is challenging because metadata may be missing, inconsistent, or incomplete [1, 10, 32, 47]. Hence, table search for those domains must rely on the data available within given tables instead of relying on well-curated metadata [7, 23].

In this work, we focus on *table union search*, which is the problem of finding tables that can union with a query table and possibly extend it with more rows. Prior work by Nargesian et al. [33] considers two tables to be *unionable* if "they share attributes from the same domain". We believe that this is a necessary, but not sufficient condition. In particular, we argue that the relationships modeled by pairs of attributes in a table should share a common semantic meaning. We illustrate this with an example.

Example 1. *Consider Tables (a), (b) and (c) in Fig. 1 describing (a) parks, (b) films shown in different parks, and (c) the birth places of famous people, respectively. Suppose the user supplies Query Table (a) and Tables (b), (c) reside in a data lake. The user wants to find data lake tables that union with the Query Table (a). Typically, union search has been defined to permit either the full query table or a projection of it to union with the full data lake table or a projection of it [3]. Now, if we only consider attribute unionability, Table (c) may be considered the most unionable table with three unionable attributes (*`Person`, `Birthplace`, `Country`*) having high attribute unionability scores with the attributes (*`Supervisor`, `City` *and* `Country`*) in the query table. This may be true if we are considering data values [33] or if we consider the attribute names and values [3, 21, 24]. However, if we consider the relationships in the tables, then we would notice that in Query Table (a),* `City` *is where the* `Park` *is located (and the person in* `Supervisor` *works), but in Table (c),* `Birthplace` *contains cities where the people in the* `Person` *column were born. Deciding unionability based only on attribute unionability, without considering the semantics of the relationships between attributes, can lead to false positives. Even worse, it can lead to the omission of good answers like Table (b) which has information about parks and the city in which they are located, but might be considered less unionable than (c) based solely on column semantics.*

We present a new definition of unionability based on both the attribute semantics and the *relationship semantics between attributes*. Relationships have been used to a limited extent in web table search. Their tables are often assumed to be "entity tables" (i.e., all attributes are properties of a known common entity represented by a single attribute) [7, 8, 28] or they use a subject column and a single relationship from that column to search for tables [43]. However, we do not assume

Searching Data Lakes for Nested and Joined Data

Yi Zhang*
AWS AI Labs
imyi@amazon.com

Peter Baile Chen*
MIT
peterbc@mit.edu

Zachary G. Ives
University of Pennsylvania
zives@cis.upenn.edu

ABSTRACT

Exploratory data science is driving new platforms that assist data scientists with everyday tasks, such as integration and wrangling, to assemble training datasets. Such tools take scientists' work-in-progress data as a *search object* (table or JSON) and find relevant supplementary data from an organizational *data lake*, which can be unioned or joined with the current data. Existing data lake search tools find *single*, relational tables to match or join with a search object. Yet many data science applications revolve around hierarchical data, which can only be matched by creating views that simultaneously *join and transform several* tables in the data lake. In this paper, we extend the Juneau data lake search system [46] for this broader class of matches *at scale*. Our contribution is a *general* framework for efficiently merging ranked results to match hierarchical data, leveraging novel techniques for indexing and sketching, and incorporating existing single-table search techniques and ranking functions. We experimentally validate our methods' benefits and broad applicability using real data from data science computational notebooks. Our results indicate that, with different ranking functions, our approach can return the optimal set of views up to 4.8x faster and 43% more related compared to heuristics, and increase the data domain coverage by up to 28%. In a case study to show the utility of our results to data science downstream tasks, we reduce regression error by up to 6.6%, and improve classification accuracy by up to 19.5%.

PVLDB Reference Format:
Yi Zhang, Peter Baile Chen, and Zachary G. Ives. Searching Data Lakes for Nested and Joined Data. PVLDB, 17(11): 3346 - 3359, 2024.
doi:10.14778/3681954.3682005

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/juneau-project/juneau.

1 INTRODUCTION

Data scientists are increasingly using tools that assist with data exploration [19], machine learning model training, and workflow definition tasks. These scientists search over tables and code from public repositories such as GitHub [44], as well as enterprise or public data lakes [34, 46–48] to find supplemental data that can augment their working dataset(s). In the same spirit as Code Llama [39] or GitHub CoPilot [18] for software engineering, such tools assist programmers with data engineering tasks such as discovery and data augmentation [34, 46], as well as data wrangling and cleaning steps [6, 46]. Such tools complement large language models like GPT, Claude, and Gemini. Unlike LLMs that make suggestions based on their parametric knowledge, search tools do not "hallucinate" results. Moreover, they can incorporate policies for domain-specific data types (e.g., gene sequence similarity) or for preferring one source over another (e.g., based on freshness).

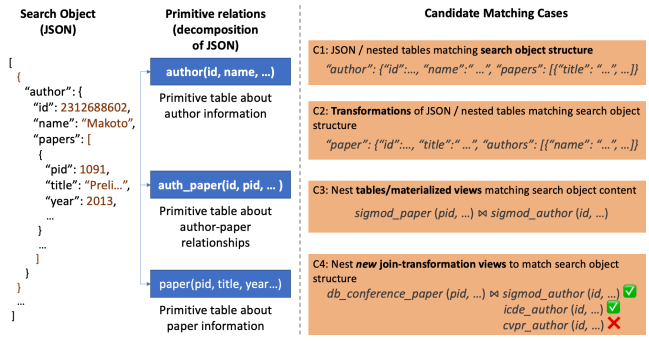


Figure 1: Schema-compliant matches to a JSON search object may come directly or through joins followed by nesting transformations.

The core of such tools is a *search system over data lakes* [13, 32, 34]: Here, the user invokes a query via a *search object* (in existing systems, a table) and the system queries data lake resources for (1) union-compatible tables [13, 33, 46, 48] that can extend training or test datasets with new tuples, (2) joinable tables [16, 46, 47] that integrate data or add machine learning features, or (3) common computational steps found as used in prior workflows [22, 24, 44, 46]. In contrast to web table search [5, 37], the critical challenges of developing such tools lie in understanding the (semantic) compatibility of the schema and detecting the primary-key overlap with the query table [13, 46] . There are two broad classes of existing systems: union-compatible search strives to find tables that *match the search object's schema* while providing novel rows; join-compatible search tends to be non-goal-directed, opportunistically finding tables that appear to join with the existing search table. Recent work [26] shows how to use *full disjunction* to knit the results together to form one output schema.

However, the paradigm of single-table matches to a search object is overly restrictive. Data scientists frequently work with JSON files, nested Dataframes, and composite tabular results (views) that join, pivot, and explode imported tables. Here, to provide the best search results — *the search system must combine subsets of indexed data into new hierarchical or joined data matching the search object structure.* New combinations of indexed tables (i.e., automatically discovered views) better leverage existing data and provide additional insights.

*This work was done while the authors were at the University of Pennsylvania.
This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 17, No. 11 ISSN 2150-8097.
doi:10.14778/3681954.3682005

This requires that the data lake system can *not only find 1NF tables* but *explore ways of creating query transformations over combinations of indexed tables to match the query.*

Example 1.1. Snapshots of the popular DBLP publications database (dblp.org) have been exported for use in data science tasks, e.g., in notebooks shared on kaggle.com. These snapshots are often subsets of the entire database, taken at different points in time, and their representation may range from flat CSVs[1] to nested JSON[2]. A data lake could also have complementary publication data (e.g., venues not indexed in DBLP) extracted from conference pages or the ACM Digital Library. Now (see Figure 1), if a data scientist takes a JSON snapshot of DBLP and searches for similar data to train a classifier that recognizes CS papers, we would like to return *all* relevant author-publication data (converted to the appropriate structure), including (C1) objects that directly match the nested schema, authors with nested papers; (C2) "pivoted" JSON, in a different structure with semantically compatible content (e.g., papers with nested authors); (C3) nesting of flat tables or views representing the same concepts as the JSON; (C4) nesting of join queries over 1NF relations, matching the JSON schema.

In a similar fashion, a data scientist may take portions of the above data and create an intermediate table for machine learning training, which joins data from different levels of the DBLP hierarchy. At this point, they may want to find matches, which, while "flat," can be matched by joining 1NF data (which, in turn, could have been gathered by decomposing other JSON structures). Unlike prior join-compatible search approaches, this task is *goal-directed* in terms of what joins; unlike prior union-compatible search approaches, this task requires more than single-table matches.

Conventional table search within a data lake introduces two key challenges: identifying semantic and value overlap across thousands to millions of tables, which has generally been addressed using sketch and embedding techniques [16, 45, 47]; and efficiently ranking the results, especially if there are multiple ranking measures [46]. Extending these ideas to heterogeneous and hierarchical *compositions* of tables and JSON/XML results in three additional critical challenges:

(1) Matches to the search object may be *partial*, requiring joins to produce a complete search result. In our example, the JSON can be normalized into three "primitive" relations: *author*, *paper*, *auth_paper*. Individual data objects might contain the information from all these (C1-3 in Figure 1); or might require us to find *other* tables to join, to form a complete match (C4). Efficiently returning the top-k results, when each result has an irregular number of joins, explodes the search space.
(2) Simply querying the primitive relations in top-k order, then joining them, does not produce the top-k answers: intuitively, high-scoring matches to papers may not actually join with high-scoring matches to authors (as per database papers with CVPR authors in Figure 1.) Therefore, the ranking function should measure the compatibility of a composition of primitive relations, and the score should be independent of the order of evaluation.
(3) Automatically and efficiently identifying which relations should join (without introducing large numbers of false positives) is nontrivial – this is aided tremendously by identifying fields that are good keys and also highly indicative of semantic content, e.g., DOIs, SSNs, etc.

[1]https://www.kaggle.com/jakboss/chunk-of-dblp-dataset
[2]https://www.kaggle.com/mathurinache/citation-network-dataset

Searching data lakes for nested and joined data requires novel indexing and pruning strategies. Broadly, as in Figure 1, the problem requires that we decompose the nesting structure of our hierarchical data and find matches to each nesting level (columns in 1NF tables, or in decompositions of other nested tables); then we return a ranked set of queries (views) over these matches as the answers. Note that among the search cases we have discussed, C4 is particularly challenging, as queries need to be created first. To address these problems, we must innovate in how we index content in tables, and perform top-k computations, with the assistance of the indices, across multiple tables while joining them.

A significant body of empirical work has been developed on finding the best features for ranking individual tables considering schema overlap, table overlap, and joinability (see Section 2). In this paper, we develop a general query system that can "plug in" existing individual table ranking strategies (with different degrees of benefits depending on the specific setup), and our focus is on scaling up running times of returning the (new) views over these tables. To facilitate such a query system, we address the following challenges.

Enumerating plans to join and nest data decomposed into 1NF, for a broad class of ranking functions. We *compose* partial matches (1NF primitive tables) from our data lake using nesting and join operations to match search object structure — favoring joins of results that originally came from the same sources. However, our system also considers cross-source joins, which are critical to applications in Linked Open Data and scientific settings with private data referencing entries in public databases.

Incremental maintenance of sketches and profiles. Existing data lake search systems developed *in-memory* sketches and data profiles. To support data compositions in millions of tables, these must be persisted and incrementally maintained.

Automatic index selection. To scale up the exploration of potential joins to produce a match, we develop solutions to determine combinations of fields to index across the data lake, in the form of automatic *data profile* selection.

To sum up, this paper makes the following contributions:

- A new class of data lake search tools over joined and nested data, whose task is to generate unions of nest-join queries over data lake tables to match a search object, with *customizable ranking functions or modules.*
- Identification of key requirements for ranking functions over candidate *views (over multiple tables)*, enabling an efficient *correlated top-k matching algorithm* to rank them.
- *Automatic* selection techniques for *profiling and sketching* millions of tables.
- Experimental validation of the viability and the generality of our methods, over a diverse benchmark suite of real data and notebooks, using state-of-the-art table scoring functions.

Semantics-aware Dataset Discovery from Data Lakes with Contextualized Column-based Representation Learning

Grace Fan
Northeastern University
United States
fan.gr@northeastern.edu

Jin Wang
Megagon Labs
United States
jin@megagon.ai

Yuliang Li
Megagon Labs
United States
yuliang@megagon.ai

Dan Zhang
Megagon Labs
United States
dan_z@megagon.ai

Renée J. Miller
Northeastern University
United States
miller@northeastern.edu

ABSTRACT

Dataset discovery from data lakes is essential in many real application scenarios. In this paper, we propose Starmie, an end-to-end framework for dataset discovery from data lakes (with table union search as the main use case). Our proposed framework features a contrastive learning method to train column encoders from pre-trained language models in a fully unsupervised manner. The column encoder of Starmie captures the rich contextual semantic information within tables by leveraging a contrastive multi-column pre-training strategy. We utilize the cosine similarity between column embedding vectors as the column unionability score and propose a filter-and-verification framework that allows exploring a variety of design choices to compute the unionability score between two tables accordingly. Empirical results on real table benchmarks show that Starmie outperforms the best-known solutions in the effectiveness of table union search by 6.8 in MAP and recall. Moreover, Starmie is the first to employ the HNSW (Hierarchical Navigable Small World) index to accelerate query processing of table union search which provides a 3,000X performance gain over the linear scan baseline and a 400X performance gain over an LSH index (the state-of-the-art solution for data lake indexing).

PVLDB Reference Format:
Grace Fan, Jin Wang, Yuliang Li, Dan Zhang, and Renée J. Miller. Semantics-aware Dataset Discovery from Data Lakes with Contextualized Column-based Representation Learning. PVLDB, 16(7): 1726 - 1739, 2023.
doi:10.14778/3587136.3587146

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/megagonlabs/starmie.

1 INTRODUCTION

The growing number of open datasets from governments, academic institutions, and companies have brought new opportunities for innovation, economic growth, and societal benefits. To integrate and analyze such datasets, researchers in both academia and industry have built a number of dataset search engines to support the application of dataset discovery [3, 7, 17, 19, 33, 40, 45]. One popular example is Google's dataset search [3] which provides keyword search on the metadata. However, for open datasets, simple keyword search might suffer from data quality issues of incomplete and inconsistent metadata across different datasets and publishers [1, 16, 41, 42]. Thus it is essential to support *table search* over open datasets, and more generally data lake tables (including private enterprise data lakes), to boost dataset discovery applications, such as finding related tables, domain discovery, and column clustering.

Finding related tables from data lakes [11, 25, 39, 46, 57] has a wide spectrum of real application scenarios. There are two sub-tasks of finding related tables, namely table union search and joinable table search. In this paper, we mainly focus on the problem of *table union search*, which has been recognized as a crucial task in dataset discovery from data lakes [2, 24, 39, 41, 42, 57, 61]. Given a query table and a collection of data lake tables, table union search aims to find all tables that are unionable with the query table. To determine whether two tables are unionable, existing solutions first identify all pairs of unionable columns from the two tables based on column representations, such as bag of tokens or bag of word embeddings. They then devise some mechanism to aggregate the column-level results to compute the table unionability score.

State-of-the-art: Early work on finding unionable tables used table clustering followed by simple syntactic measures such as the difference in column mean string length and cosine similarities to determine if two tables are unionable [4]. Table union search [42] improved on this by applying a rich collection of column representations including syntactic, semantic (leveraging ontologies), and natural language (based on word-embeddings) column representations. Two important innovations of this work were the modeling of data lake context to create an *ensemble unionability score* which models the surprisingness of a score given the score distributions within a data lake and the use of LSH indices to make table union search fast over large data lakes [42]. More recently D^3L [2] added additional column representations based on regular expression matching and SANTOS [24] added to the column representations, representations of binary relationships. In parallel to these search-based approaches, the mighty hammer of deep learning has been applied to the problem of column matching (determining the semantic type of a column) [22, 56]. Since these

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 16, No. 7 ISSN 2150-8097.
doi:10.14778/3587136.3587146

Table A:

Name	Mode of Travel	Purpose	Destination	Day	Month	Year	Expense
Philip Duffy	Air	Regional Meeting	London	10	April	2019	189.06
Jeremy Oppenheim	Taxi	Exchange Visit	Ottawa	30	Jul	2019	8.08
Mark Sedwill	Air	Evening Meal	Bristol	02	September	2019	50

Table B:

Name	Date	Destination	Purpose
Clark	23/07	France	Discuss EU
Gyimah	03/09	Belgium	Build Relations
Harrington	05/08	China	Discuss Productivity

Table C:

Bird Name	Scientific Name	Date	Location
Pine Siskin	Carduelis Pinus	2019	Ottawa
American Robin	Turdus migratorius	2019	Ottawa
Northern Flicker	Colaptes auratus	2019	London

Figure 1: An example of table union search on Open Data.

approaches are supervised, they can only be applied to finding a limited set of semantic types (78 in their experiments), and while not a general solution for unionability in data lakes, they can be used in an offline fashion to find unionable tables containing the types on which they are trained.

However, there are still plenty of opportunities to further improve the performance of table union search. One important issue is to learn sufficient contextual information between columns in tables so as to determine the unionability. This point can be illustrated in the following motivation example.

Example 1.1. Figure 1 shows an example of finding unionable tables. Given the query Table A, existing approaches first find unionable columns. In this example, the column `Destination` in Table A will be deemed more unionable with `Location` from Table C than with `Destination` from Table B. This is because the syntactic similarity score, e.g. overlap and containment Jaccard, between the two `Destination` columns is 0; while the average word embedding of cities (Table A) is also not as close to that of nations (Table B). Similarly, if an ontology is used, Table A and Table C shares the same class while the values in B are in different (though related) classes. Meanwhile, looking at the tables as a whole we observe that Table A is actually irrelevant to Table C. But as existing solutions only look at the pair of single columns when calculating column unionability score, the columns `Year/Date` and `Destination/Location` of the two tables might be wrongly aligned together. Even techniques that look at relationships [24] can be fooled by the value overlap in this relationship and determine the relationship `Year-Destination` in Table A to be unionable with `Date-Location` in Table C. This kind of mistake can be avoided by looking at a table's context, i.e. information carried by other columns within a table. Looking at the table as a whole, a method should be able to recognize that the `Year` in Table A is part of a travel date while in Table C it is the date of discovery of a bird; and `Destination` in Table A refers to the cities to which the officers are traveling; whereas `Location` in Table C is the city where a bird is found.

From the above example, we focus on the following challenges in proposing a new solution. Firstly, it is essential to learn richer semantics of columns based on natural language domain. To this end, we require a more powerful approach to learn the column representation so as to capture richer information instead of relying on simple methods like the average over bag of word embeddings utilized in previous studies [2, 14] or even the similarity of the word embedding distributions [42]. Secondly, we argue that it is crucial to utilize the contextual information within a table to learn the representation of each column, which is ignored by previous studies. Even proposals for capturing relationship semantics do not use contextual information to learn column representations [24]. Finally, due to the large volume of data lake tables, it is also a great challenge to develop a scalable and memory-efficient solution.

We propose Starmie, an end-to-end framework for dataset discovery from data lakes with table union search as the main use case. Starmie uses pre-trained language models (LMs) such as BERT [13] to obtain semantics-aware representations for columns of data lake tables. While pre-trained LMs have been shown to achieve state-of-the-art results in table understanding applications [12, 31, 47], their good performance heavily relies on high-quality labeled training data. For the problem setting of table union search [41, 42], we must come up with a fully unsupervised approach in order to apply pre-trained LMs to such applications, something not yet supported by previous studies. Starmie addresses this issue by leveraging *contrastive representation learning* [10] to learn column representations in a **self-supervised manner**. An innovation of this approach is to assume that two randomly selected columns in a data lake can be used as negative training examples. For positive examples, we propose and use novel data augmentation methods. The framework defines a learning objective that connects the same or similar columns in the representation space while separating distinct columns. As such, Starmie can apply the pre-trained representation model in downstream tasks such as table union search without requiring any labels. We also propose to combine the learning algorithm with a novel multi-column table transformer model to learn *contextualized* column embeddings that model the column semantics depending on not only the column values, but also their context within a table. While a recent study SANTOS [24] can reach a similar goal by employing a knowledge base, our proposed methods can automatically capture such contextual information from tables in an unsupervised manner without relying on any external knowledge or labels.

Based on the proposed column encoders, we use cosine similarity between column embeddings as the column unionability score and develop a bipartite matching based method to calculate the table unionability score. We propose a filter-and-verification framework that enables the use of different indexing and pruning techniques to reduce the number of computations of the expensive bipartite matching. While most previous studies employed LSH index to improve the search performance, we also make use of HNSW (Hierarchical Navigable Small World) index [36] to accelerate query processing. Experimental results show that HNSW can significantly improve the query time while only slightly reducing the MAP/recall scores. Besides table union search, we further conduct two case studies to show that Starmie can also support other dataset discovery applications such as joinable table search and column clustering. We believe these results show great promise in the use of contextualized, self-supervised embeddings for many table understanding tasks.

Our contributions can be summarized as the following.

- We propose Starmie, an end-to-end framework to support dataset discovery over data lakes with table union search as the main use case.
- We develop a contrastive learning framework to learn contextualized column representations for data lake tables without requiring labeled training instances. Starmie achieves an improvement of 6.8% in both MAP and recall compared with the best state-of-the-art method, with a MAP of 99%, a significant margin compared with previous studies.

Table Discovery in Data Lakes: State-of-the-art and Future Directions

Grace Fan
Northeastern University
fan.gr@northeastern.edu

Jin Wang
Megagon Labs
jin@megagon.ai

Yuliang Li
Megagon Labs
yuliang@megagon.ai

Renée J. Miller
Northeastern University
miller@northeastern.edu

ABSTRACT

Data discovery refers to a set of tasks that enable users and downstream applications to explore and gain insights from massive collections of data sources such as data lakes. In this tutorial, we will provide a comprehensive overview of the most recent table discovery techniques developed by the data management community. We will cover table understanding tasks such as domain discovery, table annotation, and table representation learning which help data lake systems capture semantics of tables. We will also cover techniques enabling various query-driven discovery and table exploration tasks, as well as how table discovery can support key data science applications such as machine learning and knowledge base construction. Finally, we will discuss future research directions on developing new table discovery paradigms by combining structured knowledge and dense table representations, as well as improving the efficiency of discovery using state-of-the-art indexing techniques, and more.

CCS CONCEPTS

• **Information systems → Top-k retrieval in databases**; *Mediators and data integration*; *Database design and models.*

KEYWORDS

Dataset Discovery, Data Lake, Data Integration, Unionable Tables

ACM Reference Format:
Grace Fan, Jin Wang, Yuliang Li, and Renée J. Miller. 2023. Table Discovery in Data Lakes: State-of-the-art and Future Directions. In *Companion of the 2023 International Conference on Management of Data (SIGMOD-Companion '23), June 18–23, 2023, Seattle, WA, USA.* ACM, New York, NY, USA, 7 pages. https://doi.org/10.1145/3555041.3589409

1 INTRODUCTION

We have witnessed in the last decades a drastic growth in the number of open and shared datasets coming from governments, academic institutes, and private companies. These massive collections of structured or unstructured datasets, which we refer to as *data lakes*, open up new opportunities for innovation, economic growth, and social benefits [39]. As of 2022, the size of US open data (data.gov) has reached a total of 335k datasets contributing to $3 trillion of the US economy [9]. WebDataCommons [30] has also made 233M tables extracted from the open web publicly available. Data lakes store a wide variety of open-domain knowledge from sources such as Wikipedia, news articles, and other online sources, as well as data that is private to an organization or purchased from data brokers. However, while data lakes are invaluable data sources, their large size and complexity can make it difficult for users to find and access the specific data that is required by specific downstream applications. To address this issue, the data management community has been building *data discovery systems* [3, 6, 18, 20, 34, 40, 48], with table discovery from data lakes as the primary application. Data discovery systems help users quickly and easily explore and analyze large datasets, enabling them to gain insights and knowledge that can support decision making and other activities.

This tutorial aims at providing a comprehensive overview of the most recent developments from the database community that enable the aforementioned data discovery methodologies and systems. While there are some previous studies about discovery from other data formats, such as images and JSON [36, 55], we focus on the discovery from tabular data, which is a primary data format in data lakes. We will organize our tutorial by taking an architectural view of table discovery systems depicted in Figure 1. The architecture highlights key system components (in green) and sub-tasks (in blue) that are essential building blocks of an end-to-end data discovery pipeline from raw data lake tables to end-users and downstream applications.

Firstly, we will have an overview of several table understanding tasks. Given an input data lake, table discovery systems typically enrich the raw data lake tables via automatically annotating and indexing techniques to enable better understanding of table semantics. Annotations such as semantic column types can be further used to improve the performance of downstream applications (e.g., table search). We will focus on the tasks of (1) *table annotation* [34] for generating useful meta-data about tables, (2) *domain discovery* [33, 45] for identifying novel domains beyond standard DB data types, (3) machine learning techniques for learning *embeddings* [14, 50] of data items in tables (including columns, rows, and entire tables).

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
SIGMOD-Companion '23, June 18–23, 2023, Seattle, WA, USA
© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-9507-6/23/06…$15.00
https://doi.org/10.1145/3555041.3589409

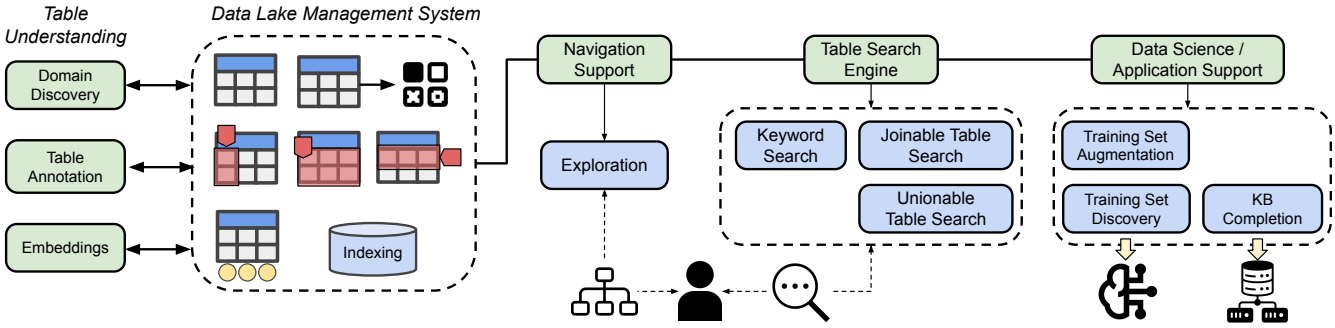


Figure 1: Overview of table discovery in data lakes.

The table discovery system serves the data lake tables to the end-users (e.g., data scientists) by enabling them to query and explore tables that are relevant to their downstream analytic tasks. This is typically achieved via *query driven discovery* [43]: given a user query, retrieve the top-k relevant tables from the data lake. Alternatively, some discovery systems support *data navigation* allowing a user to navigate through the tables in a data lake using a (discovered) organization for the tables [42] or by exploring individual tables followed by tables related through a foreign key or other relationships [18]. And query and navigation can be combined to create organizations over the set of tables retrieved by search [46] in an online manner.

For query driven discovery, we will survey the classic query types includingkeyword search or meta-data search, which often use table schema including attribute names [4, 47, 54] as supported by popular systems such as Google dataset search [3, 21]. Importantly, rather than keywords or table schema, an emerging trend in table discovery systems is to support querying data lakes using table values (e.g., columns or an entire table). Previous studies have identified two types of important search queries: joinable table search [12, 15, 16, 48, 59, 60] and unionable table search [2, 12, 17, 27, 44]. Joinable table search aims at augmenting a query table with additional attributes, while unionable table search is motivated by the need to find tables with new tuples to extend a query table. We will be covering the classic schema-based and value-based approaches, as well as the latest approaches based on knowledge graphs [27] and pre-trained language models [17] that achieve state-of-the-art performance.

We will also cover an emerging trend of performing data discovery to support a variety of data science tasks. One typical example is the creation and augmentation of training datasets. High-quality training datasets are often times a prerequisite for the success of ML models. Data lakes provide a unique opportunity for harvesting training data. Among those techniques, we will primarily introduce (1) data augmentation [8] that aims at finding new features to improve the ML models' performance and (2) training set discovery/construction [58] which focuses on constructing training data from a fixed set of tables via learning representations.

Finally, we will present challenges and future research directions for table discovery. We will cover four main aspects. First, for table understanding which is core to table discovery, we will discuss the possibility of combining two lines of already successful approaches: *knowledge* based approaches [27, 53] and *machine learning* based approaches [14, 17, 23]. We will challenge the community to find synergies between the two approaches by bringing together both rich structured knowledge from KB's and highly effective dense representations from ML. Second, we will further discuss the challenge of boosting the scalability of discovery systems, especially the issues related to indexing. We will elaborate on why indexing is still a challenging unsolved problem for data lakes and how some recently developed techniques on data structures [25], sketching [48], and vector indices [38] can potentially help. Finally, we will discuss the challenge of query-time table annotation and a potential research direction for connecting table discovery with graph mining.

2 TUTORIAL OUTLINE

In this section, we give an overview of the topics that will be covered in the tutorial. They are depicted in Figure 1 as the main components of a data discovery system (in green) and applications (in blue) that are related to table discovery.

2.1 Overview

A popular methodology for table discovery is query-driven discovery [12, 43, 60], where the user starts with a query and aims to find dataset(s) relevant to the query. The query could be either keywords or tables, while the query type could be exact match, joinable, unionable, etc. Existing studies support keyword search over the metadata of tables for users who intend to find tables relevant to a topic (Section 2.3). However, the schemas of tables in data lakes are often unreliable, either missing or incomplete often due to data being shared in primitive formats like CSV [43]. Even when metadata is available, the tables often come from a huge variety of sources meaning the metadata may be inconsistent using different vocabularies and naming conventions. Thus, two important lines of work have emerged. The first we call *Table Understanding*, which are tasks that seek to recover some of the semantics of tables. The second is data-driven table discovery based on a query table (rather than metadata), specifically joinable table search and unionable table search. Yet, data-driven table discovery still has ambiguity in the definitions, which have evolved over time. For example, early work [12] defines related tables as schema-complements and entity-complements based on a defined subject attribute that explains

Table Overlap Estimation through Graph Embeddings

FRANCESCO PUGNALONI, Hasso Plattner Institute, University of Potsdam, Germany
LUCA ZECCHINI, University of Modena and Reggio Emilia, Italy
MATTEO PAGANELLI, University of Modena and Reggio Emilia, Italy
MATTEO LISSANDRINI, University of Verona, Italy
FELIX NAUMANN, Hasso Plattner Institute, University of Potsdam, Germany
GIOVANNI SIMONINI, University of Modena and Reggio Emilia, Italy

Discovering duplicate or high-overlapping tables in table collections is a crucial task for eliminating redundant information, detecting inconsistencies in the evolution of a table across its multiple versions produced over time, and identifying related tables. Candidate duplicate or related tables to support this task can be identified via the estimation of the largest table overlap. Unfortunately, current solutions for finding it present serious scalability issues for heavy workloads: Sloth, the state-of-the-art framework for its estimation, requires more than three days of machine time for computing 100k table overlaps.

In this paper, we introduce Armadillo, an approach based on graph neural networks that learns table embeddings whose cosine similarity approximates the *overlap ratio* between tables, i.e., the ratio between the area of their largest table overlap and the area of the smaller table in the pair. We also introduce two new annotated datasets based on GitTables and a Wikipedia table corpus containing 1.32 million table pairs overall labeled with their overlap. Evaluating the performance of Armadillo on these datasets, we observed that it is able to calculate overlaps between pairs of tables several times faster than the state-of-the-art method while maintaining a good quality in approximating the exact result.

CCS Concepts: • **Information systems** → *Information retrieval*; *Information integration*; *Deduplication*.

Additional Key Words and Phrases: table representation learning, graph neural networks, data lakes, web tables, table overlap, table matching

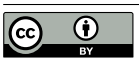
ACM Reference Format:
Francesco Pugnaloni, Luca Zecchini, Matteo Paganelli, Matteo Lissandrini, Felix Naumann, and Giovanni Simonini. 2025. Table Overlap Estimation through Graph Embeddings. *Proc. ACM Manag. Data* 3, 3 (SIGMOD), Article 228 (June 2025), 25 pages. https://doi.org/10.1145/3725365

1 Estimating table overlap

Tables are one of the most common formats used in data lakes and Web sources for organizing information that refers to a common schema [39]. For instance, a large amount of tabular data can be found in GitHub, which contains more than 10 million tables stored as CSV files [25], and Wikipedia, where more than 3 million HTML tables have been created throughout its history [2, 5].

In these large table corpora, it is frequent to find pairs of tables that share the same content across multiple cells, i.e., *overlapping* tables. This overlap, depending on its size and shape, can be a signal of table relatedness. For instance, a *vertical* overlap spanning several rows across a

Authors' Contact Information: Francesco Pugnaloni, Hasso Plattner Institute, University of Potsdam, Germany, francesco.pugnaloni@hpi.de; Luca Zecchini, University of Modena and Reggio Emilia, Italy, luca.zecchini@unimore.it; Matteo Paganelli, University of Modena and Reggio Emilia, Italy, matteo.paganelli@unimore.it; Matteo Lissandrini, University of Verona, Italy, matteo.lissandrini@univr.it; Felix Naumann, Hasso Plattner Institute, University of Potsdam, Germany, felix.naumann@hpi.de; Giovanni Simonini, University of Modena and Reggio Emilia, Italy, giovanni.simonini@unimore.it.



This work is licensed under a Creative Commons Attribution 4.0 International License.
© 2025 Copyright held by the owner/author(s).
ACM 2836-6573/2025/6-ART228
https://doi.org/10.1145/3725365

R	Monument	City	Country	Year
r_1	TV Tower	Berlin	Germany	1965-1969
r_2	Big Ben	London	England	1843-1869
r_3	St. Davids	-	England	1915-1921
r_4	Colosseum	Rome	Italy	70-80
r_5	AquaClaudia	Rome	Unknown	52
r_6	Well of Life	Zagreb	Croatia	Unknown

S	Nation	Population	City
s_1	England	8,866,180	London
s_2	Italy	2 752 908	Rome
s_3	England	1 348	St.Davids
s_4	Unknown	52	Hum
s_5	Croatia	Unknown	Rastoke
s_6	Germany	3.755.251	Berlin

Fig. 1. Pair of tables that overlap in different ways. ($\langle$"*Unknown*", "52"$\rangle$, $\langle$"*Croatia*", "*Unknown*"$\rangle$) has area four and is the largest overlap obtainable without reordering rows and columns, while ($\langle$"*Berlin*", "*Germany*"$\rangle$, $\langle$"*Rome*", "*Italy*"$\rangle$, $\langle$"*London*", "*England*"$\rangle$) is the real largest overlap, it has area six and is obtained through a permutation of rows and columns. The overlap ratio is 6/18 (largest overlap area / area of the smaller table).

few columns may denote a pair of joinable tables. Similarly, a *horizontal* overlap can be a sign of unionability. Further, an overlap covering many rows and columns might represent a case of duplicate tables, table containment, or different versions of the same table [3].

Detecting overlaps is a challenging task for traditional set-based methods for data discovery, as it requires considering the alignment of cell values in a table and the structure of the tables themselves. To fill this gap, the concept of *largest overlap*, i.e., the largest common *rectangular* subtable existing between two tables under any row and column reordering, was recently introduced [54]. We present an illustrative example of largest overlap in Figure 1.

Identifying tables for which the largest overlap exceeds a certain area and/or presents a specific shape finds application in multiple real-world scenarios, from the detection of largely overlapping tables in a data lake, e.g., for subsequent operations of change propagation [3] or elimination of redundant information, to more specific tasks, e.g., the detection of copying dependencies across multiple sources [35] or the automated discovery of multi-column joins [54]. In addition, table overlap not only serves user-facing applications, but can also be used as a filtering or blocking method for downstream tasks, such as table reclamation [16].

In practice, all proposed use cases fall under two main scenarios: *(i) table matching*, i.e., retrieving all the clusters of related tables in a corpus, and *(ii) table querying*, i.e., given a *query table*, identifying a set of related tables in a corpus by building a ranking or using a similarity threshold. Since the first scenario requires comparing all possible pairs, it has a quadratic complexity, and even when working with datasets of a few thousand elements, a large number of comparisons needs to be performed. The second, instead, is linear with respect to the number of tables in the corpus. However, it can be equally challenging if computed in a brute-force manner, indeed Sloth [54], the only prior approach available for estimating the largest table overlap, takes more than two hours of machine time to compare 1 table with a data lake of 10K tables. Thus, in this paper, we analyze the problem of efficiently approximating the *maximum normalized table overlap*, referring to it as *overlap ratio*.

1.1 Table overlap ratio

We define the overlap ratio between two tables by extending the definition of the largest table overlap [54]. The intuition is to identify the sub-tables of maximum area, in terms of number of cells, that are shared between the two tables and normalize their area with respect to the area of the smaller table.

Given two tables R and S defined on the schemas X and Y respectively, we first define an attribute mapping M as the bijective function that links a subset of columns of the first table $X_M \subseteq X$ to another set $Y_M \subseteq Y$ of the second table. Each mapping determines a table overlap O_M, which corresponds

The Battleship Approach to the Low Resource Entity Matching Problem

BAR GENOSSAR, Technion – Israel Institute of Technology, Israel
AVIGDOR GAL, Technion – Israel Institute of Technology, Israel
ROEE SHRAGA, Worcester Polytechnic Institute, USA

Entity matching, a core data integration problem, is the task of deciding whether two data tuples refer to the same real-world entity. Recent advances in deep learning methods, using pre-trained language models, were proposed for resolving entity matching. Although demonstrating unprecedented results, these solutions suffer from a major drawback as they require large amounts of labeled data for training, and, as such, are inadequate to be applied to *low resource entity matching* problems. To overcome the challenge of obtaining sufficient labeled data we offer a new *active learning* approach, focusing on a selection mechanism that exploits unique properties of entity matching. We argue that a distributed representation of a tuple pair indicates its informativeness when considered among other pairs. This is used consequently in our approach that iteratively utilizes space-aware considerations. Bringing it all together, we treat the low resource entity matching problem as a Battleship game, hunting indicative samples, focusing on positive ones, through awareness of the latent space along with careful planning of next sampling iterations. An extensive experimental analysis shows that the proposed algorithm outperforms state-of-the-art active learning solutions to low resource entity matching, and although using less samples, can be as successful as state-of-the-art fully trained known algorithms.

CCS Concepts: • **Information systems** → **Entity resolution**; • **Computing methodologies** → **Active learning settings**.

Additional Key Words and Phrases: entity resolution, entity matching, active learning

ACM Reference Format:
Bar Genossar, Avigdor Gal, and Roee Shraga. 2023. The Battleship Approach to the Low Resource Entity Matching Problem. *Proc. ACM Manag. Data* 1, 4 (SIGMOD), Article 224 (December 2023), 25 pages. https://doi.org/10.1145/3626711

1 INTRODUCTION

Entity matching, with variations as entity resolution, record linkage, record deduplication and more, is a core data integration task that plays a crucial role in any data project life cycle [8, 15]. Essentially, entity matching aims to identify dataset tuples that refer to the same real-world entities.

Traditional methods for entity matching and its variations focused on string similarity [20, 21, 27, 31] and probabilistic approaches [10], followed by rule-based methods [58, 59]. Learning-based approaches were also suggested [4, 26] with an increased focus on deep learning in recent years [13, 14, 24, 25, 28, 41, 68], culminating in the use of pre-trained language models, and specifically BERT [9], fine-tuned for entity matching [7, 29, 30, 45].

Authors' addresses: Bar Genossar, Technion – Israel Institute of Technology, Haifa, Israel, sbargen@campus.technion.ac.il, sbargen@campus.technion.ac.il; Avigdor Gal, Technion – Israel Institute of Technology, Haifa, Israel, avigal@technion.ac.il; Roee Shraga, Worcester Polytechnic Institute, Worcester, Massachusetts, USA, rshraga@wpi.edu.

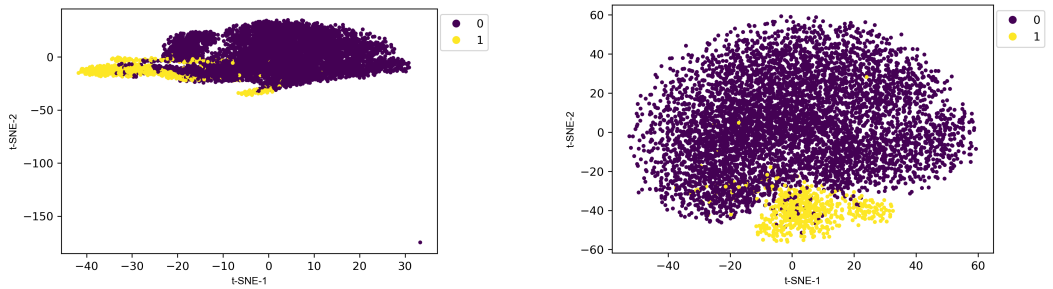
Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
2836-6573/2023/12-ART224 $15.00
https://doi.org/10.1145/3626711

Machine learning approaches to entity matching are by-and-large supervised, requiring labeled data for training, and in the case of deep learning, massive amounts of them [34, 46, 60]. Obtaining labeled data is challenging, mostly due to the need for intensive human expert labor that accompanies the labeling process [33, 34, 55]. This, in turn, calls for methods that reduce the labeling process load while maintaining an accurate trained matcher.

Several approaches were suggested over the years to overcome the dependence on large amounts of data, among which, unsupervised [3, 65, 66] and active learning methods [47, 53]. Active learning, the most prevalent approach to the insufficient labeled data problem, offers methods for training using a limited amount of labeled data. This is done by zeroing in on informative samples to be labeled and used for training. An active learning framework uses selection criteria (*e.g.*, variations of uncertainty [25, 38, 48] and centrality [67, 69]) to pinpoint the samples that are expected to improve the model performance the most. These selected samples are then sent to a labeling oracle and added to the available training set in following iterations.

A key tool which we use to quantify both uncertainty and centrality of a sample (a record pair in our case) is its vector-space latent representation. Traditional entity matching approaches constructed pair feature vector using widely used similarity measures [4, 26, 65] or learnable attribute-wise similarity vector using neural networks [24, 41]. Contemporary solutions [7, 29, 45], which demonstrate superior performance and do not require feature engineering, yield an embedding vector as alternative to feature vectors when representing the input (pair) with respect to the task.

In this work we offer a solution to low resource entity matching tasks, where there is a limited budget for labeling. In particular, we attend to the labeled data scarcity problem using active learning, proposing a novel strategy tailored to the specific characteristics of entity matching. Entity matching usually suffers from inherent label imbalance, where the number of negative (non-match) pairs overshadows the number of positive (match) ones [2]. When training with insufficient amount of data, this property is often problematic since the model may struggle to generalize to unseen minority class samples (in this case, matching pairs). To tackle this challenge, we ensure a balanced sampling of positive and negative samples using high-dimensional latent space encoding for tuple pairs. Specifically, we argue that vector-space similarity between pair representations reflects, with high probability, agreement of their labels. We next use this notion to carefully guarantee a balanced sampling, although positive pairs are harder to be found.



(a) Amazon-Google (b) Walmart-Amazon

Fig. 1. Visualization of pair distribution by t-SNE, partitioned into match and non-match pairs. Using representative vectors (with dimension of 768) of a fully trained models we demonstrate that match pairs tend to gather together.

Through the Fairness Lens: Experimental Analysis and Evaluation of Entity Matching

Nima Shahbazi
University of Illinois Chicago
nshahb3@uic.edu

Nikola Danevski
University of Rochester
ndanevsk@u.rochester.edu

Fatemeh Nargesian
University of Rochester
fnargesian@rochester.edu

Abolfazl Asudeh
University of Illinois Chicago
asudeh@uic.edu

Divesh Srivastava
AT&T Chief Data Office
divesh@research.att.com

ABSTRACT

Entity matching (EM) is a challenging problem studied by different communities for over half a century. Algorithmic fairness has also become a timely topic to address machine bias and its societal impacts. Despite extensive research on these two topics, little attention has been paid to the fairness of entity matching.

Towards addressing this gap, we perform an extensive experimental evaluation of a variety of EM techniques in this paper. We generated two social datasets from publicly available datasets for the purpose of auditing EM through the lens of fairness. Our findings underscore potential unfairness under two common conditions in real-world societies: (i) when some demographic groups are over-represented, and (ii) when names are more similar in some groups compared to others. Among our many findings, it is noteworthy to mention that while various fairness definitions are valuable for different settings, due to EM's class imbalance nature, measures such as positive predictive value parity and true positive rate parity are, in general, more capable of revealing EM unfairness.

PVLDB Reference Format:
Nima Shahbazi, Nikola Danevski, Fatemeh Nargesian, Abolfazl Asudeh, and Divesh Srivastava. Through the Fairness Lens: Experimental Analysis and Evaluation of Entity Matching. PVLDB, 16(11): 3279 - 3292, 2023.
doi:10.14778/3611479.3611525

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/UIC-InDeXLab/fair_entity_matching.

1 INTRODUCTION

Entity matching (EM) seeks to match pairs of entity records from (the same or different) data sources that refer to the same real-world entity. EM is very useful in many applications domains, including (a) healthcare, where matching of patient records from different healthcare facilities (e.g., emergency rooms, hospitals, etc.) can be used to determine if they refer to the same real-world person; (b) airline security, where airline passenger records are matched against no-fly list records to identify people who should be prevented from boarding flights or should undergo additional screening; (c) e-commerce, where product records from different retailers' websites can be matched to identify popular products and fraudulent knockoffs; and so on.

EM is a challenging problem that has been extensively investigated for over half a century by different communities, e.g., statistics, databases (DB), natural language processing (NLP), and machine learning (ML), resulting in a variety of techniques proposed in the literature for addressing this problem. These challenges arise because entities in autonomous data sources can be represented in a variety of ways (e.g., highly structured records versus textual descriptions), using different conventions (e.g., the many ways in which person names and postal addresses are represented), data quality issues (e.g., misspellings, missing values), and so on. A consequence is that, despite significant advances in recent years (especially with recent neural techniques like Ditto [36]), EM techniques still result in both false positives (non-matching record pairs that are declared as matches) and false negatives (matching record pairs that are declared as non-matches). These errors can have serious consequences in practice, as seen in the following examples.

Example 1: (No-fly list) Consider the airline security application, which aims to identify passengers that are likely to be dangerous (e.g. terrorists) for screening and potentially preventing them from boarding the flights. Using a dataset of criminal records called the no-fly list, passenger names (and other information) are matched against the no-fly list for this purpose. False positives in airline security can lead to significantly inconveniencing passengers. On the other hand, false negatives can result in known terrorists being permitted to board flights with undesirable consequences. Due to historical biases, the no-fly list datasets could over-represent some minority groups in comparison to society's population distribution. This, as we shall evaluate in our experiments, can result in higher false positive rates for those demographic groups. Another potential issue is that some demographic groups have more similar names. Hence, passengers from those groups may have a higher chance of having the same or similar information to those of known terrorists, which in turn will cause higher false positive rates for them. □

In Example 1, getting (falsely) matched is harmful. We next show an example where not getting matched can be harmful.

Example 2: (High-value customers list) Upselling to potentially high-value customers is critical for many businesses such as the fashion industry, airlines, and tourism. Suppose business A acquires

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 16, No. 11 ISSN 2150-8097.
doi:10.14778/3611479.3611525

a list L of names of high-value customers from other businesses. Business A uses EM techniques to match its own customers against list L and sends exclusive offers to upsell to the matching cases in order to prioritize them. Unlike the previous example, in this case, over-representation in the high-value customer's list L is beneficial. However, unlike the no-fly list, the privileged group(s) are likely to be over-represented and minority groups under-represented in the list L. As a result, a lower (true and false) positive match rate is expected for minority groups and they end up receiving fewer exclusive offers, resulting in biased advertising [4, 9, 40]. □

When such disparities (e.g., false positives) occur in a systematic way for some demographic (sub-)groups, thereby disadvantaging them over others, concerns about the *fairness* of EM techniques arise. While the fairness of ML models has been the topic of much recent work in the literature [12, 23, 25, 25, 28, 34, 61, 62, 64], not much attention has been paid to the fairness of EM techniques.

In this paper, we seek to address this gap in the literature and perform an extensive experimental evaluation and analysis of a variety of EM techniques on a range of datasets through the *fairness lens*. Traditionally, blocking may precede matching to reduce the space of possible matching candidates from quadratic to sub-quadratic, e.g., linear. A rich body of research focuses on blocking algorithms [19, 35, 46, 47]. In this paper, our goal is to audit off-the-shelf entity matching systems used in practice. As such, our evaluation and analysis are performed on end-to-end matching systems which may include their own built-in blocking algorithms.

Summary of Contributions: In summary, we make the following technical contributions in this paper:

- Given the pairwise nature of EM, we propose the use of *single fairness* and *pairwise fairness* to evaluate entity matchers. We adopt 11 popular fairness measures from the literature for this task and analyze their suitability for EM.
- We select a suite of 13 EM techniques (including 1 declarative rule-based technique, 7 non-neural ML techniques, and 5 neural ML techniques) and 6 benchmark datasets (including 2 structured datasets, 2 textual datasets, and 2 dirty datasets) that have been used in prior work on entity matching for fairness evaluation. Using publicly available individual-level data, we also created and used two semi-synthetic matching *social datasets* for fairness evaluation. These datasets are shared publicly as benchmarks for auditing the fairness of matchers.
- We evaluated all combinations of EM techniques, datasets, and fairness measures and analyzed the outcomes. We classified the results into four cases based on whether an (EM technique, dataset, or fairness measure) yielded (i) accurate or inaccurate matching results, and (ii) fair or unfair matching results.
  Some of our findings in this study are as follows:
  - Our results on social data confirm matching unfairness when (i) there are higher similarities among records of a certain group, (ii) the representation of demographic groups in data is biased.
  - Our results underscore that responsible EM requires training data that is representative of different possibilities from various (demographic) groups.
  - While different fairness measures are valuable for different settings, due to the class-imbalance property of EM, measures such as *positive predictive value parity* and *true positive rate parity* are more capable of revealing EM unfairness.
  - Significantly relying on proxy attributes such as name, can cause unfairness in non-neural models. On the other hand, relying on pre-trained language models and embeddings, or not fully considering the dataset structure can cause unfairness in neural matchers.

  We use our findings and lessons learned to put together a set of rules of thumb for responsible entity matching (Table 8).

2 RELATED WORK

Fairness in entity resolution (ER) has briefly been studied in the literature. In [24], a constraint-based formulation for fairness is proposed to mitigate bias in ER tasks by ensuring that all (sub-)groups have the same opportunity to be resolved. Furthermore, [38] proposes a (sub-)group-based training for different ethnicities in order to increase both accuracy and fairness in SVM-based ER which is consistent with our suggestion to use ensemble learning for EM. Finally, in a parallel work [43], the authors propose an AUC-based fairness definition for EM and ER tasks and try to resolve the bias issues through a data augmentation solution. To the best of our knowledge, we are the first to comprehensively audit off-the-shelf entity matching models for fairness and propose proper measures, datasets, and comparison angles fitting the problem settings given the inherent differences with typical machine learning tasks.

3 FAIRNESS EVALUATION FRAMEWORK

3.1 Background

Given two sets of records A and B, the EM problem is to identify all correspondences between record pairs in $A \times B$ that correspond to the same real-world entity. A correspondence $c = (e_i, e_j, s)$ interrelates two records e_i and e_j with a confidence value $s \in [0, 1]$ that indicates the similarity of e_i and e_j or the confidence of a matcher about e_i and e_j referring to the same entity [33]. To decide whether the record pair of $c = (e_i, e_j, s)$ is a *match* or *non-match*, matchers often apply a threshold on s [11, 60]. We decouple the choice of a threshold from the outcome of the matching and consider the outcome of an EM task as pairs of matching and non-matching records. Formally, we consider the following EM problem:

Definition 1 (Entity Matching Problem). *Consider two sets of records A and B. For every pair of records $(e_i, e_j) \in A \times B$, let y_{ij} be the ground-truth label indicating if e_i and e_j refer to the same entity. Given all pairs $(e_i, e_j) \in A \times B$, the EM problem is to predict y_{ij} with a label h_{ij}. That is, h_{ij} refers to the decision of the matcher about the label of e_i and e_j (match or non-match).*

In a fairness-sensitive setting, records are accompanied by sensitive attributes (e.g. `gender`, `country`, `race`, etc.). Let $\mathcal{A} = \{A_1, \ldots, A_n\}$ be the sensitive attributes, $dom(A_i)$ be the domain of A_i, and $\mathcal{G} = \{g_1, \ldots, g_m\}$ be the set of all groups of interest, i.e. $\mathcal{G} = \bigcup_{A_i \in \mathcal{A}} dom(A_i)$. The mapping $L(e_i)$ relates a record to its associated groups $G_i \subseteq \mathcal{G}$. In other words, G_i is the group that e_i belongs to. Given two sets of records A and B and the set $[(e_i, e_j, G_i, G_j, h_{ij}, y_{ij})]_{\forall (e_i, e_j) \in A \times B}$, we would like to audit the fairness of a matcher with respect to groups.

Unicorn: A Unified Multi-tasking Model for Supporting Matching Tasks in Data Integration

JIANHONG TU, Renmin University of China, China
JU FAN*, Renmin University of China, China
NAN TANG, QCRI / HKUST (GZ), Qatar / China
PENG WANG, Renmin University of China, China
GUOLIANG LI, Tsinghua University, China
XIAOYONG DU, Renmin University of China, China
XIAOFENG JIA, Beijing Big Data Centre, China
SONG GAO, Beijing Big Data Centre, China

Data matching – which decides whether two data elements (*e.g.,* string, tuple, column, or knowledge graph entity) are the "same" (*a.k.a.* a match) – is a key concept in data integration, such as entity matching and schema matching. The widely used practice is to build task-specific or even dataset-specific solutions, which are hard to generalize and disable the opportunities of knowledge sharing that can be learned from different datasets and multiple tasks. In this paper, we propose **Unicorn**, a unified model for generally supporting common data matching tasks. **Unicorn** can enable knowledge sharing by learning from multiple tasks and multiple datasets, and can also support zero-shot prediction for new tasks with zero labeled matching/non-matching pairs. However, building such a unified model is challenging due to heterogeneous formats of input data elements and various matching semantics of multiple tasks. To address the challenges, **Unicorn** employs one generic `Encoder` that converts any pair of data elements (a, b) into a learned representation, and uses a `Matcher`, which is a binary classifier, to decide whether a matches b. To align matching semantics of multiple tasks, **Unicorn** adopts a mixture-of-experts model that enhances the learned representation into a better representation. We conduct extensive experiments using 20 datasets on seven well-studied data matching tasks, and find that our unified model can achieve better performance on most tasks and on average, compared with the state-of-the-art specific models trained for ad-hoc tasks and datasets separately. Moreover, **Unicorn** can also well serve new matching tasks with zero-shot learning.

CCS Concepts: • **Information systems → Mediators and data integration**.

Additional Key Words and Phrases: data matching, data integration, multi-task learning

ACM Reference Format:
Jianhong Tu, Ju Fan, Nan Tang, Peng Wang, Guoliang Li, Xiaoyong Du, Xiaofeng Jia, and Song Gao. 2023. Unicorn: A Unified Multi-tasking Model for Supporting Matching Tasks in Data Integration. *Proc. ACM Manag. Data* 1, 1, Article 84 (May 2023), 26 pages. https://doi.org/10.1145/3588938

*Ju Fan is the corresponding author.

Authors' addresses: Jianhong Tu, Renmin University of China, Beijing, China, tujh@ruc.edu.cn; Ju Fan, Renmin University of China, Beijing, China, fanj@ruc.edu.cn; Nan Tang, QCRI / HKUST (GZ), Doha / Guangzhou, Qatar / China, ntang@hbku.edu.qa; Peng Wang, Renmin University of China, Beijing, China, lisa_wang@ruc.edu.cn; Guoliang Li, Tsinghua University, Beijing, China, liguoliang@tsinghua.edu.cn; Xiaoyong Du, Renmin University of China, Beijing, China, duyong@ruc.edu.cn; Xiaofeng Jia, Beijing Big Data Centre, Beijing, China, jiaxf@jxj.beijing.gov.cn; Song Gao, Beijing Big Data Centre, Beijing, China, gaos@jxj.beijing.gov.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
2836-6573/2023/5-ART84 $15.00
https://doi.org/10.1145/3588938

1 INTRODUCTION

Data matching generally refers to the process of deciding whether two data elements are the "same" (*a.k.a.* a match) or not, where each data element could be of different classes such as string, tuple, column, and so on. Data matching is a key concept in data integration [13] and data preparation [9] that includes a wide spectrum of tasks. In this paper, we consider seven common data matching tasks, namely entity matching, entity linking, entity alignment, string matching, column type annotation, schema matching, and ontology matching, as shown in Figure 1.

Due to their importance, almost all the aforementioned matching tasks have been studied for decades, and still remain to be important research topics. With the tremendous successes of deep learning, deep neural networks have been widely used for tackling various matching tasks, such as DeepMatcher [34] and Ditto [30] for entity matching (see a recent tutorial [4]), and TURL [10] and HNN [5] for column type annotation. However, these current deep learning based solutions are task-specific or even dataset-specific, which are referred to as **specific models** in this paper.

There are two main **limitations of specific models**. First, they can only learn knowledge from specific tasks or datasets. That is, the learned knowledge cannot be shared across different models. For example, the knowledge learned by a column type annotation model (*e.g.,* TURL [10]) cannot be shared with an entity matching model (*e.g.,* Ditto [30]) due to different neural network designs, although very likely the two models can help each other. Second, one model has to be trained (or fine-tuned) only on the labeled examples of each task or dataset, which is inefficient and has a high monetary cost. For example, DeepMatcher [34] and Ditto [30] have to be fine-tuned on a new entity matching dataset with at least hundreds of labeled matching/non-matching pairs.

In this paper, we propose **`Unicorn`**, a unified model to support multiple data matching tasks, as shown in Figure 1. Compared with aforementioned specific models, **`Unicorn`** has the following notable advantages.

(1) ***Task unification*** that generalizes task-specific solutions into one unified model, thus achieving lower maintenance complexity and smaller model sizes, compared with specific models.
(2) ***Multi-task learning*** that enables the unified model to learn from multiple tasks and multiple datasets to make full use of *knowledge sharing*, which even outperforms specific models trained only on their own datasets separately.
(3) ***Zero-shot prediction*** that allows the model to make predictions for a new task or a new dataset with *zero* labeled matching/non-matching pairs.

Although several unified models have been recently studied in the NLP and CV communities [3, 41, 42], building such a unified model for supporting multiple data matching tasks is challenging.

(1) Data elements in the matching tasks can take ***heterogeneous formats***, such as tuples and columns in tabular data, entities in knowledge graphs, and plain text, which will increase the difficulty of task unification.
(2) Each data matching task may have its ***unique matching semantics***. For example, entity matching that matches two tuples is different from schema matching that matches two columns. Thus, it is non-trivial to enable knowledge sharing among multiple tasks.

To address the challenges, we develop a general framework **`Unicorn`**, which consists of three key modules: an `Encoder`, a `Mixture-of-Experts` layer, and a `Matcher` (see Figure 1). First, the `Encoder` converts any pair of elements (a, b) with heterogeneous formats into a learned representation. To this end, we propose to serialize any pair of elements into text while still preserving their inherent structure, and employ a unified pre-trained language model for effective encoding. Second, to align matching semantics of various tasks or datasets, the `Mixture-of-Experts` layer enhances the learned representation into a better representation by combining the knowledge from multiple "experts" controlled by a learned gating network. Third, the `Matcher` is a typical binary classifier,

Check for updates

Auto-Tables: Relationalize Tables without Using Examples

Peng Li*
Georgia Tech
Atlanta, GA
pengli@gatech.edu

Yeye He
Microsoft Research
Redmond, WA
yeyehe@microsoft.com

Cong Yan
Microsoft Research
Redmond, WA
coyan@microsoft.com

Yue Wang
Microsoft Research
Redmond, WA
wanyue@microsoft.com

Surajit Chaudhuri
Microsoft Research
Redmond, WA
surajitc@microsoft.com

ABSTRACT

Relational tables, where each row corresponds to an entity and each column corresponds to an attribute, have been the standard for tables in relational databases. However, such a standard cannot be taken for granted when dealing with tables "in the wild". Our survey of real spreadsheet-tables and web-tables shows that over 30% of such tables do not conform to the relational standard, for which complex table-restructuring transformations are needed before these tables can be queried easily using SQL-based tools. Unfortunately, the required transformations are non-trivial to program, which has become a substantial pain point for technical and non-technical users alike, as evidenced by large numbers of forum questions in places like StackOverflow and Excel/Tableau forums.

We develop an Auto-Tables system that can automatically synthesize pipelines with multi-step transformations (in Python or other languages), to transform non-relational tables into standard relational forms for downstream analytics, obviating the need for users to manually program transformations. We compile an extensive benchmark for this new task, with 244 real test cases collected from user spreadsheets and online forums. Our evaluation suggests that Auto-Tables can successfully synthesize transformations for over 70% of test cases at interactive speeds, without requiring any input from users, making this an effective tool for users to prepare data for downstream analytics.

1. INTRODUCTION

Modern data analytics like SQL and BI are predicated on a standard format of relational tables, where each row corresponds to a distinct "entity", and each column corresponds to an "attribute" for the entities that contains homogeneous data-values. While such tables are de-facto standards in relational databases, to the extent that we as database people may take them for granted, we would like the highlight that a significant fraction of tables "in the wild" actually fail to conform to such standards, making it considerably more challenging to query them using SQL-based tools.

Non-relational tables are common, but hard to query. Real tables in the wild, such as spreadsheet-tables or web-tables, can often be "non-relational" and hard to query, unlike tables that we expect to find in relational databases. We randomly sampled hundreds of user spreadsheets (in Excel), and web tables (from Wikipedia), and found around 30-50% tables to have such issues. Figure 1 and Figure 2 show real samples taken from spreadsheets and the web, respectively, to demonstrate these common issues. (We emphasize that the problem is prevalent at a very large scale, since there are millions of tables like these in spreadsheets and on the web.)

Take Figure 1(a) for example. The table on the left is not a standard relational table, because each column marked in green contains sales numbers for only a single day ("`19-Oct`", "`20-Oct`", etc.), making these column values highly homogeneous in the horizontal direction (while in typical relational tables, we expect values in columns to be homogeneous in the vertical direction). Although this specific table format makes it easy for humans to eyeball changes day-over-day by reading horizontally, it is unfortunately hard to analyze using SQL. Imagine that one needs to compute the 14-day average of sales, starting from "`20-Oct`" – for this table, one has to write: `SELECT SUM("20-Oct", "21-Oct", "22-Oct", ...) FROM T`, across 14 different columns, which is long and unwieldy to write. Now imagine we need 14-day moving averages with every day in October as the starting date – the resulting SQL is highly repetitive and hard to manage.

In contrast, consider a transformed version of this table, shown on the right of Figure 1(a). Here the homogeneous columns in the original table (marked in green) are transformed into only two new columns: "`Date`" and "`Units Sold`", using a transformation operator called "`stack`" (listed in the first row of Table 1). This transformed table contains the same information as the original table, but is much easier to query – e.g., the same 14-day moving average can be computed using a succinct range-predicate on the '`Date`" column, where the starting date "`20-Oct`" is a literal parameter that can be easily changed into other values.

There are many such spreadsheet tables that require different kinds of transformations before they are ready for SQL-based analysis. Figure 1(b) shows another example, where every 3 columns form a repeating group, representing "`Revenue/Units Sold/Margin`" for a different year (marked in red/green/blue in the figure). Tables with these repeat-

*Part of work done while at Microsoft.

Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment. This is a minor revision of the paper entitled "Auto-Tables: Synthesizing Multi-Step Transformations to Relationalize Tables without Using Examples", published in PVLDB, Vol. 16, No. 11, 2150-8097. DOI: https://doi.org/10.14778/3611479.3611534

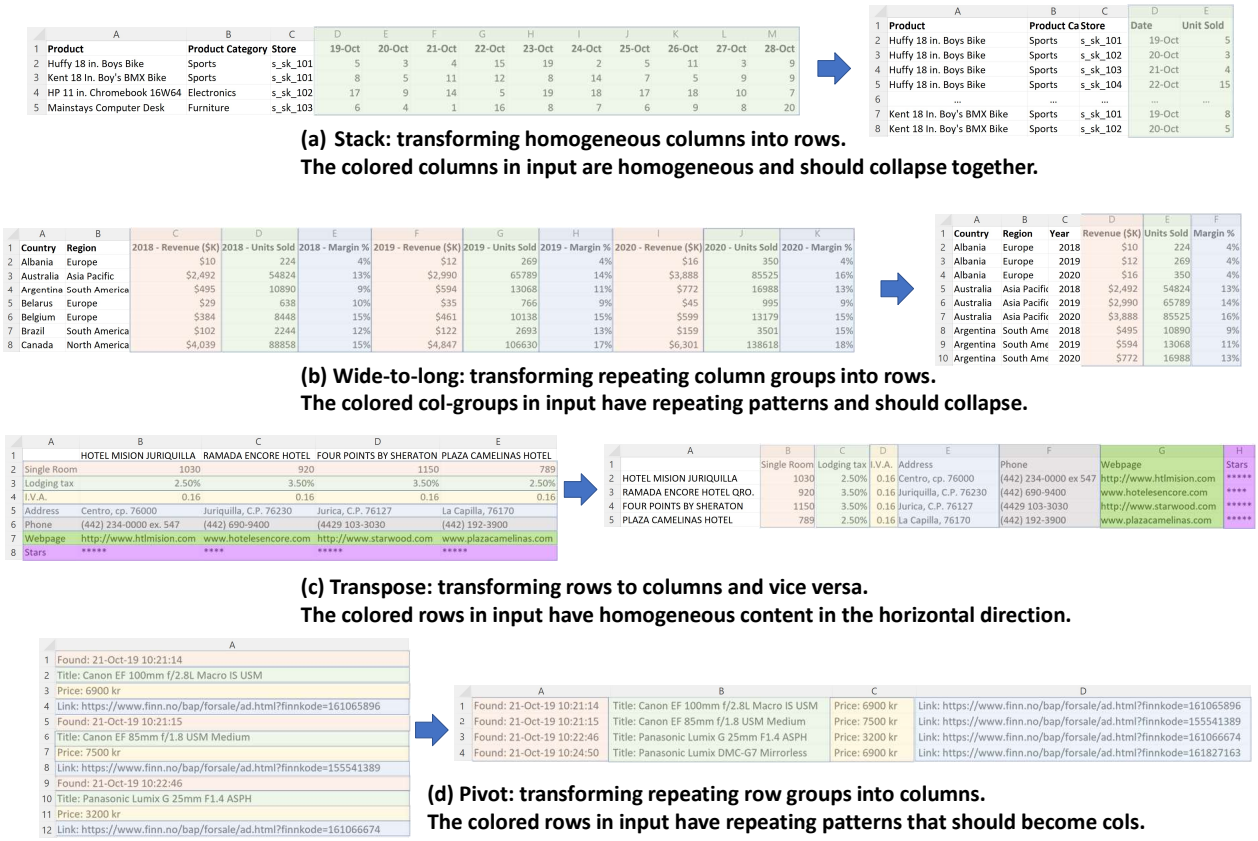


Figure 1: Example input/output tables for 4 operators in AUTO-TABLES: (a) Stack, (b) Wide-to-long, (c) Transpose, (d) Pivot. The input-tables (on the left) are not relational and hard to query, which need to be transformed to produce corresponding output-tables (on the right) that are relational and easy to query. Observe that the color-coded, repeating row/column-groups are "visual" in nature, motivating a CNN-like architecture like used in computer vision for object-detection.

ing column-groups are also hard to query, just like Figure 1(a), but in this case the required transformation operator is called "`wide-to-long`" (second row of Table 1).

Figure 1(c) shows yet another example, where each hotel corresponds to a column (whose names are in row-1), and each "attribute" of these hotels corresponds to a row. Note that in this case values in the same rows are homogeneous (marked in different colors), unlike relational tables where values in the same columns are homogeneous. A transformation called "`transpose`" is required in this case (listed in the third row of Table 1), to make the resulting table, shown on the right of the figure, easy to query – for instance, a query to sum up the total number of hotel rooms is hard to write on the original table, but can be easily achieved using a simple SUM query on the "`Single Room`" column in the transformed table.

Figure 1(d) shows another example where columns are represented as rows in the table on the left. This is similar to Figure 1(c), except that the rows in this case are "repeating" in groups, thus requiring a different transformation operator called "`pivot`" (listed in the fourth row of Table 1) as opposed to "`transpose`". The resulting table is shown on the right, which becomes easy to query.

While the examples so far are all taken from spreadsheets, we note that similar structural issues are also widespread in Web tables. Figure 2 shows real examples from Wikipedia, which share similar characteristics as the spreadsheet tables in Figure 1, which all require transformations before these tables can be queried effectively.

Non-relational tables are hard to "relationalize". We mentioned that the example tables in Figure 1 and Figure 2 require different transformation operators. Table 1 shows 8 such transformation operators commonly needed to relationalize tables (where the first 4 operators correspond to the examples we see in Figure 1).

The first column of Table 1 shows the name of the "logical operator", which may be instantiated differently in different languages (e.g., in Python or R), with different names and syntax. The second column of the table shows the equivalent Pandas operator in Python [11], which is a popular API for manipulating tables among developers and data scientists, that readers may be familiar with.

While the operations listed in Table 1 already exist in languages such as R and Python, they are not easy for users to invoke correctly, because users need to:

1. Visually identify different structural issues in an input table that make it hard to query (e.g., repeating row-/column-groups shown in Fig. 1(a-d)), which is not obvious to non-expert users;
2. Map the visual pattern identified from the input table, to a corresponding operator in Table 1 that can handle such issues. This is hard for users not familiar with the exact terminologies to describe these transformations (e.g., `pivot` vs. `stack`);

Benchmarking Filtering Techniques for Entity Resolution

George Papadakis
National and Kapodistrian University of Athens, Greece
gpapadis@di.uoa.gr

Marco Fisichella
L3S Research Center, Germany
mfisichella@l3s.de

Franziska Schoger
L3S Research Center, Germany
schoger@l3s.de

George Mandilaras
National and Kapodistrian University of Athens, Greece
gmandi@di.uoa.gr

Nikolaus Augsten
University of Salzburg, Austria
nikolaus.augsten@plus.ac.at

Wolfgang Nejdl
L3S Research Center, Germany
nejdl@l3s.de

2023 IEEE 39th International Conference on Data Engineering (ICDE) | 979-8-3503-2227-9/23/$31.00 ©2023 IEEE | DOI: 10.1109/ICDE55515.2023.00389

***Abstract*—Entity Resolution is the task of identifying pairs of entity profiles that represent the same real-world object. To avoid checking a quadratic number of entity pairs, various filtering techniques have been proposed that fall into two main categories: (i) *blocking workflows* group together entity profiles with identical or similar signatures, and (ii) *nearest-neighbor methods* convert all entity profiles into vectors and identify the closest ones to every query entity. Unfortunately, the main techniques from these two categories have rarely been compared in the literature and, thus, their relative performance is unknown. We perform the first systematic experimental study that investigates the relative performance of the main representatives per category over numerous established datasets. Comparing techniques from different categories turns out to be a non-trivial task due to the various configuration parameters that are hard to fine-tune, but have a significant impact on performance. We consider a plethora of parameter configurations, optimizing each technique with respect to recall and precision targets. Both schema-agnostic and schema-based settings are evaluated. The experimental results provide novel insights into the effectiveness, the time efficiency and the scalability of the considered techniques.**

***Index Terms*—nearest neighbors, record linkage, deduplication**

I. Introduction

Entity Resolution (ER) is a well-studied problem that aims to identify so-called *duplicates* or *matches*, i.e., different entity profiles that describe the same real-world object [1]. ER constitutes a crucial task in a number of data integration tasks, which range from Link Discovery for interlinking the sources of the Linked Open Data Cloud to data analytics, query answering and object-oriented searching [2]–[4].

Due to its quadratic time complexity, ER typically scales to large data through a Filtering-Verification framework [1], [3]. The first step (*filtering*) constitutes a coarse-grained, rapid phase that restricts the computational cost to the most promising matches, a.k.a. *candidate pairs*. This is followed by the *verification* step, which examines every candidate pair to decide whether it is a duplicate.

Verification is called *matching* in the ER literature [2], [3], [5]. Numerous matching methods have been proposed; most of them rely on similarity functions that compare the textual values of entity profiles [2], [3], [5], [6]. Early attempts were rule-based, comparing similarity values with thresholds, but more recent techniques rely on learning, i.e., they usually model matching as a binary classification task (`match`, `non-match`) [6]. Using a labelled training dataset, supervised, active, and deep learning techniques are adapted to ER [7]. In some cases, a clustering step is subsequently applied on the resulting similarity scores to refine the output [8].

In this work, we are interested in filtering methods, which significantly reduce the search space of ER. The performance of a filtering technique is assessed along three dimensions: recall, precision and time efficiency. A good filter stands out by:

1) *High recall.* The candidate pairs should involve many duplicates to reduce the number of false negatives.
2) *High precision.* The candidate pairs should involve few false positives (i.e., non-matching pairs) to significantly reduce the search space.
3) *Low run-time.* The overhead added by the filtering step to the overall run-time of ER should be low.

Ideally, filtering should also be directly applicable to the input data. For this reason, we exclusively consider methods that require no labelled training instances, which are often not available or expensive to produce [9]. We also focus on techniques for textual entity profiles. These methods are organized into two types [10]:

1) *Blocking workflows* extract one or more signatures from every entity profile and form clusters of profiles with identical or similar signatures [3], [11], [12].
2) *Nearest neighbor (NN) methods* index part or all of the input data and treat every entity profile as a query, to which they return the most similar candidates. They rely on sparse [13], [14] or dense numeric vectors [15].

Although the two types of filtering techniques follow very different approaches, they all receive the same input (the entity profiles) and produce the same output (candidate pairs).

To the best of our knowledge, no previous work systematically examines the relative performance across the two different kinds of methods. The main blocking methods are empirically evaluated in [11], [12], [16]. The studies on sparse [13], [14], [17] and dense vector-based NN methods [15] evaluate run-time and approximation quality, but do not evaluate the performance on ER tasks (cf. Section II). Even in the few cases where blocking is used as baseline for an NN method (e.g., Standard/Token Blocking is compared to DeepBlocker in [9]), the comparison is not fair: the blocking method is treated

2375-026X/23/$31.00 ©2023 IEEE
DOI 10.1109/ICDE55515.2023.00389

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:57:37 UTC from IEEE Xplore. Restrictions apply.

as an independent approach, instead of applying it as part of a blocking workflow, which is common in the literature [11].

Comparing techniques from different categories turns out to be a challenging task, due to their fundamentally different functionality and the diversity of configuration parameters that significantly affect their performance. Yet, there is *no* systematic fine-tuning approach that is generic enough to apply to all filtering methods – e.g., the step-by-step configuration optimization in [11] applies to blocking pipelines, but not to the single-stage NN methods.

In this work, we perform the first thorough and systematic experimental study that covers both types of filtering methods. To ensure a fair comparison, every approach is represented by its best performance per dataset, as it is determined after an exhaustive fine-tuning with respect to a common performance target that considers several thousand different parameter configurations. This approach is applied to 5 blocking workflows and 8 NN methods over 10 real-world datasets. For each filter type, we also consider two baseline methods with default parameters. We explore both *schema-agnostic* and *schema-based settings*: the latter focus exclusively on the values of the most informative attribute, while the former take into account all information within an entity, essentially treating it as a long textual value – as a result, they are inherently applicable to datasets with heterogeneous schemata.

Overall, we make the following contributions:

- We perform the first thorough experimental analysis on filtering techniques from different categories. We evaluate 14 state-of-the-art filters and 4 baselines on 10 real-world datasets in both schema-based and schema-agnostic settings.
- Our configuration optimization process enables the meaningful comparison of blocking workflows and NN methods in the context of serial processing (single-core execution).
- We present a qualitative analysis of the filtering techniques based on their *scope* and *internal functionality*.
- We perform a thorough scalability analysis that involves seven synthetic datasets of increasing size.
- Our work provides new insights into the relative performance and scalability of the considered techniques. We show that the blocking workflows and the cardinality-based sparse NN methods consistently excel in performance.
- Two of the tested NN methods, SCANN and kNN-Join, are applied to ER for the first time. SCANN is one of the most scalable techniques, and kNN-Join one of the best performing ones, while sticking out by its intuitive fine-tuning.
- All code and data used in this work are publicly available through a new, open initiative that is called Continuous Benchmark of Filtering methods for ER: https://github.com/gpapadis/ContinuousFilteringBenchmark.

The main part of the paper is structured as follows: Section II discusses the related works, while Section III defines formally the configuration optimization task. We elaborate on the filtering methods in Section IV and present their qualitative and quantitative analyses in Sections V and VI, resp. Section VII concludes with the main findings of our experimental analysis along with directions for future research.

II. Related Work

There has been a plethora of works examining the relative performance of blocking methods. The earliest systematic studies were presented in [3], [12]. They focus exclusively on schema-based settings in combination with several user-defined parameter configurations. They also consider exclusively the first step of blocking workflows: block building.

These studies were extended in [16], which examines the same block building methods and configurations, but applies them to schema-agnostic settings, too. The experimental outcomes suggest that recall raises significantly, when compared to the schema-based settings, while requiring no background knowledge about the given data and the quality of its schema. They also suggest that the sensitivity to parameter configuration is significantly reduced.

Building on these works, the experimental analysis in [11] examines the relative performance of the blocking workflows in schema-agnostic settings. In particular, it considers blocking workflows formed by exactly three steps: block building, block filtering and comparison cleaning. In our work, we extend this analysis by considering the top performing block building methods, all of which cluster together entities that share identical signatures. We combine them with three consecutive, but optional steps: block purging, block filtering and comparison cleaning. These steps give rise to seven different filtering pipelines, out of which only one was examined in [11].

Our work also differs from [11] in that it has only one dataset in common – excluding the scalability ones. Most importantly, [11] optimizes the configuration parameters in a heuristic step-by-step manner: first, the performance of block building is heuristically optimized and then, block filtering is heuristically fine-tuned by receiving as input the output of optimized block building as so on for comparison cleaning. In contrast, we consider a holistic approach to configuration optimization, simultaneously fine-tuning all steps in a blocking workflow. As explained in [18], [19], this approach consistently outperforms the step-by-step fine-tuning, because it is not confined to local maxima per workflow step, while it considers a significantly larger set of possible configurations.

Finally, we also go beyond [3], [11], [11], [12] in two more ways: (i) we systematically fine-tune blocking workflows in the context schema-based settings, and (ii) we compare blocking workflows with sparse and dense NN methods.

The sparse vector-based NN methods essentially correspond to similarity joins. The relative performance of the main methods is examined in [13], [14], [17] with respect to runtime. Recall and precision are not considered, because they are identical across all approaches, i.e., all methods retrieve the same pairs of entities that exceed a similarity threshold. These pairs, which are not necessarily matching, have not been evaluated with respect to the recall and precision of ER. As a result, these studies are not useful in assessing the performance of string similarity joins for ER. Note also that none of the sparse NN methods we consider has been examined in prior experimental analyses. The reason is that the local kNN-Join

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:57:37 UTC from IEEE Xplore. Restrictions apply.

BLEND: A Unified Data Discovery System

1st Mahdi Esmailoghli
TU Berlin & BIFOLD
Berlin, Germany
esmailoghli@tu-berlin.de

2nd Christoph Schnell
Leibniz Universität Hannover
Hannover, Germany
schnell@dbs.uni-hannover.de

3rd Renée J. Miller
University of Waterloo
Waterloo, Canada
rjmiller@uwaterloo.ca

4th Ziawasch Abedjan
TU Berlin & BIFOLD
Berlin, Germany
abedjan@tu-berlin.de

2025 IEEE 41st International Conference on Data Engineering (ICDE) | 979-8-3315-3603-9/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICDE65448.2025.00061

***Abstract*—Most research on data discovery has so far focused on improving individual discovery operators such as join, correlation, or union discovery. However, in practice, a combination of these techniques and their corresponding indexes may be necessary to support arbitrary discovery tasks. We propose BLEND, a comprehensive data discovery system that supports existing operators and enables their flexible pipelining. BLEND is based on a set of lower-level operators that serve as fundamental building blocks for more complex and sophisticated user tasks. To reduce the execution runtime of discovery pipelines, we propose a unified index structure and a rule- and cost-based optimizer that rewrites SQL statements into low-level operators when possible. We show the superior flexibility and efficiency of our system compared to ad-hoc discovery pipelines and stand-alone solutions.**

I. Introduction

The potential for large structured data lakes, such as governmental data lakes [19], [61], enterprise-level corpora [10], [15], and public-access lakes [22], [14], [33], has spurred various avenues of research, all aimed at empowering data scientists with the ability to extract further insights for their diverse tasks, such as machine learning (ML) [23], exploratory data analysis [41], and data cleaning [40]. To support these tasks, various data discovery operatorion have been studied, such as **join discovery** [24], [69], [63], [62], [59], [27], [70], [20], [16], [56], [50], **union search** [45], [11], [36], [13], [25], [34], and **correlation discovery** [23], [49], [9], [17].

Existing research has focused on each operator individually without considering possible opportunities for combining, pipelining, and optimizing them. Yet, fulfilling a real-world data discovery task often requires a combined application of these operators. For instance, in the case of data enrichment for ML, the user needs to use a combination of join, union, and correlation discovery [9], [45]. To achieve this, we need a unified data discovery system.

Requirements for a unified discovery system. A trivial approach to building such a unified system is to federate all indexes and algorithms via a common interface. Even after overcoming the inherent software and interface heterogeneity among prototypes, the execution of complex data discovery tasks that combines these operators across systems is inefficient and inflexible in terms of both storage and maintenance. This stems from several key challenges:

Lack of Flexibility. Introducing new criteria or discovery operators is challenging since new indexes and algorithms must be integrated into an already complex system.

Storage and Maintenance Overhead. Multiple independently developed index structures must be generated and stored. The aggregated system will comprise an incoherent set of potentially redundant structures needing maintenance.

Lack of Optimization. The federation of independent operators under the same abstraction does not lend itself to optimization. To showcase the operational complexity of a rather simple discovery task, consider following example:

Example 1. Fig. 1 illustrates a user table S along with three tables $T1$, $T2$, and $T3$ from a data lake. S contains information about departments in a company and the name of their corresponding leaders, i.e.,"Head". The user wants to fill in missing values of S but already knows that the head of the IT department, "Tom Riddle", has left the company. Therefore, any table that contains the projection ("IT","Tom Riddle") is outdated. Thus, the user defines the data discovery for missing values as follows: *Find top-*1 *table that contains ("HR" and "Firenze" in a row and "HR", "Marketing", "Finance", "IT", ... in a column but does not contain rows with "IT" and "Tom Riddle").*

To solve this task with existing discovery operators, one could use single-column join discovery, such as Josie [69], to look up overlapping column values, and multi-column join discovery, such as MATE [24], to find overlapping column combination values. With MATE, one could find tables containing the projection ("HR", "Firenze") as result set rs_1. Also, MATE can be used to identify tables containing the tuple ("IT", "Tom Riddle") as result set rs_2. Josie can be employed to find tables that join only on the department values "HR", "Marketing", "Finance", ... as result set rs_3. Finally, the equation $((rs_1 \cap rs_3) - rs_2)$ is the answer to the user's query. According to Fig. 1, $rs_1 = \{T2, T3\}$, $rs_2 = \{T2\}$, and $rs_3 = \{T1, T2, T3\}$. Thus, the answer to the discovery query is $\{T3\}$, which contains amendable values for the "Head" of the department column in S. •

The example illustrates that running independent discovery redundant and inefficient. For each look-up, combinations of the three tables $T1$, $T2$, and $T3$ may be processed. However, knowing that $T1$ and $T2$ are not desired tables, we could exclude them from follow-up operators: first, by checking all three tables to find those containing ("HR" and "Firenze"); then, by focusing only on $T3$ and $T2$ and dropping the outdated $T2$; and finally, by verifying $T3$ for overlapping department names. Intuitively, we could obtain the same results by resorting to application code. We aim to

2375-026X/25/$31.00 ©2025 IEEE
DOI 10.1109/ICDE65448.2025.00061

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:39:53 UTC from IEEE Xplore. Restrictions apply.

S

Dep.	Head
HR	Firenze
Marketing	NULL
Finance	NULL
IT	NULL
R&D	NULL
Sales	NULL

T1

Team	Size
Finance	31
Marketing	28
HR	33
IT	92
Sales	80

T2

Lead	Year	Team
Tom Riddle	2022	IT
Draco Malfoy	2022	Marketing
Harry Potter	2022	Finance
Cho Chang	2022	R&D
Luna Lovegood	2022	Sales
Firenze	2022	HR

T3

Lead	Year	Team
Ronald Weasley	2024	IT
Draco Malfoy	2024	Marketing
Harry Potter	2024	Finance
Cho Chang	2024	R&D
Luna Lovegood	2024	Sales
Firenze	2024	HR

Fig. 1: Discovering up-to-date tables to fill in the NULLs.

achieve the same flexibility within a discovery system with a unified language to optimize custom discovery processes. Such abstraction becomes necessary in case of complex queries with a variety of operators and result sets with millions of tables.

Contributions We introduce BLEND, a unified system that enables users to create complex discovery plans. With no need for compiling, installing, or adapting additional discovery systems, BLEND streamlines the discovery process. Given a data discovery plan, a data lake, and a parameter k, BLEND retrieves the top-k tables from the lake that are most relevant to the defined plan. Our key contributions are as follows:

1) **Introduction of an abstraction layer:** We design a novel abstraction that unifies the building blocks of different discovery tasks. We propose a set of atomic search operators, which users can declaratively compose to efficiently discover relevant tables. These building-blocks are implemented using primitive database functions, allowing the majority of computations to run within the database.
2) **Optimized execution of tasks:** We develop a two-phase plan optimizer that combines rule- and cost-based query optimization techniques. It re-orders the execution of operators based on predefined rules, ensuring efficient execution while preserving the intended logic of the discovery task. Furthermore, BLEND optimizes individual operators by rewriting corresponding queries using intermediate results.
3) **Integrated index:** We studied fifty-three index structures from thirty data discovery papers and propose a unified index that efficiently supports the execution of the introduced operators and their compositions. The index is designed as a single database table, enabling the system to additionally benefit from the database-level query optimizations.
4) **Simplification of complex tasks using SQL operations:** Our proposed index structure allows for the translation of BLEND's operators into SQL queries. Even complex tasks, such as correlation discovery, are reduced to SQL-based operations, eliminating the need for custom algorithms or external implementations. By integrating multiple operators into declarative pipelines, BLEND minimizes user effort and simplifies the creation of discovery tasks.
5) **User study:** We conducted a user study with data experts to understand their discovery preferences and evaluate the effectiveness of BLEND in addressing these needs.

II. Problem Statement

Given the requirements formulated in the introduction, we formalize the following problem statement. Let $\mathcal{D} = \{T_1, T_2, \ldots, T_n\}$ be a data lake, where each T_i is a table. Given a query table S with insufficient data and a discovery plan P by the user, the goal of BLEND is to enable the formulation of the plan and its optimized compilation for execution. The result of the execution is the subset $\mathcal{T}' \subseteq \mathcal{D}$ that enriches S. With this functionality in mind, we strive to solve four technical problems (**Pr.1-4**):.

First, the language for defining a plan should support the formulation of the most prominent discovery operations from the literature (**Pr.1**):

- **Keyword search**: discovered tables must overlap with a given set of keywords.
- **Join discovery**: discovered tables must be joinable with S on one or more columns.
- **Union discovery**: rows in discovered tables must be unionable with rows in S.
- **Correlation discovery**: a column in each discovered table must correlate with the user-defined numerical target in S.

Second, the system should also support any set-based aggregation of these operation's results (**Pr.2**).

Third, we require an index that serves all of the possible discovery operations but is leaner than the ensemble of the required individual indexes. More formally, assuming that an independent system exists for each of the aforementioned discovery operations M_{op_i}, where $1 < i < n$, the storage required for our index to execute any combination of operators in OP must be smaller than $\sum_{i=1}^{n} \text{Storage}(M_{op_i})$, where $Storage(M_{op_i})$ is the storage size required for the state-of-the-art index in M_{op_i} (**Pr.3**).

Finally, the system should satisfy two runtime constraints. The implementation of each (M_{op_i}) in the unified system should yield a comparable runtime while any aggregation and combination of more than two M_{op_i} should yield superior runtime compared to their execution as concatenated black box discovery system. Formally, assuming a discovery plan P is an arbitrary combination of operators in OP, i.e, $P \in OP^*$, the execution runtime of P in BLEND should satisfy

$$\text{Time}(P_{\text{BLEND}}) \leq \sum_{j|op_j \in P} \text{Time}(M_{op_j}).$$

Satisfying the previous constraint should not be a burden on how the user formulates the plan. BLEND must identify a good (near optimal) ordering of the operations based on the expected runtime and intermediate result size (**Pr.4**)

III. System Overview

Fig. 2 depicts an overview of BLEND's architecture depicting its online and offline phases. The online phase involves four stages: task definition, plan formulation, plan optimization, and execution.

First, the user defines their data discovery task using BLEND's API (Fig. 2a). Here, the plan *find_dep_heads* is

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:39:53 UTC from IEEE Xplore. Restrictions apply.

Description-Similarity Rules: Towards Flexible Feature Engineering for Entity Matching

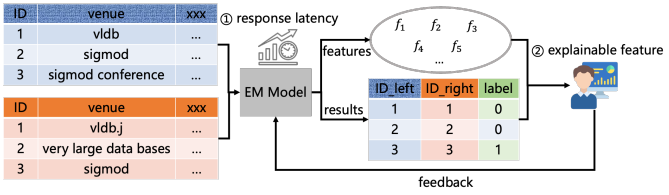
Yafeng Tang[1], Zheng Liang[1], Hongzhi Wang[1,✉], Xiaoou Ding[1], Tianyu Mu[1], Huan Hu[2]
[1] School of Computer Science, Harbin Institute of Technology, Harbin, China
[2] Huawei Cloud Computing Technologies Co., Ltd, Hangzhou, China
tangyf@stu.hit.edu.cn,{lz20, wangzh, dingxiaoou,mutianyu}@hit.edu.cn, huhuan18@huawei.com

***Abstract*—Entity Matching (EM) is a crucial task in data integration. Compared to deep learning-based EM solutions, tree-based machine learning models are more computationally effective and explainable, making them more applicable in real-world EM scenarios. However, Random Forest-based EM methods select features with a static feature engineering rule set for all attributes. Consequently, they suffer model retraining cost to select features, and can hardly customize to different EM tasks. To tackle this problem, we propose Description-Similarity Rules (DSR) for EM feature engineering. DSR introduces diverse attribute value distribution metrics and data-driven thresholds to traditional EM feature engineering rules. Unfortunately, both the DSR search space and its online model retraining costs are exponential. By pushing the model retraining to the offline stage, our DSR set mining algorithm is one order of magnitude faster than the baseline algorithms, taking only seconds for online selection. Empirically, DSR outperforms the feature engineering methods by 3.75% on average F1 score, while reaching the state-of-the-art EM performance on several datasets.**

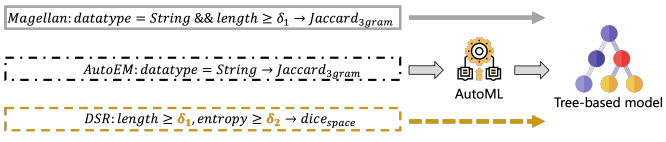
***Index Terms*—Entity Matching, Feature Engineering, Rule Mining**

I. Introduction

Entity matching [1]–[3] is the problem of identifying co-referent manifestations that refer to the same real-world entity from different data sources. For example, attributes of the same person from multiple sources are often complicated [4] in the real world. EM can improve data quality by reducing such duplicates among tables [5].



(a) An interactive EM pipeline



(b) Rule-based EM model

Fig. 1: A typical EM pipeline.

✉ Hongzhi Wang is the corresponding author.

EM has been an open problem for decades. Conventional EM solutions are based on similarity functions, which quantify the similarity between pairwise attribute values of an entity [6]. Rule-based [7]–[10] EM methods use certain thresholds for attribute similarities to make matching decisions, where experts specify the thresholds (e.g., the value of δ_1 in Figure 1(b)). Machine learning-based methods [6] train machine learning models on pairs of labeled entities with similarities as features. They search for the feature combination with the best evaluation metric (e.g., F1 score). Deep Learning methods [1], [3], [11]–[14] incorporate domain knowledge in language-model-based encoders. However, real-world EM tasks pose flexibility requirements beyond the grasp of these methods, which can be specified in the following aspects [15]. (1) **Response latency.** Entity Matching is rarely a one-time job. As Figure 1(a) shows, a typical EM pipeline requires data scientists to go through the cycle of exploring data [10], changing EM the solution, and checking results several times [16]. Also, EM becomes even more time-intensive on multi-source data which need to be split and matched separately. Methods with low response latency are necessary for refining and validating EM solutions in a pipeline as Figure 1. (2) **Explainable features.** To understand and reconfigure a machine learning model in the EM pipeline, one needs to understand how the model is influenced by its input, i.e., the feature value. Even for explainable EM models like Random Forest, system-selected features still need to be traceable from data [17].

To the best of our knowledge, few EM solutions can address the above two issues simultaneously. Rule-based methods like Autofuzzyjoin [18] may not perform well with less structured datasets and long textual data since they rely on limited word level similarity metrics and TF-IDF scores [19]. State-of-the-art deep learning methods like Sudowoodo [13] make it difficult for users to understand why entities are matched or unmatched. They often require a high cost for debugging the model: try numerous configurations at each iteration, and need expert experience to make decisions. Mainstream machine learning-based methods are easier to understand, but their efficiency is limited by searching models and features. Therefore, to meet the flexibility demands mentioned above, lightweight machine learning models such as Random Forest (RF) [20] have been generally adopted for EM [21] in commercial data integration pipelines [6], [15], [22] for their good explainability and satisfactory performance.

2375-026X/25/$31.00 ©2025 IEEE
DOI 10.1109/ICDE65448.2025.00008

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:45:22 UTC from IEEE Xplore. Restrictions apply.

2025 IEEE 41st International Conference on Data Engineering (ICDE) | 979-8-3315-3603-9/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICDE65448.2025.00008

Example 1. *Consider a training process of a Random Forest-based EM model, a data scientist needs to construct features (i.e. similarities) for each attribute pair.*

In Figure 1(b), Magellan [23] simply uses length and data types to recommend similarities, so the selected features result in poor performance (Section V). AutoML-EM [6] relaxes the rule set of Magellan and recommends redundant features, which are subsequently trimmed by auto-sklearn [24]. The general feature selection methods [8], [25]–[29] search for the optimal feature at the cost of exhaustive model retraining. Considering efficiency, in this paper, we focus on rule-based feature selection methods.

Challenges. From Example 1, we can see that the bottleneck of the RF-based EM pipeline is the drawback of rule-based feature selection, which can be summarized as follows.

Exhaustive Rule Candidate Space. In a typical EM pipeline, rule-based similarity selection methods typically cost extensive computational resources as expert knowledge is unavailable. The threshold of descriptions [1] can take any value within a continuous range. Actually, we need to consider rules with conjunction of multiple descriptions as antecedents, and all similarities can be consequent. As a result, the rule candidate space is overwhelming, and searching for each rule candidate requires an EM model retraining. What's worse, an EM pipeline mostly lacks labeled data, so the hyperparameter search algorithms [30] fail to solve this problem.

Rule set is too rigid. The rule set of existing solutions only considers `length` and `datatype` of attribute pair, while the threshold is fixed for all EM tasks. They fail to offer users enough flexibility to give feedback and perform an EM task at hand. Consequently, such rigid rule sets with fixed thresholds recommend too few or insufficient features for complex datasets (as shown in Example 1). Users can barely understand EM data and give constructive feedback to the pipeline in such cases.

Description-Similarity Rule. Rule-based FE methods fall short in practice due to the above limitations. To unlock the flexibility of EM models, we need to design a flexible substitute of feature engineering rules without too much computational overhead. Motivated by this, we propose the Description-Similarity Rule (DSR). DSR helps users in EM pipeline to better understand data with the newly introduced "description". Also, DSR has a flexible threshold, which can be discovered from data without expert knowledge.

Contribution. The contributions of our work can be summarized as follows.

• We propose the Description-Similarity Rule (DSR), a set of data-driven rules for flexible Feature Engineering on Entity Matching models. We design *descriptions* to support common data types including sets, vectors, strings, and boolean data.

• To discover the optimal DSR set for a given EM task, we design a DSR mining algorithm accelerating the baseline approach by one order of magnitude (Section IV). Also, we address the problem of exponential online retraining by sampling historical datasets with a theoretical guarantee on error probability(Section III).

• Empirically, DSR beats the state-of-the-art EM solution by 3.75% on average F1 score. DSR is also statistically comparable to black-box Deep Learning methods (Section V).

[1]Description is a set metric to compute certain attribute value distributions. We will go into details in Definition 2

TABLE I: Frequently used notations.

Symbol	Meaning
T_1, T_2	A pair of tables for entity matching
$A_1, A_2, \ldots$	The attribute set of tables
t_1, t_2	An entity pair from T_1 and T_2
M	The entity matching model
p	The evaluation metric for the EM task
$D = \{d_1, d_2, d_3, d_4\}$	A set of descriptions
D'	A subset of descriptions
$S = \{S_1, S_2, \ldots\}$	A set of similarities
$R = \{r_1, r_2, \ldots\}$	A set of description-similarity rules
δ_i	The threshold for i-th description
θ	The threshold for lowest acceptable performance
ϕ	An operator
$dom(d_i)$	The domain of valid value of description
E	The expectation of a variable
n	The budget of EM model retraining times

II. Overview

In this section, we define essential concepts for Description-Similarity Rules (Section II-A), elaborate the design details of *Descriptions* (Section II-B), present the DSR recommendation problem(Section II-C), and give an overview of our solution (Section II-D).

A. Background

Entity Matching solutions for structured data consist of at least two phases: blocking and matching [31], and often assume that upstream tasks have prepared datasets with aligned attributes [6], [32]. In this paper, we focus on the feature engineering of entity matching for relational data and treat entity blocking as an orthogonal problem.

Definition 1. ***Entity Matching (EM):*** *Given two relational tables T_1 and T_2 with the aligned attribute set $A_1, A_2, ...$ with the same schema, Entity Matching determines whether each pair of records $t_1, t_2 \in T_1 \times T_2$ refers to the same entity in the real world.*

In this paper, we focus on the Random Forest(RF)-based EM solver, as RF is a widely-adopted machine learning model in real-world EM applications [6], [12], [22], [33]. For example, consider the entity pair with $ID = 3$ in Figure 1(a), "sigmod conference" and "sigmod" have a Jaccard similarity of 0.5, which is used by RF-based EM solvers as one of the features to represent this entity pair for RF training. Therefore, the core of *RF-based EM* is **similarity selection** [7], [34], [35], and we follow SOTA RF-based EM methods [6], [23] to exploit similarity selection rules.

A similarity function quantifies the similarity between two strings (or numeric or Boolean values). There are four types of similarity functions: set-based, sequence-based, vector-based,

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:45:22 UTC from IEEE Xplore. Restrictions apply.

FEATPILOT: Automatic Feature Augmentation on Tabular Data

Jiaming Liang*
University of Pennsylvania
liangjm@seas.upenn.edu

Chuan Lei, Xiao Qin, Jiani Zhang, Asterios Katsifodimos,
Christos Faloutsos, Huzefa Rangwala
Amazon Web Services
chuanlei, drxqin, zhajiani, akatsifo, faloutso, rhuzefa@amazon.com

***Abstract*—Tabular data within enterprises or open data repositories provide a huge opportunity for feature augmentation. Using these data sources to augment training data often boosts model performance, which is crucial in data-centric AutoML systems. Recent works on automatic feature augmentation have limited capabilities in utilizing useful features that cannot be joined with the base table without connecting through intermediate tables. We present FEATPILOT, a novel framework that explores and integrates high-quality features in tabular data for ML models. FEATPILOT evaluates a candidate feature from two aspects: (1) the efficacy of a join path connecting the feature to the base table and (2) the intrinsic value of a feature towards an ML task. FEATPILOT efficiently identifies high-quality features and their optimized join paths to augment the base table. Our experimental results show that FEATPILOT achieves up to a 10.27% improvement in ML model performance compared to state-of-the-art solutions across six public datasets.**

I. INTRODUCTION

Feature augmentation on tabular data is crucial for improving machine learning (ML) model performance [1]–[5]. With additional features, models can capture more complex patterns and relationships within the data, leading to better prediction and generalization results. By exploring and integrating features [6], [7] from open tabular data repositories (e.g., Google Dataset Search[1], Kaggle[2], and OpenML[3]), ML models [8] can avoid overfitting [9] and improve performance on unseen data [3]–[5].

Given a base table with an ML task defined on it and a set of candidate tables that can be joined with the base table via one-hop or multi-hop paths, feature augmentation is the task of selecting feature columns from the candidate tables that can maximize the performance of the ML task. This process typically involves identifying useful features that can reveal additional insights or patterns for the given ML task, and determining the optimal paths to join these candidate features with the base table to enrich the initial training data.

Motivating Example. An ML engineer is developing a classification model to predict whether a product should be recommended or not. The base table `product` contains only a limited set of features for building a high-quality model (shown in Fig. 1). Fortunately, useful features exist in related tables such as `address` in the `customer` table. To augment the base table, the ML engineer needs to find a high-quality join path to augment the base table. Among three join paths in Fig. 1, `customer` ⋈ `order` ⋈ `product` (indicated by the thick red line) provides more feature instances (i.e., `address`) compared to paths involving the `rating` or `review` table, as customers may not provide ratings or reviews for every purchased product. As a result, the ML engineer can exploit this augmented table to improve the classification model.

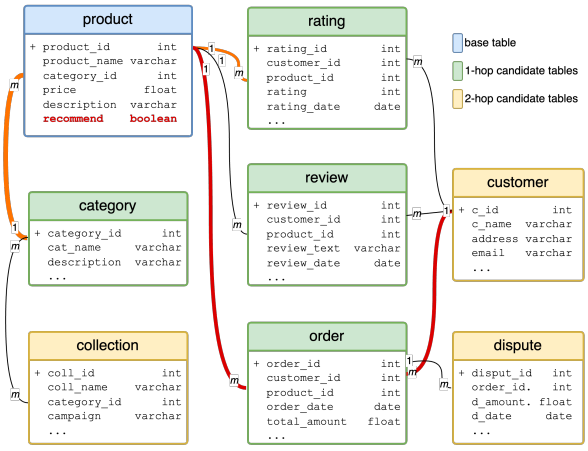


Fig. 1: Example of feature augmentation. The base table, `product`, is in blue. The task is to predict the values in the Boolean type column 'recommend'. The tables in green and yellow are 1-hop and 2-hop joinable with the base table.

Challenges. In this paper, we address the following three key challenges in feature augmentation.

(1) *Multiple Join Paths Selection.* In real-world open data settings, a candidate feature can be integrated with the base table via multiple join paths, resulting in different integration quality as explained in the motivating example. Hence, it is critical to identify the optimal join path to achieve the full potential of a candidate feature.

(2) *Expensive Join Operations.* To find out the actual benefit of augmenting the base table with a candidate feature, expensive joins need to be executed, which is computationally expensive and time consuming, especially when a long join path involves multiple joins. Candidate features in open data repositories often have similar join paths. Therefore, sharing join operations is crucial for efficient feature augmentation.

*The work was done when Jiaming Ling was with Amazon Web Services.
[1]https://datasetsearch.research.google.com/
[2]https://www.kaggle.com/datasets
[3]https://www.openml.org/

2025 IEEE 41st International Conference on Data Engineering (ICDE) | 979-8-3315-3603-9/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICDE65448.2025.00163

2375-026X/25/$31.00 ©2025 IEEE
DOI 10.1109/ICDE65448.2025.00163

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:44:54 UTC from IEEE Xplore. Restrictions apply.

(3) *Search Complexity.* The search space of all candidate tables and their join paths is vast and intractable, making it prohibitively expensive to find the optimal set of candidate tables. To address this challenge, we need effective strategies to reduce the search space while preserving optimality.

State-of-the-art Approaches. Prior approaches [3], [10] augment the base table by joining other tables using either their relevancy to the base table or the improvement of the candidate features bring to the ML models. Recently, ALITE [11] and DIALITE [12] are introduced to integrate tables based on joinability or unionability, which can improve the quality of downstream ML models. However, these approaches assume that the features and their respective relevance to the base table are specified. Hence, they fail to leverage all possible candidate features and consequently do not achieve optimal model performance. Reinforcement learning (RL) [13], [14] has emerged as an effective approach to balance between feature exploration and exploitation. Several studies [4], [5], [15] leverage RL techniques such as multi-armed bandits and Deep Q-Networks (DQNs) to explore the large search space of candidate tables. Although these RL-based methods have achieved promising results, they still face critical challenges. Specifically, METAM [5] is limited to directly joinable candidate tables and AutoFeature [4] does not provide fine-grained optimization on integration quality. Moreover, RL often requires extensive training data and tends to overfit, reducing its effectiveness in real-world applications.

Proposed Solution. To cope with the above mentioned challenges, we propose FEATPILOT, an efficient and effective framework for automatic feature augmentation. Given a base table, a set of candidate tables and an ML task, FEATPILOT first selects a subset of candidate tables using a clustering algorithm based on table characteristics. FEATPILOT further chooses representative join paths from these candidate tables to the base table. This allows FEATPILOT to discover the actual performance gain led by the candidate tables following the join paths (*Challenge 1*). Consequently, FEATPILOT learns an integration quality model and a feature importance estimator to calculate the performance gain of any unexplored candidate tables and their respective join paths (*Challenge 2*). Lastly, FEATPILOT's join-path search algorithm prunes the search space based on both integration quality and feature importance, and finds the optimized integration paths for the selected candidate tables in polynomial time (*Challenge 3*).

Contributions. Our main contributions are the following.

(1) We introduce FEATPILOT, a novel feature augmentation framework for ML tasks over tabular data repositories, by decomposing the intractable problem of feature augmentation into feature importance and integration quality.

(2) We design an effective feature exploration module consisting of a clustering-based feature importance estimation and a learning-based integration quality model.

(3) We propose an efficient feature path search algorithm that exploits both estimated feature importance and integration quality to identify high-quality features and their optimized join paths for integration.

(4) Our experiments show that FEATPILOT achieves up to a 10.27% improvement in ML model performance compared to state-of-the-art solutions across six public datasets.

Reproducibility. The source code of FEATPILOT is available at `https://anonymous.4open.science/r/FeaturePilot-C308`.

II. Preliminaries and System Overview

A. Basic Notions

Definition 1 (Base Table): The base table, T_{base} = $\{c_1, c_2, \ldots, c_n, c_l\}$, refers to the original table that includes the initial set of features and the label column before any augmentation. Each c_i ($i \in [1, n]$) represents a column and c_l denotes the label column, which stores the prediction target for each row.

Definition 2 (Candidate Tables): The candidate tables, $\mathcal{T}$ = $\{T_1, T_2, \ldots, T_m\}$, are m joinable tables that can be used to augment a base table T_{base}. Each table is denoted by T_j = $\{f_{j,1}, f_{j,2}, \ldots, f_{j,k}\}$ ($j \in [1, m]$), where $f_{j,k}$ denotes the k-th column (a.k.a feature)[4] in table T_j.

Definition 3 (Join Path): Given a set of tables $T_1, T_2, \ldots, T_n$, a join path P can be defined as a sequence of join operations between pairs of tables. A join operation between any two tables T_i and T_j can be denoted as $T_i \bowtie T_j$. The join path P can be represented as: $P = T_1 \bowtie T_2 \bowtie T_3 \ldots \bowtie T_n$.

Without loss of generality, we allow different types of join operators between T_{base} and $T_j \in \mathcal{T}$, including inner join, left outer join, right outer join, etc.

Definition 4 (Augmented Table): The augmented table $T_{aug} = T_{base} \bowtie_P T_j$ is the result of joining the base table T_{base} and a candidate table T_j following a join path P.

Note that there can be multiple join paths between the base table and a candidate table. Hence the augmented table could be different depending on the chosen join path.

Definition 5 (Machine Learning Task): A machine learning (ML) task is defined as a function f that maps an input space $\mathcal{X}$ to an output space $\mathcal{Y}$. Given a dataset $D = \{(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)\}$, where $x_i \in \mathcal{X}$ and $y_i \in \mathcal{Y}$, the objective is to learn the function f that best approximates the true underlying relationship from the inputs to the outputs.

Definition 6 (Utility Gain): Given a ML task, the utility score, $US(T)$, is defined as the performance metric (e.g., accuracy, precision, recall, etc.) of the task on a table T achieved by a specific ML model[5]. Let $US(T_{base})$ be the performance metric on the base table, and $US(T_{aug}) = US(T_{base} \bowtie_P T_j)$ be the performance metric on the augmented table. The utility gain UG is defined as the improvement in the performance metric: $UG = US(T_{aug}) - US(T_{base})$.

Problem Statement. Given a base table T_{base}, a set of candidate tables $\mathcal{T}$, and a ML task, our goal is to select features from these candidate tables $\mathcal{T}$ and to identify the best join paths to augment the base table T_{base} with the selected

[4]We use two terms, column and feature, alternately in this paper.

[5]$US(T)$ uses the negation of a metric value when smaller values are preferred (e.g., mean squared error (MSE), mean absolute error (MAE), etc.).

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:44:54 UTC from IEEE Xplore. Restrictions apply.

LIFTus: An Adaptive Multi-aspect Column Representation Learning for Table Union Search

Ermu Qiu[†], Jun Gao[†*], Yaofeng Tu[‡], and Jingru Yang[§*]
[†]Key Laboratory of High Confidence Software Technologies, CS, Peking University, China [‡]ZTE Corporation,
[§]National Key Laboratory of Data Space Technology and System
qem@stu.pku.edu.cn, gaojun@pku.edu.cn, tu.yaofeng@zte.com.cn, okiyang@pku.edu.cn

2025 IEEE 41st International Conference on Data Engineering (ICDE) | 979-8-3315-3603-9/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICDE65448.2025.00165

***Abstract*—Table union search (TUS) represents a fundamental operation in data lakes to find tables unionable to the given one. Recent approaches to TUS mainly learn column representations for searching by introducing Pre-trained Language Models (PLMs), especially on columns with linguistic data. However, a significant amount of non-linguistic data, notably represented by domain-specific strings and numerical data in the data lake, are still under-explored in the existing methods. To address this issue, we propose LIFTus, an adaptive multi-aspect column representation for table unionable search, where aspect refers to a concept more flexible than data types, so that a single column can exhibit multiple aspects simultaneously. LIFTus aims at combining different aspects of a column (including both linguistic and non-linguistic aspects) to promote the effectiveness and generalization of TUS in a self-supervised manner. Specifically, besides employing PLMs to extract the linguistic aspects from an individual column, LIFTus trains a pattern encoder to learn possible character-level sequential patterns for the column, and builds a number encoder to capture numerical aspects of the column, including the distribution and magnitude features. LIFTus further utilizes a hierarchical cross-attention aided by aspect-relevant statistics to combine these aspects adaptively in producing the final column representations, which are indexed by vector retrieval techniques to achieve efficient search. Extensive experimental results demonstrate that LIFTus has outperformed the current state-of-the-art methods in terms of effectiveness, and achieved much better generalization capability to support unseen data.**

***Index Terms*—Table Union Search, Data Lake, Aspect, Non-Linguistic Column, Generalization**

I. Introduction

With the rapid growth of data from various sources, data lake [1]–[3] becomes an attractive infrastructure to accommodate flexible types of data in their raw form directly. Unlike data warehouses [4], data lakes allow storing data including relational tables (sometimes with metadata unavailable) and other types without transforming into a predefined schema [5]. Thus, a fundamental step for an analytic operation in data lake is to locate the relevant data. TUS is such an operation to identify a set of unionable tables in a data lake that share similar columns with the given query Table [6].

TUS methods attempt to achieve effectiveness, efficiency and generalization. Data instances collected in tables within a data lake often exhibit considerable heterogeneity, encompassing linguistic data (*e.g.* words, long texts, dates), non-linguistic data (*e.g.* domain-specific strings, numerical values), and even their combinations [2], [7], [8]. It is challenging to find the desired unionable target tables effectively. In addition, the column representation learning, a key step in the TUS, needs to capture features of flexible data instances and utilize external knowledge, indicating high time consumption. Moreover, TUS methods should be adaptive to new and evolving data, as the volume of data in the lake grows rapidly. Consequently, a generalized model for column representation is highly desirable.

In the context of the TUS problem, unionability refers to the potential for attributes or tables to be unioned based on shared domain features [9]. Early approaches to TUS have focused on designing rules to measure the unionability between two tables [2], [9], [10], but face challenges in capturing latent linguistic information. With the development of PLMs such as BERT [11] and its variants [12], [13], recent works [14]–[17] have employed these models into TUS, as the data in the lake have flexible format including the textual features [18]. Besides PLMs, knowledge bases (KBs) [19], [20] can also provide information for the determination of unionability [7]. Some studies further notice the roles of the inter-column relationships and compose a token sequence by sampling data from different columns into the BERT-like models, thereby learning the context-aware column representations [14], [15]. The learned column representations are then fed into the vector index for fast similarity search [14]. Some studies calculate pairwise similarities of columns to build a semantic graph for TUS [19], achieving promising performance but at the cost of additional preprocessing and online search time [7].

While PLM-based methods become mainstream in TUS, their preference for linguistic data imposes potential limitations on the performances, especially in the context of the relational tables. We illustrate these issues in Fig. 1. Table A and Table B can be unionable as they both pertain to the entities of washing machines manufactured by the same company and their corresponding columns are unionable, while Table C serves as an example that is prone to model misjudgment. We can see that three tables possess linguistic data in their *Location* and *Description* columns, and Table C also includes the *Color* column as part of the linguistic data. Besides, Fig. 1 illustrates the widespread presence of non-linguistic data in the data lake, including but not limited to identifiers (*e.g. Product Code* column) and numerical data (*e.g. ID* column, *Capacity* column, *Price* column, etc.). For

* Corresponding authors

2375-026X/25/$31.00 ©2025 IEEE
DOI 10.1109/ICDE65448.2025.00165

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:40:17 UTC from IEEE Xplore. Restrictions apply.

example, the strings from the *Product Code* and *Function Abbr* columns (*e.g.* "SDL/JHX/BP/ZH" and "MB65V35E") are not tokens in natural language but highly domain-specific, as they are codes designed by washing machine manufacturers.

Table A

ID	Product Code	Capacity(L)	Function Abbr	Location	Price	Description
20001	EMS75B37mate6	7.5	SDL/JHX/BP/ZH	WalMart	3100	This is a top-loading washing machine that deli...
20002	ES100B35Mate5	10	DDL/JHX/DP	WalMart	2950	Featuring a 10L capacity, it can handle large loa...
20003	ES90B26Mate6	9	SDL/BP/FCR	WalMart	2200	The machine's smart sensors adjust the water...

Table B

ID	Product Code	Capacity(L)	Function Abbr	Location	Price	Description
20981	EMS65B26Mate6	6.5	DDL/DP/DLCJ	Carrefour	1850	With a variety of wash cycles, including a quick...
20982	EB80B37Mate5	8	DDL/DP	Carrefour	2100	Its durable stainless steel drum is gentle on fab...
20983	ES95B36PLUS5	9.5	SDL/BP/FCR/ZH	Carrefour	3500	Equip with a self-cleaning function, eliminating...

Table C

ID	Product Code	Power(W)	Color	Location	Price	Description
301	MB65V35E	340	White	WalMart	1900	The machine's sleek design and compact size m...
302	MB75V35E	400	Silver	WalMart	2150	For those with allergies, the UltraClean 3000's...
303	MB65V55E	360	Black	WalMart	2350	With a quiet operation and a 5-year warranty, t...

Fig. 1. An example of data lake. In this data lake, Table A and Table B are unionable with each other, while Table C is easily misjudged as being unionable with Table A. This misjudgment often stems from an inadequate understanding of non-linguistic columns (*e.g. ID*, *Product Code*, *Function Abbr*), which are widely distributed across the data lake and play a crucial role in determining the unionability of tables. The metadata present here is for easy understanding, but not used in the following processing.

We illustrate the issues faced by the existing PLM-based methods. First, the domain-specific strings, represented by commonly used identifiers, fall outside the typical token set in the PLMs [11]–[13], and cannot be captured by PLMs effectively. Taking Fig. 1 as an example, as the token sets in the PLMs do not cover the values in *Product Code* column and *Function Abbr*, PLMs handle them using the frequent sub-strings, like BPE [21], computed from the large common corpus, which may be different from the key sub-parts in these domain-specific strings. We can see that the similarity between "EMS100B37mate6" and "EMS65B26Mate6" should be significantly greater than that between "EMS100B37mate6" and "MB65V35E". However, PLMs may decompose the string into different parts (*e.g.* "E", "MS1"), which damages the understanding of the domain-specific string. It is imperative to employ character-level sequential modeling for domain-specific strings to capture their similarities.

Second, the accurate perception of the numerical values to match columns is an open problem [22]. It is a live issue for PLMs to adequately understand numerical values [23]. PLMs' token sets can not cover infinite numerical values distributed in continuous space. Thus, the PLM-based methods [14]–[16] struggle to fully understand real numbers, which leads to a loss in the similarity computation over arbitrary numbers in real-life applications. The set-coverage-based methods [9] may yield Jaccard similarity scores nearly zero (*e.g.* {7.5, 10, 9} in *Capacity* column in Table A, {6.5, 8, 9.5} in *Capacity* column in Table B in Fig. 1), despite the fact that these numbers differ only slightly. We also notice that there has been research on encoding numerical types in tables [24]. However, there remains a significant gap between encoding individual numbers and encoding entire numerical columns in TUS, where the latter needs to capture the crucial criteria to determine the similarity among columns, including distribution and magnitude features.

Third, columns may include combined features, and relying solely on data-type-specific approaches may be too coarse to accurately capture column features. As we can see from Fig. 1, some columns in a data lake are rich in semantic information, characterized by a high proportion of natural language words, such as the *Description* columns, which rely more on external corpora or PLMs. In contrast, the *Function Abbr* columns in Table A and Table B should guide the model to prioritize character-level similarity during unionable search. Additionally, some columns are predominantly numerical, such as the *ID* and *Price* columns across the three tables. In such a case, the determination should focus on the distribution and magnitude features of the numbers. Moreover, certain columns contain a mixture of numerical data and non-language strings, such as the *Product Code* columns in the three tables. We can see that even if we can obtain the data type for the column, the data-type-specific methods are not sufficient to capture the flexible features inside the column.

To overcome the aforementioned challenges and issues of the existing methods, we make the following design choices. (1) We introduce a notation of *column aspect*, which is finer than column data type, as each column has only one data type but with multiple aspects. Thus, instead of invoking one method after determining the column data type, we apply multiple aspect extractors on one column simultaneously, so as to handle the different forms of the same typed column or mixed-typed data. We also expect that the full exploration of aspects in an individual column can enhance the model's generalization to unseen data. (2) We do not solely rely on PLM to handle all kinds of columns, but introduce effective methods to capture distinct features in columns, including the linguistic data, domain-specific strings, and numerical values. For aggregating these aspects, we design a learning method rather than heuristic rules to enable robust adaptation to scenarios involving tables with diverse information. (3) We mainly study the exploration of features in one column, and leave the inter-column relationships into future work, since different features inside one single column, we believe, are still underexplored. (4) The method will yield the column embedding, instead of the pairwise similarity model in order to leverage the existing vector index like HNSW (Hierarchical Navigable Small World) [25] to improve the efficiency of online search.

This paper proposes **LIFTus**, an adaptive mu**l**ti-aspect column representat**i**on **f**or **t**able **u**nionable **s**earch. The contributions are as follows:

- We design methods to capture the key non-linguistic aspects in LIFTus, including a pattern encoder for domain-specific strings to infer similarities between strings, and a number encoder to convert the numerical aspect into embedding that preserves the distribution and magnitude features.
- We devise a hierarchical cross-attention integrator, to adaptively combine different aspects of columns, in which

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:40:17 UTC from IEEE Xplore. Restrictions apply.

METAM: Goal-Oriented Data Discovery

Sainyam Galhotra, Yue Gong, Raul Castro Fernandez
Department of Computer Science, The University of Chicago
Email: {sainyam, yuegong, raulcf}@uchicago.edu

2023 IEEE 39th International Conference on Data Engineering (ICDE) | 979-8-3503-2227-9/23/$31.00 ©2023 IEEE | DOI: 10.1109/ICDE55515.2023.00213

***Abstract*—Data is a central component of machine learning and causal inference tasks. The availability of large amounts of data from sources such as open data repositories, data lakes and data marketplaces creates an opportunity to augment data and boost those tasks' performance. However, augmentation techniques rely on a user manually discovering and shortlisting useful candidate augmentations. Existing solutions do not leverage the synergy between discovery and augmentation, thus underexploiting data.**

In this paper, we introduce METAM, a novel goal-oriented framework that queries the downstream task with a candidate dataset, forming a feedback loop that automatically steers the discovery and augmentation process. To select candidates efficiently, METAM leverages properties of the: i) data, ii) utility function, and iii) solution set size. We show METAM's theoretical guarantees and demonstrate those empirically on a broad set of tasks. All in all, we demonstrate the promise of goal-oriented data discovery to modern data science applications.

I. INTRODUCTION

Augmenting a dataset by joining it with others can improve the utility of data-driven tasks such as causal inference and supervised machine learning. The abundance of tables in open repositories [1], lakes in organizations [2], and even data markets [3], [4] translates into an abundance of *augmentation candidates*. Identifying good augmentation candidates among many is a data discovery problem. To solve this problem, one could use a traditional data discovery system to identify what tables join with the input data, and then, separately, identify what joins increase the task's utility. This *discover-then-augment* approach works when the discovery system returns candidates relevant to the task. Unfortunately, in practice, it is hard to guarantee the discovery system identifies good augmentations because: i) relevant augmentations depend on the task; and ii) analysts may not know what properties make an augmentation relevant, e.g., what features augment the predictive power of a classifier. The disconnection between data discovery and augmentation presents a research opportunity.

We harness that opportunity with a new approach we call **goal-oriented data discovery**, where data discovery is not treated as a one-time process. Instead, it adapts to the task by performing interventions: a process to augment the initial dataset and validate its utility. *Interventional* queries help to identify augmentations that *cause* an increase in the task's utility, thus steering the discovery process to automatically maximize the task's utility. This achieves two objectives: first, analysts do not need to know what criteria make an augmentation good because the approach is automatic. Second, any downstream task with a utility function benefits from this discovery approach. The consequences of goal-oriented data discovery are significant; consider the following anecdote. We used goal-oriented data discovery to predict "housing prices" in a geographical area. Our technique identified some obvious datasets that a social scientist would have been able to identify using a discovery system, such as "income of people staying in the neighborhood" and "crime stats". But crucially, it also identified non-obvious datasets correlated with housing prices such as "presence of grocery stores" and "number of taxi trips" from those areas. Indeed, many sociologists and economists leverage external data to infer causal relationships between attributes of interest [5], [6], [7]. But they rely on domain knowledge and manual effort to identify those relationships. *Goal-oriented data discovery* paves the way to identify new causal relationships from large data repositories automatically.

A trivial but computationally prohibitive way of solving goal-oriented data discovery is to measure the utility of every augmentation candidate and choose the best. The key technical contributions of our paper focus on developing *interventional querying algorithms* for goal-oriented data discovery that exploit the structure of the data, the utility function, and the solution to adaptively prioritize the candidates for querying.

Interventional Queries for Goal-Oriented Data Discovery. Our technical contributions leverage the following properties to optimize the complexity of discovery without loss of quality.

Properties of the Data. A key insight is that *similar* augmentations perform similarly on the downstream task. We exploit this insight by clustering augmentations and judiciously choosing them from different clusters, thus skipping computation. To cluster augmentations, we represent each with a vector of *data profiles*, which are task-independent measures of data and include semantic similarity, correlation, and mutual information, among others. When a combination of data profiles is correlated with the task's utility, clustering narrows down the number of augmentation candidates.

Properties of the Utility function. A task's optimal utility is given by a set of augmentations. Enumerating all subsets is infeasible. Our insight is that when utility functions are monotonic, i.e., utility never decreases with new augmentations, it is possible to find augmentations efficiently, by considering them one by one. And we can make any function monotonic by ignoring augmentations that harm utility using a wrapper around the user-provided task implementation.

Properties of the Solution set. The optimal set of augmentations is a collection of join paths over different datasets. Most augmentations are irrelevant for a downstream task, and useful augmentations are a small subset of the candidate set. We leverage this property to prioritize small subsets of join

2375-026X/23/$31.00 ©2023 IEEE
DOI 10.1109/ICDE55515.2023.00213

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:59:06 UTC from IEEE Xplore. Restrictions apply.

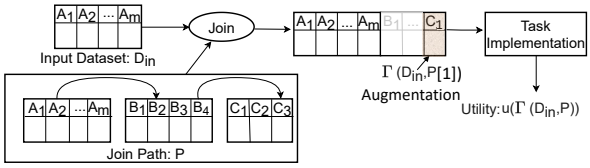


Fig. 1: Notation of a data augmentation pipeline.

paths over larger ones by using combinatorial testing.

METAM: Goal-oriented data discovery We combine the aforementioned properties into an anytime algorithm that finds augmentation candidates by adaptively querying the task. Although goal-oriented data discovery is NP-hard, the algorithm is guaranteed to identify an approximate solution under reasonable assumptions which hold in practice. We also show the efficiency guarantees of the algorithm, and implement it as part of METAM, an end-to-end data discovery approach.

Evaluation Results. METAM automatically finds useful augmentations among millions of them within minutes. We evaluate METAM on large repositories with data from cities in the United States and Kaggle, and for prescriptive analytics tasks such as *what-if* and *how-to* analysis in causal inference, classification and regression in supervised ML, as well as entity linking, and clustering.

Outline. We present notation and discuss the problem in Section II. We present insights for interventional queries in Section III, algorithm in Section IV, and it's analysis in Section V. We present evaluation in Section VI, and related work in Section VII.

II. Goal-Oriented Data Discovery

In this section, we introduce notation and problem statement in Section II-A, and preliminaries in Section II-C.

A. Problem Definition

Figure 1 presents a summary of the notation used in the paper. Let $\mathcal{R}(A_1, \ldots, A_m)$ denote a relation schema over m attributes, where A_i denotes the i^{th} attribute. D comprises a schema $\mathcal{R}(A_1, \ldots, A_m)$ and a list of tuples T where each tuple $t \in T$ is a specific instance of the schema. Using this notation, we now define a noisy dataset and a data repository.

Definition 1 (Noisy Structured Data)**.** *A noisy structured dataset D is characterized by incomplete schema information $\mathcal{R}(A_1, \ldots, A_m)$ where $A_i = \phi$ denotes missing header values and a list of tuples T which may contain less than m values. Additionally, certain tuples may contain duplicate values.*

Definition 2 (Data Repository)**.** *A data repository contains a set of datasets $\mathcal{D} = \{D_1, \ldots, D_n\}$, where different datasets D_i, D_j may contain overlapping values. Additionally, the datasets may lack schema and contain missing values.*

The different datasets that are joined with the input dataset D_{in} are denoted as a join path, defined below.

Definition 3 (Join Path)**.** *A join path P is defined as an ordered list of noisy datasets $P \equiv \{D_1, \ldots, D_t\}$ such that datasets D_j and D_{j+1} join for all $j < t$ with each other forming a chain of join operations. The dataset formed by joining these t datasets is considered to be augmented with the original dataset D_{in}.*

The augmentation of a join path P to a dataset D_{in} in denoted by $\Gamma(D_{in}, P)$. Augmenting a join path P with D_{in} can augment multiple new columns, some of which may not be required by the downstream task. To distinguish between these different augmented columns, we define augmentation as the projection of the join consisting of a single column.

Definition 4 (Augmentation)**.** *An augmentation $\Gamma(D_{in}, P[j])$ is defined as the j^{th} column that is added after joining the path P with the input dataset D_{in}.*

Note that $\Gamma(D_{in}, P) \equiv \{\Gamma(D_{in}, P[j]), \forall j \in \{1, 2, \ldots, l\}\}$, where l is the number of columns after materializing the join-path P. Γ considers materializing a join path to add a new attribute. Other types of data augmentation include unions (addition of additional rows), relational embedding based augmentations, spatio-temporal augmentations, etc. Empirically, we observed that join-path based addition of new attributes is commonly used for popular applications. However, our framework extends to any general set of augmentations. A task t is a black box consisting of the analysis algorithm that takes a dataset (D_{in} or an augmented version of D_{in}) as input and outputs the performance measure of the task. We call "query" the process of obtaining a task's utility on a dataset. We formally define the utility score of the task as follows.

Definition 5 (Utility score)**.** *Given a task t that operates on a dataset D, the utility score $u_t(D)$ is defined as the objective (or the evaluation metric) of the task when operated with D.*

Without loss of generality, we assume that the utility score is normalized $u_t(\cdot) \in [0, 1]$ and that higher utility score means better task quality. Different sets of augmentations may improve the task's utility. We seek a *minimal* set that contains only the augmentations that help to maintain high utility.

Definition 6 (Minimal set of Augmentations)**.** *Given an initial dataset D_{in}, a mechanism to compute $u_t(\cdot)$, a set of augmentations $\mathcal{P} = \{P_1[i_1], \ldots, P_j[i_j]\}$ is considered minimal if removing any attribute from $\Gamma(D_{in}, \mathcal{P})$ reduces task utility.*

Problem II.1 (Goal-oriented discovery)**.** *Given an initial dataset D_{in}, a task t that has a mechanism to compute $u(D')$ for any table D', the problem of identifying augmentations that optimize the task is to identify a minimum augmentation set $\mathcal{T}$ such that the augmented dataset has utility $u(\Gamma(D_{in}, \mathcal{T})) \geq \theta$.*

B. Task implementation and utility calculation

A task takes a dataset as input and returns a utility score that depends, at least in part, on the input dataset. Some examples:

1) Predictive analytics: The task trains a machine learning model over an input training dataset. The training code could be simple, e.g., a random forest, or complex, such as an AutoML solution. The utility correspond to a model-relevant metric such as accuracy, F1 score and may also include fairness and robustness metrics.

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:59:06 UTC from IEEE Xplore. Restrictions apply.

Rottnest: Indexing Data Lakes for Search

Ziheng Wang*+ Stanford University
Sasha Krassovsky* Anthropic PBC
Conor Kennedy*, Alex Aiken Stanford University
Weston Pace LanceDB, Inc.
Rain Jiang*, Huayi Zhang, Chenyu Jiang, Wei Xu Bytedance, Inc.

***Abstract*—Data lakes have become widely popular in managing enterprise data. Their widespread integration with query engines has allowed them to displace specialized data warehouses as the single source of truth for enterprise data. While the columnar storage format and block min-max indices allow query engines to achieve competitive performance on relational data analytics queries, they are not yet suitable for other search-oriented queries like full text and vector nearest neighbor search. We present Rottnest, a general system that builds additional lightweight indices on top of data lakes. We show that our system is more cost efficient compared to un-indexed data lakes or specialized databases across several orders of magnitude of total query loads and operating time horizons.**

I. Introduction

Data lakes have become mainstream in analytical data processing. They typically consist of Parquet files on object storage such as AWS S3 organized by a table format like Delta Lake, Iceberg or Hudi [1]–[5]. The Parquet files are queried on-demand with engines such as Trino, Databricks, Snowflake, or DuckDB [6]–[10]. For OLAP SQL workloads, the performance of such query engines is often better than specialized data warehouses [11], [12]. The widespread support of Parquet among data science and machine learning tools have made data lakes ideal for the analytical "single source of truth" in modern enterprise data stacks.

However, data lakes are still unsuitable for emerging workloads that are **search-oriented**. These workloads, such as high-cardinality time series, text, and embedding analytics typically need to quickly drill down to a very small subset of the data and perform complex aggregations on this subset. In most cases, the minimal indexing available in current Parquet-based data lakes do not allow efficient evaluation of the filter conditions (e.g. UUID match, text regex, approximate nearest neighbors), making these workloads inefficient for query engines like SparkSQL or Trino [6], [13]. As a result, the lakehouse paradigm breaks for these workloads – organizations have to duplicate their data to a specialized data management system like Clickhouse, ElasticSearch or Qdrant to perform efficient search analytics, as shown in Figure 1.

Modern data lakes store data primarily as immutable Parquet files in object storage, optimized for large-scale analytics. Data lakes like Delta Lake and Apache Iceberg added transactional support to data lakes by maintaining lightweight metadata files alongside the Parquet files [4], [5], [14]. These systems achieve ACID properties efficiently through a combination of metadata logging and background compaction operations - transaction logs track file-level changes while periodic compactions merge small files to optimize read performance. Building on these principles, we explore if we could support novel search workloads through additional lightweight index files on top of transactional data lakes. We build Rottnest, a system that augments data lakes with external indices resident on object storage for workloads such as high cardinality analytics, full text search, and vector embedding search.

Although individual components of Rottnest's novel architecture such as inverted indices and LSM-style compaction are well studied, our key insight is that combining them enables efficient search over data lakes with low query cost and storage overhead. Rottnest's architecture is enabled by a novel consistent-on-demand indexing protocol that can be bolted on to any data lake and object storage backend. We also propose a novel evaluation methodology that demonstrates Rottnest's effectiveness across a wide range of operational scenarios.

Just as data lakes provide "good enough" transactional semantics for offline big data workloads without replacing OLTP databases, Rottnest provides "good enough" search capabilities for offline workloads without aiming to replace specialized search systems in latency-sensitive online serving. We believe it can be the most compelling option for most offline workloads such as historical log analytics and LLM pretraining data exploration.

In summary, we make three contributions:

1) A novel bolt-on, consistent-on-demand protocol for concurrent index maintenance. Our protocol, with formal correctness proofs, enables consistent indexing even as the underlying data lake undergoes operations like compaction and deletion. Unlike existing alternatives, it only relies on strong read-after-write consistency, making it compatible with all major cloud providers.
2) A new system architecture to support search-oriented workloads over data lakes using our lazy indexing protocol that combines in-situ querying of Parquet files directly and object-store native indices to deliver low query cost and low storage overhead.
3) A novel evaluation framework using physics-inspired *phase diagrams* to analyze total cost of ownership (TCO) across different query loads and time horizons. This methodology reveals insights about system trade-offs that would be difficult to obtain through traditional performance evaluations, and can be applied to evaluate other cloud data systems where TCO is the relevant metric.

* Equal contribution. + Currently at Anthropic PBC.

2025 IEEE 41st International Conference on Data Engineering (ICDE) | 979-8-3315-3603-9/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICDE65448.2025.00139

2375-026X/25/$31.00 ©2025 IEEE
DOI 10.1109/ICDE65448.2025.00139

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:38:49 UTC from IEEE Xplore. Restrictions apply.

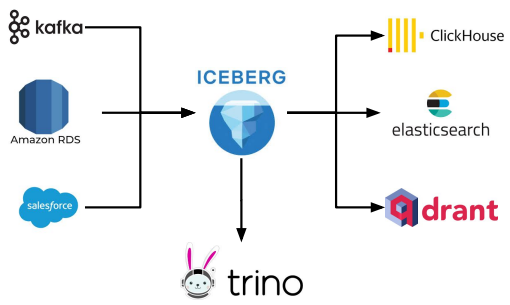


Fig. 1. Typical enterprise data stack. For workloads where directly querying the data lake with a query engine like Trino is too inefficient, a specialized system like Clickhouse, ElasticSearch or Qdrant is used.

II. Motivations

In this section, we describe the access characteristics of Parquet-based data lakes to explain why search-based workloads can be challenging. We also describe existing approaches commonly taken to address those challenges.

A. Parquet Data Lakes on Object Storage

Organizations have started using object-storage-based data lakes based on Parquet files as the single source of truth for enterprise data, due to their wide ecosystem integrations, compelling cost characteristics and transactional semantics. As depicted in Figure 1, operational data is first ETLed into the data lake from OLTP databases, CRMs, and custom connectors. Once the data lands in the data lake, a query engine such as Trino or SparkSQL is used to query the data lake directly for analytical workloads.

Parquet files are well optimized for OLAP workloads against object storage. A Parquet file can consist of multiple row groups, where each row group in turn consists of a collection of column chunks. A column chunk in a row group consists of all the values of a column in the row range corresponding to the row group. Min-max statistics on the column chunks assist query engines in performing predicate pushdown, reading only column chunks necessary for a particular query. These column chunks, which range from several to hundreds of MB in size, can be downloaded in parallel via byte-range requests against object storage. With VM instance bandwidth reaching 100Gbps in major public clouds, analytics on Parquet files residing in object storage has reached interactive speeds.

B. Search Workloads

While data lakes are designed for SQL OLAP use cases, they still fall short in emerging search-oriented workloads, such as:

- **High-cardinality filtering**, such as filtering on a UUID column. Examples include observability workloads (filtering by Kubernetes pod name) and blockchain analytics (filtering by transaction hash).
- **Substring search**, such as log search or text analytics. For example, to detect if evaluation datasets are leaked in the pretraining corpus, one could perform a substring search against the training records, which might be stored as a text column in a data lake.
- **Vector embedding search**, i.e. approximate nearest neighbor search. Examples include retrieval-augmented-generation (RAG) and recommendation systems.

While query engines can efficiently use Parquet-based data lakes for OLAP workloads, they struggle to do so on these new workloads. There are two main issues:

1) **Useless indices.** In search workloads, it is often infeasible to keep the data sorted based on a high-cardinality column and maintain insertion performance. For example, time series data such as IT monitoring and blockchain logs are naturally sorted by timestamp, and sorting them based on UUID tags amount to an extremely expensive transpose. Natural sort-orders do not exist for text or vector embeddings. These two factors render min-max indices on column chunks useless for search workloads.
2) **Read granularity.** Even if an appropriate index exists, column chunks corresponding to text or binary data types typically tend to be tens to hundreds of MBs in size due to Parquet's design (further explored in Section V-A). This means for highly selective search queries, current query engines have to retrieve many MBs of data from object storage just to return a result that is tens of bytes.

"Lakehouse" query engines today like Spark, Trino, or DuckDB today typically have to perform expensive full-column scans which read GBs to TBs of data to perform search queries, making them very expensive.

C. Existing Approaches

Today practitioners commonly employ two approaches to tackle such search workloads, copy data (ETL) and brute force scan. We also mention an emerging third approach, which is new data formats.

1) Copy Data: Practitioners today commonly copy data from the data lake back into specialized systems tailor-made for their workload, such as Clickhouse [15], ElasticSearch [16], vector databases like Qdrant [17] (Figure 1). While this approach guarantees the best performance for those queries by leveraging specialized systems, it largely negates the benefits of the data lake for the data in question. These systems tend to be compute-storage integrated and are expensive to manage. In addition, the data pipelines duplicate data and reintroduce data consistency and staleness issues that data lakes seek to address in the first place.

While we believe that this is the best approach if the search queries are frequent or require strict latency SLAs, e.g. a search engine like Google, we observe that many important workloads do not have such requirements. For example, an LLM pretraining data exploration system might serve only up to 100 internal users while query latencies up to a minute are acceptable. Similarly, a log management system might serve only a particular SRE team where query latencies up to a few seconds are acceptable.

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:38:49 UTC from IEEE Xplore. Restrictions apply.

2023 IEEE 39th International Conference on Data Engineering (ICDE) | 979-8-3503-2227-9/23/$31.00 ©2023 IEEE | DOI: 10.1109/ICDE55515.2023.00123

Schema Matching using Pre-Trained Language Models

Yunjia Zhang
University of Wisconsin-Madison
yunjia@cs.wisc.edu

Avrilia Floratou
Microsoft
avflor@microsoft.com

Joyce Cahoon
Microsoft
jcahoon@microsoft.com

Subru Krishnan
Microsoft
subru@microsoft.com

Andreas C. Müller
Microsoft
amueller@microsoft.com

Dalitso Banda*
Sentry
dalitso.banda@sentry.io

Fotis Psallidas
Microsoft
fopsalli@microsoft.com

Jignesh M. Patel
University of Wisconsin-Madison
jignesh@cs.wisc.edu

***Abstract*—Schema matching over relational data has been studied for more than two decades. However, the state-of-the-art methods do not address key modern-day challenges encountered in real customer scenarios, namely: 1) no access to the source (customer) data due to privacy constraints, 2) target schema with a much larger number of entities and attributes compared to the source schema, and 3) different but semantically equivalent entity and attribute names in the source and target schemata. In this paper, we address these shortcomings. Using real-world customer schemata, we demonstrate that existing linguistic matching approaches have low accuracy. Next, we propose the Learned Schema Mapper (LSM), a novel linguistic schema matching system that leverages the natural language understanding capabilities of pre-trained language models to improve the overall accuracy. Combining this with active learning and a smart attribute selection strategy that selects the most informative attributes for users to label, LSM can significantly reduce the overall human labeling cost. Experimental results demonstrate that users can correctly match their full schema while saving as much as 81% of the labeling cost compared to manual labeling.**

I. Introduction

More and more organizations in various industries (retail, healthcare, manufacturing, etc.) are adopting a data-driven approach to discover non-trivial and strategic insights. At the same time, the amount of information collected has never been greater [1], highlighting the need for modern data analytics tools that can keep up with the volume, variety, and velocity of data associated with different industries. To that end, multiple companies are offering industry-specific analytics solutions [2]–[4] tailored to meet customers' domain-specific needs. Azure Industry solutions [2], for example, offers a variety of services that aim to optimize industry-specific processes, provide enhanced collaboration capabilities and reduce the time to actionable insights. Enabling these enhanced experiences often requires a substantial understanding of customer data. To tackle this problem, the service operator usually maps the customer's data to a known industry-specific schema (ISS) whose semantics are clearly defined, and then builds tools on top of it to streamline various processes.

*Work done while at Microsoft.

In this paper, we study the schema matching problem in the context of known ISSs and customer data in the form of a relational schema. While the problem has been studied for over two decades, we find that existing solutions are not adequate in our environment.

First, information about customer data is often limited due to privacy considerations. In particular, we find that customers are reluctant to grant access to individual data records [5], [6]. As a result, the service operator cannot leverage customer data records when performing schema matching. The same considerations do not constrain their access to the customer schemata. Indeed, since the outcome of schema matching reveals what entities and attributes in the known ISS are present in the customer data, further obfuscating the customer schema from the service operator is not valuable [1]. As a result, customers are generally willing to provide schema information to the operator to facilitate schema matching.

Second, ISSs are typically large (in terms of entities and attributes) as they try to capture a wide variety of concepts associated with a given industry. The customer schema, however, does not necessarily encapsulate all these concepts and is often much smaller than the target ISS. This discrepancy can complicate the schema matching process as it results in a large number of available (and potentially irrelevant) candidate matches for each attribute of the customer schema.

Finally, the entity and attribute names of the customer schema are often hard to understand due to abbreviations and/or customer-specific terminology, further complicating the automation of schema matching for the service operator.

The externally-imposed customer constraints along with the challenges associated with real customer schemata significantly limit the accuracy of existing schema matching approaches (see Section III). In this work, we propose the Learned Schema Matcher (LSM), a novel human-in-the-loop linguistic schema matching approach. LSM does not require access to the individual data records (data-free schema matching) but only to the source schema, respecting the customer

[1]There are other scenarios where both the source and target schemata need to be obfuscated from the service operator [7]. Extending our approach to meet these more stringent privacy requirements is left for future work.

2375-026X/23/$31.00 ©2023 IEEE
DOI 10.1109/ICDE55515.2023.00123

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:57:05 UTC from IEEE Xplore. Restrictions apply.

requirements. At the core of our approach lies a fine-tuned language model that is able to scale to large ISSs and allows us to better handle noise in the source schema. Further, by combining this model with active learning and a smart interaction strategy to obtain targeted feedback from the user, we can fully map a source schema to an ISS while incurring significantly less human labeling cost than prior work.

Our contributions are the following:

- We study the state-of-the-art linguistic schema matching approaches on real-world schemata from Microsoft customers and demonstrate that they have low accuracy.
- We develop LSM, a novel matching approach that relies on a fine-tuned pre-trained language model to tackle data-free schema matching. To the best of our knowledge, this is the first work demonstrating that leveraging the natural language capabilities of pre-trained language models can improve the accuracy of data-free schema matching. Previous methods that are either not data-free [8]–[10] or do not leverage natural language understanding [11]–[15].
- The externally-imposed customer constraints render near-100% accuracy virtually impossible without humans-in-the-loop. We thus propose to combine our schema matching model with an active learning strategy. Our method employs a novel attribute selection strategy to obtain targeted feedback from the user with the goal of reducing the human labeling cost.
- We evaluate LSM on real customer schemata and publicly available datasets. We show that with LSM, users can correctly match their full schema to ISS with a 81% reduction in labeling cost compared to manual labeling.

The remainder of the paper is organized as follows: Section II contains the problem statement. Section III presents an evaluation of the state-of-the-art linguistic matching approaches on real customer schemata. Section IV describes the design of our proposed LSM method. Section V contains our experimental evaluation, and Section VI discusses our experiences in designing LSM. Finally, Section VII presents the related work, and our conclusions are stated in Section VIII.

II. Problem Statement

We now define the problem of schema matching in our environment. Given a source (customer) schema and a known target (ISS) schema that both follow the E/R model [16], our goal is to map each attribute of the source schema to an attribute at the target schema. Figure 1 shows an example of an ISS and a customer schema and their corresponding match.

More formally, we are given a source schema S_s consisting of a set of entities E_s, a set of attributes A_s, and a set of PK/FK relationships R_s. Similarly, we also have a known target schema S_t that consists of a set of entities E_t, a set of attributes A_t, and a set of PK/FK relationships R_t. We assume that each attribute in A_s (A_t) belongs to a single entity in E_s (E_t).

Each entity e ($e \in E_s \cup E_t$) has a name $e.name$, a primary key $e.pk$, and a set of foreign keys $e.fks$. Each attribute a ($a \in A_s \cup A_t$) has a name $a.name$, a data type $a.dtype$, and optionally a natural language description $a.desc$ associated with it.

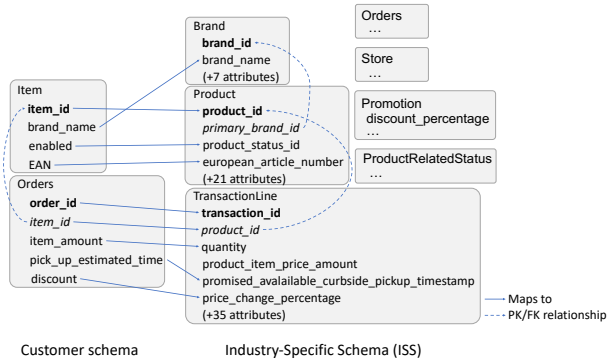


Fig. 1: An example customer schema and corresponding ISS.

We define an attribute correspondence as $r_{ij} = (a_i, a_j)$ where a_i is an attribute in the source schema S_s and a_j is an attribute at the target schema S_t. As discussed in [17], intuitively, an attribute correspondence specifies a relationship between a_i and a_j. The correspondence may specify that the two attributes are equal to each other or that there may be a transformation function involved (e.g., from dollars to euros). In this work, we focus on correspondences that denote equality and leave more complex transformations as part of future work.

Definition 1. *An entity match $\mathcal{M}$ is a triple (e_s, e_t, m), where e_s is an entity in E_s, e_t is an entity in E_t, m is a set of attribute correspondences between e_s and e_t, and each source and target attribute occurs in at most one correspondence in m.*

Based on the above definition, we now define the schema matching problem addressed in this paper.

Definition 2. *The result $\overline{\mathcal{M}}$ of the schema matching process is a set of entity matches between entities in E_s and in E_s, where each attribute in S_s and S_t appears in at most one entity match.*

As we discuss in Section IV, our approach generates $\overline{\mathcal{M}}$ by utilizing all the available information in the source and target schemata (i.e., entities, attributes, and PK/FK relationships).

III. Motivation

Given that in our setting, we do not have access to the customer data, we evaluated the applicability of the state-of-the-art schema matching approaches that rely solely on schema-level information. In this section, we demonstrate that prior approaches have poor accuracy on our real customer schemata. We then identify a set of challenges that motivate the design of the Learned Schema Matcher.

State-of-the-art Methods. We choose four representative schema-based matching methods: CUPID [11], COMA [18],

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:57:05 UTC from IEEE Xplore. Restrictions apply.

Selecting Sub-tables for Data Exploration

Yael Amsterdamer
Bar-Ilan University
amstery@cs.biu.ac.il

Susan B. Davidson
University of Pennsylvania
susan@cis.upenn.edu

Tova Milo
Tel Aviv University
milo@post.tau.ac.il

Kathy Razmadze
Tel Aviv University
kathyr@mail.tau.ac.il

Amit Somech
Bar-Ilan University
somecha@cs.biu.ac.il

2023 IEEE 39th International Conference on Data Engineering (ICDE) | 979-8-3503-2227-9/23/$31.00 ©2023 IEEE | DOI: 10.1109/ICDE55515.2023.00192

***Abstract*—Data scientists frequently examine the raw content of large tables when exploring an unknown dataset. In such cases, small subsets of the full tables (*sub-tables*) that accurately capture table contents are useful. We present a framework which, given a large data table T, creates a sub-table of small, fixed dimensions by selecting a subset of T's rows and projecting them over a subset of T's columns. The question is: Which rows and columns should be selected to yield an *informative* sub-table?**

Our first contribution is an informativeness metric for sub-tables with two complementary dimensions: *cell coverage*, which measures how well the sub-table captures prominent data patterns in T, and *diversity*. We use association rules as the patterns captured by sub-tables, and show that computing optimal sub-tables directly using this metric is infeasible. We then develop an efficient algorithm that indirectly accounts for association rules using *table embedding*. The resulting framework produces sub-tables for the full table as well as for the results of queries over the table, enabling the user to quickly understand results and determine subsequent queries. Experimental results show that high-quality sub-tables can be efficiently computed,and verify the soundness of our metrics as well as the usefulness of selected sub-tables through user studies.

***Index Terms*—Interactive data exploration and discovery**

I. Introduction

Data exploration is the process of understanding an unfamiliar dataset and determining what part of the data is relevant to an analysis task. The process is often done by iteratively applying exploratory queries, examining their results, and interpreting them with data visualizations.

Numerous lines of work have studied solutions for different aspects of the exploration process, such as query suggestions (e.g., [1], [2]), query formulation for non-programmers [3]–[6], insights-discovery [7]–[9], and visualization recommendations [10]–[12].

In this work, we focus on the task of *tabular data display*: before applying additional queries or visualizations, users often manually examine the raw table or query results. This is done using commands such as the table display of the popular Pandas library[1], which shows a small subset of the raw table rows and columns that is easier for a human user to inspect than the full table. However, a disadvantage of the Pandas tabular display is that its choice of rows and columns to display is arbitrary: it includes a few of the first and last rows and columns. As a result, the sub-tables may include arbitrary data values and cross-column value combinations rather than ones that reflect prominent trends; it may also elide columns that are important for further exploration and analysis.

[1]Pandas: Python Data Analysis Library. https://Pandas.pydata.org/

Example I.1. *Consider a table T taken from the Kaggle* flights *dataset [13] which contains 31 columns and ∼6M rows. T may be used for analysis tasks such as predicting flight delays and cancellations.*

The analyst starts by inspecting the data using Pandas display(T)*, which yields the sub-table displayed at the top left of Figure 1.*

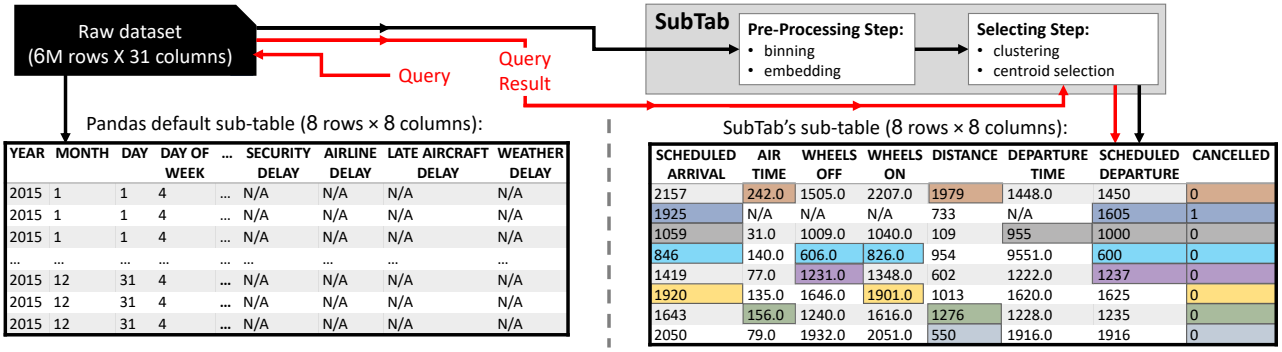
The usefulness of this display for data exploration is intuitively limited: for instance, the first rows, last rows, and last columns are repetitive, and the displayed values are arbitrary. In addition, the chosen columns may not be relevant for the analyst's goals: for instance, the analyst may be interested in flight cancellations, yet there is no reason to assume that the columns in the Pandas *sub-table are relevant for this goal.* □

Our goal is to develop a foundational approach for an informed selection of sub-tables that characterizes a given table as a whole and captures multiple diverse patterns. Concretely, given a table T with n rows and m columns, our goal is to create a sub-table T_{sub} with $k << n$ rows and $l << m$ columns which is a subset of k rows projected over a subset of l columns of T. To do this, we introduce a novel notion of sub-table informativeness which reflects (1) the extent to which the sub-table captures prominent *data patterns* and trends in the full table, including cross-column patterns. This is based on *association rules* (ARs) as a standard type of pattern that applies to a subset of a row's cells and hence allows several such patterns to be captured is a small sub-table. (See a discussion of other types of patterns in Section VII.); (2) the *diversity* in the sub-table, including representative values from each selected column and minimizing data repetitions. We also allow users to specify *target columns* that affect the choice of sub-tables so that it captures columns and ARs relevant to the target column.

Example I.2. *Continuing with the* flights *dataset, an* informative *sub-table is shown at the right of Figure 1. Unlike the sub-table at the top left, this sub-table captures several prominent ARs that hold over the input table and that include the target column* CANCELLED*. For purposes of illustration, we show this by highlighting cells that participate in one of the ARs that hold for the row (many additional ARs hold). For example, the highlighted rule in the first row states that long flights (*AIR_TIME$\in [198.0, 422.0]$ *and* DISTANCE$\in [1546.0, 2724.0]$*) are likely* not *to be cancelled (*CANCELLED$= 0$*). The highlighted rule in the second row states that short afternoon flights (according to the* SCHEDULED_DEPARTURE *and* SCHED-

2375-026X/23/$31.00 ©2023 IEEE
DOI 10.1109/ICDE55515.2023.00192

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:58:35 UTC from IEEE Xplore. Restrictions apply.



YEAR	MONTH	DAY	DAY OF WEEK	...	SECURITY DELAY	AIRLINE DELAY	LATE AIRCRAFT DELAY	WEATHER DELAY
2015	1	1	4	...	N/A	N/A	N/A	N/A
2015	1	1	4	...	N/A	N/A	N/A	N/A
2015	1	1	4	...	N/A	N/A	N/A	N/A
...	...	...	...	...	...	...	...	...
2015	12	31	4	...	N/A	N/A	N/A	N/A
2015	12	31	4	...	N/A	N/A	N/A	N/A
2015	12	31	4	...	N/A	N/A	N/A	N/A

SCHEDULED ARRIVAL	AIR TIME	WHEELS OFF	WHEELS ON	DISTANCE	DEPARTURE TIME	SCHEDULED DEPARTURE	CANCELLED
2157	242.0	1505.0	2207.0	1979	1448.0	1450	0
1925	N/A	N/A	N/A	733	N/A	1605	1
1059	31.0	1009.0	1040.0	109	955	1000	0
846	140.0	606.0	826.0	954	9551.0	600	0
1419	77.0	1231.0	1348.0	602	1222.0	1237	0
1920	135.0	1646.0	1901.0	1013	1620.0	1625	0
1643	156.0	1240.0	1616.0	1276	1228.0	1235	0
2050	79.0	1932.0	2051.0	550	1916.0	1916	0

Fig. 1: System Workflow. SubTab takes as input a raw table used throughout the EDA session. The sub-table on the left is the output of the display command in Pandas, a random subset of the table's rows and columns. The sub-table on the right is the output of SubTab, in which some of the ARs are highlighted for purposes of illustration. SubTab has two-steps: (1) Pre-Processing Step - the table is binned and embedded into vectors; (2) Selecting Step - triggered by a display of the input table or as the result of a query over the table, an *informative sub-table* is displayed. This is done by clustering the row and column vectors of the query result in the embedded space and selecting the centroids as representatives.

ULED_ARRIVAL columns) are likely to be canceled. The sub-table also shows diverse values per column by including representatives of diverse sub-groups in the value distribution, e.g., morning, afternoon and night arrivals/departures, short, medium and long flight distances, etc. □

We formalize (Section II) our notion of informativeness based on a combination of two complementary metrics: *cell coverage* and *diversity*. Cell coverage measures how well the sub-table T_{sub} represents data patterns in the full table T, which are defined via ARs.

Concretely, given a set of prominent ARs that hold over the full table T,[2] our metric reflects the number of cells in T that are describable by ARs that are captured in T_{sub}. If one or more *target columns* are known in advance to be the focus of the analysis, they will be included in the l selected columns, and we measure cell coverage only according to ARs that include one or more target columns. For the second metric, *diversity*, we split the value distribution of a column (continuous or categorical) into meaningful *bins*. We then compute a Jaccard-like row similarity score based on these bins, and use the average pairwise row distance as the diversity score. Intuitively, in a small sub-table we cannot capture the full distribution of each column, and our metric is geared towards increasing the number of represented bins and reducing bin repetitions. We use a score combining these metrics to measure the informativeness of a sub-table.

Unfortunately, we show (Section III) that optimizing our informativeness measures directly is generally infeasible, and discuss the limitations of approximate solutions. In particular, although there are several efficient techniques for mining ARs (e.g., [14]–[18]), they are still too time-consuming for large datasets in an interactive setting.

We therefore propose a sub-table selection method which *indirectly* accounts for ARs using *table embedding* [19], [20], and that allows us to avoid calculating ARs thereby speeding up and simplifying the entire process (Section IV). Concretely, given a table T we use *binning* [15] to split each column's values into a small set of meaningful groups. We then compute an embedding of table cells as vectors. As we demonstrate in Section IV-B, the embedding captures bin co-occurrences, and therefore implicitly corresponds to ARs. To select rows and columns for a sub-table we derive from the cell vectors a vector representation for rows and columns, cluster them (separately), and select the centroids as rows and columns that represent diverse patterns in the data.

An important benefit of our solution design is in *responding to queries over* T: during the exploratory data analysis (EDA) session, users typically issue different queries on a given table T (red arrows of Figure 1). Our computation of cell embedding may be viewed as a part of the pre-processing step of a given data table T, along with the binning of its values (first blue box in Figure 1). This step is executed once upon loading the table. Then, for subsequent selection-projection query Q on T, we can compute the vector representation of rows and columns in $Q(T)$ based on the cells of T that appear in them, and re-execute clustering and centroid selection (*Selecting* step in Figure 1, shown as the second blue box). This significantly speeds up sub-table computation compared with computing everything from scratch (a few seconds instead of up to a minute for large tables).

Contributions: Main contributions of this work include:

1) A formal notion of *informativeness* for sub-tables, designed to measure how well a sub-table characterizes the full table. The capturing of prominent data patterns in the sub-table is measured by *cell coverage* (for ARs), while the ratio of non-repetitive, representative values is measured by *diversity*.

2) A *complexity analysis* of the problem of selecting an optimal sub-table, which shows the infeasibility of computing sub-tables that optimize our informativeness metrics.

3) A *greedy algorithm* (CCSG), which traverses column combinations, and greedily selects rows for each such combination. This algorithm approximates the optimal cell coverage if all column combinations can be traversed within a given

[2]There are standard metrics we can use to measure the prominence of ARs in T, such as Support and Confidence [14]. Also see Section V-D.

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:58:35 UTC from IEEE Xplore. Restrictions apply.

Sudowoodo: Contrastive Self-supervised Learning for Multi-purpose Data Integration and Preparation

Runhui Wang[§]
Computer Science Department
Rutgers University
New Brunswick, United States
runhui.wang@rutgers.edu

Yuliang Li
Megagon Labs
Mountain View, United States
yuliang@megagon.ai

Jin Wang
Megagon Labs
Mountain View, United States
jin@megagon.ai

2023 IEEE 39th International Conference on Data Engineering (ICDE) | 979-8-3503-2227-9/23/$31.00 ©2023 IEEE | DOI: 10.1109/ICDE55515.2023.00391

***Abstract*—Machine learning (ML) is playing an increasingly important role in data management tasks, particularly in Data Integration and Preparation (DI&P). The success of ML-based approaches, however, heavily relies on the availability of large-scale, high-quality labeled datasets for different tasks. Moreover, the wide variety of DI&P tasks and pipelines oftentimes requires customizing ML solutions at a significant cost for model engineering and experimentation. These factors inevitably hold back the adoption of ML-based approaches to new domains and tasks.**

In this paper, we propose Sudowoodo, a multi-purpose DI&P framework based on contrastive representation learning. Sudowoodo features a unified, matching-based problem definition capturing a wide range of DI&P tasks including Entity Matching (EM) in data integration, error correction in data cleaning, semantic type detection in data discovery, and more. Contrastive learning enables Sudowoodo to learn similarity-aware data representations from a large corpus of data items (e.g., entity entries, table columns) without using any labels. The learned representations can later be either directly used or facilitate fine-tuning with only a few labels to support different DI&P tasks. Our experiment results show that Sudowoodo achieves multiple state-of-the-art results on different levels of supervision and outperforms previous best specialized blocking or matching solutions for EM. Sudowoodo also achieves promising results in data cleaning and column matching tasks showing its versatility in DI&P applications.

I. Introduction

Machine learning, particularly deep learning (DL), is playing an increasingly important role in almost all fields in computer science including data management [1], [2], [3], [4]. More recently, this trend has extended to Data Integration and Preparation (DI&P) [5], [6], [7], [8] with learning-based solutions making promising progress and advancing the state-of-the-art in multiple tasks, including entity matching [9], [10], data cleaning [11], [12], [13], [14], and table annotation [15], [16], [17]. For example, Entity Matching (EM) solutions based on pre-trained language models such as BERT [18] achieve new state-of-the-art results across multiple matching tasks [19], [10]. While shown to be effective, as we describe next, existing learning-based solutions for DI&P suffer from two major challenges making them less attractive in practice.

Challenge 1: Label Requirement. The success of ML-based approaches comes at the cost of creating large-scale, high-quality annotated datasets. For example, in Entity Matching, learning-based methods typically require around 10k or more labeled entity pairs to achieve the best pairwise matching performance [9], [10]. Creating high-quality datasets of such cardinality can be quite expensive considering the required human efforts, which make ML-based solutions less accessible to new DI&P tasks and domains.

Challenge 2: Task Variety. There is a wide scope of DI&P tasks requiring ML models for different problem formulations. This means that oftentimes, practitioners have to build a specialized ML solution for each task, resulting in extra costs of model engineering. For example, Entity Matching typically consists of two tasks, *blocking* and *matching* [20], with the goal of (1) filtering out candidate matching pairs and (2) performing the pairwise comparison, respectively. The two tasks are closely related, but because of their different problem formulations, existing studies typically propose specialized solutions [21], [9], [19], [22], [10] that require separate modeling, data annotation, and experimentation efforts. for each task. This has also been the case for other DI&P tasks in data cleaning and data discovery [11], [12], [13], [14], [15], [16], [17].

To address these two challenges, we proposed Sudowoodo, a multi-purpose DI&P framework based on *contrastive representation learning* [23]. Sudowoodo features a generic DI&P problem definition of data matching: given a collection of data items (e.g., entity entries, table columns, cell values), identify all pairs of compatible items. This generic definition allows Sudowoodo to support a wide range of tasks in data integration, cleaning, and discovery. As Figure 1 illustrates, EM can be formulated as a task of matching entity entries; error correction in data cleaning [12], [13], [14] can be formulated as matching erroneous cell values with candidate corrections; and column matching, which is widely used in data discovery [24], [15], can be formulated as matching table columns that have the same semantic type. The matching criteria are also customizable w.r.t the specific application. In this work, we consider any criteria that can be cast into a general binary relation, such as whether two entity entries refer to the same real-world entity for EM.

Sudowoodo addresses the label requirement challenge by

[§]This work was done during the internship at Megagon Labs

2375-026X/23/$31.00 ©2023 IEEE
DOI 10.1109/ICDE55515.2023.00391

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:56:35 UTC from IEEE Xplore. Restrictions apply.

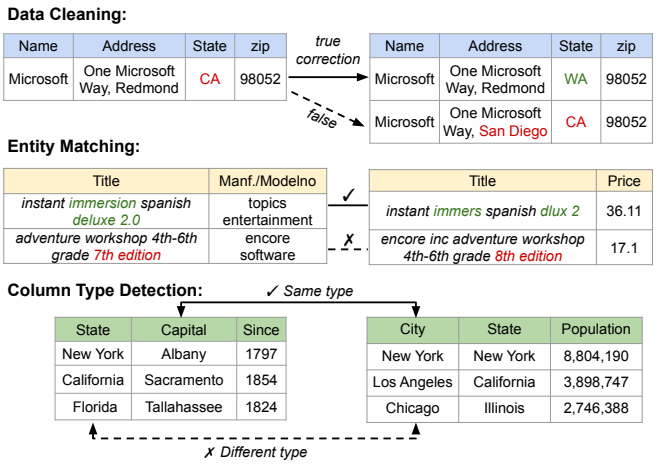


Fig. 1: Sudowoodo supports any DI&P tasks that can be formulated as a general form of matching task.

leveraging contrastive learning to learn a data representation model from a large collection of unlabeled data items, such as corpora of entity entries or table columns. Unlike that of language models such as BERT [18], the effectiveness of Sudowoodo's pre-training process relies on its contrastive learning objective instead of learning linguistic characteristics. The contrastive objective allows Sudowoodo to learn how to distinguish pairs of similar data items from dissimilar ones that are likely to be distinct. In this way, without using any labels, Sudowoodo learns meaningful representations where similar data items are close in the representation space while pairs of dissimilar data items are far apart.

After the pre-training process, the learned data representation model is accessible to downstream tasks either directly in an unsupervised manner or by *fine-tuning* using a few labels. Unlike that in language models, fine-tuning is not always necessary if the downstream task is similarity-based for which Sudowoodo already provides a reliable metrics space. When fine-tuning is needed, Sudowoodo provides an optional *pseudo labeling* step that extracts training signals with high confidence from the learned representations useful for further boosting the fine-tuning performance. As we will illustrate, pseudo labeling together with a customized fine-tuning architecture significantly improves Sudowoodo's performance on DI&P tasks in low-resourced settings.

Example 1: We illustrate the Sudowoodo pipeline in Figure 2 using Entity Matching (EM), a typical application of data integration. The upper part of the figure shows a standard EM pipeline consisting of the blocking and matching sub-tasks. Given two collections of entity entries (Tables A and B), the goal of blocking is to select a candidate set of matching pairs from the cross product to avoid the quadratic-size pairwise comparison. Next, it trains a matching model on a labeled dataset sampled from the candidate set, before applying the model to all the candidate pairs for the final matching results.

Sudowoodo starts with contrastive learning to learn the similarity-aware data representations of all entity entries from the two input tables. Because its model already captures entity similarity, we can solve the blocking task by applying

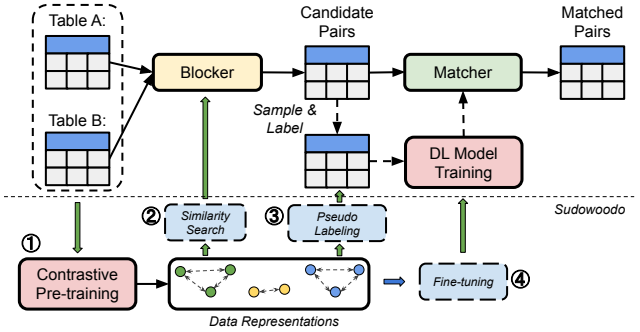


Fig. 2: The Sudowoodo pipeline for EM (blocking+matching).

nearest neighbor search to identify the top-k most similar entity entries for all input entries as the candidate set. For the matching task, practitioners can then fine-tune the pre-trained representation model on a labeled training set. For this step, Sudowoodo's pseudo labeling module allows automatic creation of additional high-quality labels by extracting match/unmatch pairs with high confidence from the learned representations. By doing so, Sudowoodo significantly reduces the label requirement for training the matching model.

Further optimizations. Based on the standard contrastive learning framework, Sudowoodo features several optimizations to further boost the pre-training effectiveness. First, the quality of learned representations heavily depends on the generated pairs of data items that are semantic-preserving. Sudowoodo obtain such pairs via data transformation such as synonym replacement or span deletion. To obtain more fine-grained transformations, Sudowoodo supports the *cutoff* data augmentation (DA) technique [25] that transforms input data items at the word-embedding level. Combining cutoff with a package of DI&P data augmentation operators, Sudowoodo obtains more diverse views for contrastive pre-training.

Another challenge for applying contrastive learning to DI&P tasks is how to obtain more effective negative examples. Negative examples allow the representation model to learn how to separate dissimilar data items. The standard contrastive learning approach uses uniformly sampled pairs as negative examples. This simple approach can be insufficient when random pairs fail to provide strong enough negative examples because they are easily separable. To address this issue, Sudowoodo splits the unlabeled data items into clusters and samples negative pairs within each cluster. By doing so, Sudowoodo allows the representation model to learn "harder" by distinguishing syntactically similar items.

In summary, this paper makes the following contributions:

- We propose a generic matching problem definition that captures a wide variety of DI&P tasks including (1) EM in data integration, (2) Error correction in data cleaning, (3) Semantic type detection in data discovery, and more.
- We propose Sudowoodo, a contrastive representation learning framework for the supported DI&P tasks. Sudowoodo pre-trains a representation model from a large collection of unlabeled data items. Practitioners can then apply the representations to downstream tasks either directly or by

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:56:35 UTC from IEEE Xplore. Restrictions apply.

TabSketchFM: Sketch-based Tabular Representation Learning for Data Discovery over Data Lakes

Aamod Khatiwada[⋆▶] Harsha Kokel[⋆] Ibrahim Abdelaziz Subhajit Chaudhury Julian Dolby
Northeastern University *IBM Research* *IBM Research* *IBM Research* *IBM Research*
khatiwada.a@northeastern.edu harsha.kokel@ibm.com ibrahim.abdelaziz1@ibm.com subhajit@ibm.com dolby@us.ibm.com

Oktie Hassanzadeh Zhenhan Huang[▶] Tejaswini Pedapati Horst Samulowitz Kavitha Srinivas
IBM Research *Rensselaer Polytechnic Institute* *IBM Research* *IBM Research* *IBM Research*
hassanzadeh@us.ibm.com huangz12@rpi.edu tejaswinip@us.ibm.com samulowitz@us.ibm.com kavitha.srinivas@ibm.com

2025 IEEE 41st International Conference on Data Engineering (ICDE) | 979-8-3315-3603-9/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICDE65448.2025.00118

***Abstract*—Enterprises have a growing need to identify relevant tables in data lakes; e.g. tables that are unionable, joinable, or subsets of each other. Tabular neural models can be helpful for such data discovery tasks. In this paper, we present TabSketchFM, a neural tabular model for data discovery over data lakes. First, we propose novel pre-training: a sketch-based approach to enhance the effectiveness of data discovery in neural tabular models. Second, we finetune the pretrained model for identifying unionable, joinable, and subset table pairs and show significant improvement over previous tabular neural models. Third, we present a detailed ablation study to highlight which sketches are crucial for which tasks. Fourth, we use these finetuned models to perform table search; i.e., given a query table, find other tables in a corpus that are unionable, joinable, or that are subsets of the query. Our results demonstrate significant improvements in F1 scores for search compared to state-of-the-art techniques. Finally, we show significant transfer across datasets and tasks establishing that our model can generalize across different tasks and over different data lakes.**

I. Introduction

Enterprises store critical data in *data lakes*, large repositories of tabular data, for both governance and analytics [1]. It is essential to effectively find relevant tables (e.g., joinable, unionable, subsets) within lakes for reporting, decision-making, statistical analysis, and more [2], [3]. For instance, *Unionable table search* helps to augment an existing table with new rows, and to enrich analytics over it [4]. *Joinable table* search is useful to run analytics that needs data in columns from multiple tables, e.g., to identify how many customers in some region purchased a product due to an email campaign. Also, identifying *subsets* or *supersets* of a table is essential, for example, when searching for potential copies of personal data to comply with the General Data Privacy Regulation of the European Union[1]. Motivated by such use cases, we focus on the problem of identifying tables from the data lakes that can be *unionable, joinable, and/or subsets of each other* [4]–[6]. Specifically, in this work, we propose a pre-trained tabular model that could help in enhancing the effectiveness of data discovery techniques.

⋆ Joint first-authors ▶ Work done while at IBM Research.

[1]https://gdpr-info.eu

Central to the problem of identifying relevant tables in the data lakes is identifying similar columns in tables. However, the columns in a table could be ambiguous. For instance, a column called `Age` is different when it is in a table about buildings from when it is in a table about people. Furthermore, values within a column `Age` play a key role in determining semantics; `Age` of buildings in New York are unlikely to match `Age` of historical buildings in Egypt. Contextualizing columns could be useful in resolving such ambiguities. Language Models (LMs) are shown to be successful in contextualizing words in a sentence [7]. Consequently, we use them to contextualize columns and use such contextualization to determine similar columns or tables.

Existing pre-trained tabular models [8]–[11] based on LMs [7] do not focus on identifying relevant tables. Instead, they focus on language-style tasks such as fact checking [10], question answering [12], converting natural language to SQL [11], table cell content population [8], and table metadata prediction [8], [9]. In addition, input to these models is the actual cell values or a row because of the focus on language-style tasks. However, this has two limitations for understanding relevant tables: (a) only a small number of values can be passed in as input because of severe limits on token length in encoder-only models that can generate embeddings for search (e.g. 512 tokens in BERT [7]), and this potentially limits the information available to the model to understand the input table [2] ; (b) treating numerical values in the table as text tends to lose their semantics; it may be better to pass numerical information directly as vectors into the model.

To address these limitations on input length and numerical representation, we introduce a novel transformer-based tabular model—*TabSketchFM*—exploiting both table metadata and

[2] Decoder only models have larger context lengths. However, it has been difficult until recently to use decoder-only models for embeddings [13]. Moreover, these models are expensive to deploy and provide only a minimal improvement in accuracy over encoder-only models such as BERT. This is why embeddings for search are mostly based on BERT-style models. Further, even with large context lengths, it is not practical to serialize tables with millions of rows as sentences.

2375-026X/25/$31.00 ©2025 IEEE
DOI 10.1109/ICDE65448.2025.00118

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:39:19 UTC from IEEE Xplore. Restrictions apply.

cell values that creates different sketches over them instead of linearizing cell values. Specifically, we abstract column cell values using data sketches, as has been used in the data discovery literature (e.g., MinHash sketches [14], and numerical sketches [15]). Such sketches capture tabular features, and we input them into the neural models. Inputting numerical sketches bypasses feeding table cell values to the model, addressing the token limit issue, as well as the numerical representation of cell values.

After pretraining using sketches, we finetune the models for individual data discovery tasks such as union, join, and finding subsets. The embeddings given by the finetuned models are then used for the respective retrieval tasks. While some in the literature [16] have used self supervision for fine tuning, self supervision applies better for some tasks such as unions compared to say joinability. To finetune models for the data discovery tasks, we used a suite of 8 benchmarks across the 3 tasks *LakeBench* that we have open sourced for use [17]. On the fine tuning tasks, *TabSketchFM* outperformed other neural methods, by as much as 55% on F1. We then used the fine tuned models for search tasks, and showed that *TabSketchFM* outperforms the state-of-the-art neural and traditional methods by as much as 70% on F1.

Importantly, we also compare the performance of specialized traditional and neural models for data discovery against an off-the-shelf pretrained model that encodes column values as a single sentence. To our surprise, the off-the-shelf model performs much better than existing systems - in fact, for certain tasks such as union, it is sufficient to use that model for discovering unionable tables; our tabular specific model does worse, as do all other models that target unionability. The insight from this finding is that while some data discovery tasks need values to match across columns (e.g. join or subsets), for union, it is sufficient when columns have the same semantic meaning; indeed there is no need for any value overlap. Sentence embeddings capture those semantics well. Because certain tasks seem to benefit from these sentence embeddings (e.g. union), and other do not (e.g., subset), we propose a way to concatenate the embeddings from sentence encoders with tabular model embeddings, and show that one can augment tabular embeddings with value embeddings to improve performance, at least in some cases.

In summary, our contributions are as follows:

- **A novel tabular pretrained model.** We are the first to present a sketch-based tabular pretrained model, *TabSketchFM*, which contextualizes columns based on their data sketches and tabular metadata. The sketches help to better handle input token limits and numerical cell values. To achieve this, we modified the text-based transformer architecture to take in numerical and MinHash sketches as input. A purely sketch based approach though misses the opportunity to include table semantics from column values. Given that the inputs to the transformer architecture in our system are numeric vectors, it is difficult to combine these numeric inputs with tokens derived from cell values. We therefore combine the sketch based model embeddings with column value embeddings from existing pretrained sentence transformer models. We show that this combination boosts performance on some data discovery tasks.
- **Finetuning models for downstream tasks.** We develop cross-encoders from pretrained TabSketchFM, which we fine-tune for a range of downstream tasks from the LakeBench collection [17]. We fine tune not only *TabSketchFM* but other tabular pretrained models to see if they can perform well on these downstream tasks and present the comparision.
- **Detailed ablation study.** We assess the effectiveness of different sketches for identifying different types of table relevance. Our study presents interesting insights such as Minhash sketches of column values are crucial for identifying joins, but numerical sketches are crucial for identifying subsets; once again highlighting the value of using sketches for table representation.
- **Demonstration of TabSketchFM on search.** We compared the performance of *TabSketchFM*'s fine tuned models against neural and non-neural baselines. In the case of join search, *TabSketchFM* outperformed the nearest systems geared for join search by almost 70% in F1 scores, but tellingly, the sentence encoding model outperformed existing systems as well by 63%. By combining the two embeddings (from TabSketchFM and sentence encodings), we gain almost 73% over other systems in F1.
- **Illustrate generalization.** Finally, we illustrate the main advantage of learning a pretrained foundational model by showing the generalization of search performance across tasks. For example, we fine tune the pretrained model on join identification on one dataset and use it for union search on another dataset. Our evaluation shows that our finetuned models generalize well.

II. Related Work

Tabular Pretrained Models. Neural models are pretrained on tabular datasets to either recover masked tokens in the table or to detect corrupt values [8]–[11]. [18] surveys neural models for tabular data representation and their applications. [8] combined the Masked Language Model (MLM) objective from BERT [7] with a novel Masked Entity Recovery to train TURL. [10] use MLM with novel Cell-level Cloze and Table Context Retrival for TUTA. [11] used Masked Column Prediction and Cell Value Recovery for TABERT while [9] repurposed ELECTRA's objective function [19] for TABBIE. These models are finetuned and evaluated for different downstream tasks such as table metadata prediction, table content population, fact-checking, question answering, semantic parsing, and so on. For example, TABERT is evaluated on neural semantic parsing; TABBIE for column, row, and column type population tasks; and TUTA is finetuned and evaluated for cell and table type classification tasks. These downstream tasks take either a table or a table with a question as input; they do not consider the inter-table tasks such as dataset discovery, which we consider in this paper. Furthermore, they adopt

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:39:19 UTC from IEEE Xplore. Restrictions apply.

Ver: View Discovery in the Wild

Yue Gong*, Zhiru Zhu*, Sainyam Galhotra, Raul Castro Fernandez
Department of Computer Science, The University of Chicago
Email: {yuegong, zhiru, sainyam, raulcf}@uchicago.edu
*Yue Gong and Zhiru Zhu contributed equally

2023 IEEE 39th International Conference on Data Engineering (ICDE) | 979-8-3503-2227-9/23/$31.00 ©2023 IEEE | DOI: 10.1109/ICDE55515.2023.00045

***Abstract*—We present `Ver`[1], a data discovery system that identifies project-join views over large repositories of tables that do not contain join path information, and even when input queries are inaccurate. `Ver` implements a reference architecture to solve both the technical (scale and search) and human (semantic ambiguity, navigating a large number of results) problems of view discovery. We demonstrate users find the view they want when using `Ver` with a user study and we demonstrate its performance with large-scale end-to-end experiments on real-world datasets containing tens of millions of join paths.**

I. Introduction

The existence of large repositories of data such as those that originate from the combination of disparate databases [1], data lakes [2], open data portals [3], and cloud repositories [4] has the upside of offering opportunities to find valuable data for tasks such as machine learning, reporting, and data analytics. The downside is the resulting data discovery problem: identifying a combination of datasets useful for the downstream task even when these reside in different databases. For example, a machine learning engineer may need a training dataset that requires combining a table in a database with the one sitting on the enterprise data lake. Large volumes of often incomplete and noisy data without any join information, which we call *pathless table collections*, make solving data discovery difficult and thus hampers productivity.

There are several approaches to identify project-join views (PJ-views) over pathless table collections. Discovery systems such as Aurum [5], Goods [6], Auctus [7], Juneau [8], Josie [9], Table-Union [10], D3L [11] and libraries such as LSHEnsemble [12], and Lazo [13] help with identifying datasets that satisfy some relevance criteria whether via keywords [6] or programs [5]. Analysts can then combine the datasets to verify that it satisfies the view they need. Another approach is query-by-example (QBE) interfaces [14], [15] that lets users provide examples of the view they need. These can be adapted, with effort, to run on top of data discovery systems. Whether via keywords, programs, or QBE interfaces, the result of discovery queries over pathless table collections leads to ambiguous results that include semantically distinct results, duplicates or near-duplicates, and different versions of the data. This ambiguity further complicates identifying the right view. More generally, solving view discovery in the wild requires addressing the following challenges:

- **Challenge 1. Noisy Queries.** Users provide queries that represent their best knowledge of the data in the form of keywords, programs, and examples. User-provided input may or may not appear in the table collection and they may be noisy and incorrect.
- **Challenge 2. Noisy Join Paths.** Pathless table collections do not include join paths. Identifying true join paths automatically is impossible. We resort to identifying inclusion dependencies, a proxy to join paths, i.e., a join path implies an inclusion dependency but not the other way around.
- **Challenge 3. Large Number of Join Paths.** Large volumes of data result in a large number of join paths that require efficient data structures to represent and navigate them.
- **Challenge 4. Noisy Result Views.** A large number of join paths implies many views may satisfy a user query. Such result views will be noisy due to ambiguity in the user query and noisy join paths. Concretely, there will be duplicate result views, views that are contained within each other, others that are complementary, and others that will show contradictory values for the same key.
- **Challenge 5. Result View Navigation.** Result views will contain semantically ambiguous results, e.g., views with "work address" and "home address". Only users know the right context so the challenge is to elicit that context and use it to choose the view they need among all result views.

In this paper, we introduce a *reference architecture to identify PJ-views in the wild*. Discovering a PJ-view over pathless table collections requires understanding human preferences and requirements (which we refer to as a *human* problem) and solving a *technical* problem. The architecture divides the larger problem into smaller ones, each of which we tackle with a different component. Reference architectures help conceptualize problems and have been influential in advancing the field. For example, [16], [17] for relational databases and [18], [19], [20], [21] for data integration. The reference architecture we propose tackles the five challenges above. While challenges 1-3 are addressed by existing work, we introduce new techniques to address Challenge 4 and 5 in this paper, and demonstrate them as part of an implementation of the reference architecture. We make the following contributions:

- An end-to-end system, **`Ver`**, that identifies PJ-views among tens of millions of join paths. **`Ver`** implements the reference architecture for QBE-based interfaces. We choose QBE because it permits users to declare the table they would like to find, even when they do not know examples and can only provide attribute names (Section III). **`Ver`** relies on existing work [5] to address Challenges 1-3.

[1]code is available at https://github.com/TheDataStation/ver

2375-026X/23/$31.00 ©2023 IEEE
DOI 10.1109/ICDE55515.2023.00045

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:58:08 UTC from IEEE Xplore. Restrictions apply.

- A **view presentation** component that helps humans to identify a good view among many. The approach uses a novel bandit-based approach to *learn* user-specific discovery preferences, and it uses abduction-based reasoning to quickly narrow down the search space, reducing the number of human interactions (Section IV).
- A **view distillation** component that automatically filters out views by classifying them into **4C categories**: *compatible*, *contained*, *complementary*, and *contradictory*. Besides helping with filtering, these categories produce information used by the view presentation component (Section V).

We conduct an IRB-approved user study (Institutional Review Board) [22] to validate **Ver**'s approach to the human problem. We conduct thorough experiments on queries from open data repositories that lack join paths.

II. Definitions and Problem Statement

$\mathcal{R}(A_1, \ldots, A_m)$ is a relation schema over m attributes, where A_i denotes the i^{th} attribute. A table D comprises a schema $\mathcal{R}(A_1, \ldots, A_m)$ and a set of tuples T where each tuple $t \in T$ is a specific instance of the schema.

In practice, tables do not look like the ideal defined above because they may *lack header information*, have *ambiguous names* and contain *dirty and noisy data*. More formally:

Definition 1 (Noisy structured data): A noisy data D is characterized by an incomplete schema information $\mathcal{R}(A_1, \ldots, A_m)$ where $A_i = \phi$ for missing header values and tuples T such that each tuple $t \in T$ contains at most m values.

In addition, a pathless collection may contain tables with contradictory values, e.g., two census tables with different population counts for the same states of the country. Formally, two tables D_i and D_j contradict if the tables contain different values for the same key. We discuss the detection of contradictions in Section V.

Definition 2 (Pathless table collection): A pathless table collection contains a set of noisy tables $\mathcal{D} = \{D_1, \ldots, D_n\}$ where each D_i is a noisy table and tables D_i, D_j may contain contradictory values.

PJ-example-query. A PJ-query (hereafter called query) contains (possibly noisy) example tuples of the desired output. The examples are a proxy to user's discovery requirements. The quality of examples depends on user's knowledge. Given a query q, there may be many tables that contain the input examples, and many combinations of these may satisfy q, resulting in a large number of candidate PJ-views.

Definition 3 (Noisy query): An input query q is a noisy table consisting of l example tuples, $\chi = \{\chi_1, \ldots, \chi_l\}$ where each χ_i is a noisy tuple denoting example values that are expected to be present in the desired output. The different columns in the examples χ are denoted by $\chi.A_i, \forall i \in \{1, \ldots, \tau\}$, where τ is the number of attributes in the input query.

PJ-views are constructed by joining datasets through keys. We first define a join path and then use it to discuss the effects of noise, followed by a formal problem statement.

Definition 4 (Join path): A join path P is defined as an ordered set of noisy tables $P \equiv \{D_1, \ldots, D_t\}$ such that tables D_j and D_{j+1} join for all $j < t$ via a key column $k \in D_j, D_{j+1}$ forming a chain of join operations.

Joinable datasets can be identified in the presence of key information, which is generally absent in pathless scenarios. A PJ-view V is the dataset formed after materializing the path P followed by projection, i.e., choosing the relevant columns.

Problem 1 (Project-Join view discovery over pathless table collections): Consider a pathless table collection $\mathcal{D}$ and a query q with examples χ. The goal is to construct a minimal candidate set of PJ-views $\mathcal{D}'$ that satisfy the user requirements.

Users may require the PJ-view to contain all examples χ, or any of them, depending on user and application. Formalizing these requirements is outside the scope of this work.

III. A Reference Architecture

Why a Reference Architecture? From a systems engineering perspective, a reference architecture is the materialization of a "divide and conquer" strategy that splits complex engineering problems into smaller parts. Thus, a reference architecture describes a collection of components and their interactions. In making a reference architecture concrete, we state our understanding of the problem and represent it as a concrete artifact that the community can scrutinize and improve.

Design Overview

Finding a PJ-view in the wild requires solving a *human* and a *technical* problem. We present a reference architecture for view discovery that contains components targeting both *human* and *technical* problems. Along with each component, we discuss implementation options, including those made by **Ver**. We also briefly discuss two novel components, View Distillation and View Presentation.

Overview. Algorithm 1 shows pseudocode (human components are highlighted in blue) and Fig. 1 shows **Ver**'s architecture inside a funnel, denoting the gradual reduction of views as data flows downstream. **Ver** builds a discovery index offline.

Discovery Engine and Index Creation (technical). *This component builds indices over pathless table collections: i) a join path index, which can be approximate; ii) retrieval indices over table names, values, attribute names and column similarity. The indices are available online, via the Engine's API to other components. (**Challenge 2**).*

The indices can be built using state-of-the-art methods such as Aurum [5] (that **Ver**'s implementation uses), Auctus [7], JOSIE [9], LSHEnsemble [12]. After the indices are built, users design and submit queries via a VIEW-SPECIFICATION component (line 1).

View Specification (human). Discovery interfaces include spreadsheet-style, keyword search, APIs, natural language, and combinations of these. The reference architecture supports these interfaces via the View-Specification component. For QBE-based interfaces, as implemented by **Ver**, the input is a set of examples, χ, and the output of this stage is a set of example attributes and values. VIEW-SPECIFICATION

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:58:08 UTC from IEEE Xplore. Restrictions apply.